Análisis y Diseño de Algoritmos

Dr. José Martínez Carranza

Otoño 2026 - Bryan Ezequiel Violante Arriaga

Índice

1. Grafos **2**

2. Teoría de grafos: terminología, conexidad y Euler **17**

3. Hamilton, bipartitos, cliques y árboles **34**

4. Representaciones, BFS y DFS a fondo **53**

5. Aplicaciones de DFS **76**

6. Árboles de expansión mínima **96**

7. Caminos más cortos desde una fuente **110**

8. Demostraciones y problemas de caminos más cortos **132**

9. Caminos más cortos entre todos los pares **151**

10. Flujo máximo: Ford-Fulkerson y Edmonds-Karp **172**

11. Emparejamiento bipartito máximo **189**

12. Empujar y reetiquetar **203**

1. Grafos

22 de octubre de 2026

Resumen rápido. Un grafo es un par de conjuntos, vértices y aristas, y modela cualquier cosa hecha de objetos y relaciones entre pares de ellos: ciudades y carreteras, páginas y ligas, estados de un juego y jugadas. Se representa con matriz o con listas de adyacencia, y se clasifica por sus aristas: dirigido o no, pesado o no, cíclico o acíclico, simple o no, encajado o topológico, implícito o explícito, etiquetado o no. Se vieron los dos recorridos, búsqueda en anchura (con cola) y en profundidad (con recursión o pila), y tres problemas: camino más corto con Dijkstra y Floyd, detección de ciclos con BFS y árbol de expansión mínima con Kruskal. Todos los ejemplos usan los mismos tres grafos.

Esta clase es la base de la parte de grafos: presenta cada tema con un ejemplo y la idea de por qué funciona. Las pruebas, las variantes y los temas que no cupieron están en las secciones de complemento que siguen, cada una sobre un bloque:

- Sección 2: terminología completa (caminatas, senderos, caminos, circuitos y ciclos), conexidad y recorridos de Euler.
- Sección 3: ciclos de Hamilton, grafos bipartitos, cliques y árboles.
- Sección 4: las representaciones y BFS y DFS a fondo: colores, tiempos, pruebas de corrección y DFS iterativo.
- Sección 5: orden topológico, ciclos dirigidos y componentes fuertemente conexas con DFS.
- Sección 6: árboles de expansión mínima: la propiedad del corte, Kruskal y Prim.
- Sección 7: caminos más cortos desde una fuente: relajación, Bellman-Ford, caminos en DAG y Dijkstra con su prueba.
- Sección 8: las demostraciones de las propiedades de la relajación y problemas de caminos más cortos.
- Sección 9: caminos más cortos entre todos los pares: Floyd-Warshall con predecesores, cerradura transitiva y Johnson.
- Sección 10: flujo máximo con Ford-Fulkerson y Edmonds-Karp.
- Sección 11: emparejamiento bipartito máximo.
- Sección 12: empujar y reetiquetar, la otra familia de algoritmos de flujo máximo.

Definición formal

Definición 1.1 (Grafo)**.** Un **grafo** es un par $G = (V, E)$ donde V es un conjunto finito de **vértices** y E un conjunto de **aristas**. En un grafo no dirigido cada arista es un par no ordenado de vértices distintos,

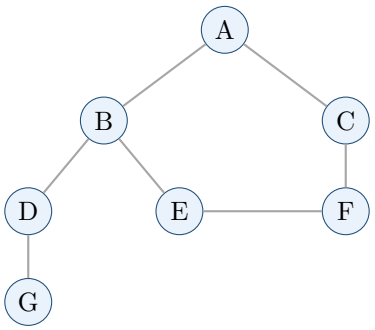
$$E \subseteq \{\{u, v\} : u, v \in V,\ u \neq v\}.$$

Se escribe $n = |V|$ y $m = |E|$.

Si $\{u, v\} \in E$, los vértices u y v son **adyacentes** y la arista es **incidente** en los dos. El **grado** de un vértice, $\mathrm{gr}(v)$, es el número de aristas incidentes en él. Otras fuentes, como Cormen, escriben $\deg(v)$.

Ejemplo 1.2 (El grafo G de esta clase)**.**

$$V = \{A, B, C, D, E, F, G\}, \qquad E = \{\{A, B\}, \{A, C\}, \{B, D\}, \{B, E\}, \{C, F\}, \{E, F\}, \{D, G\}\}.$$



$n = 7$ y $m = 7$. Los grados son $\mathrm{gr}(B) = 3$, $\mathrm{gr}(G) = 1$ y 2 para los demás. Este grafo se usa en toda la clase.

Observación. Cada arista suma 1 al grado de sus dos extremos, así que $\sum_{v \in V} \mathrm{gr}(v) = 2m$. En G: $2{+}3{+}2{+}2{+}2{+}2{+}1 = 14 = 2{\cdot}7$. Y como hay a lo más un par por cada dos vértices, $0 \leq m \leq n(n-1)/2$.

Matriz y listas de adyacencia

Definición 1.3 (Matriz de adyacencia)**.** Con los vértices numerados de 1 a n, la matriz de adyacencia es la matriz A de $n \times n$ con

$$A[i][j] = \begin{cases} 1 & \text{si } \{i, j\} \in E, \\ 0 & \text{si no.} \end{cases}$$

Para G, en el orden $A, \ldots, G$:

	A	B	C	D	E	F	G
A	0	1	1	0	0	0	0
B	1	0	0	1	1	0	0
C	1	0	0	0	0	1	0
D	0	1	0	0	0	0	1
E	0	1	0	0	0	1	0
F	0	0	1	0	1	0	0
G	0	0	0	1	0	0	0

En un grafo no dirigido la matriz es simétrica y la suma de cada renglón es el grado del vértice. La diagonal es cero porque la definición no admite lazos. En un grafo dirigido las dos cosas cambian, como se ve más abajo.

La alternativa son las **listas de adyacencia**: para cada vértice, la lista de sus vecinos.

$A : B, C$ $\qquad$ $E : B, F$
$B : A, D, E$ $\qquad$ $F : C, E$
$C : A, F$ $\qquad$ $G : D$
$D : B, G$

Operación	**Matriz**	**Listas**
Memoria	$\Theta(n^2)$	$\Theta(n + m)$
¿Es $\{u, v\}$ arista?	$\mathcal{O}(1)$	$\mathcal{O}(\mathrm{gr}(u))$
Recorrer los vecinos de u	$\Theta(n)$	$\Theta(\mathrm{gr}(u))$
Recorrer todo el grafo	$\Theta(n^2)$	$\Theta(n + m)$

Idea clave

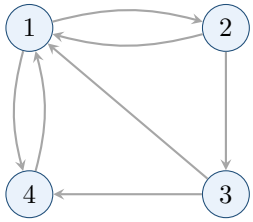
Casi todos los grafos reales son **dispersos**: m es mucho menor que n^2. Un mapa de carreteras con un millón de cruces tiene unos cuantos millones de tramos, no 10^{12}. Ahí la matriz gasta memoria en ceros y las listas ganan. La matriz conviene en grafos **densos** o cuando se pregunta mucho si una arista existe. Las complejidades de esta clase suponen listas.

Tipos de grafos

La clasificación es la de Skiena. Cada par describe una propiedad de las aristas o de cómo se guarda el grafo, y cambia qué algoritmo se puede usar.

Dirigido o no dirigido

En un **grafo dirigido** las aristas son pares ordenados, $E \subseteq V \times V$: la arista (u, v) va de u a v y no al revés. Cada vértice tiene **grado de entrada** $\text{gr}^-(v)$, las aristas que llegan a él, y **grado de salida** $\text{gr}^+(v)$, las que salen, y la matriz de adyacencia deja de ser simétrica. Una calle de un solo sentido o *A sigue a B* en una red social son dirigidas; una amistad no.

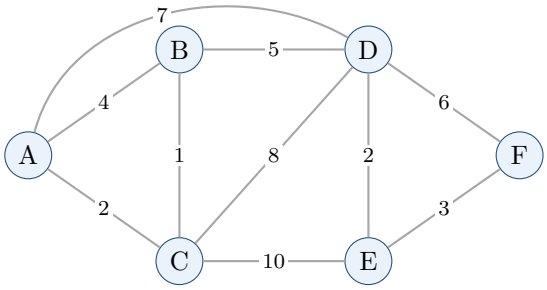


	1	2	3	4
1	0	1	0	1
2	1	0	1	0
3	1	0	0	1
4	1	0	0	0

$A[2][3] = 1$ pero $A[3][2] = 0$: de 2 se llega a 3 en un paso, de 3 a 2 no. El renglón de un vértice suma su grado de salida y su columna, el de entrada: el vértice 1 tiene $\text{gr}^+(1) = 2$ y $\text{gr}^-(1) = 3$. Este grafo se usa en Floyd.

Pesado o no pesado

En un **grafo pesado** cada arista tiene un número, $w : E \to \mathbb{R}$: distancia, tiempo, costo. En la matriz se guarda el peso en lugar del 1, y ∞ donde no hay arista. El 0 de la diagonal de la matriz de abajo no es parte de la representación: es la convención de Floyd, para quien ir de un vértice a sí mismo cuesta 0. Otros problemas llenan distinto las casillas vacías; en flujo, por ejemplo, una arista ausente es una capacidad 0.



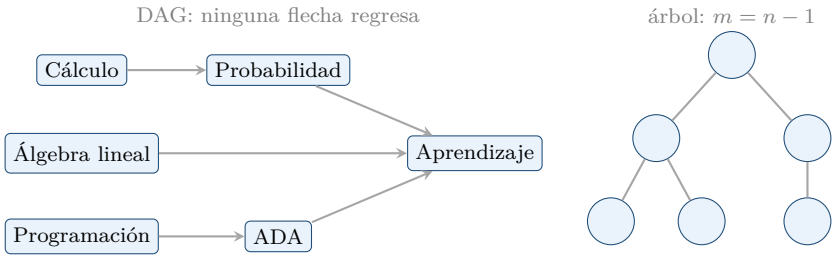
	A	B	C	D	E	F
A	0	4	2	7	∞	∞
B	4	0	1	5	∞	∞
C	2	1	0	8	10	∞
D	7	5	8	0	2	6
E	∞	∞	10	2	0	3
F	∞	∞	∞	6	3	0

En un grafo no pesado el camino más corto es el de menos aristas, y lo encuentra BFS. En uno pesado el de menos aristas puede no ser el más barato: de A a D la arista directa pesa 7, y A, C, B, D pesa $2 + 1 + 5 = 8$. Hacen falta Dijkstra o Floyd. Este grafo se usa en Dijkstra y en Kruskal.

Cíclico o acíclico

Una **caminata** es una sucesión de vértices en la que cada uno es adyacente al siguiente, y puede repetir lo que sea. Si no repite aristas es un **sendero**; si no repite vértices, un **camino**, que es el sentido de "camino" en toda esta clase. Un sendero que regresa a su vértice de inicio es un **circuito**, y un **ciclo** es un circuito que tampoco repite vértices, salvo el primero, que es también el último. G tiene uno: A, B, E, F, C, A. La diferencia importa: un ocho que pasa dos veces por el mismo vértice es circuito pero no ciclo, y en un grafo simple no dirigido todo ciclo tiene al menos tres aristas. En un grafo dirigido el ciclo sigue el sentido de las flechas. La sección 2 desarrolla esta terminología.

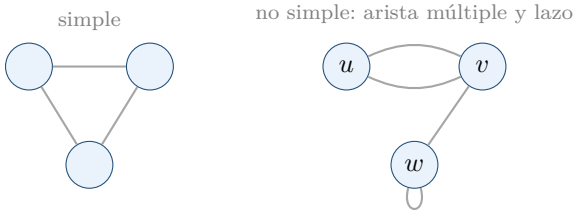
Un grafo sin ciclos es **acíclico**. Un no dirigido, conexo y acíclico es un **árbol**; un dirigido acíclico es un **DAG** (*directed acyclic graph*), como las seriaciones de un plan de estudios.



Un DAG siempre tiene un **orden topológico**: una forma de poner los vértices en fila con todas las flechas hacia adelante, el orden en que se pueden cursar las materias. Un grafo con un ciclo dirigido no lo tiene.

Simple o no simple

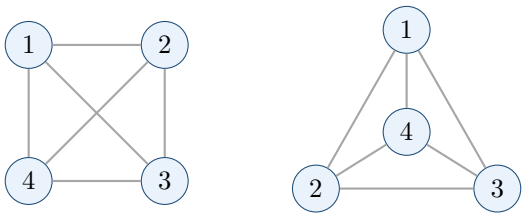
Un grafo es **simple** si no tiene **lazos** (aristas de un vértice a sí mismo) ni **aristas múltiples** (dos aristas entre el mismo par). La definición no dirigida de arriba, con conjuntos, solo admite grafos simples: $\{u, u\}$ no es un par y un conjunto no repite elementos. La dirigida, $E \subseteq V \times V$, sí admite lazos (u, u), aunque tampoco aristas múltiples; un lazo pone un 1 en la diagonal de la matriz y suma 1 a cada grado de su vértice, al de entrada y al de salida. Para tener aristas múltiples E se vuelve un multiconjunto.



Una arista múltiple aparece, por ejemplo, con dos vuelos distintos entre las mismas ciudades. Casi todos los algoritmos suponen grafos simples: con aristas múltiples pesadas basta quedarse con la más barata.

Encajado o topológico

Un grafo es **encajado** (*embedded*) si a sus vértices y aristas se les asignan posiciones geométricas: un punto en el plano para cada vértice y una curva para cada arista. Un grafo es **topológico** si solo importa quién está unido con quién. Los dos dibujos de abajo son el mismo grafo topológico, K_4, con cuatro vértices todos unidos entre sí, en dos encajes distintos:

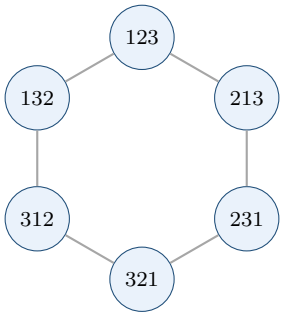


A la izquierda dos aristas se cruzan; a la derecha ninguna. Como grafo topológico es lo mismo; como grafo encajado no. En un mapa la geometría es parte del problema: la distancia entre dos cruces sale de sus coordenadas, y se puede saber si dos calles se cruzan.

Implícito o explícito

Un grafo **explícito** se construye completo y se guarda en una matriz o en listas antes de recorrerlo. Uno **implícito** no se guarda nunca: los vecinos de un vértice se generan cuando se necesitan, con una regla.

Ejemplo 1.4 (Permutaciones como grafo implícito)**.** Los vértices son las permutaciones de $1, 2, 3$, y dos son vecinas si una sale de la otra intercambiando dos elementos contiguos:



Los vecinos de 123 son 213 y 132, y se obtienen con la regla sin haber guardado nada. Con k elementos el grafo tiene $k!$ vértices: con $k = 10$ son 3 628 800, y solo se generan los que el recorrido alcanza a visitar.

Los grafos implícitos aparecen en búsqueda: los estados de un rompecabezas, las posiciones de un juego, las soluciones parciales de un problema de backtracking.

Etiquetado o no etiquetado

En un grafo **etiquetado** cada vértice tiene un nombre propio, como A a G en G, y dos vértices con nombres distintos son distintos. En uno **no etiquetado** los nombres no importan, solo la estructura. La pregunta natural es si dos grafos son el mismo salvo los nombres: si existe una biyección $f : V_1 \to V_2$ con $\{u, v\} \in E_1$ si y solo si $\{f(u), f(v)\} \in E_2$. Es la **prueba de isomorfismo**. No se conoce un algoritmo polinomial para decidirla; en la práctica se hace con búsqueda con retroceso, que descarta pronto las asignaciones imposibles comparando invariantes como el número de aristas o la lista de grados. Los dos dibujos de K_4 de arriba tienen las mismas etiquetas y las mismas aristas, así que son el mismo grafo aun como grafos etiquetados; el isomorfismo hace falta cuando los nombres no coinciden.

Búsqueda en anchura (BFS)

Recorrer un grafo es visitar todos los vértices alcanzables desde uno inicial s, cada uno una sola vez. La **búsqueda en anchura** (*breadth-first search*) visita por capas: primero s, luego todos sus vecinos,

luego los vecinos de esos que no se han visto, y así. Para respetar ese orden usa una **cola**: el primer vértice descubierto es el primero que se procesa.

Algoritmo 1.1 BFS(G, s)

```
1:  para todo v ∈ V hacer
2:      visto[v] ← falso,    d[v] ← ∞,    π[v] ← nulo
3:  fin para
4:  visto[s] ← verdadero,    d[s] ← 0
5:  Q ← cola vacía;  ENCOLAR(Q, s)
6:  mientras Q no está vacía hacer
7:      u ← DESENCOLAR(Q)
8:      para todo v en la lista de adyacencia de u hacer
9:          si no visto[v] entonces
10:             visto[v] ← verdadero,    d[v] ← d[u] + 1,    π[v] ← u
11:             ENCOLAR(Q, v)
12:         fin si
13:     fin para
14: fin mientras
```

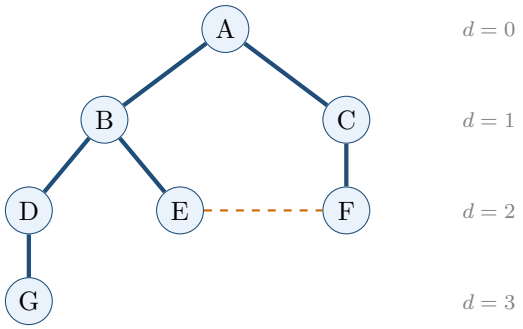
$d[v]$ es la distancia en aristas desde s y $\pi[v]$ el vértice desde el que se descubrió v, su **padre**.

Observación. Cormen escribe los atributos con punto, $v.d$ y $v.\pi$, y NIL para el predecesor vacío; aquí son $d[v]$, $\pi[v]$ y nulo. En DFS Cormen usa $u.d$ y $u.f$ para los tiempos de descubrimiento y de terminación; aquí son entra$[u]$ y sale$[u]$, porque $d[v]$ ya es la distancia de BFS. Dentro de la notación asintótica Cormen escribe V y E por $|V|$ y $|E|$: su $O(V + E)$ es el $\mathcal{O}(n + m)$ de aquí.

Ejemplo 1.5 (BFS sobre G desde A)**.** Los vecinos se recorren en orden alfabético.

Paso	Sale	Descubre	Cola después
1	A	B, C	[B, C]
2	B	D, E (A ya visto)	[C, D, E]
3	C	F (A ya visto)	[D, E, F]
4	D	G (B ya visto)	[E, F, G]
5	E	nada: B y F ya vistos	[F, G]
6	F	nada: C y E ya vistos	[G]
7	G	nada	[ ]

Orden de visita: A, B, C, D, E, F, G. Las aristas por las que se descubrió cada vértice forman el **árbol BFS**:



Cada renglón es una capa: todos los vértices a la misma distancia de A. La arista $\{E, F\}$, punteada,

no está en el árbol: cuando se procesó E, F ya estaba visto.

La inicialización cuesta $\Theta(n)$. Después, cada vértice alcanzable entra y sale de la cola una sola vez, porque se marca al descubrirlo, y su lista se recorre una vez cuando sale. Las listas suman $2m$ entradas en un grafo no dirigido, porque cada arista aparece en las listas de sus dos extremos, y m en uno dirigido. En total BFS cuesta $\mathcal{O}(n+m)$ con listas. Es $\Theta(n+m)$ cuando s alcanza a todos, como en G; si no, puede ser menos, porque las listas de los vértices inalcanzables nunca se recorren. Con matriz, encontrar los vecinos de cada vértice que sale cuesta $\Theta(n)$, y el total es $\mathcal{O}(n^2)$.

Idea clave

En un grafo no pesado, $d[v]$ es la longitud del camino más corto de s a v, y siguiendo π de v hacia atrás se reconstruye ese camino. BFS ya resuelve el camino más corto cuando todas las aristas valen lo mismo. La prueba está en la sección 4.

Ojito...

En clase se dijo que la complejidad depende del problema, y que en un grafo completamente conectado es factorial. Hay que separar dos cosas. BFS procesa cada vértice una vez y cada arista desde sus dos extremos: en el grafo completo K_n, con $m = n(n-1)/2$, cuesta $\Theta(n^2)$, no factorial. Lo factorial aparece cuando el problema exige recorrer **caminos** y no vértices, como probar todas las rutas posibles: desde un vértice de K_5 salen 65 caminos, contando el trivial de longitud cero, y desde uno de K_{10}, 986 410, alrededor de $e \cdot (n-1)!$. O cuando el grafo es implícito y sus vértices son permutaciones, como el ejemplo de arriba: BFS sigue siendo lineal en el tamaño del grafo, pero el grafo mide $k!$.

Búsqueda en profundidad (DFS)

La **búsqueda en profundidad** (*depth-first search*) se mete lo más hondo que puede por un camino y solo regresa cuando se atora: cuando el vértice actual ya no tiene vecinos sin visitar. Se escribe de forma natural con recursión; la pila de llamadas hace el papel que la cola hace en BFS. La versión iterativa usa una pila explícita, con dos trampas que se ven en la sección 4.

Algoritmo 1.2 DFS-Visitar(G, u)

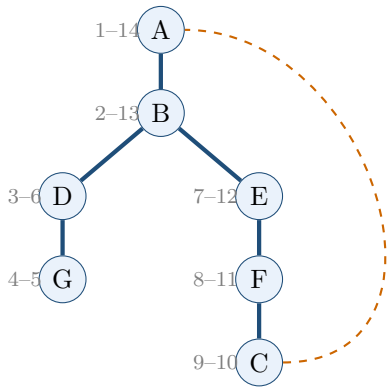
```
1: t ← t + 1,    entra[u] ← t,    visto[u] ← verdadero
2: para todo v en la lista de adyacencia de u hacer
3:     si no visto[v] entonces
4:         π[v] ← u
5:         DFS-Visitar(G, v)
6:     fin si
7: fin para
8: t ← t + 1,    sale[u] ← t
```

t es un reloj global que empieza en 0. entra$[u]$ marca cuándo se descubrió u y sale$[u]$ cuándo se terminó con todo lo que cuelga de él. Antes de la primera llamada se pone visto$[v] \leftarrow$ falso para todo v y $t \leftarrow 0$. Una llamada desde u solo alcanza lo que es accesible desde u; para recorrer un grafo desconectado se lanza DFS-Visitar desde cada vértice que siga sin visitar, y el resultado es un bosque. Ese procedimiento externo, DFS(G), está en la sección 4.

Ejemplo 1.6 (DFS sobre G desde A)**.** Con los vecinos en orden alfabético, como en BFS:

- A → B → D → G. G no tiene vecinos nuevos: se termina, y se regresa a D, que tampoco tiene; se regresa a B.
- B → E → F → C. Los vecinos de C son A y F, los dos vistos: se termina C, luego F, E, B y A.

Orden de visita: A, B, D, G, E, F, C. El de BFS era A, B, C, D, E, F, G.



Junto a cada vértice, entra–sale. Este árbol DFS, el de este orden de las listas, es largo y angosto; el BFS, ancho y bajo. La arista $\{A, C\}$, punteada, es una **arista de retroceso**: se explora primero desde C, cuando A sigue abierto, y va de C a un antecesor suyo en el árbol.

DFS también cuesta $\mathcal{O}(n+m)$ con listas, pero la cuenta es otra, porque no hay cola: DFS-Visitar se llama una vez por vértice, ya que lo primero que hace es marcarlo, y cada llamada recorre su lista una vez. Recorrer todo el grafo cuesta $\Theta(n+m)$.

	BFS	**DFS**
Estructura	cola	pila o recursión
Orden	por capas, de cerca a lejos	un camino hasta el fondo
Árbol	ancho y bajo	largo y angosto
Útil para	camino más corto sin pesos	ciclos dirigidos, orden topológico, componentes
Costo con listas	$\mathcal{O}(n+m)$	$\mathcal{O}(n+m)$

Las aplicaciones de DFS de la tabla se usan aquí sin prueba: un grafo dirigido tiene ciclo si y solo si DFS encuentra una arista de retroceso; en un DAG, ordenar por sale decreciente da un orden topológico; en uno no dirigido, los árboles DFS son las componentes conexas (secciones 5 y 4).

Camino más corto: Dijkstra

Dado un grafo pesado con pesos **no negativos** y un vértice s, el algoritmo de Dijkstra (1959) encuentra la distancia $\delta(s, v)$ de s a todos los demás: el peso del camino más barato. Mantiene una estimación $d[v]$ que empieza en ∞ y nunca baja de $\delta(s, v)$, porque siempre es el peso de una caminata de s a v. En cada paso fija el vértice no fijado con la d más chica: esa ya es su distancia definitiva. Luego **relaja** las aristas que salen de él: si llegar a un vecino pasando por u es más barato que lo que se tenía, se actualiza.

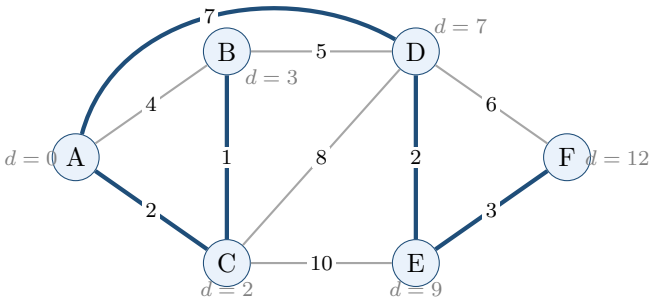
Algoritmo 1.3 Dijkstra(G, w, s)

```
1:  para todo v ∈ V hacer
2:      d[v] ← ∞,    π[v] ← nulo
3:  fin para
4:  d[s] ← 0
5:  Q ← montículo de mínimos con todos los vértices, con llave d
6:  mientras Q no está vacío hacer
7:      u ← Extraer-Min(Q)                          // d[u] queda fija
8:      para todo v adyacente a u hacer
9:          si d[u] + w(u, v) < d[v] entonces       // relajar
10:             d[v] ← d[u] + w(u, v),    π[v] ← u
11:             Decrementar-Llave(Q, v, d[v])
12:         fin si
13:     fin para
14: fin mientras
```

Ejemplo 1.7 (Dijkstra sobre el grafo pesado desde A)**.** Cada renglón muestra las d después de procesar el vértice que sale; en negritas, las ya fijas.

Paso	Sale	A	B	C	D	E	F	Relajaciones
0		0	∞	∞	∞	∞	∞	
1	A	**0**	4	2	7	∞	∞	B, C, D desde A
2	C	**0**	3	**2**	7	12	∞	B: $2+1=3<4$; D: 10, no
3	B	**0**	**3**	**2**	7	12	∞	D: $3+5=8$, no mejora
4	D	**0**	**3**	**2**	**7**	9	13	E: $7+2=9<12$
5	E	**0**	**3**	**2**	**7**	**9**	12	F: $9+3=12<13$
6	F	**0**	**3**	**2**	**7**	**9**	**12**	

B empezó en 4 por la arista directa y bajó a 3 pasando por C. Los padres quedan $\pi[B] = C$, $\pi[C] = A$, $\pi[D] = A$, $\pi[E] = D$, $\pi[F] = E$, y el camino a F se lee de atrás hacia adelante: F $\leftarrow$ E $\leftarrow$ D $\leftarrow$ A, o sea A, D, E, F, con $7 + 2 + 3 = 12$.



En azul, el **árbol de caminos más cortos**: las aristas $\{\pi[v], v\}$. Que el camino de A a cada vértice dentro de este árbol sea uno más corto en el grafo se demuestra en la sección 8.

Por qué funciona. Supóngase que todos los vértices fijados antes que u tienen ya su distancia correcta, y tómese un camino más corto P de s a u. (Si $u = s$, o si no hay camino a u, no hay nada que probar.) Sea y el primer vértice de P que no está fijo, que puede ser el propio u, y x el anterior, que sí lo está. Cuando x se fijó tenía $d[x] = \delta(s, x)$ y relajó la arista hacia y; como esa arista está en un camino más corto, desde entonces $d[y] = \delta(s, y)$. Esto es lo esencial: no basta con que y tenga $d[y] \geq d[u]$, hace falta

que su d ya sea exacta. El tramo de P que va de y a u no resta, porque los pesos son no negativos, así que

$$d[y] = \delta(s, y) \leq \delta(s, u) \leq d[u],$$

y como el algoritmo eligió u, también $d[u] \leq d[y]$. Todo es igual y $d[u] = \delta(s, u)$. Es un algoritmo voraz: la elección del momento nunca se revisa, y con pesos no negativos no hace falta. La prueba completa, con las propiedades de la relajación que usa, está en la sección 7.

Dos consecuencias. Con $w \geq 0$ la condición del **si** nunca se cumple para un vértice que ya salió: si v está fijo, $d[v] = \delta(s, v) \leq \delta(s, u) + w(u, v) = d[u] + w(u, v)$. Así que Decrementar-Llave siempre actúa sobre un vértice que sigue en el montículo. Y si solo interesa un destino t, se puede parar en cuanto t sale del montículo, no cuando apenas se descubre, porque su d todavía puede bajar, como la de B en el ejemplo.

Costo. Cada vértice sale del montículo una sola vez, y solo entonces se recorre su lista. Las listas suman $2m$ entradas (m en un dirigido), así que hay $\mathcal{O}(m)$ relajaciones y cada una hace a lo más un Decrementar-Llave. Con montículo binario, construirlo cuesta $\mathcal{O}(n)$ y las n extracciones y los $\mathcal{O}(m)$ decrementos cuestan $\mathcal{O}(\log n)$ cada uno: $\mathcal{O}((n + m) \log n)$ en total. Es el montículo de la clase del 17 de septiembre, con un detalle: para decrementar la llave de v hay que saber dónde está v dentro del arreglo, así que cada vértice guarda su posición y el montículo la actualiza en cada intercambio. Con un arreglo indexado por vértice, decrementar es una asignación pero buscar el mínimo cuesta $\Theta(n)$, y el total es $\Theta(n^2)$. El montículo gana cuando $m = o(n^2/\log n)$; en un grafo denso, $m = \Theta(n^2)$, cuesta $\mathcal{O}(n^2 \log n)$ y gana el arreglo. Con montículo de Fibonacci se llega a $\mathcal{O}(n \log n + m)$; esa y las demás colas están en la sección 7.

Ojito...

Con un peso negativo Dijkstra deja de ser confiable. Con las aristas dirigidas (A, B) de peso 2, (A, C) de peso 3, (C, B) de peso -2 y (B, D) de peso 2, desde A se fija B con $d = 2$ antes de procesar C, y B relaja (B, D): $d[D] = 4$. Al procesar C, la relajación de (C, B) baja $d[B]$ a $3 - 2 = 1$, porque el pseudocódigo de arriba relaja aunque el vecino ya esté fijo, pero nadie vuelve a relajar (B, D). D termina con 4, y el camino A, C, B, D cuesta 3. Sin la arista (B, D) el resultado final es correcto gracias a esa relajación tardía; lo que falla es que B se dio por fija con un valor que luego cambió, y que se llamó Decrementar-Llave sobre un vértice que ya no estaba en el montículo. El argumento de arriba suponía que el tramo de y a u no resta; aquí, cuando sale B, ese tramo es la arista (C, B). Para pesos negativos están Bellman-Ford (sección 7) y, entre todos los pares, Floyd y Johnson (sección 9).

Camino más corto entre todos los pares: Floyd

Dijkstra da las distancias desde un vértice. Para tener las de todos contra todos se puede correr n veces, una por fuente: con montículo binario son $\mathcal{O}(n(n + m) \log n)$. Floyd (Floyd-Warshall, 1962) lo resuelve con programación dinámica en tres ciclos, en $\Theta(n^3)$, y admite pesos negativos siempre que no haya ciclos de peso negativo. Ninguno gana siempre. En un grafo disperso, $m = \mathcal{O}(n)$, las n corridas de Dijkstra cuestan $\mathcal{O}(n^2 \log n)$ y le ganan a Floyd; en uno denso, $m = \Theta(n^2)$, cuestan $\mathcal{O}(n^3 \log n)$, y Floyd gana además por sencillez.

Con los vértices numerados de 1 a n, un vértice **intermedio** de un camino es cualquiera de sus vértices salvo los dos extremos. Sea $d_{ij}^{(k)}$ el **peso** de un camino más corto de i a j cuyos intermedios están todos en $\{1, \ldots, k\}$. Con $k = 0$ no se permite ningún intermedio y solo queda la arista directa. Para pasar de $k - 1$ a k hay dos opciones: el camino no usa k, y queda igual; o lo usa. Como no hay ciclos negativos,

ese camino se puede tomar sin vértices repetidos, así que k aparece una sola vez y lo parte en dos tramos, de i a k y de k a j, cuyos intermedios están en $\{1, \ldots, k-1\}$. Los dos ya están calculados:

$$d_{ij}^{(k)} = \min\left(d_{ij}^{(k-1)},\ d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right),$$

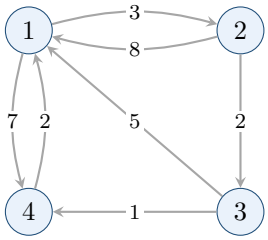
con $d_{ij}^{(0)} = w(i,j)$, 0 en la diagonal e ∞ si no hay arista. La respuesta es $D^{(n)}$, porque con $k = n$ ya se permite cualquier intermedio. La hipótesis de no tener ciclos negativos se usa justo en el segundo caso: si un ciclo negativo pasa por k, cada vuelta abarata el camino, no hay camino más corto y la recurrencia ya no describe nada. Floyd lo delata: al terminar, algún $D[i][i]$ es negativo (sección 9).

Algoritmo 1.4 Floyd(W, n)

```
1: D ← W
2: para k ← 1 hasta n hacer
3:     para i ← 1 hasta n hacer
4:         para j ← 1 hasta n hacer
5:             D[i][j] ← mín(D[i][j], D[i][k] + D[k][j])
6:         fin para
7:     fin para
8: fin para
9: devolver D
```

Son $\Theta(n^3)$ operaciones y $\Theta(n^2)$ de memoria, porque la misma matriz se puede actualizar en su lugar. En la vuelta k el renglón k y la columna k no cambian: sin ciclos negativos $D[k][k] = 0$, y entonces $D[i][k] + D[k][k] = D[i][k]$. Como $D[i][k]$ y $D[k][j]$ son las únicas casillas que se leen además de $D[i][j]$, da igual leerlas antes o después de actualizarlas.

Ejemplo 1.8 (Floyd sobre el grafo dirigido)**.** Los pesos de las aristas del grafo dirigido de arriba:



En negritas, las casillas que cambian en cada paso:

$$D^{(0)} = \begin{pmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{pmatrix} \qquad D^{(1)} = \begin{pmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \mathbf{15} \\ 5 & \mathbf{8} & 0 & 1 \\ 2 & \mathbf{5} & \infty & 0 \end{pmatrix} \qquad D^{(2)} = \begin{pmatrix} 0 & 3 & \mathbf{5} & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & \mathbf{7} & 0 \end{pmatrix}$$

$$D^{(3)} = \begin{pmatrix} 0 & 3 & 5 & \mathbf{6} \\ \mathbf{7} & 0 & 2 & \mathbf{3} \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{pmatrix} \qquad D^{(4)} = \begin{pmatrix} 0 & 3 & 5 & 6 \\ \mathbf{5} & 0 & 2 & 3 \\ \mathbf{3} & \mathbf{6} & 0 & 1 \\ 2 & 5 & 7 & 0 \end{pmatrix}$$

Algunos cambios, uno por paso:

- $k = 1$: de 3 a 2 no había arista; pasando por 1, $d_{31} + d_{12} = 5 + 3 = 8$.

- $k = 2$: de 1 a 3, $d_{12} + d_{23} = 3 + 2 = 5$.
- $k = 3$: de 2 a 4 bajó de 15 a $d_{23} + d_{34} = 2 + 1 = 3$.
- $k = 4$: de 3 a 1 bajó de 5 a $d_{34} + d_{41} = 1 + 2 = 3$.

$D^{(4)}$ tiene todas las distancias. Por ejemplo $d_{32} = 6$ es el camino $3 \to 4 \to 1 \to 2$, con $1 + 2 + 3 = 6$. Ese camino se lee del dibujo: el pseudocódigo solo calcula pesos. Para que el algoritmo lo entregue se guarda además una matriz de predecesores, que está en la sección 9.

	Dijkstra	**Floyd**
Responde	desde un vértice	todos los pares
Pesos negativos	no	sí, sin ciclos negativos
Detecta ciclos negativos	no	sí, con la diagonal
Técnica	voraz	programación dinámica
Costo de una corrida	$\mathcal{O}((n+m)\log n)$	$\Theta(n^3)$
Costo para todos los pares	n corridas: $\mathcal{O}(n(n+m)\log n)$	$\Theta(n^3)$
Conviene en	grafos dispersos	grafos densos

Encontrar ciclos con BFS

En un grafo no dirigido, BFS encuentra un ciclo en cuanto ve una arista que no es del árbol: al procesar u, un vecino v que ya fue visto y que no es el padre de u. El padre no cuenta porque la arista $\{u, \pi[u]\}$ es justo por la que se llegó a u. Basta cambiar el **si** de BFS:

Algoritmo 1.5 Cambio a BFS para detectar un ciclo

```
1: para todo v en la lista de adyacencia de u hacer
2:     si no visto[v] entonces
3:         ... igual que en BFS ...
4:     si no si v ≠ π[u] entonces
5:         devolver hay un ciclo que pasa por la arista {u, v}
6:     fin si
7: fin para
```

Ejemplo 1.9 (El ciclo de G)**.** En el paso 5 de BFS sale E y su vecino F ya está visto, pero $\pi[E] = B \neq F$. Hay ciclo. Para recuperarlo se sube por los padres desde los dos extremos hasta que se encuentran:

$$E \to B \to A, \qquad F \to C \to A.$$

Se juntan en A, y el ciclo es A, B, E, F, C, A: la arista punteada del árbol BFS cierra la vuelta.

Ojito...

En un grafo dirigido esta regla da falsos positivos. Con las aristas (A, B), (A, C) y (B, C), BFS desde A llega a C dos veces y la regla dice ciclo, pero no hay ningún ciclo dirigido. Para grafos dirigidos se usa DFS: hay un ciclo si y solo si DFS encuentra una arista de retroceso, una que llega a un vértice todavía abierto, que ya entró y no ha salido. No basta visto: en el mismo ejemplo, al regresar a A y revisar (A, C), C está visto pero ya terminado. Por eso DFS usa tres colores (sección 4); la prueba está en la sección 5. La otra opción es el algoritmo de Kahn: con una cola se van quitando los vértices de grado de entrada 0; si alguno nunca llega a 0, hay ciclo. En el grafo de Floyd ningún vértice empieza con $\text{gr}^-(v) = 0$, así que la cola nace vacía: tiene ciclos, como $1 \to 2 \to 1$.

Árbol de expansión mínima: Kruskal

Definición 1.10 (Árbol de expansión mínima)**.** En un grafo no dirigido, conexo y pesado, un **árbol de expansión** es un subconjunto de aristas que conecta todos los vértices sin formar ciclos; tiene exactamente $n-1$ aristas. El de **expansión mínima** (MST, *minimum spanning tree*) es el de menor peso total.

Es el problema de conectar n ciudades con cable gastando lo menos posible. El algoritmo de Kruskal (1956) es voraz: ordena las aristas de la más ligera a la más pesada y las agrega en ese orden, saltándose las que cerrarían un ciclo.

Algoritmo 1.6 Kruskal(G, w)

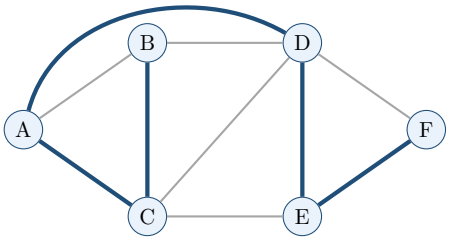
```
1:  T ← ∅
2:  para todo v ∈ V hacer
3:      Crear-Conjunto(v)
4:  fin para
5:  ordenar E por peso, de menor a mayor
6:  para todo {u, v} ∈ E, en ese orden hacer
7:      si Encontrar(u) ≠ Encontrar(v) entonces
8:          T ← T ∪ {{u, v}}
9:          Unir(u, v)
10:     fin si
11: fin para
12: devolver T
```

Para saber si una arista cierra un ciclo se usa una estructura de **conjuntos disjuntos** (*union-find*): cada vértice empieza en su propio conjunto, Encontrar dice en qué conjunto está un vértice y Unir junta dos. Si u y v están en el mismo conjunto, hay un camino entre ellos y la arista cerraría un ciclo.

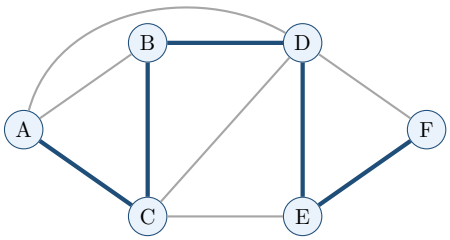
Ejemplo 1.11 (Kruskal sobre el grafo pesado)**.** Las aristas en orden de peso:

Arista	Peso	¿Entra?	Componentes después
$\{B, C\}$	1	sí	{A} {B, C} {D} {E} {F}
$\{A, C\}$	2	sí	{A, B, C} {D} {E} {F}
$\{D, E\}$	2	sí	{A, B, C} {D, E} {F}
$\{E, F\}$	3	sí	{A, B, C} {D, E, F}
$\{A, B\}$	4	no: A y B ya están juntos	
$\{B, D\}$	5	sí	{A, B, C, D, E, F}

Con $n-1=5$ aristas el árbol está completo, y las que faltan ($\{D, F\}$, $\{A, D\}$, $\{C, D\}$, $\{C, E\}$) cerrarían ciclos. Peso total: $1+2+2+3+5=13$.



Caminos más cortos desde A: peso 15



Expansión mínima: peso 13

Ojito...

El árbol de caminos más cortos de Dijkstra y el árbol de expansión mínima no son lo mismo, aunque los dos tengan $n-1$ aristas. El de Dijkstra minimiza la distancia de A a cada vértice; el MST, el peso total. En el MST el camino de A a D es A, C, B, D, con 8; en el de Dijkstra es la arista directa, con 7. Pero el MST pesa 13 y el de Dijkstra 15.

Por qué funciona. Se parte V en dos lados de modo que ninguna arista ya aceptada cruce de un lado al otro. La arista más ligera que cruza, e, es **segura**: si las aceptadas caben en un MST T, también caben junto con e en uno. Si T no usa e, agregarla cierra un ciclo que cruza el corte por otra arista e', que no es de las aceptadas; cambiar e' por e da un árbol de expansión que pesa $w(T) - w(e') + w(e) \leq w(T)$. El intercambio no empeora, y con empates empata en lugar de mejorar; así que el árbol nuevo también es mínimo y contiene a e y a las aceptadas. Pedir que el corte no separe aristas aceptadas es lo que permite encadenar los pasos: no basta con que e esté en algún MST, tiene que estar en el mismo que las anteriores. Cada arista que Kruskal acepta es la más ligera que sale del componente de uno de sus extremos, porque las más ligeras ya se revisaron y quedaron dentro de algún componente; así que es segura. Es la misma idea voraz de la clase del 8 de octubre, y la sección 6 la demuestra completa como la propiedad del corte.

Costo. Ordenar las aristas cuesta $\mathcal{O}(m \log m)$. Con unión por rango y compresión de caminos, la sucesión completa de operaciones de conjuntos disjuntos (n de Crear-Conjunto, $2m$ de Encontrar y $n-1$ de Unir) cuesta $\mathcal{O}((n+m)\,\alpha(n))$, donde α es la inversa de la función de Ackermann: crece tan despacio que $\alpha(n) \leq 4$ para cualquier n concebible en la práctica (Cormen, sección 21.4). Como G es conexo, $m \geq n-1$ y eso es $\mathcal{O}(m\,\alpha(n))$. Domina el ordenamiento: $\mathcal{O}(m \log m) = \mathcal{O}(m \log n)$, porque $m < n^2$ implica $\log m < 2 \log n$.

Observación. El otro algoritmo clásico es el de Prim: hace crecer un solo árbol desde un vértice, agregando siempre la arista más ligera que sale de él. Es casi idéntico a Dijkstra: la llave de v pasa de $d[u] + w(u,v)$ a solo $w(u,v)$, y se actualiza solo si v sigue en Q. Esa condición sobra en Dijkstra, donde con pesos no negativos la relajación nunca dispara hacia un vértice fijo, pero en Prim es indispensable. Sin ella, en el grafo pesado desde A, al sacar B (llave 1) se compara $w(B,C) = 1$ con la llave 2 de C, que ya está en el árbol, y se cambia $\pi[C]$ a B cuando $\pi[B] = C$. Al final los padres forman dos ciclos, B con C y D con E, y ningún vértice cuelga de A. Con montículo binario Prim cuesta $\mathcal{O}(m \log n)$, como Kruskal; está en la sección 6.

Resumen

Algoritmo	**Resuelve**	**Estructura**	**Costo**
BFS	recorrido por capas; camino más corto sin pesos; ciclos en no dirigidos	cola	$\mathcal{O}(n+m)$
DFS	recorrido en profundidad; ciclos dirigidos; orden topológico	pila o recursión	$\mathcal{O}(n+m)$
Dijkstra	caminos más cortos desde un vértice, pesos ≥ 0	montículo binario	$\mathcal{O}((n+m)\log n)$
		arreglo	$\Theta(n^2)$
Floyd	caminos más cortos entre todos los pares, sin ciclos negativos	matriz	$\Theta(n^3)$
Kruskal	árbol de expansión mínima	conjuntos disjuntos	$\mathcal{O}(m \log n)$
Prim	árbol de expansión mínima	montículo binario	$\mathcal{O}(m \log n)$

BFS y DFS cuestan $\Theta(n+m)$ cuando recorren todo el grafo. Floyd usa $\Theta(n^2)$ de memoria. El resumen de toda la parte de grafos, con lo que agregan las secciones de complemento, está al final de la sección 12.

2. Teoría de grafos: terminología, conexidad y Euler *Complemento: Rosen, cap. 10*

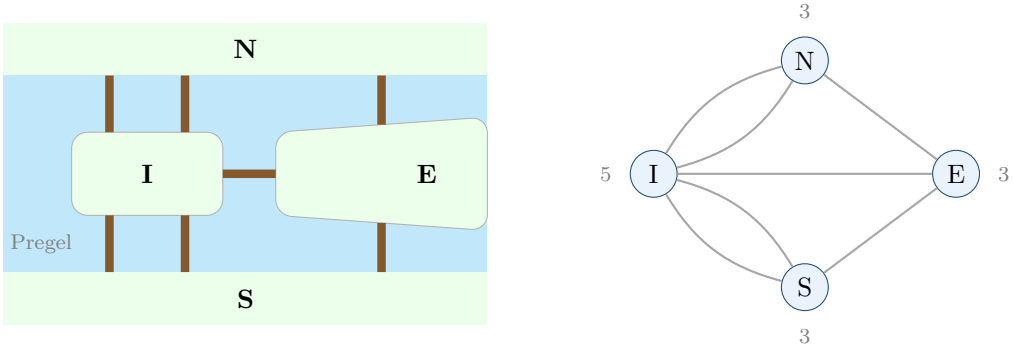
Resumen rápido. Esta sección arma el vocabulario que el resto de la parte de grafos da por sabido. Parte del problema de los puentes de Königsberg y nombra lo que la clase del 22 de octubre dejó sin nombre: multigrafos y pseudografos, el grado con lazos y el teorema del apretón de manos, subgrafos, la matriz de incidencia y el isomorfismo. Fija la convención de caminata, sendero, camino, circuito y ciclo que usan todas las secciones siguientes, define conexidad, componentes, puentes y vértices de corte, y cierra con los grafos eulerianos: el teorema de Euler con su prueba completa, el caso dirigido y el algoritmo de Hierholzer, que construye el recorrido en $\Theta(n+m)$. La idea que se lleva el lector: decidir si existe un recorrido que use cada arista una vez, y construirlo, cuesta lo mismo que leer el grafo.

La materia prima son las diapositivas *Grafos 1* de la Dra. Kelsey Ramírez, que siguen de cerca a Rosen (cap. 10) y recomiendan además a Grimaldi (cap. 11). Son teoría pura: no traen pseudocódigo ni complejidades. Las pruebas de suficiencia, el caso dirigido y el algoritmo de Hierholzer son agregados, para que la libreta responda con un procedimiento lo que la clase decide mirando el dibujo. Lo que ya está en la sección 1 (la definición formal, adyacencia, incidencia y grado en grafos simples, la matriz y las listas de adyacencia, la clasificación de Skiena) no se repite: se cita y se complementa.

Motivación: los siete puentes de Königsberg

En el siglo XVIII la ciudad de Königsberg (hoy Kaliningrado) estaba partida por el río Pregel en cuatro regiones: la orilla norte N, la orilla sur S, la isla Kneiphof I y la región del este E, entre los dos brazos del río. Siete puentes las unían. La pregunta era si se podía dar un paseo que cruzara cada puente exactamente una vez y regresara al punto de partida.

Euler (1736) notó que la respuesta no depende de la forma de las islas, de las distancias ni del tamaño de los puentes: solo importa qué regiones une cada puente. Cada región se vuelve un vértice y cada puente una arista. Es el primer grafo de la historia, y la figura muestra las dos versiones: a la izquierda el mapa, redibujado de forma esquemática; a la derecha el grafo, con el grado de cada vértice.



El grafo tiene $n = 4$ y $m = 7$, y no es simple: entre I y N hay dos puentes, y otros dos entre I y S. Sus aristas forman el multiconjunto

$$E(K) = \{\{I,N\},\{I,N\},\{I,S\},\{I,S\},\{I,E\},\{N,E\},\{S,E\}\}.$$

La E de la región este es la de las diapositivas; para no confundirla con el conjunto de aristas, a este se le llama $E(K)$. La pregunta de Euler se vuelve: ¿hay un recorrido cerrado que use cada arista exactamente una vez? La respuesta, al final de la sección, sale de los números junto a cada vértice.

Multigrafos, pseudografos y grado

La sección 1 define el grafo simple, donde E es un conjunto de pares no ordenados de vértices distintos, y nota que un lazo o dos aristas entre el mismo par no caben en esa definición. Para admitirlos, E se vuelve un multiconjunto. Rosen les pone nombre según tres preguntas: ¿las aristas tienen dirección?, ¿puede haber varias entre el mismo par de vértices?, ¿puede haber lazos?

Tipo	Dirigido	Aristas múltiples	Lazos
Grafo simple	no	no	no
Multigrafo	no	sí	no
Pseudografo	no	sí	sí
Grafo dirigido simple	sí	no	no
Grafo dirigido	sí	no	sí
Multigrafo dirigido	sí	sí	sí

Definición 2.1 (Multigrafo y pseudografo)**.** Un **multigrafo** es un par $G = (V, E)$ en el que E es un multiconjunto de pares $\{u, v\}$ con $u \neq v$: dos aristas distintas pueden tener los mismos extremos, y se llaman **múltiples** o **paralelas**. Un **pseudografo** admite además **lazos**, aristas $\{v, v\}$ cuyos dos extremos son el mismo vértice.

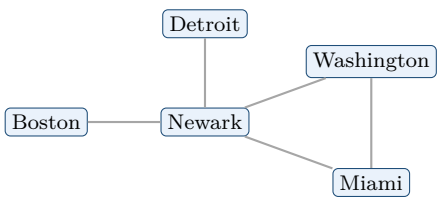
Como dos aristas paralelas no se distinguen por sus extremos, en un multigrafo se les pone nombre, $e_1, \ldots, e_m$, y cada una tiene sus dos extremos. Todo grafo simple es multigrafo y todo multigrafo es pseudografo, así que un teorema probado para pseudografos vale para los tres. Rosen agrega el **grafo mixto**, con aristas dirigidas y no dirigidas a la vez; no aparece en esta parte.

Ojito...

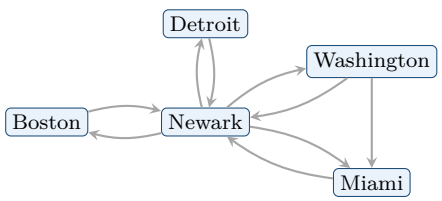
En un grafo dirigido, los arcos (c, d) y (d, c) **no** son múltiples: tienen sentidos opuestos y son arcos distintos, como las dos flechas entre 1 y 2 del grafo dirigido de la sección 1. Múltiples son dos arcos con el mismo origen y el mismo destino. La tabla de las diapositivas dice que un grafo dirigido "puede tener" aristas múltiples; eso contradice a Rosen, que reserva ese caso para el multigrafo dirigido.

Modelar es decidir qué se guarda. El ejemplo de las rutas aéreas de las diapositivas lo muestra con los mismos datos y dos grafos distintos.

Ejemplo 2.2 (Rutas aéreas, Rosen §10.1, ejercicio 1)**.** Una aerolínea tiene estos vuelos diarios: Boston a Newark 4, Newark a Boston 2, Newark a Miami 3, Miami a Newark 2, Newark a Detroit 1, Detroit a Newark 2, Newark a Washington 3, Washington a Newark 2, Washington a Miami 1. Si solo interesa qué ciudades tienen vuelo entre sí, en cualquier sentido, el modelo es un grafo simple no dirigido con 5 aristas (izquierda). Si importa el sentido, es un grafo dirigido simple con 9 arcos (derecha).



simple no dirigido, $m = 5$



dirigido simple, 9 arcos

El de la derecha responde que hay vuelo de Washington a Miami pero no de regreso; el de la izquierda no puede. Ninguno de los dos sabe cuántos vuelos hay: para eso haría falta un multigrafo con una arista por vuelo, 20 en total, o un grafo pesado con el número de vuelos como peso.

El grado y el teorema del apretón de manos

En un pseudografo el grado necesita una precisión: un lazo toca a su vértice por los dos extremos.

Definición 2.3 (Grado)**.** El **grado** $\mathrm{gr}(v)$ de un vértice de un pseudografo es el número de extremos de aristas que caen en v: cada arista $\{u, v\}$ con $u \neq v$ suma 1 al grado de cada extremo, y cada lazo en v suma 2 al grado de v. Un vértice de grado 0 es **aislado** y uno de grado 1 es una **hoja**.

Rosen, Cormen y las diapositivas escriben $\deg(v)$; aquí se conserva el $\mathrm{gr}(v)$ de la sección 1. Ahí mismo aparece, como observación, la suma de los grados en un grafo simple. Vale en general y tiene nombre.

Teorema 2.4 (Apretón de manos)**.** *En todo pseudografo $G = (V, E)$,*

$$\sum_{v \in V} gr(v) = 2m.$$

Demostración. Se cuentan de dos maneras los pares (vértice, extremo de arista que cae en él). Contando por vértices, cada v aporta $\mathrm{gr}(v)$ pares, y el total es la suma de los grados. Contando por aristas, cada arista tiene exactamente dos extremos, sea o no un lazo, y aporta 2. Las dos cuentas cuentan lo mismo, así que coinciden. □

El nombre viene de una fiesta: si cada quien anota cuántas manos estrechó, la suma de las anotaciones es el doble del número de apretones. En el multigrafo de Königsberg, $\mathrm{gr}(I) = 5$ y las otras tres regiones tienen grado 3; suman $14 = 2 \cdot 7$, uno por cada extremo de los siete puentes.

Ojito...

Las diapositivas definen el grado como "el número de aristas incidentes". Leída al pie de la letra, esa definición cuenta un lazo una sola vez, y entonces el teorema falla: un grafo con un solo vértice y un lazo tendría suma de grados 1 y no 2. La versión de Rosen, con el lazo contado dos veces, es la que hace funcionar todo lo que sigue.

Corolario 2.5. *En todo pseudografo, el número de vértices de grado impar es par.*

Demostración. Se parte la suma de los grados en la de los vértices de grado par y la de los de grado impar. El total es $2m$, par, y la primera parte es par por ser suma de pares; luego la suma de los grados impares también es par. Una suma de números impares es par solo si tiene un número par de sumandos. □

El corolario parece una curiosidad. Es lo que hace exhaustiva la clasificación de los grafos eulerianos al final de la sección.

Grado de entrada y de salida

En un grafo dirigido, $\mathrm{gr}^-(v)$ es el **grado de entrada**, el número de arcos que llegan a v, y $\mathrm{gr}^+(v)$ el **grado de salida**, el número de arcos que salen de v. Un lazo (v, v) suma 1 a cada uno.

Proposición 2.6. *En todo grafo dirigido, $\sum_v gr^-(v) = \sum_v gr^+(v) = m$.*

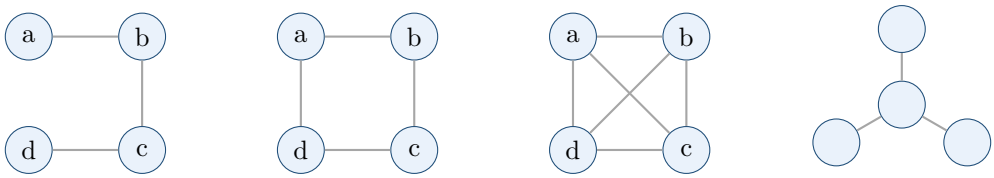
Demostración. Cada arco tiene exactamente un origen y un destino: aporta 1 a la suma de grados de salida y 1 a la de entrada. □

En el grafo dirigido de la sección 1, con arcos $1 \to 2$, $2 \to 1$, $1 \to 4$, $4 \to 1$, $2 \to 3$, $3 \to 1$ y $3 \to 4$, los grados de salida son $2, 2, 2, 1$ y los de entrada $3, 1, 1, 2$ para los vértices 1 a 4. Las dos listas suman 7, el número de arcos.

Familias con nombre

Algunos grafos aparecen tanto que tienen nombre propio. Las diapositivas usan cuatro, y las secciones siguientes también.

Ejemplo 2.7 (Familias de grafos)**.** El **camino** P_n tiene n vértices en fila y $n-1$ aristas; el **ciclo** C_n, con $n \geq 3$, cierra la fila con una arista más; el **completo** K_n une cada par de vértices, y la **estrella** une un centro con k hojas.



P_4 C_4 K_4 estrella, $k = 3$

En P_n los extremos tienen grado 1 y los demás 2; en C_n todos tienen grado 2 y $m = n$; en K_n todos tienen grado $n-1$ y $m = n(n-1)/2$, el máximo posible en un grafo simple; en la estrella el centro tiene grado k y $m = k$. La sección 3 vuelve a la estrella como el bipartito completo $K_{1,k}$.

Subgrafos

Definición 2.8 (Subgrafo)**.** $H = (W, F)$ es un **subgrafo** de $G = (V, E)$ si $W \subseteq V$, $F \subseteq E$ y cada arista de F tiene sus dos extremos en W. Es el **subgrafo inducido** por W, y se escribe $G[W]$, si F contiene *todas* las aristas de E con los dos extremos en W. Es un **subgrafo generador** (o de expansión) si $W = V$.

Dos subgrafos se usan a cada rato. $G - e$ es el grafo sin la arista e, con todos los vértices. $G - v$ es el grafo sin el vértice v *y sin todas las aristas que inciden en* v, porque una arista sin uno de sus extremos deja de ser arista.

Ojito...

La diapositiva 46 define subgrafo con $W \subseteq V$ y $F \subseteq E$, sin la tercera condición. Sin ella, $W = \{A\}$ y $F = \{\{A, B\}\}$ cumplirían la definición, y eso no es un grafo: la arista tiene un extremo que no está.

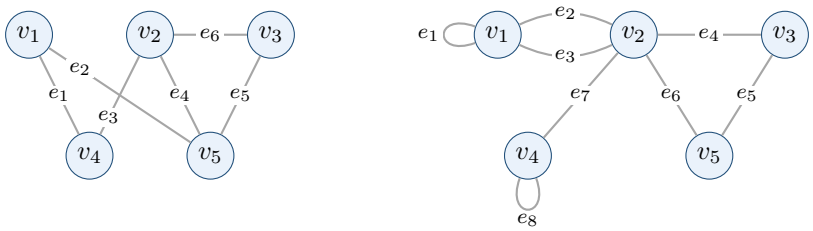
Ejemplo 2.9 (Subgrafos del grafo G de la libreta)**.** En el grafo G de la sección 1, el subgrafo inducido por $\{A, B, C, E, F\}$ tiene las aristas $\{A, B\}$, $\{A, C\}$, $\{B, E\}$, $\{C, F\}$ y $\{E, F\}$: es el ciclo C_5 que la clase encuentra con BFS. El subgrafo $G - \{E, F\}$ es generador, porque conserva los siete vértices, y es exactamente el árbol BFS de esa clase. En general, los árboles de BFS, de DFS y de expansión mínima son subgrafos generadores.

Matriz de incidencia

La matriz de adyacencia de la sección 1 relaciona vértices con vértices. La de incidencia relaciona vértices con aristas, y por eso admite aristas paralelas sin perder información.

Definición 2.10 (Matriz de incidencia)**.** Con los vértices $v_1, \ldots, v_n$ y las aristas $e_1, \ldots, e_m$ numerados, la matriz de incidencia es la matriz M de $n \times m$ con $M_{ij} = 1$ si v_i es extremo de e_j, y $M_{ij} = 0$ si no.

Ejemplo 2.11 (Diapositivas 41 y 44)**.** A la izquierda, un grafo simple con cinco vértices y seis aristas; a la derecha, un pseudografo con lazos e_1 y e_8 y aristas paralelas e_2 y e_3.



Para el grafo simple, su matriz de incidencia M y el producto MM^T:

$$M = \begin{array}{c|cccccc} & e_1 & e_2 & e_3 & e_4 & e_5 & e_6 \\ \hline v_1 & 1 & 1 & 0 & 0 & 0 & 0 \\ v_2 & 0 & 0 & 1 & 1 & 0 & 1 \\ v_3 & 0 & 0 & 0 & 0 & 1 & 1 \\ v_4 & 1 & 0 & 1 & 0 & 0 & 0 \\ v_5 & 0 & 1 & 0 & 1 & 1 & 0 \end{array} \qquad MM^T = \begin{array}{c|ccccc} & v_1 & v_2 & v_3 & v_4 & v_5 \\ \hline v_1 & 2 & 0 & 0 & 1 & 1 \\ v_2 & 0 & 3 & 1 & 1 & 1 \\ v_3 & 0 & 1 & 2 & 0 & 1 \\ v_4 & 1 & 1 & 0 & 2 & 0 \\ v_5 & 1 & 1 & 1 & 0 & 3 \end{array}$$

Para el pseudografo:

$$M = \begin{array}{c|cccccccc} & e_1 & e_2 & e_3 & e_4 & e_5 & e_6 & e_7 & e_8 \\ \hline v_1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ v_2 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ v_3 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ v_4 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ v_5 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \end{array}$$

Las columnas de los lazos tienen un solo 1, y las de las aristas paralelas e_2 y e_3 son iguales.

Proposición 2.12. *Sea M la matriz de incidencia de un multigrafo sin lazos y A su matriz de adyacencia, con A_{ij} igual al número de aristas entre v_i y v_j. Entonces cada columna de M tiene exactamente dos unos, la suma del renglón i es $gr(v_i)$, y*

$$MM^T = D + A,$$

donde D es la matriz diagonal con los grados.

Demostración. Las dos primeras afirmaciones son la definición leída por columnas y por renglones. Para la tercera, $(MM^T)_{ij} = \sum_k M_{ik}M_{jk}$, y el sumando k vale 1 justo cuando e_k incide a la vez en v_i y en v_j. Con $i = j$ eso cuenta las aristas que inciden en v_i, su grado; con $i \neq j$ cuenta las aristas entre v_i y v_j, que es A_{ij}. □

En el ejemplo, la diagonal de MM^T es $2, 3, 2, 2, 3$, los grados, y fuera de ella está la matriz de adyacencia. Con lazos la suma del renglón deja de ser el grado: en el pseudografo el renglón de v_1 suma 3 y $\mathrm{gr}(v_1) = 4$, y el de v_4 suma 2 con $\mathrm{gr}(v_4) = 3$. Algunos textos escriben un 2 en la entrada del lazo para que la suma sí dé el grado.

Idea clave

La matriz de incidencia ocupa $\Theta(nm)$ de memoria, más que la de adyacencia en cuanto $m > n$, y preguntar si $\{u, v\}$ es arista cuesta $\Theta(m)$: hay que recorrer el renglón de u buscando una columna que también tenga 1 en v. Por eso casi ningún algoritmo la usa. Su valor es teórico: identidades como $MM^T = D + A$, y que se generaliza sin cambios a aristas con más de dos extremos. La versión dirigida, con -1 en el origen y 1 en el destino, está en la sección 4 (Cormen, ejercicio 22.1-7).

Isomorfismo

La sección 1 distingue el grafo encajado del topológico: dos dibujos distintos son el mismo grafo topológico si unen a los mismos pares. Cuando además los vértices tienen nombres distintos, "el mismo grafo" se precisa con una biyección.

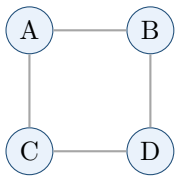
Definición 2.13 (Isomorfismo)**.** Dos grafos simples $G_1 = (V_1, E_1)$ y $G_2 = (V_2, E_2)$ son **isomorfos**, $G_1 \cong G_2$, si existe una biyección $f : V_1 \to V_2$ tal que

$$\{u, v\} \in E_1 \iff \{f(u), f(v)\} \in E_2.$$

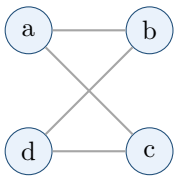
Una propiedad que conserva todo isomorfismo es un **invariante**.

El número de vértices, el de aristas y la secuencia de grados son invariantes, y también lo son la conexidad, el número de ciclos de cada longitud y el grado de los vecinos de cada vértice: la biyección manda vecinos en vecinos. Esto da la regla para trabajar. Si dos grafos difieren en un invariante, no son isomorfos. Si coinciden en todos los que se revisaron, no se sabe nada: para afirmar que lo son hay que exhibir la biyección.

Ejemplo 2.14 (De la matriz al grafo, diapositivas 36 a 38)**.** La diapositiva 36 da una matriz de adyacencia en el orden a, b, c, d que, leída renglón por renglón, tiene las aristas $\{a, b\}$, $\{a, c\}$, $\{b, d\}$, $\{c, d\}$. La diapositiva 38 la dibuja con dos aristas cruzadas (derecha), y parece otra cosa que el cuadrado C_4 (izquierda).



C_4



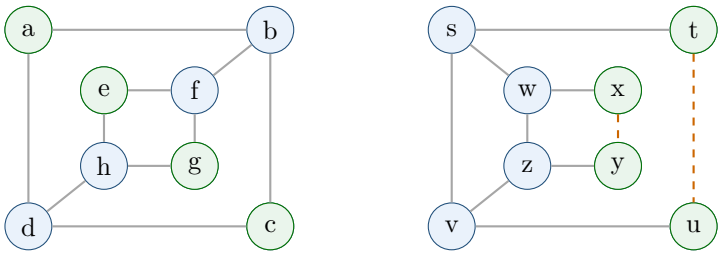
diapositiva 38

La biyección $f(A) = a$, $f(B) = b$, $f(C) = c$, $f(D) = d$ manda las cuatro aristas de la izquierda en las cuatro de la derecha: son isomorfos. El cruce es del dibujo, no del grafo.

Ejemplo 2.15 (Dos grafos que se parecen, diapositivas 48 a 51)**.** G y H tienen ocho vértices cada uno, un cuadrado exterior y uno interior unidos por dos aristas:

$$E_G = \{ab, bc, cd, da, ef, fg, gh, he, bf, dh\}, \qquad E_H = \{st, tu, uv, vs, wx, xy, yz, zw, sw, vz\}.$$

En verde, los vértices de grado 2; punteadas, las aristas de H que unen dos vértices de grado 2.



G H

Los invariantes de la diapositiva 49 coinciden: ocho vértices, diez aristas y la misma secuencia de grados, $2, 2, 2, 2, 3, 3, 3, 3$. Los dos últimos renglones de la tabla no coinciden.

Invariante	G	H
Vértices, aristas	8, 10	8, 10
Secuencia de grados	$2, 2, 2, 2, 3, 3, 3, 3$	$2, 2, 2, 2, 3, 3, 3, 3$
Vértices de grado 2	a, c, e, g	t, u, x, y
Aristas entre vértices de grado 2	0	2 (tu, xy)
Ciclos de longitud 4	2	3 ($stuv$, $wxyz$, $swzv$)

El argumento de la clase va por vértices. Un isomorfismo conserva el grado y la adyacencia, así que a, de grado 2, tendría que ir a un vértice de grado 2 de H cuyos dos vecinos tengan grado 3, como los de a. En H no hay ninguno: cada vértice de grado 2 tiene un vecino de grado 2. No existe a dónde mandar a, así que G y H **no son isomorfos**.

Ojito...

La diapositiva 51 termina en la observación sobre los vecinos y no escribe la conclusión; la de arriba se confirmó probando las $8! = 40\,320$ biyecciones. De paso, la diapositiva 52 que sigue es un ejemplo de grado fuera de lugar, no la respuesta. Mismo dibujo no implica mismo grafo, ni al revés: la diapositiva 38 es C_4 con un cruce, y G y H se parecen mucho sin ser isomorfos.

Probar las $n!$ biyecciones cuesta $\Omega(n!)$. No se conoce un algoritmo polinomial para decidir isomorfismo en general, ni se sabe que el problema sea NP-completo; el mejor resultado, de Babai (2015), es cuasipolinomial. En la práctica se descarta con invariantes baratos y solo se busca la biyección cuando todos coinciden.

Caminatas, senderos, caminos, circuitos y ciclos

Moverse dentro de un grafo tiene cinco nombres, cada uno con una restricción más que el anterior. La terminología cambia de un libro a otro, así que esta subsección fija la que usa toda la parte de grafos de la libreta.

Definición 2.16 (Caminata)**.** En un pseudografo, una **caminata** de longitud k de u a v es una secuencia alternada de vértices y aristas

$$v_0, e_1, v_1, e_2, \ldots, e_k, v_k$$

con $v_0 = u$, $v_k = v$ y cada e_i una arista de extremos v_{i-1} y v_i. Es **cerrada** si $v_0 = v_k$. Un solo vértice es una caminata de longitud 0.

En un grafo simple cada arista queda determinada por sus extremos, y la caminata se escribe solo con vértices, $v_0, v_1, \ldots, v_k$. Las diapositivas usan n para la longitud; aquí es k, porque n es el número de vértices.

Definición 2.17 (Sendero, camino, circuito y ciclo)**.** Una caminata es

- un **sendero** si no repite aristas;
- un **camino** si no repite vértices;
- un **circuito** si es un sendero cerrado de longitud al menos 1;
- un **ciclo** si es un circuito en el que $v_0, v_1, \ldots, v_{k-1}$ son distintos: un camino cerrado, que no repite vértices salvo el primero y el último.

	¿Repite vértices?	¿Repite aristas?	¿Cerrada?
Caminata	puede	puede	puede
Sendero	puede	no	puede
Camino	no	no	solo con $k = 0$
Circuito	puede	no	sí
Ciclo	solo $v_0 = v_k$	no	sí

Todo camino es sendero: si no se repiten vértices, una arista no puede repetirse, porque sus extremos se repetirían. Por lo mismo, todo ciclo es circuito. El recíproco falla: en dos triángulos pegados por un vértice (la "mariposa" de la diapositiva 119, más abajo) el recorrido c, a, b, c, d, e, c es un circuito que pasa dos veces por c, y no es ciclo.

Observación. En un grafo simple todo ciclo tiene longitud al menos 3: uno de longitud 1 sería un lazo, y uno de longitud 2, como a, b, a, usaría dos veces la misma arista. En un multigrafo dos aristas paralelas a, e_1, b, e_2, a sí forman un ciclo de longitud 2, y en un pseudografo un lazo es un ciclo de longitud 1.

La definición de ciclo de la sección 1 y el "ciclo" que Kruskal evita cerrar son ciclos en este sentido estricto.

Ojito...

La profesora llama **circuito** a cualquier caminata cerrada (diapositiva 81), aunque repita aristas, y lo advierte: "depende de la convención". Grimaldi, que ella misma recomienda, exige que el circuito no repita aristas, y esta libreta sigue a Grimaldi. Hay dos razones. Así los nombres van en pares, abierto y cerrado: sendero y circuito, camino y ciclo. Y así un circuito euleriano es literalmente un circuito. El precio es que una respuesta de la actividad de la clase cambia (la secuencia 3 de la actividad de recorridos). Con los libros en inglés hay que traducir con cuidado: el *simple path* de Rosen es un sendero y el de Cormen es un camino, y los "caminos simples" de la sección 1 son caminos a secas.

Libreta	Diapositivas	Grimaldi	Rosen	Cormen
caminata	caminata	*walk*	*path*	*path*
sendero	sendero	*trail*	*simple path*	sin nombre
camino	camino, trayectoria	*path*	sin nombre	*simple path*
caminata cerrada	circuito	*closed walk*	*circuit*	*cycle* (dirigido)
circuito	circuito	*circuit*	*simple circuit*	sin nombre
ciclo	ciclo	*cycle*	sin nombre	*simple cycle*

Repetir vértices nunca ayuda a llegar: siempre se puede quitar la vuelta.

Lema 2.18. *Si hay una caminata de u a v, hay un camino de u a v, y su longitud no es mayor.*

Demostración. Si la caminata repite un vértice x, se borra el tramo entre la primera y la última

aparición de x, que es una vuelta cerrada. Lo que queda sigue siendo una caminata de u a v y es más corta. Como la longitud no puede bajar para siempre, el proceso termina en una caminata sin vértices repetidos: un camino. □

En particular, la caminata más corta de u a v es un camino. Por eso la $d[v]$ de BFS, que mide la caminata más corta, es la longitud de un camino, y por eso con pesos no negativos siempre hay un camino más corto que no repite vértices: quitar una vuelta no aumenta el peso.

La matriz de adyacencia también cuenta caminatas, y eso enlaza esta subsección con la representación.

Proposición 2.19. *Si A es la matriz de adyacencia de un multigrafo, con A_{ij} igual al número de aristas entre v_i y v_j, entonces $(A^k)_{ij}$ es el número de caminatas de longitud k de v_i a v_j.*

Demostración. Por inducción sobre k. Con $k = 1$ es la definición de A. Una caminata de longitud $k+1$ de v_i a v_j es una caminata de longitud k de v_i a algún v_l seguida de una arista de v_l a v_j; por hipótesis hay $(A^k)_{il}$ de las primeras y A_{lj} de las segundas, y sumando sobre l queda $\sum_l (A^k)_{il} A_{lj} = (A^{k+1})_{ij}$. □

Tómese el grafo de la actividad de recorridos de la clase:

$$V = \{a, b, c, d, e, f\}, \qquad E = \{ab, bc, de, ef, ad, be, cf, ae, bf, cd\}.$$

Ahí $(A^2)_{ac} = 2$, por las caminatas a, b, c y a, d, c, y la diagonal de A^2 es $3, 4, 3, 3, 4, 3$, los grados: una caminata cerrada de longitud 2 es ir y volver por una arista. La sección 9 usa la misma idea con pesos, cambiando la suma por un mínimo y el producto por una suma.

En un grafo dirigido las definiciones son las mismas con arcos: cada paso va de v_{i-1} a v_i siguiendo la flecha, $(v_{i-1}, v_i) \in E$. En el grafo dirigido de la sección 1, $3 \to 4 \to 1 \to 2$ es un camino dirigido (el que da $d_{32} = 6$ en Floyd) y $2 \to 3 \to 1 \to 2$ es un ciclo dirigido de longitud 3. Ahí $1 \to 2 \to 1$ es un ciclo de longitud 2: los arcos $(1, 2)$ y $(2, 1)$ son distintos, así que no se repite ninguno.

Conexidad

Definición 2.20 (Conexo)**.** Un grafo no dirigido es **conexo** si hay un camino entre cada par de vértices, y **desconectado** si no.

Por el lema 2.18 da lo mismo pedir caminos o caminatas. La sección 1 ya usa la palabra: un árbol es un grafo no dirigido, conexo y acíclico, y el árbol de expansión mínima supone un grafo conexo. Ahora las dos palabras tienen definición, y los árboles se estudian en la sección 3.

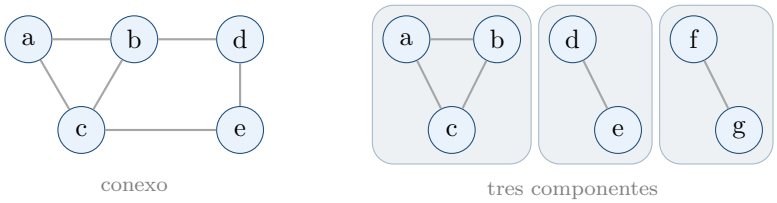
Proposición 2.21. *La relación "u y v están conectados", es decir, hay un camino de u a v, es de equivalencia en V.*

Demostración. Es reflexiva por el camino de longitud 0, y simétrica porque un camino se recorre al revés. Es transitiva porque un camino de u a v seguido de uno de v a w es una caminata de u a w, y el lema 2.18 la convierte en camino. □

Definición 2.22 (Componente conexa)**.** Las **componentes conexas** de G son los subgrafos inducidos por las clases de esa relación. Equivalentemente, son los subgrafos conexos maximales: conexos, y sin poder agregar un vértice más sin dejar de serlo.

Las componentes parten a V, cada arista queda dentro de una sola componente, y G es conexo si y solo si tiene una sola. Un vértice aislado es una componente de un solo vértice.

Ejemplo 2.23 (Diapositivas 95 y 100)**.** A la izquierda, un grafo conexo: a y e no son adyacentes, pero el camino a, c, e los une. A la derecha, un grafo con tres componentes, $\{a, b, c\}$, $\{d, e\}$ y $\{f, g\}$: no hay camino de a a d ni de e a g.



Las componentes se calculan con los recorridos de la sección 1. Una BFS desde s visita exactamente la componente de s: todo lo que alcanza está conectado con s, y todo lo conectado con s se alcanza. Repetida desde cada vértice que siga sin visitar, etiqueta todas las componentes en $\Theta(n + m)$, porque cada vértice entra a la cola una sola vez en total y cada lista se recorre una sola vez, no una vez por componente. La sección 4 hace lo mismo con DFS.

En un grafo dirigido hay dos nociones. Es **débilmente conexo** si, olvidando el sentido de los arcos, el grafo no dirigido que queda es conexo; es **fuertemente conexo** si hay un camino dirigido de cada vértice a cada otro. El grafo dirigido de Floyd en la sección 1 es fuertemente conexo. El DAG de seriaciones es débilmente conexo y no fuertemente: ninguna flecha regresa. Las componentes fuertemente conexas y su algoritmo están en la sección 5.

Puentes y vértices de corte

Hay aristas y vértices de los que depende la conexidad: si fallan, la red se parte.

Definición 2.24 (Puente y vértice de corte)**.** Una arista e es un **puente** (o arista de corte) si $G - e$ tiene más componentes conexas que G. Un vértice v es un **vértice de corte** (o punto de articulación) si $G - v$ tiene más componentes conexas que G.

Recuérdese que $G - v$ quita también las aristas de v. En el grafo G de la sección 1, $\{B, D\}$ y $\{D, G\}$ son puentes y B y D son vértices de corte; ninguna arista del ciclo A, B, E, F, C, A es puente. Esa última observación es general.

Teorema 2.25. *Una arista es puente si y solo si no pertenece a ningún ciclo.*

Demostración. Sea e una arista de extremos u y v. Si e está en un ciclo $u, v, x_1, \ldots, x_r, u$, el resto del ciclo, $v, x_1, \ldots, x_r, u$, es un camino de v a u que no usa e. Cualquier caminata de G que pase por e se desvía por ese camino, así que $G - e$ tiene las mismas componentes que G, y e no es puente. Esto cubre los lazos, que son ciclos de longitud 1.

Si e no está en ningún ciclo, $u \neq v$ y $G - e$ no tiene camino de u a v: ese camino, seguido de e, cerraría un ciclo que contiene a e (de longitud 2 si el camino es una arista paralela a e). Entonces la componente de G que contiene a u y a v se parte en dos al quitar e, las demás no cambian, y $G - e$ tiene una componente más. □

Para vértices no hay una regla así: estar en un ciclo no protege a un vértice. Lo que sí se tiene es un puente entre la definición de puente y la de vértice de corte.

Corolario 2.26. *Si $\{u, v\}$ es un puente y u tiene otro vecino $w \neq v$, entonces u es vértice de corte.*

Demostración. Sea $e = \{u, v\}$. En $G - e$, el vértice w sigue unido a u y queda en una componente distinta de la de v. Un camino de w a v en $G - u$ no pasaría por u, y por lo tanto tampoco por e: sería un camino en $G - e$, que no existe. Así que al quitar u la componente de u se parte al menos en la parte de w y la de v, y las demás no cambian. □

Ojito...

Tres detalles que las diapositivas dejan pasar. Primero, la diapositiva 104 dice que una arista en un ciclo "no suele ser" crítica: por el teorema 2.25, nunca lo es. Segundo, la diapositiva 102 señala un puente, $\{b, d\}$, pero $\{d, e\}$ también lo es: toda arista que llega a una hoja es puente. Del mismo modo, en la diapositiva 105 también d es vértice de corte, además de b. Tercero, dos aristas paralelas nunca son puentes, porque juntas forman un ciclo de longitud 2; eso importa en multigrafos como el de Königsberg, que no tiene ningún puente.

Por la definición, saber si una arista es puente cuesta una BFS sobre $G - e$, $\Theta(n + m)$; encontrar todos los puentes así cuesta $\Theta(m(n + m))$, y todos los vértices de corte $\Theta(n(n + m))$. Una sola DFS con un valor más por vértice, low$[v]$, encuentra todos en $\Theta(n + m)$; ese algoritmo está en la sección 5.

Grafos eulerianos

La diapositiva 111 plantea dos preguntas gemelas: ¿hay un recorrido que pase por cada *arista* exactamente una vez?, ¿hay uno que pase por cada *vértice* exactamente una vez? La primera es la de Euler y se resuelve aquí. La segunda es la de Hamilton, y está en la sección 3; se parecen tanto que uno esperaría resolverlas igual, y no.

Definición 2.27 (Recorridos eulerianos)**.** Un **sendero euleriano** de G es un sendero que usa cada arista de G exactamente una vez; un **circuito euleriano** es un sendero euleriano cerrado. G es **euleriano** si tiene circuito euleriano, **semi-euleriano** si tiene sendero euleriano pero no circuito, y **no euleriano** si no tiene ninguno.

Ojito...

Las diapositivas y Rosen dicen "camino euleriano" (*Euler path*). Con la convención de esta libreta no es un camino, porque casi siempre repite vértices: el recorrido b, a, d, c, b, d de abajo pasa dos veces por b y por d. Grimaldi lo llama *Euler trail*, sendero euleriano, y es el nombre que se usa aquí. Cormen llama *Euler tour* al circuito euleriano de un grafo dirigido y lo describe como un *cycle* que puede repetir vértices (Cormen, problema 22-3): para esta libreta es un circuito.

La intuición está en un vértice intermedio. Cada vez que el recorrido pasa por él usa dos de sus aristas, una para entrar y otra para salir. Si el recorrido usa todas las aristas una vez, las de cada vértice quedan agrupadas en pares, salvo en los extremos de un recorrido abierto.

El teorema de Euler

Teorema 2.28 (Euler)**.** *Sea G un pseudografo con al menos una arista, conexo salvo vértices aislados. G tiene un circuito euleriano si y solo si todos sus vértices tienen grado par.*

"Conexo salvo vértices aislados" significa que todas las aristas están en una misma componente. Los vértices aislados no estorban: no tienen aristas que recorrer.

Demostración. **Necesidad.** Sea $v_0, e_1, v_1, \ldots, e_m, v_m$ un circuito euleriano, con $v_m = v_0$, y sea $e_{m+1} = e_1$. Cada índice $i \in \{1, \ldots, m\}$ empareja dos extremos que caen en v_i: el final de e_i, por donde se entra,

y el inicio de e_{i+1}, por donde se sale. Para $i = m$ el par es la última llegada a v_0 y la primera salida. Como cada arista aparece una sola vez, cada extremo que cae en un vértice v queda en exactamente un par, así que $\mathrm{gr}(v)$ es dos veces el número de apariciones de v entre $v_1, \ldots, v_m$. Un lazo en v también encaja: se sale y se regresa a v en el mismo paso, y sus dos extremos quedan en pares distintos.

Un sendero solo se atora en su inicio. Supóngase que todos los grados de un pseudografo son pares, y que desde un vértice s se camina usando cada vez una arista todavía no usada, mientras la haya. Estando en un $v \neq s$, el sendero ha entrado a v una vez más de las que ha salido, así que ha usado un número impar de los extremos de v (un lazo, entrado y salido en un paso, usa dos). Como $\mathrm{gr}(v)$ es par, queda un extremo libre y se puede seguir. Las aristas son finitas, así que la caminata se detiene, y solo puede hacerlo en s: el resultado es un circuito.

Suficiencia. Sea s un vértice con aristas. Por el paso anterior, desde s sale un circuito C. Si C usa todas las aristas, terminamos. Si no, alguna arista sin usar toca un vértice x de C. En efecto, si una arista sin usar $\{y, z\}$ no toca a C, se toma un camino de s a y, que existe porque todas las aristas están en una componente; las aristas de C unen vértices de C y y no es uno de ellos, así que la primera arista del camino que no está en C sale de un vértice x de C y está sin usar.

Sea R el grafo de las aristas sin usar. En R todos los grados siguen siendo pares, porque C usa en cada vértice un número par de extremos (es la necesidad aplicada a C). Desde x sale entonces un circuito C' en R, que regresa a x. Se **empalma** C' en C: en una aparición de x en C se inserta C' completo. El resultado es un circuito, porque C y C' no comparten aristas, y tiene más aristas que C. Se repite; como m es finito, al final el circuito usa todas las aristas. □

La prueba de la suficiencia es constructiva: camina, se atora, busca un vértice con aristas libres y empalma. Es exactamente el algoritmo de Hierholzer (1873), que se ve más abajo. Las diapositivas solo dan la necesidad.

Teorema 2.29 (Sendero euleriano)**.** *Sea G un pseudografo conexo salvo vértices aislados. G tiene un sendero euleriano que no es cerrado si y solo si tiene exactamente dos vértices de grado impar, y en ese caso todo sendero euleriano empieza en uno de ellos y termina en el otro.*

Demostración. Si hay un sendero euleriano de u a v con $u \neq v$, las apariciones intermedias de cada vértice emparejan sus extremos como en la prueba anterior; en u sobra la primera salida y en v la última llegada. Así que u y v tienen grado impar y los demás par.

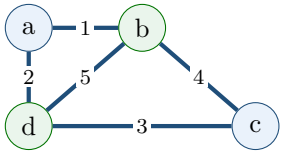
Al revés, si u y v son los únicos vértices de grado impar, se agrega una arista nueva $e^* = \{u, v\}$, que puede quedar paralela a otra; por eso el teorema se enuncia para multigrafos. Ahora todos los grados son pares y, por el teorema 2.28, hay un circuito euleriano. Como es cerrado, se puede empezar a leer justo después de e^*: queda $u, \ldots, v, e^*, u$. Borrando e^* queda un sendero de u a v que usa cada arista original una vez. □

Con los dos teoremas y el corolario 2.5, la clasificación de un grafo conexo salvo aislados se lee contando vértices de grado impar:

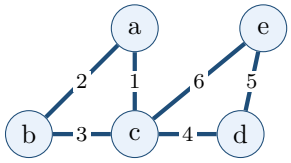
Vértices impares	**Tipo**	**Recorrido**
0	euleriano	circuito desde cualquier vértice con aristas
2	semi-euleriano	sendero de un impar al otro
$4, 6, \ldots$	no euleriano	ninguno
$1, 3, 5, \ldots$	imposible	por el corolario 2.5

La tabla de la diapositiva 124 dice “más de 2”; el último renglón explica por qué eso significa 4 o más y por qué la tabla no deja casos fuera.

Ejemplo 2.30 (Diapositivas 115 y 119)**.** A la izquierda, el grafo con $E = \{ab, ad, bd, bc, dc\}$ tiene grados $\mathrm{gr}(a) = \mathrm{gr}(c) = 2$ y $\mathrm{gr}(b) = \mathrm{gr}(d) = 3$: es semi-euleriano, y el sendero b, a, d, c, b, d va de un impar (en verde) al otro. A la derecha, la mariposa con $E = \{ab, ac, bc, cd, ce, de\}$ tiene $\mathrm{gr}(c) = 4$ y 2 en los demás: es euleriana, y c, a, b, c, d, e, c es un circuito euleriano. El número sobre cada arista es el orden en que se usa.



sendero b, a, d, c, b, d



circuito c, a, b, c, d, e, c

El circuito de la mariposa pasa dos veces por c: es circuito, no ciclo.

Ojito…

La hipótesis de conexidad no sobra, pero solo se pide entre los vértices que tienen aristas. Dos triángulos ajenos tienen todos sus grados pares y ningún circuito euleriano: no hay forma de pasar de uno al otro. En cambio, un triángulo más un vértice suelto sí tiene circuito euleriano.

Ya se puede contestar la pregunta de Euler. En Königsberg, $\mathrm{gr}(I) = 5$ y $\mathrm{gr}(N) = \mathrm{gr}(S) = \mathrm{gr}(E) = 3$.

Resultado

Los cuatro vértices del grafo de Königsberg tienen grado impar. No hay circuito euleriano ni sendero euleriano: no se pueden cruzar los siete puentes una sola vez cada uno, ni regresando al punto de partida ni sin regresar.

El caso dirigido

En un grafo dirigido el recorrido tiene que seguir las flechas, y el par entrada-salida de cada vértice intermedio se vuelve un arco que llega y uno que sale. La paridad se cambia por el balance.

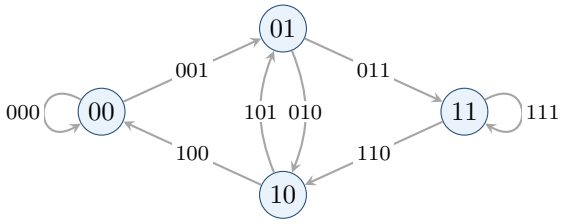
Teorema 2.31 (Euler, caso dirigido)**.** *Sea G un grafo dirigido con al menos un arco, débilmente conexo salvo vértices aislados.*

1. *G tiene un circuito euleriano si y solo si $gr^-(v) = gr^+(v)$ para todo v.*
2. *G tiene un sendero euleriano no cerrado si y solo si hay un vértice s con $gr^+(s) - gr^-(s) = 1$, uno t con $gr^-(t) - gr^+(t) = 1$, y todos los demás balanceados. El sendero va de s a t.*

Demostración. Es la misma prueba con dos ajustes. En la necesidad, cada paso por un vértice usa un arco que llega y uno que sale, así que en un circuito cada vértice queda balanceado; en un sendero abierto, el inicio tiene una salida de más y el final una llegada de más. En el paso de atorarse, estando en $v \neq s$ se ha llegado a v una vez más de las que se ha salido, y como v está balanceado le queda un arco de salida libre. En el empalme, la conexidad débil da un arco libre que toca un vértice x del circuito, pero podría llegar a x en lugar de salir; como en los arcos libres todo sigue balanceado, si a x le llega un arco libre también le sale uno. El caso del sendero se reduce al del circuito agregando el arco (t, s). □

Cormen enuncia la parte 1 para grafos fuertemente conexos (problema 22-3). No es una hipótesis más fuerte en el fondo: si el grafo está balanceado y es débilmente conexo, el circuito euleriano pasa por todos los vértices con arcos, y siguiéndolo se llega de cualquiera de ellos a cualquier otro. Balanceado y débilmente conexo implica fuertemente conexo.

Ejemplo 2.32 (Una cadena de De Bruijn)**.** Se busca la cadena binaria cíclica más corta que contenga, como ventana de tres símbolos consecutivos, cada una de las ocho cadenas de 3 bits. Una cerradura de dos botones que se abre con los últimos tres oprimidos cae tecleando esa cadena: 10 pulsaciones en lugar de 24. El grafo tiene un vértice por cada cadena de 2 bits y, por cada cadena xyz de 3 bits, un arco de xy a yz, con lazos para 000 y 111.



Cada vértice tiene $\mathrm{gr}^- = \mathrm{gr}^+ = 2$ y el grafo es conexo: hay circuito euleriano. Desde 00, con las listas en orden, el algoritmo de Hierholzer de abajo da $00, 00, 01, 10, 01, 11, 11, 10, 00$. Empezando con 00 y agregando el último bit de cada arco sale 0001011100; como el circuito regresa a 00, los dos últimos símbolos repiten los dos primeros, y la cadena cíclica es 00010111. Sus ocho ventanas, dando la vuelta al final, son $000, 001, 010, 101, 011, 111, 110, 100$, cada cadena una vez. No hay una más corta, porque ocho ventanas necesitan ocho posiciones. La misma construcción, con fragmentos de ADN en lugar de bits, es la base de varios ensambladores de genomas.

El algoritmo de Hierholzer

Decidir el tipo cuesta lo mismo que contar grados y correr una BFS.

Algoritmo 2.1 Tipo-Euler(G)

```
1:  procedimiento Tipo-Euler(G)
2:      I ← {v ∈ V : gr(v) es impar}
3:      correr BFS desde un vértice s con gr(s) > 0
4:      si algún v con gr(v) > 0 quedó sin visitar entonces
5:          devolver no euleriano                    // aristas en dos componentes
6:      si no si |I| = 0 entonces
7:          devolver euleriano: circuito desde cualquier s con gr(s) > 0
8:      si no si |I| = 2 entonces
9:          devolver semi-euleriano: sendero de un impar al otro
10:     si no
11:         devolver no euleriano                    // |I| ≥ 4
12:     fin si
13: fin procedimiento
```

Los grados salen de las longitudes de las listas en $\Theta(n+m)$, y la BFS cuesta $\Theta(n+m)$. La rama final es la única que queda, porque $|I|$ es par. Si $m = 0$ no hay nada que recorrer y no hace falta llamar al algoritmo.

Construir el recorrido también es lineal. La prueba del teorema 2.28 empalma subcircuitos; Hierholzer lo organiza con una pila para que el empalme salga solo, sin manipular listas.

Algoritmo 2.2 Hierholzer(G, s)

```
1:  procedimiento HIERHOLZER(G, s)
2:      para todo e ∈ E hacer
3:          usada[e] ← falso
4:      fin para
5:      para todo v ∈ V hacer
6:          sig[v] ← primera entrada de la lista de v
7:      fin para
8:      P ← pila vacía;  APILAR(P, s)
9:      C ← lista vacía
10:     mientras P no está vacía hacer
11:         u ← TOPE(P)
12:         mientras sig[u] apunta a una arista usada hacer
13:             avanzar sig[u]
14:         fin mientras
15:         si sig[u] apunta a una arista e = {u, w} entonces
16:             usada[e] ← verdadero
17:             APILAR(P, w)                          // seguir caminando
18:         si no                                     // u ya no tiene aristas libres
19:             DESAPILAR(P);  agregar u al frente de C
20:         fin si
21:     fin mientras
22:     devolver C
23: fin procedimiento
```

Si el grafo es euleriano se llama con cualquier s de grado positivo; si es semi-euleriano, **obligatoriamente** con uno de los dos impares. Las líneas que no son obvias:

- Líneas 12 a 14: sig$[u]$ es un apuntador que avanza sobre la lista de u y nunca retrocede. Cada arista $\{u, w\}$ aparece en la lista de u y en la de w, con el mismo identificador; al usarla desde u se marca, y la copia en la lista de w se salta cuando el apuntador de w pase por ella.
- Líneas 15 a 17: mientras el tope tenga aristas libres se camina, y la pila guarda la caminata actual.
- Línea 19: un vértice sin aristas libres ya no puede extender nada, y pasa al frente de C. Al desapilar se regresa por la caminata; si aparece un vértice con aristas libres, desde él sale un subcircuito, cuyos vértices se desapilarán más tarde y quedarán en C justo delante de ese vértice. Ese es el empalme de la prueba.

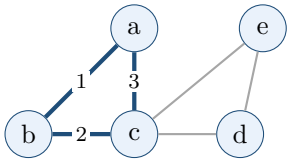
Por qué el subcircuito regresa a su vértice de arranque x: en ese momento las aristas usadas forman un sendero del inicio a x (la pila) y otro de x al final (lo que ya está en C). Por la paridad de la prueba, en las aristas libres todos los grados son pares, y el paso de atorarse dice que la nueva caminata solo se detiene en x. La primera caminata es la excepción en el caso semi-euleriano: sale de un impar y se atora en el otro. C termina con $m + 1$ vértices, en el orden del recorrido.

Ejemplo 2.33 (Hierholzer sobre la mariposa desde a)**.** Desde c, como en la diapositiva, la primera

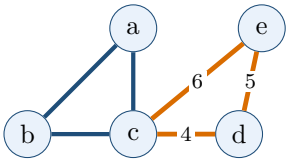
caminata ya usa todo y no se ve el empalme; desde a sí. Las listas, en orden alfabético: a: b, c; b: a, c; c: a, b, d, e; d: c, e; e: c, d.

Paso	**Tope**	**Acción**	**Pila** P	C
1	a	usa $\{a, b\}$ y apila b	a, b	
2	b	usa $\{b, c\}$ y apila c	a, b, c	
3	c	usa $\{c, a\}$ y apila a	a, b, c, a	
4	a	sin aristas libres: desapila	a, b, c	a
5	c	usa $\{c, d\}$ y apila d	a, b, c, d	a
6	d	usa $\{d, e\}$ y apila e	a, b, c, d, e	a
7	e	usa $\{e, c\}$ y apila c	a, b, c, d, e, c	a
8	c	sin aristas libres: desapila	a, b, c, d, e	c, a
9	e	desapila	a, b, c, d	e, c, a
10	d	desapila	a, b, c	d, e, c, a
11	c	desapila	a, b	c, d, e, c, a
12	b	desapila	a	b, c, d, e, c, a
13	a	desapila	vacía	a, b, c, d, e, c, a

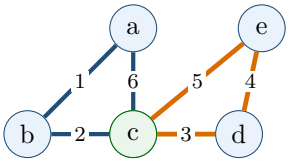
En el paso 4 la caminata se atora en $a = s$, como predice la paridad, después de cerrar a, b, c, a y con $\{c, d\}$, $\{d, e\}$, $\{e, c\}$ sin usar. Al retroceder, c todavía tiene aristas libres, y de él sale el subcircuito c, d, e, c, que queda empalmado en c. Las tres instantáneas: en azul lo recorrido antes, en naranja el subcircuito.



se atora en a: cierra a, b, c, a



desde c sale c, d, e, c



empalme: a, b, c, d, e, c, a

El resultado, a, b, c, d, e, c, a, usa las seis aristas una vez cada una.

Costo. Cada arista se marca una sola vez y provoca un solo Apilar: hay $m + 1$ apilados contando s, y como cada vértice apilado se desapila una vez, $m + 1$ desapilados. El trabajo de las líneas 12 a 14 se paga con el apuntador: sig$[u]$ recorre la lista de u una sola vez en todo el algoritmo, no una vez por visita a u, y las listas suman $2m$ entradas. Inicializar cuesta $\Theta(n + m)$. En total, $\Theta(n + m)$ con listas de adyacencia. Con matriz, guardando multiplicidades, el apuntador de cada renglón barre n columnas y el total es $\Theta(n^2 + m)$.

En un grafo dirigido el algoritmo es el mismo con la lista de arcos que salen de cada vértice. Cada arco aparece en una sola lista, la de su origen, así que ni siquiera hacen falta las marcas: basta avanzar sig$[u]$ cada vez que se usa un arco. Así se obtuvo el circuito del ejemplo 2.32.

Ojito...

Hierholzer no comprueba nada: supone que el recorrido existe y que s es un inicio válido. Primero se clasifica con Tipo-Euler y luego se elige s. En el grafo semi-euleriano del ejemplo 2.30, desde b (impar) devuelve b, a, d, b, c, d, un sendero euleriano distinto del de la diapositiva e igual de válido. Desde a (par) devuelve a, b, c, d, b, a, que repite $\{a, b\}$ y omite $\{a, d\}$: ni siquiera es un sendero. La caminata de un vértice par se atora en el otro impar, no en su inicio, y el empalme queda mal hecho.

Varios textos presentan en su lugar el algoritmo de Fleury: caminar sin cruzar un puente del grafo restante salvo que no haya otra opción. Es correcto, pero cada paso tiene que averiguar si una arista es

puente, y con la prueba por definición eso cuesta $\mathcal{O}(m(n+m))$ en total. Hierholzer evita la pregunta: no le importa atorarse, porque sabe dónde se va a atorar.

Resumen

Pregunta	**Cómo se decide**	**Costo**
¿Es conexo?, ¿cuáles son sus componentes?	BFS o DFS desde cada vértice sin visitar	$\Theta(n+m)$
¿Es e un puente?, ¿es v de corte?	quitarlo y contar componentes; todos a la vez con low (sección 5)	$\Theta(n+m)$
¿Hay circuito o sendero euleriano?	contar vértices impares y revisar conexidad	$\Theta(n+m)$
Construir el recorrido	Hierholzer	$\Theta(n+m)$
¿Son isomorfos?	descartar con invariantes; confirmar con una biyección	sin algoritmo polinomial conocido
¿Hay ciclo hamiltoniano?	sección 3	NP-completo

Idea clave

Euler contra Hamilton es el contraste que importa en un curso de algoritmos. Recorrer cada arista una vez se decide localmente, vértice por vértice, contando paridades, y se construye en tiempo lineal. Recorrer cada vértice una vez no tiene una condición local equivalente, y es NP-completo. Dos preguntas que suenan iguales quedan en los dos extremos de la escala de dificultad.

3. Hamilton, bipartitos, cliques y árboles

Complemento: Rosen, caps. 10 y 11

Resumen rápido. La sección 2 buscó recorridos que usan cada arista una vez. Esta empieza con la pregunta gemela, pasar por cada vértice una vez, y muestra que el parecido engaña: Euler se decide en tiempo lineal y Hamilton es NP-completo. Se ven las condiciones necesarias, la suficiente de Dirac, la búsqueda con retroceso y el agente viajero con la programación dinámica de Held y Karp, que baja de $(n-1)!/2$ rutas a $\Theta(n^2 2^n)$ pasos. Luego los grafos bipartitos, que son exactamente los que no tienen ciclos impares, con un algoritmo de BFS que en $\Theta(n+m)$ entrega la partición o un ciclo impar como testigo. Las cliques dan el contraste entre maximal y máxima. La segunda mitad es de árboles: sus seis caracterizaciones equivalentes, árboles con raíz, árboles k-arios llenos, casi completos y perfectos, las fórmulas de conteo, el balanceo, los tres recorridos y la evaluación de expresiones posfijas con una pila. La materia prima son las diapositivas *Grafos 2* de la Dra. Kelsey Ramírez, que siguen a Rosen; los algoritmos y sus costos son agregados.

Caminos y ciclos hamiltonianos

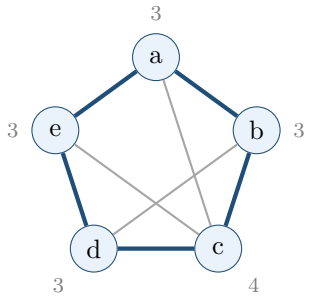
Un recorrido euleriano pasa por cada **arista** exactamente una vez. Si se cambia arista por **vértice** sale otra pregunta, que lleva el nombre de W. R. Hamilton por un juego suyo de 1857: dar la vuelta a los 20 vértices de un dodecaedro sin repetir ninguno.

Definición 3.1 (Camino y ciclo hamiltonianos)**.** Sea $G = (V, E)$ con n vértices. Un **camino hamiltoniano** es un camino $v_1, v_2, \ldots, v_n$ que pasa por todos los vértices, cada uno exactamente una vez. Un **ciclo hamiltoniano** es un ciclo $v_1, v_2, \ldots, v_n, v_1$ que pasa por todos; el único repetido es v_1, por ser inicio y final. Un grafo con ciclo hamiltoniano es un **grafo hamiltoniano**.

Camino y ciclo van en el sentido de la sección 2: no repiten vértices. En un grafo simple el ciclo hamiltoniano exige $n \geq 3$, porque en K_2 la sucesión a, b, a usaría dos veces la misma arista. Quitarle una arista a un ciclo hamiltoniano deja un camino hamiltoniano; al revés no siempre se puede cerrar.

Ejemplo 3.2 (Hamilton no usa todas las aristas)**.** Junto a cada vértice de la figura está su grado; las aristas son

$$E = \{ab, bc, cd, de, ea, ac, ce, bd\}\,.$$



El ciclo a, b, c, d, e, a, en azul, visita los cinco vértices y deja fuera las tres diagonales. Hay otros tres ciclos hamiltonianos distintos (cuatro en total, sin contar sentido ni punto de partida). Un ciclo hamiltoniano usa exactamente n aristas, tenga el grafo las que tenga; un circuito euleriano las usa todas.

Condiciones necesarias

Lo primero que se revisa antes de buscar un ciclo es lo que lo hace imposible.

Teorema 3.3 (Condiciones de grado)**.** *Sea G un grafo simple con $n \geq 3$ vértices.*

(a) Si G tiene ciclo hamiltoniano, $gr(v) \geq 2$ para todo v.
(b) Si G tiene más de dos vértices de grado 1, no tiene camino hamiltoniano.

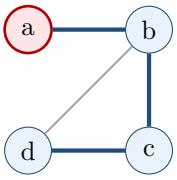
Demostración. (a) En el ciclo cada vértice tiene un antecesor y un sucesor distintos, y las aristas que lo unen con ellos son dos aristas distintas. (b) En un camino, cada vértice interior usa dos aristas del camino, así que uno de grado 1 solo puede ser extremo, y un camino tiene dos extremos. □

Teorema 3.4 (Vértices de corte)**.** *Si G tiene ciclo hamiltoniano, quitar cualquier conjunto S de $k \geq 1$ vértices deja a lo más k componentes conexas. En particular, un grafo con un vértice de corte no es hamiltoniano.*

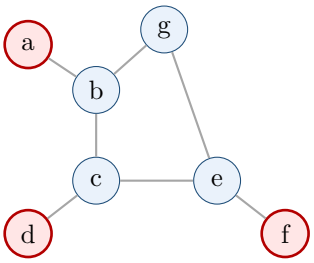
Demostración. Quitar k vértices de un ciclo lo parte en a lo más k tramos, y cada tramo es un camino, que es conexo. Los tramos cubren todos los vértices que quedan y sus aristas siguen en $G - S$, así que $G - S$ tiene a lo más k componentes. Con $k = 1$ sale lo segundo: el ciclo sin un vértice es un camino que toca a todos los demás. □

Para caminos hamiltonianos la cuenta es la misma con un tramo más: quitar k vértices deja a lo más $k + 1$ componentes.

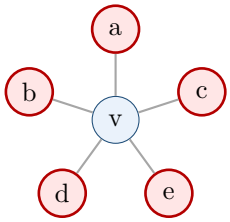
Ejemplo 3.5 (Tres grafos que no pasan)**.** Los tres grafos de las diapositivas que fallan alguna condición; en rojo, los vértices que estorban.



(1) camino a, b, c, d; sin ciclo



(2) tres vértices de grado 1



(3) la estrella $K_{1,5}$

(1) Con $E = \{ab, bc, cd, db\}$ hay camino hamiltoniano, a, b, c, d (y también a, b, d, c), pero no ciclo: $\text{gr}(a) = 1$. Además b es vértice de corte. (2) Con $E = \{ab, bg, bc, cd, ce, ge, ef\}$, los vértices a, d y f tienen grado 1: ni siquiera hay camino. (3) La estrella, un centro unido a cinco hojas, no tiene ni ciclo ni camino: las hojas tienen grado 1 y un camino admite a lo más dos como extremos.

Ojito...

Dos detalles de las diapositivas. Para el grafo (1) y para el camino P_5 (a, b, c, d, e) justifican que no hay ciclo "porque no existe la arista $\{d, a\}$" (o $\{e, a\}$). Eso solo descarta el ciclo que cerraría esa ruta en particular; podría haber otra ruta que sí cierre. El argumento que sí funciona es el del teorema 3.3: a tiene grado 1 y en un ciclo todo vértice usa dos aristas. El otro detalle: el texto llama a la estrella $K_{1,4}$ con "cuatro hojas", pero la figura dibuja cinco (a, b, c, d, e), así que es $K_{1,5}$. La conclusión no cambia; con tres hojas ya basta.

Una condición suficiente: Dirac

Las condiciones de arriba solo descartan. La que garantiza un ciclo pide muchas aristas en cada vértice.

Teorema 3.6 (Dirac, 1952)**.** *Si G es simple, $n \geq 3$ y $gr(v) \geq n/2$ para todo v, entonces G tiene ciclo hamiltoniano.*

Conviene probar algo más fuerte, el teorema de Ore (1960): si $\mathrm{gr}(u) + \mathrm{gr}(v) \geq n$ para cada par u, v no adyacente, G tiene ciclo hamiltoniano. Dirac es el caso particular en que cada sumando ya vale al menos $n/2$.

Prueba de Ore. Supóngase que hay un contraejemplo con n vértices. Agregar aristas solo sube los grados, así que la condición se conserva. Se le agregan aristas, una por una, mientras haya alguna que no lo vuelva hamiltoniano. Se llega a un G que cumple la condición, no es hamiltoniano, y se vuelve hamiltoniano al agregarle **cualquier** arista que le falte; como K_n sí es hamiltoniano, a G le falta alguna. Como $G \neq K_n$, hay u, v no adyacentes. El ciclo hamiltoniano de $G + \{u, v\}$ usa la arista nueva; sin ella es un camino hamiltoniano de G,

$$u = v_1, v_2, \ldots, v_n = v.$$

Sean $S = \{i : u \text{ es adyacente a } v_{i+1}\}$ y $T = \{i : v \text{ es adyacente a } v_i\}$, ambos dentro de $\{1, \ldots, n-1\}$. Cada vecino de u aporta un índice a S y cada vecino de v uno a T, así que $|S| + |T| = \mathrm{gr}(u) + \mathrm{gr}(v) \geq n > n - 1$, y algún i está en los dos. Para ese i,

$$v_1, \ldots, v_i,\ v_n, v_{n-1}, \ldots, v_{i+1},\ v_1$$

es un ciclo hamiltoniano de G: va por el camino hasta v_i, salta a v_n por la arista $\{v_i, v\}$, regresa por el camino hasta v_{i+1} y cierra con $\{v_{i+1}, u\}$. Contradicción. □

Ejemplo 3.7 (Suficiente, no necesaria)**.** En el grafo del ejemplo 3.2, $n = 5$ y el grado mínimo es $3 \geq 5/2$: Dirac garantiza el ciclo sin buscarlo. Las diapositivas usan ese grafo antes del teorema y no lo notan, pero es el contraste natural con el ciclo C_5: ahí cada vértice tiene grado $2 < 5/2$, Dirac no dice nada, y el grafo es él mismo un ciclo hamiltoniano. La condición de Dirac es suficiente, no necesaria.

Euler contra Hamilton

Recordatorio de la sección 2: un grafo conexo (sin contar vértices aislados) tiene circuito euleriano si y solo si todos sus grados son pares, y camino euleriano si y solo si tiene 0 o 2 vértices de grado impar. Decidirlo cuesta $\Theta(n + m)$: contar grados impares y correr una BFS. Para Hamilton no se conoce ningún algoritmo polinomial. Las dos preguntas tienen la misma forma, recorrer todo sin repetir, y solo cambia qué no se repite. Tres hechos explican la diferencia.

La condición de Euler es local. Como los vértices sí se pueden repetir, cada vértice solo necesita emparejar las aristas por las que se entra con las aristas por las que se sale, y eso se puede exactamente cuando su grado es par. Lo único global es la conexidad, y una BFS la revisa. Además los pedazos se pegan: dos circuitos sin aristas en común que comparten un vértice se funden en uno, pasando dos veces por ese vértice. Por eso Hierholzer nunca deshace una decisión.

La condición de Hamilton es global. En cada vértice hay que escoger cuáles dos de sus aristas usar, y lo que se escoge en uno restringe a todos los demás. Los pedazos no se pegan: fundir dos ciclos que comparten un vértice obliga a pasar dos veces por él, que es justo lo prohibido. Una mala elección temprana no se repara localmente; hay que regresar, y el árbol de búsqueda crece como $(n-1)!$. Ninguna condición de grados cierra el hueco entre las necesarias y la de Dirac: en medio quedan grafos como C_5, hamiltoniano sin cumplir Dirac, y el de Petersen, que cumple todas las necesarias y no es hamiltoniano.

La prueba formal es la NP-completitud. Un ciclo propuesto se verifica con n consultas a la matriz, así que el problema está en NP; y cualquier problema de NP se reduce a él (Karp, 1972; Cormen, teorema 34.13, lo hace desde cubierta de vértices). Un algoritmo polinomial para Hamilton daría uno

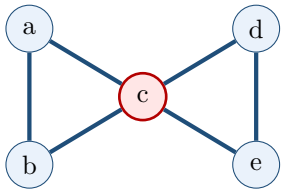
para todo NP, es decir $P = NP$. Lo mejor que se conoce es exponencial: la programación dinámica del agente viajero, más abajo, decide Hamilton en $\Theta(n^2 2^n)$.

Idea clave

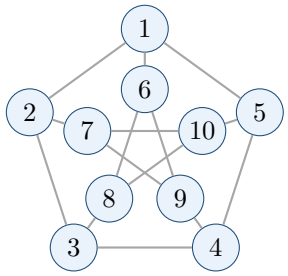
Euler se lee en los grados. Hamilton no se lee en nada local: un ciclo propuesto se verifica en tiempo lineal, pero encontrarlo, hasta donde se sabe, cuesta tiempo exponencial. Las dos propiedades son independientes, como muestra la tabla.

Grafo	n	m	**Impares**	**Euler**	**Camino H.**	**Ciclo H.**
C_5	5	5	0	circuito	sí	sí
Ejemplo 3.2	5	8	4	no	sí	sí
Corbata	5	6	0	circuito	sí	no
Camino P_5	5	4	2	camino	sí	no
Estrella $K_{1,5}$	6	5	6	no	no	no
Petersen	10	15	10	no	sí	no

La "corbata" son dos triángulos que comparten el vértice c: tiene circuito euleriano, a, b, c, d, e, c, a, pero c es vértice de corte y el teorema 3.4 descarta el ciclo hamiltoniano.



corbata: euleriano sí, hamiltoniano no (c es de corte)



Petersen: 3-regular, sin vértices de corte, sin ciclo hamiltoniano

El **grafo de Petersen** es el caso clásico. Tiene $n = 10$ y $m = 15$, todos sus vértices tienen grado 3, es conexo y quitar cualquier vértice lo deja conexo: pasa los teoremas 3.3 y 3.4. Dirac pediría grado 5 y no aplica. No tiene ciclo hamiltoniano, aunque sí camino: $1, 6, 8, 10, 5, 4, 9, 7, 2, 3$ con la numeración de la figura. Que no tiene ciclo se puede probar a mano.

Búsqueda con retroceso

Sin una caracterización, lo que queda es buscar. La búsqueda con retroceso (*backtracking*) hace crecer un camino desde un vértice fijo, y cuando se atora deshace el último paso y prueba el siguiente vecino.

Algoritmo 3.1 Ciclo-Hamiltoniano(G): búsqueda con retroceso

```
1:  procedimiento Ciclo-Hamiltoniano(G)
2:      para todo v ∈ V hacer
3:          usado[v] ← falso
4:      fin para
5:      s ← cualquier vértice;  P ← [s];  usado[s] ← verdadero
6:      devolver Extender(G, P)
7:  fin procedimiento

8:  procedimiento Extender(G, P)
9:      u ← último vértice de P
10:     si |P| = n entonces
11:         devolver {u, P[1]} ∈ E                    // ¿cierra el ciclo?
12:     fin si
13:     para todo v adyacente a u con no usado[v] hacer
14:         agregar v al final de P;  usado[v] ← verdadero
15:         si Extender(G, P) entonces
16:             devolver verdadero
17:         fin si
18:         quitar v de P;  usado[v] ← falso          // retroceder
19:     fin para
20:     devolver falso
21: fin procedimiento
```

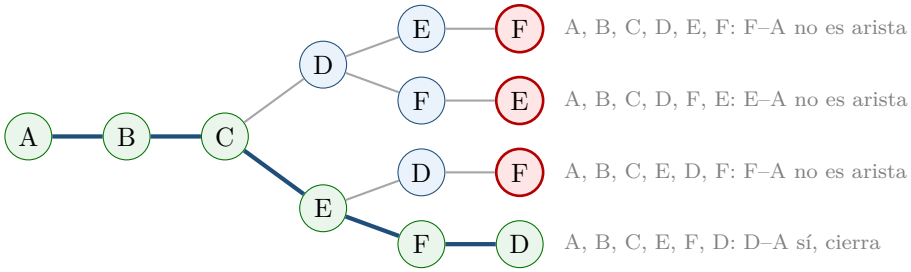
Fijar el inicio s no pierde nada: todo ciclo hamiltoniano pasa por s. Para camino hamiltoniano sí hay que probar todos los inicios, o agregar un vértice nuevo unido a todos los demás: G tiene camino hamiltoniano si y solo si G más ese vértice tiene ciclo.

Costo. Con el arreglo usado, cada consulta es $\mathcal{O}(1)$. Cada llamada a Extender corresponde a un camino que empieza en s, distinto para cada llamada, y el grafo con más caminos así es K_n:

$$\sum_{k=0}^{n-1} \frac{(n-1)!}{(n-1-k)!} = (n-1)! \sum_{j=0}^{n-1} \frac{1}{j!} \leq e \cdot (n-1)!.$$

Para K_5 son 65, el mismo número del ojito sobre los caminos que salen de un vértice, en la sección 1: es aquí donde ese factorial aparece de verdad. Cada llamada trabaja $\mathcal{O}(n)$ fuera de sus llamadas recursivas, así que el total es $\mathcal{O}(n \cdot (n-1)!) = \mathcal{O}(n!)$. Las podas de los teoremas 3.3 y 3.4 se revisan antes en $\Theta(n+m)$ y en la práctica ahorran mucho, pero no cambian el peor caso.

Ejemplo 3.8 (Retroceso sobre el grafo pesado, sin pesos)**.** Se busca un ciclo en el grafo pesado de la sección 1 ignorando los pesos, desde A y con los vecinos en orden alfabético. Cada nodo del árbol de búsqueda es el vértice que se agrega; las ramas se exploran de arriba hacia abajo.



De A el primer vecino libre es B, de B es C, y de C se prueba primero D. Las dos ramas bajo D llegan a usar los seis vértices sin poder cerrar, porque A solo es adyacente a B, C y D. Se regresa a C y se prueba E: bajo E, la rama de D falla igual y la de F termina en D, que sí es vecino de A. El ciclo es A, B, C, E, F, D, A, tras 13 llamadas a Extender.

El agente viajero

Definición 3.9 (Problema del agente viajero)**.** Dadas n ciudades y el costo $w(u, v) \geq 0$ de ir de una a otra, el **problema del agente viajero** (*traveling salesman problem*, TSP) pide la ruta que visita cada ciudad una vez, regresa al inicio y tiene el menor costo total: un ciclo hamiltoniano de costo mínimo en un grafo completo y pesado.

Si el grafo no es completo, una arista que falta vale $w = \infty$. Es al menos tan difícil como Hamilton: dado G, se arma el completo con peso 0 en las aristas de G y 1 en las demás, y G es hamiltoniano si y solo si hay una ruta de costo 0. Por eso la versión de decisión también es NP-completa (Cormen, teorema 34.14).

Fuerza bruta. Se fija la ciudad de salida y se permutan las otras $n-1$. Una ruta y su reversa cuestan lo mismo, así que hay $(n-1)!/2$ rutas distintas, cada una evaluada en $\Theta(n)$: en total $\Theta(n!)$. Para $n = 5$ son 12 rutas; para $n = 20$, unas 6.1×10^{16}.

Programación dinámica (Bellman; Held y Karp, 1962). Con las ciudades numeradas $1, \ldots, n$ y salida en 1, sea $g[S][j]$ el costo del camino más barato que sale de 1, visita exactamente los vértices de $S \subseteq \{2, \ldots, n\}$ y termina en $j \in S$. El penúltimo vértice de ese camino es algún $i \in S$ distinto de j, y lo que viene antes es un camino de 1 a i por $S \setminus \{j\}$, que debe ser el más barato. Eso da

$$g[\{j\}][j] = w(1, j), \qquad g[S][j] = \min_{i \in S \setminus \{j\}} \big(g[S \setminus \{j\}][i] + w(i, j)\big),$$

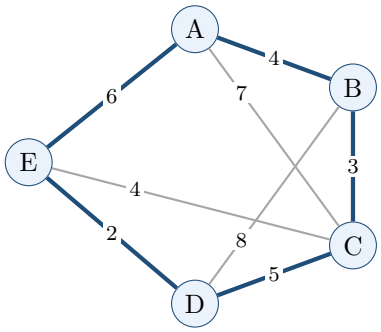
y la ruta óptima cuesta $\min_j \big(g[\{2, \ldots, n\}][j] + w(j, 1)\big)$. Es la subestructura óptima de la clase de programación dinámica: la mejor manera de terminar depende de qué falta y de dónde se está, no del orden en que se visitó S. Esa observación es la que ahorra: hay $(n-1)!$ órdenes, pero solo 2^{n-1} conjuntos.

Costo. Recorrer los conjuntos por tamaño creciente, como en el algoritmo de abajo, garantiza que $g[S \setminus \{j\}][\cdot]$ ya está calculado. Hay $(n-1)\,2^{n-2}$ pares (S, j) con $j \in S$, y cada uno toma un mínimo sobre a lo más $n-2$ términos: $\Theta\big(n^2 2^n\big)$ en tiempo y $\Theta(n 2^n)$ en memoria, porque la tabla se guarda completa. Con S guardado como máscara de bits, cada consulta es $\mathcal{O}(1)$. Para $n = 20$ son unos 4.2×10^8 pasos contra 6.1×10^{16} rutas. Sigue siendo exponencial, como se espera de un problema NP-difícil, y en la práctica la memoria es el cuello de botella.

Algoritmo 3.2 Agente-Viajero(w, n): programación dinámica de Held y Karp

```
1:  procedimiento Agente-Viajero(w, n)
2:      g[{j}][j] ← w(1, j) para j = 2, ..., n
3:      para k ← 2 hasta n − 1 hacer                    // por tamaño creciente
4:          para todo S ⊆ {2, ..., n} con |S| = k hacer
5:              para todo j ∈ S hacer
6:                  g[S][j] ← mín_{i∈S\{j}} (g[S \ {j}][i] + w(i, j))
7:              fin para
8:          fin para
9:      fin para
10:     devolver mín_{2≤j≤n} (g[{2, ..., n}][j] + w(j, 1))
11: fin procedimiento
```

Ejemplo 3.10 (El agente viajero de las diapositivas)**.** Cinco ciudades con $w(A,B) = 4$, $w(A,C) = 7$, $w(A,E) = 6$, $w(B,C) = 3$, $w(B,D) = 8$, $w(C,D) = 5$, $w(C,E) = 4$ y $w(D,E) = 2$.



óptimo: A, B, C, D, E, A, con costo 20

Faltan A–D y B–E, que valen ∞. De las 12 rutas de K_5 solo existen cuatro:

Ruta	**Costo**
A, B, C, D, E, A	$4+3+5+2+6=20$
A, B, D, E, C, A	$4+8+2+4+7=25$
A, C, B, D, E, A	$7+3+8+2+6=26$
A, B, D, C, E, A	$4+8+5+4+6=27$

La tabla de Held y Karp, con salida en A, da lo mismo sin enumerar rutas. Cada renglón es un conjunto S y cada columna el vértice final j; un guion indica $j \notin S$.

S	$j=B$	$j=C$	$j=D$	$j=E$
$\{B\}$	4	–	–	–
$\{C\}$	–	7	–	–
$\{D\}$	–	–	∞	–
$\{E\}$	–	–	–	6
$\{B,C\}$	10	7	–	–
$\{B,D\}$	∞	–	12	–
$\{B,E\}$	∞	–	–	∞
$\{C,D\}$	–	∞	12	–
$\{C,E\}$	–	10	–	11
$\{D,E\}$	–	–	8	∞

S	$j=B$	$j=C$	$j=D$	$j=E$
$\{B,C,D\}$	20	17	12	–
$\{B,C,E\}$	13	∞	–	11
$\{B,D,E\}$	16	–	∞	14
$\{C,D,E\}$	–	13	13	14
$\{B,C,D,E\}$	16	18	13	14
$+\,w(j,A)$	20	25	∞	20

Un par de entradas a mano: $g[\{B,C\}][C] = g[\{B\}][B] + w(B,C) = 4+3 = 7$, porque la única opción es A, B, C, y $g[\{B,C\}][B] = g[\{C\}][C] + w(C,B) = 7+3 = 10$. Y $g[\{B,C,D,E\}][E] = 14$ es A, B, C, D, E, con $4+3+5+2$; cerrar con $w(E,A) = 6$ da 20. Por la columna de B también sale 20: es la misma ruta al revés. Por D sale ∞ porque no hay arista D–A.

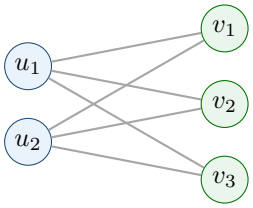
Ojito...

El grafo de las diapositivas no es completo y la diapositiva no da la respuesta. En el TSP el grafo se supone completo; una arista que falta se modela con costo ∞, y entonces una ruta "existe" solo si su costo es finito. Ojo también con la frase "la ruta más corta": no es el camino más corto entre dos ciudades de las secciones 7 a 9, que se resuelve en tiempo polinomial, sino un ciclo que toca a todas.

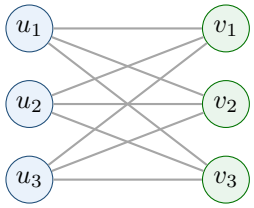
Grafos bipartitos

Definición 3.11 (Grafo bipartito)**.** Un grafo no dirigido $G = (V, E)$ es **bipartito** si V se parte en dos conjuntos disjuntos, $V = V_1 \cup V_2$, de modo que toda arista tiene un extremo en V_1 y el otro en V_2. El **bipartito completo** $K_{p,q}$ tiene $|V_1| = p$, $|V_2| = q$ y cada vértice de V_1 unido con todos los de V_2: tiene pq aristas.

Modela cualquier relación entre dos tipos de objetos: estudiantes y proyectos, trabajadores y tareas, usuarios y películas. Asignar a cada estudiante un proyecto distinto es el emparejamiento de la sección 11. Cormen llama a los lados L y R.



$K_{2,3}$: $2 \cdot 3 = 6$ aristas



$K_{3,3}$: $3 \cdot 3 = 9$ aristas

Observación. Las diapositivas y Rosen escriben $K_{m,n}$. Aquí $n = |V|$ y $m = |E|$, y con esa lectura "$K_{m,n}$ tiene mn aristas" diría $m = mn$. Por eso los lados se llaman p y q.

La caracterización por ciclos impares

Un ciclo en un bipartito va de un lado al otro en cada paso, así que para volver al lado de salida necesita un número par de pasos. Lo notable es que eso basta: lo único que impide partir un grafo en dos lados es un ciclo impar.

Teorema 3.12 (Caracterización de los bipartitos)**.** *Un grafo no dirigido es bipartito si y solo si no contiene ciclos de longitud impar.*

Demostración. ($\Rightarrow$) Sean V_1, V_2 los lados y $v_0, v_1, \ldots, v_k = v_0$ un ciclo. Cada arista cambia de lado, así que v_i está en el lado de v_0 exactamente cuando i es par. Como $v_k = v_0$, k es par.

($\Leftarrow$) Basta partir cada componente conexa por separado: no hay aristas entre componentes, así que la unión de las particiones sirve. Sea entonces G conexo y sin ciclos impares. Se corre BFS desde cualquier s y se pone

$$V_1 = \{v : d[v] \text{ es par}\}, \qquad V_2 = \{v : d[v] \text{ es impar}\}.$$

Falta ver que ninguna arista queda dentro de un lado. Supóngase que $\{u, v\} \in E$ con $d[u]$ y $d[v]$ de la misma paridad.

Primero, $d[u] = d[v]$. En BFS, $d[v] = \delta(s, v)$, el número de aristas del camino más corto (sección 1; la prueba está en la 4). Un camino más corto a u seguido de la arista $\{u, v\}$ llega a v, así que $\delta(s, v) \leq \delta(s, u) + 1$, y por simetría $|d[u] - d[v]| \leq 1$. Con la misma paridad, la diferencia es 0.

Luego, el ciclo. Sea w el ancestro común más profundo de u y v en el árbol BFS; existe, porque s es ancestro de todos. Cada arista del árbol baja exactamente un nivel, así que el camino del árbol de w a u mide $d[u] - d[w]$, y el de w a v mide $d[v] - d[w] = d[u] - d[w]$. Por ser w el más profundo, los dos caminos solo comparten a w; además $w \neq u$, porque u no puede ser ancestro de v estando en el mismo nivel. La arista $\{u, v\}$ no es del árbol, porque une dos vértices del mismo nivel. Los dos caminos y la arista forman entonces un ciclo de longitud

$$2\big(d[u] - d[w]\big) + 1,$$

impar. Contradicción. □

Observación. Da igual hablar de ciclos impares o de caminatas cerradas impares: una caminata cerrada de longitud impar siempre contiene un ciclo impar. Si la caminata repite un vértice, además de cerrar en el inicial, se parte ahí en dos caminatas cerradas más cortas cuyas longitudes suman la original; una de las dos es impar, y se repite el argumento hasta llegar a una que no repite vértices.

El algoritmo: dos lados con BFS

La prueba es un algoritmo. Se corre BFS, se asigna a cada vértice el lado que dice la paridad de su distancia, y se revisa cada arista. Si ninguna une dos vértices del mismo lado, los lados son la partición; si alguna lo hace, se reconstruye el ciclo impar subiendo por los padres desde sus dos extremos, como en la prueba. Para no chocar con color$[v]$ de la sección 4, que es blanco, gris o negro, el lado se guarda en lado$[v] \in \{0, 1\}$. Las secciones 4 (el problema de los luchadores) y 11 (emparejamiento) usan este algoritmo tal cual.

Primero la pieza que reconstruye el testigo. Se llama solo en un conflicto, cuando u y v son adyacentes y están en el mismo lado, y sube por π desde los dos hasta que se encuentran.

Algoritmo 3.3 Ciclo-Impar(u, v): el testigo de que no es bipartito

```
1: procedimiento Ciclo-Impar(u, v)                    // al llamarse, d[u] = d[v]
2:     x ← u,    y ← v,    P ← [u],    R ← [v]
3:     mientras x ≠ y hacer                           // suben a la par
4:         x ← π[x];  agregar x al final de P
5:         y ← π[y];  agregar y al final de R
6:     fin mientras
7:     devolver P seguida de R al revés, sin repetir x
8: fin procedimiento
```

El procedimiento principal es la BFS de la clase con un lado por vértice.

Algoritmo 3.4 Bipartito(G): dos lados con BFS

```
1:  procedimiento Bipartito(G)
2:      para todo v ∈ V hacer
3:          lado[v] ← nulo,    π[v] ← nulo
4:      fin para
5:      para todo s ∈ V hacer                          // una BFS por componente
6:          si lado[s] = nulo entonces
7:              lado[s] ← 0,    d[s] ← 0
8:              Q ← cola vacía;  Encolar(Q, s)
9:              mientras Q no está vacía hacer
10:                 u ← Desencolar(Q)
11:                 para todo v en la lista de adyacencia de u hacer
12:                     si lado[v] = nulo entonces
13:                         lado[v] ← 1 − lado[u],    d[v] ← d[u] + 1,    π[v] ← u
14:                         Encolar(Q, v)
15:                     si no si lado[v] = lado[u] entonces
16:                         devolver falso, Ciclo-Impar(u, v)
17:                     fin si
18:                 fin para
19:             fin mientras
20:         fin si
21:     fin para
22:     devolver verdadero, V1 = {v : lado[v] = 0}, V2 = {v : lado[v] = 1}
23: fin procedimiento
```

Detalles que no son obvios:

- El ciclo externo arranca una BFS en cada componente. Los lados de componentes distintas son independientes, y la partición final es la unión.
- El lado es la paridad de $d[v]$. Para decidir no hace falta d; se guarda para que Ciclo-Impar sepa que los dos extremos están en el mismo nivel y puedan subir a la par.
- A diferencia de la detección de ciclos de la sección 1, no hace falta excluir al padre: el padre está del otro lado y nunca dispara la condición. Una arista fuera del árbol entre niveles consecutivos cierra un ciclo **par** y es inofensiva; solo una arista dentro de un mismo nivel delata un ciclo impar.
- Ciclo-Impar devuelve $u, \ldots, w, \ldots, v$, donde w es el ancestro común; la arista $\{v, u\}$ lo cierra.

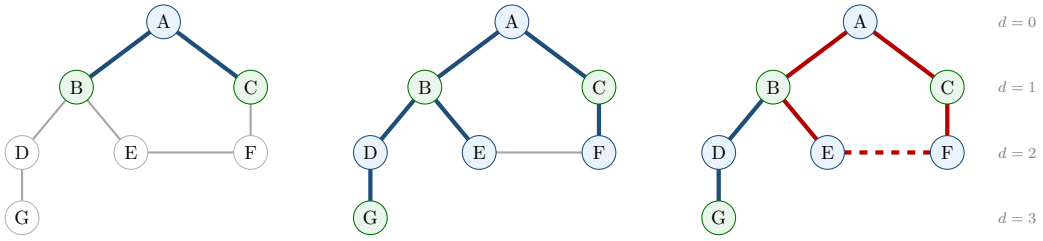
Costo. La inicialización es $\Theta(n)$. Cada vértice recibe lado una sola vez, justo antes de encolarse, y nunca cambia: n operaciones de cola de $\mathcal{O}(1)$. La lista de cada vértice se recorre una vez, cuando sale de la cola; las listas suman $2m$ entradas, cada una con una comparación de $\mathcal{O}(1)$. El ciclo externo agrega $\Theta(n)$. Total, $\Theta(n + m)$ con listas, y $\Theta(n^2)$ con matriz. Ciclo-Impar sube un nivel por iteración: $\mathcal{O}(n)$. Decidir si un grafo es bipartito no cuesta más que recorrerlo.

Idea clave

El algoritmo siempre entrega un testigo que se revisa a mano: la partición, si es bipartito, o un ciclo impar, si no. Es la diferencia con Hamilton, donde un "no" no viene con ningún certificado corto conocido.

Ejemplo 3.13 (El grafo G de la clase no es bipartito)**.** Se corre Bipartito sobre el grafo G de la sección 1, con los vecinos en orden alfabético. En la figura, azul es el lado 0 y verde el lado 1.

Paso	Sale	Asigna o revisa	Cola después
0		A: 0	[A]
1	A	B: 1, C: 1	[B, C]
2	B	A bien; D: 0, E: 0	[C, D, E]
3	C	A bien; F: 0	[D, E, F]
4	D	B bien; G: 1	[E, F, G]
5	E	B bien; F tiene 0, **igual** que E	conflicto

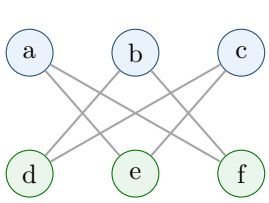


1. sale A 4. sale D 5. sale E: E y F en el lado 0

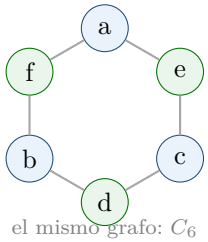
En el paso 5 sale E, del lado 0, y su vecino F también está en el lado 0: $d[E] = d[F] = 2$, la arista E–F queda dentro de un nivel. Ciclo-Impar(E, F) sube $E \to B \to A$ y $F \to C \to A$, se encuentra en A y devuelve E, B, A, C, F; con la arista F–E es el ciclo A, B, E, F, C, A, de longitud 5, en rojo. Es el mismo ciclo que la clase encontró con BFS en "Encontrar ciclos", y ahora se sabe además que es impar. G no es bipartito.

Ejemplo 3.14 (Un bipartito disfrazado)**.** El ejemplo 6 de bipartitos de las diapositivas tiene $V = \{a, b, c, d, e, f\}$ y $E = \{ae, af, bd, bf, cd, ce\}$.

Paso	Sale	Asigna o revisa	Cola
0		a: 0	[a]
1	a	e: 1, f: 1	[e, f]
2	e	a bien; c: 0	[f, c]
3	f	a bien; b: 0	[c, b]
4	c	d: 1; e bien	[b, d]
5	b	d bien, f bien	[d]
6	d	b bien, c bien	[ ]



como en la diapositiva



el mismo grafo: C_6

Nunca hay conflicto: $V_1 = \{a, b, c\}$, $V_2 = \{d, e, f\}$. El dibujo de la diapositiva esconde que el grafo es el hexágono C_6: a, e, c, d, b, f, a. Son el mismo grafo dibujado de dos maneras, isomorfos en el sentido de la sección 2. También es $K_{3,3}$ sin las aristas a–d, b–e y c–f.

Observación. En un bipartito conexo la partición es única salvo intercambiar los lados: el lado de s fija el de todos los demás. Con c componentes hay 2^c maneras de asignar los lados, porque cada componente se puede voltear por separado. Y ser bipartito ignora el sentido de las aristas: en un dirigido se pregunta por el no dirigido que queda al borrar las flechas.

Bipartitos y Hamilton

El mismo argumento de alternancia da una poda para Hamilton.

Corolario 3.15. *Sea G bipartito con lados V_1 y V_2. Si G tiene ciclo hamiltoniano, $|V_1| = |V_2|$. Si tiene camino hamiltoniano, $||V_1| - |V_2|| \leq 1$.*

Demostración. El ciclo alterna de lado, así que tiene tantos vértices de un lado como del otro, y pasa por todos. El camino también alterna: con un número par de vértices los dos lados quedan iguales, y con uno impar empieza y termina en el mismo lado, que lleva uno de más. □

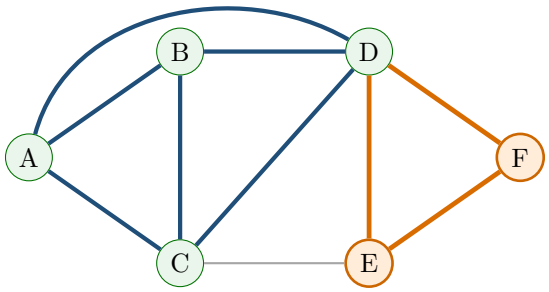
Esto liquida de un golpe la estrella $K_{1,5}$ (lados de 1 y 5: ni camino) y muestra que $K_{2,3}$ no es hamiltoniano aunque tenga el camino v_1, u_1, v_2, u_2, v_3. $K_{3,3}$ sí lo es: tiene seis ciclos hamiltonianos distintos.

Cliques

Definición 3.16 (Clique)**.** Una **clique** de G es un conjunto $C \subseteq V$ de vértices adyacentes dos a dos: si $u, v \in C$ y $u \neq v$, entonces $\{u, v\} \in E$. Su tamaño es $|C|$. El **número de clique** $\omega(G)$ es el tamaño de la clique más grande. Una clique es **maximal** si no se le puede agregar ningún vértice, porque ninguno fuera de ella es adyacente a todos los de adentro; es **máxima** si tiene tamaño $\omega(G)$.

El **subgrafo inducido** $G[C]$ tiene los vértices de C y todas las aristas de G entre ellos. C es clique exactamente cuando $G[C]$ es isomorfo al completo $K_{|C|}$, que es como lo escriben las diapositivas: $G[C] \cong K_k$. Toda clique máxima es maximal; al revés no. En redes sociales una clique es un grupo en el que todos se conocen, y buscarlas es la forma más estricta de detectar grupos densos.

Ejemplo 3.17 (Las cliques del grafo pesado)**.** En el grafo pesado de la sección 1, sin pesos, los cuatro vértices A, B, C, D están unidos todos con todos.



azul: $\{A, B, C, D\}$, máxima
naranja: $\{D, E, F\}$, maximal no máxima

$\{A, B, C, D\}$ es una clique de tamaño 4, así que $\omega(G) \geq 4$. Para la igualdad hay que descartar una de tamaño 5: una K_5 necesita cinco vértices de grado al menos 4, y los grados son $3, 3, 4, 5, 3, 2$ para A a F; solo C y D llegan. Así que $\omega(G) = 4$. Las cliques maximales son tres: $\{A, B, C, D\}$, $\{C, D, E\}$ y $\{D, E, F\}$. La última es maximal, porque ningún otro vértice es adyacente a D, E y F a la vez, pero no máxima.

Ojito...

Exhibir una clique de tamaño k solo prueba $\omega(G) \geq k$; para la igualdad falta un argumento de que no hay una más grande, como el de grados de arriba. Las diapositivas se quedan en la primera mitad. Y una clique es un **conjunto de vértices**, no un grafo: la frase "el grafo completo K_4 es una clique de tamaño 4" mezcla las dos cosas. La clique es $\{a, b, c, d\}$; su subgrafo inducido es isomorfo a K_4.

La diferencia entre maximal y máxima es algorítmica. Una clique maximal se encuentra con un barrido voraz: se recorren los vértices en cualquier orden y se agrega cada uno que sea adyacente a todos los ya escogidos. Con matriz, cada revisión cuesta $\mathcal{O}(n)$ y el total es $\mathcal{O}(n^2)$. El resultado depende del orden: en el grafo pesado, el orden A, B, ..., F da $\{A, B, C, D\}$, la máxima, y el orden F, E, ..., A da $\{F, E, D\}$, que es maximal y nada más. Encontrar la máxima es otra historia: decidir si $\omega(G) \geq k$ es el problema CLIQUE, NP-completo (Cormen, teorema 34.11), y la fuerza bruta revisa los 2^n subconjuntos. Es la misma brecha que entre Euler y Hamilton.

Observación. Si G es bipartito y tiene al menos una arista, $\omega(G) = 2$, porque un triángulo es un ciclo impar. Al revés no: el grafo G de la clase no tiene triángulos, así que $\omega(G) = 2$, y aun así no es bipartito por su ciclo de longitud 5.

Árboles

Definición y caracterizaciones

Definición 3.18 (Árbol y bosque)**.** Un **árbol** es un grafo no dirigido, conexo y sin ciclos. Un **bosque** es un grafo sin ciclos, no necesariamente conexo: cada una de sus componentes es un árbol. Una **hoja** de un árbol es un vértice de grado 1; las diapositivas también le dicen vértice terminal o colgante.

Cormen dice *free tree* para distinguirlo del árbol con raíz. Todo lo que sigue sale de dos lemas.

Lema 3.19 (Hojas)**.** *Todo árbol con $n \geq 2$ vértices tiene al menos dos hojas.*

Demostración. Sea $v_0, v_1, \ldots, v_k$ un camino de longitud máxima; $k \geq 1$ porque un árbol conexo con dos o más vértices tiene aristas. Si v_0 tuviera un vecino distinto de v_1, o ese vecino está en el camino y cierra un ciclo, o no está y alarga el camino. Ninguna de las dos se puede, así que $\text{gr}(v_0) = 1$; lo mismo para v_k. □

Lema 3.20 (Aristas)**.** *Todo árbol con n vértices tiene $n - 1$ aristas. Un bosque con n vértices y c componentes tiene $n - c$.*

Demostración. Inducción sobre n. Con $n = 1$ no hay aristas. Con $n \geq 2$ se quita una hoja v y su arista. Lo que queda sigue sin ciclos y sigue conexo, porque ningún camino entre otros dos vértices pasa por v: entraría y saldría por su única arista. Es un árbol con $n - 1$ vértices, que por hipótesis tiene $n - 2$ aristas; con la de v son $n - 1$. Para el bosque se suma sobre las componentes: $\sum(n_i - 1) = n - c$. □

Teorema 3.21 (Caracterizaciones de un árbol)**.** *Para un grafo no dirigido G con n vértices y m aristas, son equivalentes:*

(1) G es un árbol: conexo y sin ciclos.
(2) Entre cada par de vértices hay un único camino.
(3) G es conexo y $m = n - 1$.
(4) G no tiene ciclos y $m = n - 1$.

(5) G es conexo y cada arista es un puente.
(6) G no tiene ciclos y agregar cualquier arista nueva crea uno.

Demostración. (1)⇔(2). Conexo dice que hay al menos un camino. Si hubiera dos caminos distintos P y P' de u a v, sea x el último vértice común antes de que se separen y y el primer vértice de P, después de x, que vuelve a estar en P'. Los tramos de P y de P' entre x e y solo comparten sus extremos y juntos cierran un ciclo. Al revés, si hubiera un ciclo, dos vértices vecinos en él estarían unidos por la arista y por el resto del ciclo: dos caminos.

(1)⇒(3) y (1)⇒(4) son el lema 3.20.

(3)⇒(1). Si G conexo tuviera un ciclo, quitar una arista del ciclo no lo desconecta, porque sus extremos siguen unidos por el resto del ciclo. Repitiendo hasta que no queden ciclos se llega a un árbol con n vértices y menos de $n-1$ aristas, contra el lema.

(4)⇒(1). Sin ciclos es un bosque; si tiene c componentes, $m = n - c$, y con $m = n - 1$ sale $c = 1$.

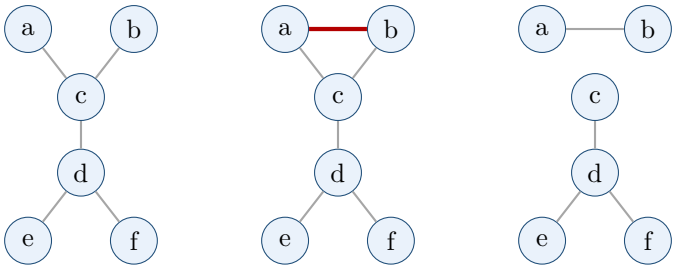
(1)⇔(5). En un árbol, quitar $\{u, v\}$ desconecta a u de v: el único camino entre ellos era esa arista. Al revés, una arista de un ciclo nunca es puente, así que si todas lo son no hay ciclos.

(1)⇔(6). En un árbol, si $\{u, v\} \notin E$, el único camino de u a v más la arista nueva es un ciclo. Al revés, si G sin ciclos no fuera conexo, una arista entre dos componentes no crearía ningún ciclo. □

Idea clave

(5) y (6) dicen lo mismo desde los dos lados: un árbol es un grafo conexo con el mínimo de aristas y un grafo acíclico con el máximo. Kruskal vive de eso: por (6), una arista entre dos vértices que ya están unidos cerraría un ciclo, y por (4), se detiene a las $n-1$ aristas (sección 6).

Ejemplo 3.22 (Árbol, ni árbol ni bosque, bosque)**.** Tres grafos sobre los mismos seis vértices.



G_1 G_2 G_3

G_1 es árbol: conexo, con 6 vértices y 5 aristas, por (3). G_2 le agrega a–b y tiene el ciclo a, b, c, a: conexo con 6 aristas, no es árbol ni bosque. G_3 no tiene ciclos pero tampoco es conexo: es un bosque de dos árboles, y en efecto $m = 4 = 6 - 2$.

Árboles con raíz

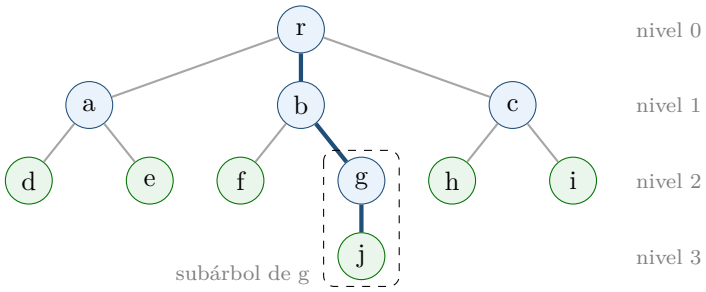
Definición 3.23 (Árbol con raíz)**.** Un **árbol con raíz** (*rooted tree*) es un árbol con un vértice distinguido, la **raíz**; las aristas se leen de arriba hacia abajo. Para un vértice v:

- si v no es la raíz, su **padre** es el vecino que está en el camino de v a la raíz; v es **hijo** de su padre, y dos vértices con el mismo padre son **hermanos**;

- sus **ancestros** son los vértices del camino de la raíz a v, sin contar a v; sus **descendientes**, los vértices que tienen a v como ancestro;
- una **hoja** es un vértice sin hijos y un vértice **interno** uno con al menos un hijo;
- el **subárbol con raíz** v tiene a v, sus descendientes y las aristas entre ellos.

El **nivel** de v es la longitud del camino de la raíz a v; la raíz está en el nivel 0. La **altura** del árbol es el mayor nivel de sus hojas.

Ejemplo 3.24 (El vocabulario sobre un árbol)**.** El árbol de las diapositivas, con raíz r; en verde, las hojas.



El padre de g es b; los hijos de b son f y g; el hermano de h es i. Las hojas son d, e, f, h, i, j y los internos r, a, b, c, g. Los ancestros de j son r, b, g, el camino en azul; los descendientes de c son h e i. El subárbol con raíz g tiene los vértices g, j y la arista $\{g, j\}$. j está en el nivel 3 y la altura es 3.

Los árboles de los recorridos ya son árboles con raíz: en el árbol BFS de la sección 1, $\pi[v]$ es el padre de v, $d[v]$ es su nivel y la altura es la mayor distancia desde s. Un árbol es **ordenado** si los hijos de cada vértice tienen un orden fijo de izquierda a derecha; dos árboles con los mismos padres pero con los hijos de r intercambiados son iguales como árboles con raíz y distintos como ordenados. En un binario ordenado se habla de hijo izquierdo e hijo derecho.

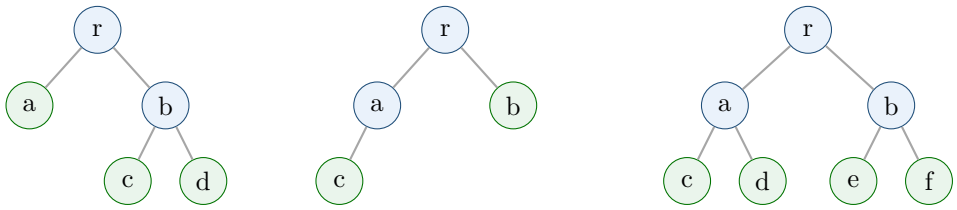
Ojito...

"Hoja" no significa lo mismo en el árbol libre y en el árbol con raíz. En el libre es grado 1; en el árbol con raíz, sin hijos. Difieren en la raíz: una raíz con un solo hijo tiene grado 1 y no es hoja. Con los ancestros pasa algo parecido: las diapositivas excluyen al propio vértice, y Cormen lo incluye y dice "ancestro propio" para excluirlo.

Árboles k-arios: lleno, casi completo y perfecto

Definición 3.25 (Árboles k-arios)**.** Un árbol con raíz es **k-ario** si cada vértice tiene a lo más k hijos; **binario** si $k = 2$ y **ternario** si $k = 3$. Un árbol k-ario es:

- **lleno** si cada vértice interno tiene exactamente k hijos;
- **casi completo** si todos sus niveles están llenos salvo tal vez el último, que se llena de izquierda a derecha: es la forma del heap;
- **perfecto** si es lleno y todas sus hojas están en el mismo nivel.



lleno, no casi completo — casi completo, no lleno — perfecto: lleno y casi completo

A la izquierda, cada interno tiene dos hijos, pero el nivel 2 se llenó por la derecha. En medio, el heap de cuatro elementos: el nivel 2 está a la izquierda, pero a tiene un solo hijo. A la derecha, el perfecto de altura 2, que es las dos cosas.

Ojito...

"Completo" tiene tres significados según la fuente, y aquí se evita a propósito. Las diapositivas, siguiendo a Rosen (*full m-ary tree*), llaman árbol n-ario completo al que aquí es **lleno**. La clase de heaps (17 de septiembre) llama árbol binario completo al que aquí es **casi completo**, que es el *nearly complete* de Cormen. Y Cormen llama *complete k-ary tree* al que aquí es **perfecto**. Leer "completo" sin saber de qué libro viene es como se confunde un heap con un árbol lleno, y las fórmulas de abajo valen para uno y no para el otro.

Teorema 3.26 (Conteo en árboles llenos)**.** *Un árbol k-ario lleno con i vértices internos tiene*

$$n = ki + 1 \text{ vértices} \qquad y \qquad \ell = (k-1)i + 1 \text{ hojas.}$$

Demostración. Todo vértice salvo la raíz es hijo de exactamente un interno, y cada interno tiene exactamente k hijos: $n - 1 = ki$. Las hojas son el resto, $\ell = n - i = (k-1)i + 1$. □

Despejando, cualquiera de n, i y ℓ da los otros dos; por ejemplo, $i = (\ell-1)/(k-1)$ y $n = (k\ell-1)/(k-1)$. Las diapositivas escriben $m = ni + 1$, con n la aridad; aquí la aridad es k para no chocar con $n = |V|$.

Definición 3.27 (Árbol balanceado)**.** Un árbol con raíz de altura h es **balanceado** si todas sus hojas están en los niveles h o $h - 1$.

El del ejemplo 3.24 es balanceado: altura 3, hojas en los niveles 2 y 3. Todo casi completo es balanceado, porque sus niveles están llenos hasta el penúltimo, y todo perfecto también. Un árbol con raíz r, hijos a y b, y una cadena b, c, d no lo es: tiene altura 3 y la hoja a está en el nivel 1.

Ojito...

Balanceado, en este sentido, no es AVL. La definición de las diapositivas es la de Rosen y solo mira dónde están las hojas. Un árbol AVL pide que en **cada** vértice las alturas de los dos subárboles difieran en a lo más 1. Hay árboles que cumplen una y no la otra; que coincidan en un ejemplo no las hace la misma definición.

Teorema 3.28 (Altura contra hojas)**.** *(a) Un árbol k-ario de altura h tiene a lo más k^h hojas; por tanto $h \geq \lceil \log_k \ell \rceil$. (b) Si además es lleno y balanceado, $h = \lceil \log_k \ell \rceil$.*

Demostración. (a) Inducción sobre h. Con $h = 0$ hay una hoja, k^0. Con $h \geq 1$, la raíz tiene a lo más k hijos y cada uno es raíz de un subárbol de altura a lo más $h - 1$, con a lo más k^{h-1} hojas. Como h es entero y $\log_k \ell \leq h$, sale la cota con techo.

(b) Con $h \geq 1$. Por ser balanceado no hay hojas antes del nivel $h-1$, y por ser lleno cada vértice de los niveles 0 a $h-2$ tiene k hijos: el nivel $h-1$ tiene exactamente k^{h-1} vértices. De ellos, $I \geq 1$ son internos (alguna hoja está en el nivel h) y cada uno aporta k hojas en el nivel h; los demás son hojas. Entonces

$$\ell = (k^{h-1} - I) + kI = k^{h-1} + (k-1)I > k^{h-1},$$

y con (a), $k^{h-1} < \ell \leq k^h$, o sea $h - 1 < \log_k \ell \leq h$. □

Observación. La parte (a) da la cota inferior $\Omega(n \lg n)$ de ordenar por comparaciones. Un algoritmo que solo compara se describe con un árbol binario de decisión: cada vértice interno es una comparación y cada hoja un orden de salida. Para ordenar bien necesita al menos $n!$ hojas, así que su altura, el número de comparaciones en el peor caso, es al menos $\lg n! = \Theta(n \lg n)$.

Ejemplo 3.29 (Torneo de eliminación directa)**.** ¿Cuántos partidos hay en un torneo de eliminación directa con 16 jugadores? El torneo es un árbol binario lleno: las hojas son los jugadores, cada vértice interno es un partido y la raíz es la final. Con $k = 2$ y $\ell = 16$, el teorema 3.26 da $i = \ell - 1 = 15$ partidos, y como el cuadro es balanceado, el teorema 3.28 da $\lceil \lg 16 \rceil = 4$ rondas. El argumento directo es más general: cada partido elimina a uno y todos salvo el campeón quedan eliminados una vez. Con 13 jugadores y pases libres hay 12 partidos y $\lceil \lg 13 \rceil = 4$ rondas.

Recorridos

Un **recorrido** de un árbol visita todos sus vértices de manera sistemática. En un árbol binario ordenado hay tres naturales, según cuándo se visita la raíz respecto a sus dos subárboles: **preorden** (raíz, izquierdo, derecho), **inorden** (izquierdo, raíz, derecho) y **postorden** (izquierdo, derecho, raíz).

Algoritmo 3.5 Recorrer(x): los tres recorridos de un árbol binario

```
1: procedimiento Recorrer(x)
2:     si x ≠ nulo entonces
3:         visitar x                          // solo en preorden
4:         Recorrer(izq[x])
5:         visitar x                          // solo en inorden
6:         Recorrer(der[x])
7:         visitar x                          // solo en postorden
8:     fin si
9: fin procedimiento
```

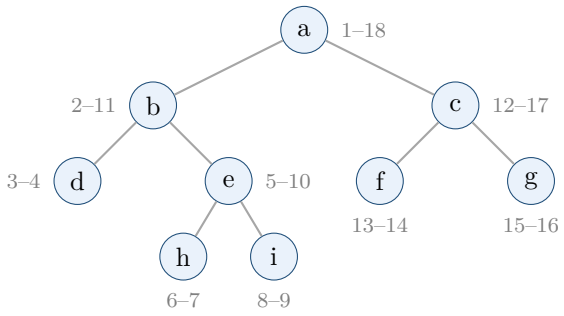
Cada recorrido conserva una sola de las tres líneas que visitan: Preorden es Recorrer con la línea 3, Inorden con la 5 y Postorden con la 7, y las llamadas recursivas se renombran igual. Lo único que cambia es en qué momento se visita la raíz.

Costo. Cada vértice se visita una vez, y cada llamada hace trabajo constante más dos llamadas recursivas. Con j vértices en el subárbol izquierdo,

$$T(n) = T(j) + T(n-j-1) + d, \qquad T(0) = c,$$

y por sustitución $T(n) = (c+d)n + c$: la hipótesis da $(c+d)j + c + (c+d)(n-j-1) + c + d = (c+d)n + c$. Los tres son $\Theta(n)$, sin importar la forma del árbol (Cormen, teorema 12.1).

Ejemplo 3.30 (Los tres recorridos y el reloj de DFS)**.** El árbol de las diapositivas, con entra–sale de DFS-Visitar junto a cada vértice, visitando primero el hijo izquierdo.

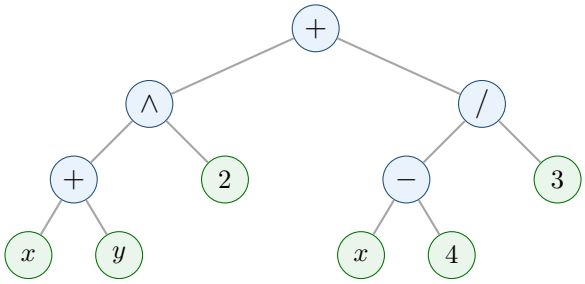


Recorrido	Resultado	Cómo sale del reloj
Preorden	a, b, d, e, h, i, c, f, g	orden creciente de entra
Inorden	d, b, h, e, i, a, f, c, g	(solo binarios)
Postorden	d, h, i, e, b, f, g, c, a	orden creciente de sale
Por niveles	a, b, c, d, e, f, g, h, i	BFS desde a

El preorden es el orden en que DFS descubre los vértices y el postorden el orden en que los termina. El inorden solo tiene sentido en binarios, porque necesita un "en medio" entre dos hijos. El recorrido por niveles es BFS.

Árboles de expresiones y notación posfija

Una expresión aritmética con operadores binarios es un árbol binario lleno: cada operador es un vértice interno con sus dos operandos como hijos, y los números y variables son las hojas. El árbol de $(x+y)^2+(x-4)/3$, con $\wedge$ para la potencia:



Los tres recorridos dan tres notaciones; las diapositivas escriben "postfija" por posfija.

Recorrido	Notación	Resultado
Preorden	**prefija** (polaca)	$+\ \wedge\ +\ x\ y\ 2\ /\ -\ x\ 4\ 3$
Postorden	**posfija** (polaca inversa)	$x\ y\ +\ 2\ \wedge\ x\ 4\ -\ 3\ /\ +$
Inorden	**infija**, con paréntesis	$((x+y)\wedge 2)+((x-4)/3)$

Ojito...

El inorden sin paréntesis, $x+y\wedge 2+x-4/3$, es ambiguo: con la precedencia usual se leería $x+y^2+x-4/3$, que es otra expresión. La infija necesita paréntesis para fijar la forma del árbol. La prefija y la posfija no: cada operador sabe que lleva exactamente dos operandos, así que la cadena determina el árbol. Las diapositivas no escriben la infija.

La posfija se evalúa con una pila, de izquierda a derecha y sin mirar adelante: los operandos se apilan, y cada operador desapila sus dos operandos y apila el resultado.

Algoritmo 3.6 Evaluar-Posfija$(t_1, \ldots, t_k)$

```
1:  procedimiento Evaluar-Posfija(t_1, ..., t_k)
2:      P ← pila vacía
3:      para i ← 1 hasta k hacer
4:          si t_i es un número o una variable entonces
5:              Apilar(P, valor(t_i))
6:          si no                                   // t_i es un operador binario ∘
7:              b ← Desapilar(P);  a ← Desapilar(P)
8:              Apilar(P, a ∘ b)
9:          fin si
10:     fin para
11:     devolver Desapilar(P)
12: fin procedimiento
```

Ejemplo 3.31 (Evaluar la posfija con $x = 1$, $y = 2$)**.** Contenido de la pila después de cada símbolo, con el fondo arriba:

x	y	$+$	2	$\wedge$	x	4	$-$	3	$/$	$+$
1	1	3	3	9	9	9	9	9	9	8
	2		2		1	1	-3	-3	-1	
						4		3		

El resultado es 8, y directamente $(1+2)^2 + (1-4)/3 = 9 - 1 = 8$.

El orden de los desapilamientos importa: el primero que sale es el operando **derecho**. En el $-$ del ejemplo sale primero el 4 y luego el 1, y el resultado es $1 - 4 = -3$, no $4 - 1$; con $-$, $/$ o $\wedge$, invertirlos cambia el resultado. Hay un apilamiento por símbolo y dos desapilamientos por operador: $\Theta(k)$. Funciona porque el postorden escribe cada operador justo después de sus dos subárboles completos, así que cuando llega el operador sus dos valores son los dos de arriba de la pila.

Resumen

Problema	**Método**	**Costo**
¿Hay recorrido euleriano?	grados impares y una BFS	$\Theta(n+m)$
¿Hay ciclo hamiltoniano?	retroceso; NP-completo	$\mathcal{O}(n!)$
Agente viajero	fuerza bruta	$\Theta(n!)$
	Held y Karp	$\Theta\big(n^2 2^n\big)$
¿Es bipartito?	BFS con dos lados; da partición o ciclo impar	$\Theta(n+m)$
Clique maximal	voraz con matriz	$\mathcal{O}\big(n^2\big)$
Clique máxima	NP-difícil; subconjuntos	$\mathcal{O}\big(2^n n^2\big)$
Recorrer un árbol	preorden, inorden, postorden	$\Theta(n)$
Evaluar una posfija de k símbolos	pila	$\Theta(k)$

4. Representaciones, BFS y DFS a fondo *Complemento: Cormen, cap. 22.1 a 22.3*

Resumen rápido. La clase del 22 de octubre presentó las dos representaciones y los dos recorridos con un ejemplo cada uno. Esta sección los retoma con el rigor de Cormen. Primero, el costo de cada operación con listas y con matriz, y seis algoritmos chicos que manipulan representaciones: grados, transpuesto, cuadrado, sumidero universal en $\mathcal{O}(n)$, matriz de incidencia y simplificación de un multigrafo. Después, BFS con colores: por qué su costo es $\mathcal{O}(n+m)$ y no $\Theta(n+m)$, la prueba de que calcula distancias, el árbol BFS y el diámetro de un árbol. Al final, DFS con colores y tiempos: el bosque DFS, los teoremas del paréntesis y del camino blanco, los cuatro tipos de arista, la versión iterativa con sus dos trampas y las componentes conexas. Lo que queda es la maquinaria de pruebas sobre la que se apoyan las aplicaciones de DFS y los caminos más cortos de las secciones que siguen.

La notación de atributos es la de la sección 1, que también da su equivalencia con la de Cormen: $d[v]$, $\pi[v]$, color$[v]$, y entra$[v]$ y sale$[v]$ para los tiempos de DFS.

Listas y matriz: el costo de cada operación

La clase del 22 de octubre definió las dos representaciones y comparó cuatro operaciones. Aquí se fija la notación y se completa la cuenta. Las **listas de adyacencia** son un arreglo Adj de n listas, una por vértice: $Adj[u]$ contiene los v con $(u,v) \in E$ en un dirigido, o con $\{u,v\} \in E$ en uno no dirigido, en un orden arbitrario. El orden no significa nada para el grafo, pero sí decide qué vecino descubren primero BFS y DFS; por eso cada ejemplo de esta sección dice qué orden usa.

Lema 4.1 (Longitud total de las listas)**.** *En un grafo dirigido $\sum_{u \in V} |Adj[u]| = m$. En uno no dirigido la suma es $2m$.*

Demostración. En un dirigido la arista (u,v) aparece una sola vez, como v en $Adj[u]$. En uno no dirigido $\{u,v\}$ aparece dos veces: como v en $Adj[u]$ y como u en $Adj[v]$. □

Es la misma cuenta que $\sum_v \mathrm{gr}(v) = 2m$ de la clase. En un dirigido se lee así: la suma de los grados de salida es m, y la de los grados de entrada también. Escribimos $\mathrm{gr}^+(u)$ para el **grado de salida** (el número de aristas (u,v)) y $\mathrm{gr}^-(u)$ para el **grado de entrada** (el de aristas (v,u)). De este lema sale el $\Theta(n+m)$ de memoria de las listas: n cabezas más m o $2m$ nodos. El término n no se puede quitar, porque un grafo con vértices aislados sigue pagando su arreglo de cabezas.

Operación	**Listas**	**Matriz**
Memoria	$\Theta(n+m)$	$\Theta(n^2)$
¿Es (u,v) arista?	$\mathcal{O}(\mathrm{gr}^+(u))$, peor caso $\mathcal{O}(n)$	$\Theta(1)$
Recorrer los vecinos de u	$\Theta(1+\mathrm{gr}^+(u))$	$\Theta(n)$
Recorrer todas las aristas	$\Theta(n+m)$	$\Theta(n^2)$
Grado de salida de u	$\Theta(\mathrm{gr}^+(u))$; $\Theta(1)$ si la lista guarda su longitud	$\Theta(n)$
Grados de entrada de todos	$\Theta(n+m)$	$\Theta(n^2)$
Agregar una arista	$\mathcal{O}(1)$, al frente y sin revisar duplicados	$\Theta(1)$
Borrar la arista (u,v)	$\mathcal{O}(\mathrm{gr}^+(u))$; en no dirigido, también $\mathcal{O}(\mathrm{gr}(v))$	$\Theta(1)$
Grafo transpuesto	$\Theta(n+m)$	$\Theta(n^2)$
Cuadrado G^2	$\mathcal{O}(nm)$	$\mathcal{O}(n^3)$, u $\mathcal{O}(n^{\lg 7})$ con Strassen

En un no dirigido, $\mathrm{gr}^+(u)$ es simplemente $\mathrm{gr}(u)$. Las cuatro filas de la clase siguen ahí; las demás las justifican los algoritmos de esta subsección.

Grafos pesados y atributos. En listas, el peso $w(u, v)$ viaja en el nodo de v dentro de $Adj[u]$; para el grafo pesado de la clase, $Adj[A] = (B, 4), (C, 2), (D, 7)$. En la matriz va en la casilla (u, v), y en las casillas sin arista se guarda lo que pida el problema: ∞ para caminos más cortos, 0 para capacidades, nulo si hay que distinguir. El 0 en la diagonal de la matriz pesada de la clase es la convención de Floyd, no parte de la representación. Los atributos de vértice, como $d[v]$ o color$[v]$, viven en arreglos indexados por vértice, con acceso $\mathcal{O}(1)$; los de arista, como un flujo $f(u, v)$, en el nodo de la lista o en una segunda matriz.

Ojito...

- $\Theta(n + m)$ no es $\Theta(m)$. Las dos coinciden cuando el grafo es no dirigido y conexo, porque entonces $m \geq n - 1$, pero no en general.
- Cada arista no dirigida vive dos veces en las listas. Borrarla o cambiarle el peso obliga a tocar $Adj[u]$ y $Adj[v]$; olvidar una copia deja el grafo inconsistente.
- La matriz cobra $\Theta(n^2)$ desde que se crea, porque inicializarla ya cuesta eso. Un algoritmo que promete $\mathcal{O}(n + m)$ no puede construir una matriz por dentro, ni siquiera para consultar aristas rápido.
- "La suma de cada renglón es el grado" y "la diagonal es cero" valen en grafos no dirigidos sin lazos. En un dirigido el renglón u suma $\text{gr}^+(u)$ y la columna u suma $\text{gr}^-(u)$; un lazo (u, u) pone un 1 en la diagonal y suma 1 a cada uno de los dos grados.
- Consultar una arista con listas no es $\mathcal{O}(1)$. En un no dirigido conviene buscar en la lista más corta: $\mathcal{O}(\text{mín}(\text{gr}(u), \text{gr}(v)))$.

Grados de entrada y de salida

Algoritmo 4.1 Grados(G)

```
1:  procedimiento Grados(G)
2:      para todo u ∈ V hacer
3:          gr⁺[u] ← 0,    gr⁻[u] ← 0
4:      fin para
5:      para todo u ∈ V hacer
6:          para todo v ∈ Adj[u] hacer
7:              gr⁺[u] ← gr⁺[u] + 1
8:              gr⁻[v] ← gr⁻[v] + 1
9:          fin para
10:     fin para
11:     devolver gr⁺, gr⁻
12: fin procedimiento
```

Las líneas 2 a 4 cuestan $\Theta(n)$. El ciclo de la línea 5 da n vueltas, cada una de costo $\Theta(1)$ más la longitud de $Adj[u]$, y por el lema 4.1 las vueltas internas suman m: total $\Theta(n + m)$. Si solo interesan los grados de salida y cada lista guarda su longitud, basta leerla, en $\Theta(n)$. Los de entrada no admiten atajo: cualquier entrada de cualquier lista puede sumarle a cualquier vértice, así que hay que verlas todas. Con matriz, $\text{gr}^+(u)$ es la suma del renglón u y $\text{gr}^-(u)$ la de la columna u: $\Theta(n)$ cada uno y $\Theta(n^2)$ para todos.

Grafo transpuesto

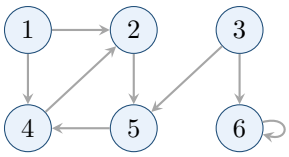
Definición 4.2 (Grafo transpuesto)**.** El **transpuesto** de un grafo dirigido $G = (V, E)$ es $G^{\mathsf{T}} = (V, E^{\mathsf{T}})$ con $E^{\mathsf{T}} = \{(v, u) : (u, v) \in E\}$: el mismo grafo con todas las aristas invertidas. Su matriz es A^{T}.

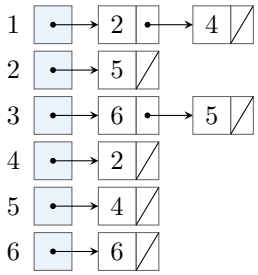
Algoritmo 4.2 Transponer-Listas(G)

```
1:  procedimiento Transponer-Listas(G)
2:      para todo u ∈ V hacer
3:          Adj^T[u] ← lista vacía
4:      fin para
5:      para todo u ∈ V, en orden creciente hacer
6:          para todo v ∈ Adj[u] hacer
7:              agregar u al final de Adj^T[v]          // (u, v) se vuelve (v, u)
8:          fin para
9:      fin para
10:     devolver Adj^T
11: fin procedimiento
```

Cada inserción cuesta $\mathcal{O}(1)$ si la lista guarda un apuntador a su último nodo, y hay m de ellas: $\Theta(n + m)$, que es también el tamaño de la salida. Hay un efecto secundario útil: como u se recorre en orden creciente y se agrega al final, cada $Adj^{\mathsf{T}}[v]$ sale ordenada. Transponer dos veces deja ordenadas las listas de G en $\Theta(n + m)$, sin una sola comparación; es un ordenamiento por cubetas disfrazado. Con matriz, transponer es intercambiar $A[i][j]$ con $A[j][i]$ arriba de la diagonal, $(n^2 - n)/2$ intercambios en su lugar, $\Theta(n^2)$. El transpuesto reaparece en las componentes fuertemente conexas de la sección 5, que corren DFS sobre G y luego sobre G^{T} y necesitan transponer en tiempo lineal.

Ejemplo 4.3 (Un grafo dirigido, sus grados y su transpuesto)**.** El grafo de la figura 22.2 de Cormen tiene seis vértices y ocho aristas: $(1, 2)$, $(1, 4)$, $(2, 5)$, $(3, 5)$, $(3, 6)$, $(4, 2)$, $(5, 4)$ y el lazo $(6, 6)$. A la izquierda el dibujo, al centro el arreglo Adj con sus listas ligadas y a la derecha la matriz.





	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

Cada arista aparece una sola vez en las listas, que suman $8 = m$. La matriz no es simétrica ($a_{12} = 1$, $a_{21} = 0$) y el lazo pone un 1 en a_{66}. Grados y Transponer-Listas recorren las listas en el mismo orden y se pueden correr en una sola pasada. Los grados de salida son $2, 1, 2, 1, 1, 1$, las longitudes de las listas o las sumas de los renglones, y los de entrada $0, 2, 0, 2, 2, 2$, las sumas de las columnas; las dos listas suman 8. Los vértices 1 y 3 no reciben ninguna arista, así que en G^{T} sus listas quedan vacías. Las demás salen $Adj^{\mathsf{T}}[2] = 1, 4$, $Adj^{\mathsf{T}}[4] = 1, 5$, $Adj^{\mathsf{T}}[5] = 2, 3$ y $Adj^{\mathsf{T}}[6] = 3, 6$, todas en orden creciente.

Cuadrado de un grafo

Definición 4.4 (Cuadrado de un grafo)**.** El **cuadrado** de un grafo dirigido $G = (V, E)$ es $G^2 = (V, E^2)$, donde $(u, v) \in E^2$ si y solo si G tiene una caminata de una o de dos aristas de u a v.

Cormen dice "un camino de a lo más dos aristas". Leído al pie de la letra eso incluye el camino de cero aristas de u a u, y entonces todo vértice tendría lazo en G^2. La lectura usual, y la de los algoritmos de abajo, es la de la definición: $(u, u) \in E^2$ solo si u tiene lazo o hay una ida y vuelta $u \to w \to u$.

Algoritmo 4.3 Cuadrado-Listas(G)

```
1:  procedimiento Cuadrado-Listas(G)
2:      para todo u ∈ V hacer
3:          Adj²[u] ← lista vacía,    marca[u] ← NULO
4:      fin para
5:      para todo u ∈ V hacer
6:          para todo w ∈ Adj[u] hacer                    // caminatas de una arista
7:              si marca[w] ≠ u entonces
8:                  marca[w] ← u;  agregar w a Adj²[u]
9:              fin si
10:             para todo v ∈ Adj[w] hacer                // de dos aristas
11:                 si marca[v] ≠ u entonces
12:                     marca[v] ← u;  agregar v a Adj²[u]
13:                 fin si
14:             fin para
15:         fin para
16:     fin para
17:     devolver Adj²
18: fin procedimiento
```

marca$[x] = u$ significa "x ya está en $Adj^2[u]$". Sin la marca, v entraría una vez por cada caminata $u \to w \to v$ y otra más si además $(u, v) \in E$. La marca no se reinicializa entre un u y el siguiente: una marca puesta por un vértice anterior nunca coincide con el u actual. Reinicializarla costaría $\Theta(n)$ por vértice, $\Theta(n^2)$ en total, y destruiría la cota.

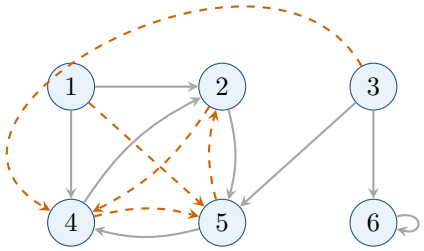
Costo. El ciclo de la línea 6 da m vueltas en total. Dentro de la vuelta (u, w), el de la línea 10 da $\mathrm{gr}^+(w)$ vueltas, y cada w aparece en $\mathrm{gr}^-(w)$ listas, así que el trabajo interno es

$$\sum_{u \in V} \sum_{w \in Adj[u]} \mathrm{gr}^+(w) = \sum_{w \in V} \mathrm{gr}^-(w)\,\mathrm{gr}^+(w) \le n \sum_{w \in V} \mathrm{gr}^-(w) = nm.$$

El total es $\mathcal{O}(n + m + nm) = \mathcal{O}(nm)$ para $m \ge 1$. La cota se alcanza: con k vértices que apuntan a un centro y el centro apuntando a otros k, se tiene $n = 2k + 1$ y $m = 2k$, pero G^2 tiene $k^2 + 2k$ aristas. La salida sola ya mide $\Theta(n^2)$ con $m = \Theta(n)$, así que no existe un algoritmo lineal para G^2.

Con matriz, la entrada (i, j) del producto entero $A \cdot A$ es $\sum_k a_{ik} a_{kj}$, y cada sumando vale 1 exactamente cuando $i \to k \to j$ es una caminata. Por inducción, $(A^k)_{ij}$ cuenta las caminatas de k aristas de i a j. Entonces E^2 son las entradas no nulas de $A + A^2$: con tres ciclos anidados cuesta $\Theta(n^3)$, y con el producto de Strassen $\mathcal{O}\big(n^{\lg 7}\big)$, con $\lg 7 \approx 2.81$, más $\Theta(n^2)$ para llevar a 1 lo no nulo. Es la semilla de los caminos más cortos por multiplicación de matrices de la sección 9.

Ejemplo 4.5 (El cuadrado del grafo dirigido)**.** El grafo es el del ejemplo 4.3. Para $u = 1$, el algoritmo recorre $w = 2$ (agrega 2) y su vecino 5 (agrega 5), luego $w = 4$ (agrega 4) y su vecino 2, que ya tiene la marca de 1 y no se repite. Así con cada u; las aristas nuevas salen punteadas.



G^2: punteadas las cinco aristas nuevas

$|E^2| = 13$: las ocho de G más $(1,5)$, $(2,4)$, $(3,4)$, $(4,5)$ y $(5,2)$. El ciclo $2 \to 5 \to 4 \to 2$ produce en G^2 el mismo ciclo recorrido al revés, porque saltar dos pasos hacia adelante en un ciclo de tres es dar uno hacia atrás.

Sumidero universal en $\mathcal{O}(n)$

Definición 4.6 (Sumidero universal)**.** En un grafo dirigido, un **sumidero universal** es un vértice con $\text{gr}^- = n - 1$ y $\text{gr}^+ = 0$: todos los demás le apuntan y él no apunta a nadie, ni a sí mismo.

Con la matriz, casi cualquier pregunta sobre un grafo cuesta $\Omega(n^2)$. Decidir si hay al menos una arista, por ejemplo, exige leer todas las casillas en el peor caso: un adversario contesta 0 a cada lectura y, si el algoritmo se detiene antes, pone un 1 en una casilla no leída. El sumidero universal es la excepción que vale la pena conocer, y escapa al argumento por este lema.

Lema 4.7. *Sean $k \neq j$. Si $a_{kj} = 1$, k no es sumidero universal, porque tiene una arista de salida. Si $a_{kj} = 0$, j no lo es, porque le falta la arista desde k.*

Cada lectura fuera de la diagonal descarta a uno de los dos vértices, no solo una casilla. Con $n - 1$ lecturas sobrevive un solo candidato.

Algoritmo 4.4 Sumidero-Universal(A, n)

```
1:  procedimiento Sumidero-Universal(A, n)
2:      k ← 1                                              // candidato
3:      para j ← 2 hasta n hacer
4:          si A[k][j] = 1 entonces                        // k tiene una arista de salida
5:              k ← j
6:          fin si
7:      fin para
8:      para j ← 1 hasta n hacer                           // verificar al candidato
9:          si A[k][j] = 1 o (j ≠ k y A[j][k] = 0) entonces
10:             devolver NULO
11:         fin si
12:     fin para
13:     devolver k
14: fin procedimiento
```

Que el algoritmo devuelva un solo vértice no deja nada fuera:

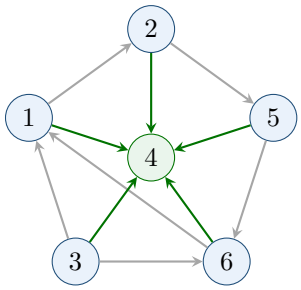
Lema 4.8. *Un grafo dirigido tiene a lo más un sumidero universal.*

Demostración. Si $k \neq k'$ fueran sumideros, k' recibiría arista de todos los demás, en particular $(k, k') \in E$, y eso contradice $\mathrm{gr}^+(k) = 0$. □

Corrección. Invariante del ciclo de la línea 3: al empezar la vuelta j, ningún vértice de $\{1, \ldots, j-1\}$ distinto de k es sumidero universal. Al inicio el conjunto es $\{1\} = \{k\}$. En la vuelta j se lee a_{kj} con $k < j$: si vale 1, k queda descartado y j es el nuevo candidato; si vale 0, j queda descartado (lema 4.7). Al terminar, el único sumidero posible es k, pero puede no serlo: la primera fase siempre entrega un candidato, haya sumidero o no. Las líneas 8 a 12 lo verifican: el renglón k completo en ceros, incluida la diagonal, que descarta un lazo, y la columna k en unos fuera de la diagonal.

Costo. La primera fase lee $n-1$ casillas; la segunda, el renglón (n) y la columna sin la diagonal $(n-1)$. A lo más $3n-2$ lecturas: $\mathcal{O}(n)$, sobre una entrada de tamaño n^2.

Ejemplo 4.9 (Sumidero universal, con y sin sumidero)**.** Seis vértices; todos tienen una arista hacia 4, y además $(1,2)$, $(2,5)$, $(3,1)$, $(3,6)$, $(5,6)$ y $(6,1)$. El vértice 4 tiene $\mathrm{gr}^- = 5$ y $\mathrm{gr}^+ = 0$. En la matriz del centro, en amarillo, las casillas que lee la primera fase; a la derecha, en verde, el renglón y la columna que revisa la segunda.



	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	1	1	0
3	1	0	0	1	0	1
4	0	0	0	0	0	0
5	0	0	0	1	0	1
6	1	0	0	1	0	0

fase 1: eliminación

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	1	1	0
3	1	0	0	1	0	1
4	0	0	0	0	0	0
5	0	0	0	1	0	1
6	1	0	0	1	0	0

fase 2: verificación de $k = 4$

La primera fase lee una casilla por columna, siempre arriba de la diagonal: un 0 avanza a la derecha y un 1 baja al renglón de esa columna.

j	k antes	a_{kj}	k después	**Qué se descarta**
2	1	1	2	1, porque tiene una arista de salida
3	2	0	2	3, porque no recibe arista de 2
4	2	1	4	2, porque tiene una arista de salida
5	4	0	4	5, porque no recibe arista de 4
6	4	0	4	6, porque no recibe arista de 4

Queda $k = 4$. La segunda fase lee su renglón, seis ceros, y su columna fuera de la diagonal, cinco unos: 4 es el sumidero. Se leyeron $5 + 6 + 5 = 16 = 3 \cdot 6 - 2$ de las 36 casillas.

Si se agrega la arista $(4, 6)$, ya no hay sumidero. La eliminación es la misma hasta $j = 6$, donde $a_{46} = 1$ deja como candidato a 6, y la verificación lo tumba en la primera lectura, porque $a_{61} = 1$. Sin la segunda fase, el algoritmo devolvería 6, que tiene $\mathrm{gr}^- = 3$ y $\mathrm{gr}^+ = 2$.

Matriz de incidencia y BB^T

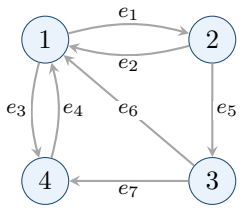
Definición 4.10 (Matriz de incidencia)**.** Para un grafo dirigido sin lazos con las aristas numeradas $e_1, \ldots, e_m$, la **matriz de incidencia** es la matriz B de $n \times m$ con $b_{ij} = -1$ si e_j sale del vértice i, $b_{ij} = 1$ si e_j entra al vértice i, y $b_{ij} = 0$ si no. Los renglones son vértices y las columnas aristas.

Proposición 4.11 (Cormen, ejercicio 22.1-7)**.** *Para un grafo dirigido sin lazos, $(BB^\mathsf{T})_{ii} = gr^-(i) + gr^+(i)$, y para $i \neq j$, $(BB^\mathsf{T})_{ij}$ es menos el número de aristas entre i y j, en cualquier sentido.*

Demostración. $(BB^{\mathsf{T}})_{ij} = \sum_e b_{ie}b_{je}$, el producto punto de los renglones i y j. Si $i = j$, cada sumando es b_{ie}^2, que vale 1 si e toca a i y 0 si no: la suma cuenta las aristas que entran o salen de i. Si $i \neq j$, el producto $b_{ie}b_{je}$ es distinto de cero solo cuando e une a i con j; en ese caso uno de los dos vale -1 y el otro 1, y el producto es -1. □

Dicho de otro modo, $BB^{\mathsf{T}} = D - (A + A^{\mathsf{T}})$, con D la diagonal de grados totales: es la **matriz laplaciana** del grafo no dirigido subyacente, contando multiplicidades. Cada renglón suma 0, porque cada columna de B tiene un -1 y un 1. La laplaciana es la base del teorema de Kirchhoff para contar árboles de expansión; aquí basta reconocerla. La definición no admite lazos: la columna de (u, u) tendría que valer -1 y 1 en la misma casilla.

Ejemplo 4.12 (Incidencia del grafo dirigido de la clase)**.** Es el grafo dirigido de la clase del 22 de octubre, el de Floyd, con sus siete aristas numeradas. Sus dos pares de aristas antiparalelas producen entradas -2.



$$B = \begin{array}{c|ccccccc} & e_1 & e_2 & e_3 & e_4 & e_5 & e_6 & e_7 \\ \hline 1 & -1 & 1 & -1 & 1 & 0 & 1 & 0 \\ 2 & 1 & -1 & 0 & 0 & -1 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 1 & -1 & -1 \\ 4 & 0 & 0 & 1 & -1 & 0 & 0 & 1 \end{array} \qquad BB^{\mathsf{T}} = \begin{pmatrix} 5 & -2 & -1 & -2 \\ -2 & 3 & -1 & 0 \\ -1 & -1 & 3 & -1 \\ -2 & 0 & -1 & 3 \end{pmatrix}$$

El 5 de la esquina son las cinco aristas que tocan a 1: $\mathrm{gr}^-(1) = 3$ más $\mathrm{gr}^+(1) = 2$. Hay -2 en $(1, 2)$ porque entre 1 y 2 hay dos aristas, una en cada sentido; -1 en $(2, 3)$ por la única arista e_5; 0 en $(2, 4)$ porque no hay ninguna.

De multigrafo a grafo simple

Un multigrafo no dirigido admite aristas repetidas y lazos. Su grafo simple equivalente $G' = (V, E')$ deja una sola arista por cada grupo de repetidas y quita los lazos (Cormen, ejercicio 22.1-4). El truco es el mismo de la marca del cuadrado.

Algoritmo 4.5 Simplificar-Multigrafo(G)

```
1:  procedimiento Simplificar-Multigrafo(G)
2:      para todo u ∈ V hacer
3:          Adj'[u] ← lista vacía,    último[u] ← nulo
4:      fin para
5:      para todo u ∈ V hacer
6:          para todo v ∈ Adj[u] hacer
7:              si v ≠ u y último[v] ≠ u entonces
8:                  último[v] ← u
9:                  agregar v a Adj'[u]
10:             fin si
11:         fin para
12:     fin para
13:     devolver Adj'
14: fin procedimiento
```

En la línea 7, $v \neq u$ quita los lazos y último$[v] \neq u$ quita las repetidas: último$[v] = u$ quiere decir que v ya entró a $Adj'[u]$. La salida es simétrica sin hacer nada especial, porque si v entra a $Adj'[u]$,

al procesar v se encontrará a u en $Adj[v]$. Por ejemplo, con dos aristas entre 1 y 2, un lazo en 1 y $Adj[1] = 2, 1, 2, 3$, el primer 2 entra y deja último[2] = 1, el lazo se omite, el segundo 2 se descarta y el 3 entra con último[3] = 1. Al pasar a $u = 2$ con $Adj[2] = 1, 3, 1$, esa marca es vieja, distinta de 2, y el 3 entra sin haber reinicializado nada. El arreglo se inicializa una sola vez y el doble ciclo hace una vuelta $\mathcal{O}(1)$ por entrada de las listas: $\Theta(n + m)$.

Búsqueda en anchura a fondo

La clase del 22 de octubre (sección 1) dio BFS con un arreglo booleano visto y lo corrió sobre G desde A. Aquí se usa la versión de Cormen, que pinta cada vértice de uno de tres colores. El resultado es el mismo; los colores sirven para las pruebas.

Definición 4.13 (Colores de BFS)**.** Un vértice es **blanco** mientras no se ha descubierto, **gris** desde que se descubre hasta que se termina de recorrer su lista, y **negro** después. Los grises forman la **frontera** entre lo descubierto y lo que falta.

Algoritmo 4.6 BFS(G, s) con colores (Cormen, sección 22.2)

```
1:  procedimiento BFS(G, s)
2:      para todo u ∈ V − {s} hacer
3:          color[u] ← BLANCO,    d[u] ← ∞,    π[u] ← NULO
4:      fin para
5:      color[s] ← GRIS,    d[s] ← 0,    π[s] ← NULO
6:      Q ← cola vacía;  ENCOLAR(Q, s)
7:      mientras Q no está vacía hacer
8:          u ← DESENCOLAR(Q)
9:          para todo v ∈ Adj[u] hacer
10:             si color[v] = BLANCO entonces                    // se descubre v
11:                 color[v] ← GRIS,    d[v] ← d[u] + 1,    π[v] ← u
12:                 ENCOLAR(Q, v)
13:             fin si
14:         fin para
15:         color[u] ← NEGRO                                     // lista de u agotada
16:     fin mientras
17: fin procedimiento
```

Tres líneas merecen comentario. La línea 10 solo pregunta si v es blanco: gris y negro se tratan igual, así que un bit por vértice basta, y ese bit es el visto de la clase (Cormen, ejercicio 22.2-3). La línea 11 pinta de gris y fija d y π en el instante en que v se descubre, no cuando sale de la cola; eso es lo que garantiza que cada vértice entre a Q una sola vez. La línea 15 no cambia el resultado: existe para el invariante siguiente, que da sentido a la palabra frontera.

Observación. Al principio de cada vuelta del ciclo de la línea 7, Q es exactamente el conjunto de vértices grises. Al inicio, s es el único gris y el único en Q; cada vez que un vértice se pinta de gris (línea 11) también se encola (línea 12), y el que se desencola en la línea 8 se pinta de negro en la 15.

Costo: $\mathcal{O}(n + m)$, y por qué no $\Theta(n + m)$

El análisis es agregado: en lugar de acotar cada vuelta del ciclo, se cuenta cuántas veces puede pasar cada cosa en toda la ejecución.

1. *Inicialización* (líneas 2 a 6): $\Theta(n)$.
2. *Operaciones de cola.* Un vértice entra a Q solo en la línea 12, solo si era blanco en la 10, y la 11 lo pinta de gris en ese mismo momento; ninguna línea lo regresa a blanco. Así que cada vértice se encola a lo más una vez, contando s, y como cada Desencolar saca algo que entró, también se desencola a lo más una vez. Con una cola de arreglo circular o de lista ligada, cada operación es $\mathcal{O}(1)$: $\mathcal{O}(n)$ en total.
3. *Recorrido de listas.* El ciclo de la línea 9 corre para u solo cuando u sale de la cola, a lo más una vez por vértice, y da $|Adj[u]|$ vueltas de costo $\mathcal{O}(1)$. Por el lema 4.1, todas las listas suman m o $2m$: $\mathcal{O}(m)$.
4. *Suma:* $\Theta(n) + \mathcal{O}(n) + \mathcal{O}(m) = \mathcal{O}(n+m)$, lineal en el tamaño de la representación.

Por eso la clase escribe $\mathcal{O}(n+m)$ y no $\Theta(n+m)$: el segundo es el costo del peor caso, que se alcanza cuando todo vértice es alcanzable desde s, pero no describe cada ejecución. Si s no tiene aristas de salida, BFS inicializa los n vértices, saca a s, mira una lista vacía y termina en $\Theta(n)$, aunque m sea del orden de n^2. El costo exacto de una ejecución es

$$\Theta\left(n + \sum_{u \text{ alcanzable desde } s} |Adj[u]|\right),$$

porque las listas de los vértices inalcanzables nunca se leen. Por eso Cormen escribe $O(V+E)$: una cota superior que vale para toda entrada. Afirmar $\Theta(n+m)$ sin más promete también una cota inferior, y esa falla en cuanto hay vértices inalcanzables con listas largas. En DFS no pasa, como se verá: su ciclo externo obliga a leer todas las listas.

Cada término depende de una estructura. El $\mathcal{O}(n)$ de la cola supone operaciones en $\mathcal{O}(1)$; con una lista de Python y `pop(0)`, que cuesta lo que mide la lista, BFS sube a $\mathcal{O}(n^2+m)$. El $\mathcal{O}(m)$ supone listas: con matriz, cada vértice que sale de la cola lee su renglón completo, n casillas, y el total es n por el número de alcanzables, $\mathcal{O}(n^2)$. Y si la cola se cambia por un montículo con llave d, sale Dijkstra con pesos unitarios, $\mathcal{O}((n+m)\log n)$: más caro y sin ganancia, porque, como se prueba abajo, la cola ya entrega los vértices en orden de d.

Distancias: por qué BFS las calcula

Definición 4.14 (Distancia de camino más corto)**.** $\delta(s,v)$ es el mínimo número de aristas de un camino de s a v, e ∞ si no hay ninguno. Un **camino más corto** de s a v es uno de longitud $\delta(s,v)$.

Aquí todas las aristas pesan 1; la sección 7 generaliza a pesos reales. La caja de idea de la clase afirma que BFS deja $d[v] = \delta(s,v)$. La prueba de Cormen tiene tres pasos: d nunca subestima, la cola está ordenada por d, y un vértice con la d equivocada lleva a una contradicción.

Lema 4.15 (Cormen, lema 22.1)**.** *Sea G dirigido o no y $s \in V$. Para toda arista $(u,v) \in E$, $\delta(s,v) \leq \delta(s,u) + 1$.*

Demostración. Si u es alcanzable, un camino más corto a u seguido de la arista (u,v) llega a v con $\delta(s,u)+1$ aristas, y el más corto a v no puede ser más largo. Si u no es alcanzable, el lado derecho es ∞. □

En un dirigido la desigualdad va en un solo sentido: una arista (f,s) puede ir de un vértice lejano a la fuente sin que eso acote nada en sentido contrario. En uno no dirigido cada arista sirve en los dos sentidos, y queda $|\delta(s,u) - \delta(s,v)| \leq 1$ para toda arista $\{u,v\}$.

Lema 4.16 (Cormen, lema 22.2)**.** *Al terminar BFS desde s, $d[v] \geq \delta(s, v)$ para todo $v \in V$.*

Demostración. Inducción sobre el número de operaciones ENCOLAR, con la hipótesis "$d[v] \geq \delta(s, v)$ para todo v". Tras encolar a s, $d[s] = 0 = \delta(s, s)$ y los demás valen ∞. Si un v blanco se descubre desde u, entonces $d[v] = d[u] + 1 \geq \delta(s, u) + 1 \geq \delta(s, v)$, por la hipótesis y el lema 4.15. Como v queda gris, la línea 11 no vuelve a tocarlo y su d ya no cambia. □

Lema 4.17 (Cormen, lema 22.3)**.** *Si en algún momento $Q = \langle v_1, \ldots, v_r \rangle$, con v_1 a la cabeza, entonces $d[v_r] \leq d[v_1] + 1$ y $d[v_i] \leq d[v_{i+1}]$ para $i = 1, \ldots, r - 1$.*

Idea de la prueba. Inducción sobre las operaciones de cola. Al desencolar v_1, la nueva cabeza es v_2 y $d[v_r] \leq d[v_1] + 1 \leq d[v_2] + 1$. Al encolar v desde u, u acaba de salir como cabeza, así que $d[u] \leq d[v_1]$ para la cabeza actual v_1, y $d[v] = d[u] + 1 \leq d[v_1] + 1$; además $d[v_r] \leq d[u] + 1 = d[v]$. Si la cola quedó vacía al sacar u, v queda solo y no hay nada que probar. □

La consecuencia que conviene recordar: la cola contiene a lo más dos valores distintos de d, consecutivos y en orden no decreciente. La columna "d en Q" de la traza del ejemplo 4.22 lo muestra paso a paso.

Corolario 4.18 (Cormen, corolario 22.4)**.** *Si v_i se encola antes que v_j, entonces $d[v_i] \leq d[v_j]$ en el momento en que v_j se encola.*

Es inmediato del lema 4.17 y de que cada vértice recibe un valor finito de d una sola vez.

Teorema 4.19 (Corrección de BFS; Cormen, teorema 22.5)**.** *Sea G dirigido o no y supóngase que BFS corre desde s. Entonces descubre todo vértice alcanzable desde s y, al terminar, $d[v] = \delta(s, v)$ para todo $v \in V$. Además, para todo $v \neq s$ alcanzable, un camino más corto de s a $\pi[v]$ seguido de la arista $(\pi[v], v)$ es un camino más corto de s a v.*

Demostración. Por contradicción. Supóngase que algún vértice termina con $d[v] \neq \delta(s, v)$, y de todos ellos tómese uno con $\delta(s, v)$ mínima. No es s, porque $d[s] = 0 = \delta(s, s)$. Por el lema 4.16, $d[v] > \delta(s, v)$, así que $\delta(s, v)$ es finita: v es alcanzable. Sea u el vértice anterior a v en un camino más corto de s a v; entonces $\delta(s, v) = \delta(s, u) + 1$, y como $\delta(s, u)$ es menor que $\delta(s, v)$, la elección de v obliga a que $d[u] = \delta(s, u)$. Juntando,

$$d[v] > \delta(s, v) = \delta(s, u) + 1 = d[u] + 1. \qquad (*)$$

Como $d[u]$ es finita, u pasó por la cola. Considérese el instante en que u sale de ella. En ese momento v es blanco, gris o negro.

- Blanco: la línea 11 fija $d[v] = d[u] + 1$.
- Negro: v ya salió de la cola antes que u, así que entró antes, y por el corolario 4.18, $d[v] \leq d[u]$.
- Gris: v se descubrió al sacar algún w que salió antes que u, así que $d[v] = d[w] + 1$ con $d[w] \leq d[u]$ por el mismo corolario, y $d[v] \leq d[u] + 1$.

Los tres casos contradicen $(*)$. Entonces $d[v] = \delta(s, v)$ para todo v. Un alcanzable no descubierto tendría $d[v] = \infty > \delta(s, v)$, así que todos se descubren. Por último, si $\pi[v] = u$, la línea 11 dio $d[v] = d[u] + 1$, es decir, $\delta(s, v) = \delta(s, u) + 1$, y un camino más corto a u más la arista (u, v) mide exactamente eso. □

El caso gris es el que obliga a la prueba a separar colores, y ocurre de verdad: en la figura 22.3, cuando t revisa su lista encuentra a x gris, ya descubierto por w.

El árbol BFS y la impresión de caminos

Definición 4.20 (Subgrafo predecesor de BFS)**.** El **subgrafo predecesor** de BFS desde s es $G_\pi = (V_\pi, E_\pi)$ con $V_\pi = \{v : \pi[v] \neq \text{nulo}\} \cup \{s\}$ y $E_\pi = \{(\pi[v], v) : v \in V_\pi - \{s\}\}$. Es un **árbol BFS** si V_π son exactamente los vértices alcanzables desde s y, para cada $v \in V_\pi$, el único camino de s a v en G_π es un camino más corto en G.

Lema 4.21 (Cormen, lema 22.6)**.** *Aplicado a un grafo dirigido o no, BFS construye π de modo que G_π es un árbol BFS.*

Idea de la prueba. $\pi[v]$ recibe un valor distinto de nulo exactamente cuando $v \neq s$ es alcanzable, y entonces $(\pi[v], v) \in E$; por eso V_π son los alcanzables. G_π es conexo, porque de cada vértice se sube por π hasta s, y tiene $|E_\pi| = |V_\pi| - 1$, así que es un árbol. A lo largo del único camino de s a v cada arista sube d en exactamente 1, y por el teorema 4.19 el camino mide $d[v] = \delta(s, v)$. □

Cormen escribe en esa prueba que $\pi[v] = u$ si y solo si $(u, v) \in E$ y $\delta(s, v) < \infty$. Leída al pie de la letra es falsa: en la figura 22.3, $(x, u) \in E$ y u es alcanzable, pero $\pi[u] = t$. Lo que quiere decir es lo que dice la idea de arriba. Para leer el camino del árbol se sube por π y se imprime al regresar de la recursión:

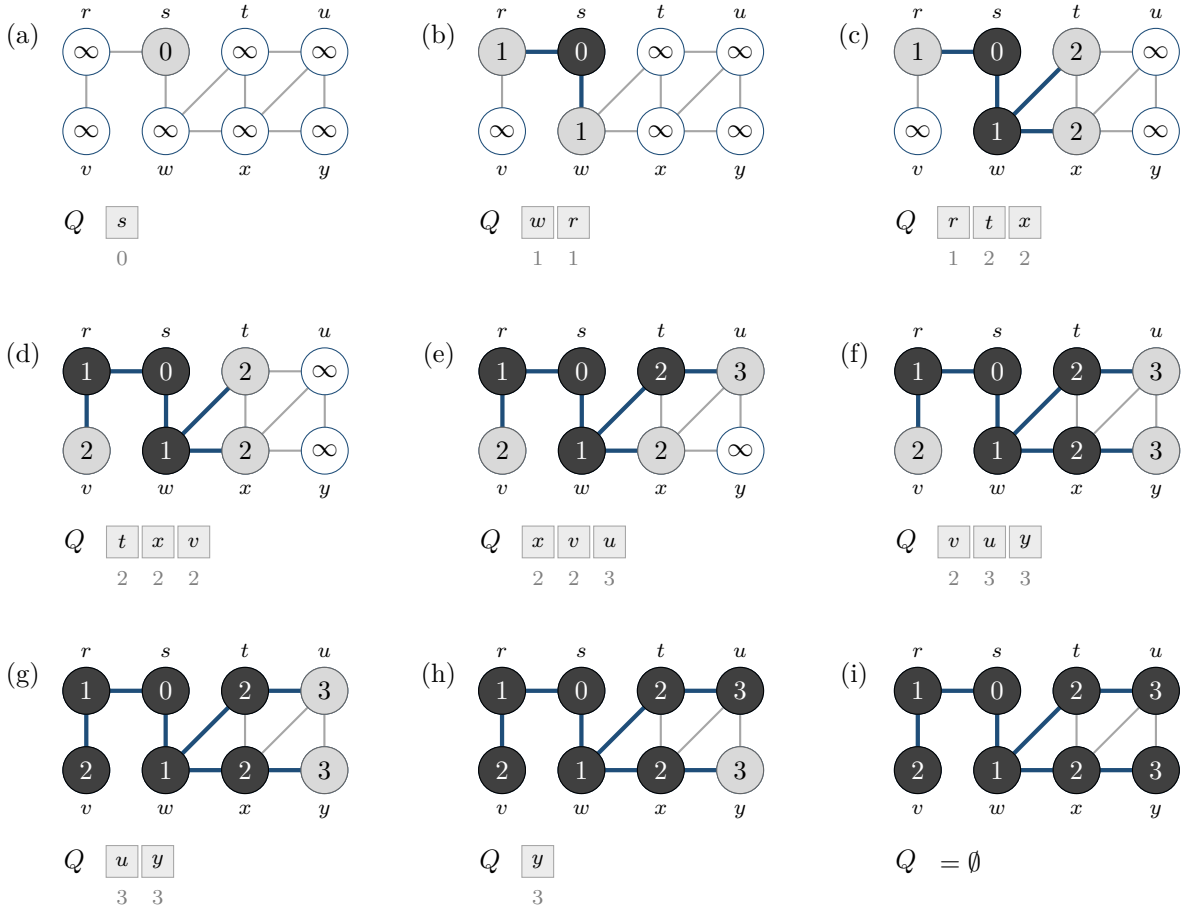
Algoritmo 4.7 Imprimir-Camino(G, s, v)

```
1:  procedimiento Imprimir-Camino(G, s, v)
2:      si v = s entonces
3:          imprimir s
4:      si no si π[v] = nulo entonces
5:          imprimir "no hay camino de s a v"
6:      si no
7:          Imprimir-Camino(G, s, π[v])
8:          imprimir v
9:      fin si
10: fin procedimiento
```

Supone que BFS ya corrió desde la misma s. Si v es alcanzable, la cadena de padres termina en s sin tocar nulo, así que la rama de la línea 4 solo puede dispararse en la primera llamada. Hace $\delta(s, v) + 1$ llamadas de costo $\mathcal{O}(1)$. Si los π vinieran de un BFS desde otra fuente s', la cadena terminaría en s' y el mensaje "no hay camino" saldría a mitad de la salida.

Observación. Las distancias no dependen del orden de las listas, porque $d[v] = \delta(s, v)$ solo depende de G (Cormen, ejercicio 22.2-5). El árbol sí puede depender. En la figura 22.3, si la lista de w es s, x, t en lugar de s, t, x, sale x antes que t y descubre a u: queda $\pi[u] = x$. De los 41 472 órdenes posibles de las listas de ese grafo solo salen dos árboles, y solo difieren en $\pi[u]$.

Ejemplo 4.22 (La figura 22.3 de Cormen)**.** El grafo es no dirigido, con $V = \{r, s, t, u, v, w, x, y\}$, diez aristas y fuente s. Cormen no imprime las listas; las que reproducen sus colas son r: s, v; s: w, r; t: w, x, u; u: t, x, y; v: r; w: s, t, x; x: w, t, u, y; y: x, u. La de s no es alfabética, porque la instantánea (b) tiene $Q = \langle w, r \rangle$. Cada instantánea muestra el estado al principio de una vuelta del ciclo: blancos con ∞, grises y negros con su d, y la cola con la d de cada vértice debajo; en azul, las aristas del árbol.

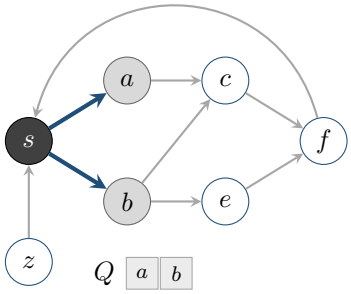


En la traza, B, G y N son el color del vecino en el momento en que se examina.

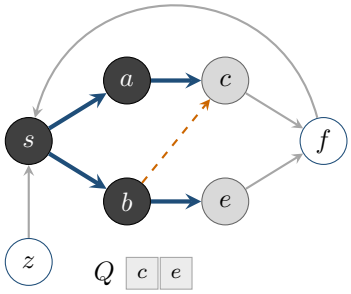
It.	Q **al inicio**	d **en** Q	**Sale**	**Examina**	**Descubre**
1	s	0	s	w B, r B	w, r con $d = 1$, $\pi = s$
2	w, r	1, 1	w	s N, t B, x B	t, x con $d = 2$, $\pi = w$
3	r, t, x	1, 2, 2	r	s N, v B	v con $d = 2$, $\pi = r$
4	t, x, v	2, 2, 2	t	w N, x G, u B	u con $d = 3$, $\pi = t$
5	x, v, u	2, 2, 3	x	w N, t N, u G, y B	y con $d = 3$, $\pi = x$
6	v, u, y	2, 3, 3	v	r N	nada
7	u, y	3, 3	u	t N, x N, y G	nada
8	y	3	y	x N, u N	nada

Quedan siete aristas de árbol, $n-1$, y tres fuera: t–x, x–u y u–y. Las tres unen vértices del mismo nivel o de niveles contiguos, como exige $|\delta(s,u) - \delta(s,v)| \leq 1$. Imprimir-Camino$(G, s, y)$ imprime s, w, x, y. En las iteraciones 4, 5 y 7 aparece un vecino gris: son los casos que la prueba del teorema 4.19 trata aparte.

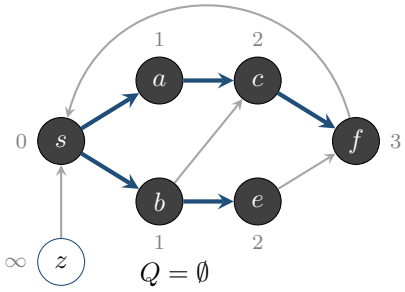
Ejemplo 4.23 (Dirigido, con un vértice inalcanzable)**.** $V = \{s, a, b, c, e, f, z\}$ y nueve aristas dirigidas: (s,a), (s,b), (a,c), (b,c), (b,e), (c,f), (e,f), (f,s) y (z,s), con las listas en orden alfabético. No hay vértice d, para no confundirlo con el atributo. Tres instantáneas: tras sacar s, tras sacar b y el estado final, con la d junto a cada vértice.



(1) sale s



(3) sale b



(6) final

Tres cosas que este ejemplo enseña y el de la clase no:

- z tiene una arista *hacia* s, pero ninguna llega a z: es inalcanzable. Termina blanco, con $d[z] = \infty$ y $\pi[z] =$ nulo, y su lista nunca se lee: salen seis vértices y se examinan ocho de las nueve entradas de las listas. Imprimir-Camino(G, s, z) imprime "no hay camino de s a z".
- La arista (f, s) baja del nivel 3 al 0. En un dirigido una arista puede regresar cualquier número de niveles; solo hacia adelante está acotada (lema 4.15).
- También aquí aparecen vecinos grises: al sacar b, c ya está en la cola (punteada en la instantánea 3), y al sacar e, f también. En la versión no dirigida, z y f quedarían a distancia 1.

Ojito...

- Marcar al descubrir, no al sacar. Si el vértice se marca cuando sale de la cola, puede entrar varias veces, una por cada vecino que lo encuentre antes de que salga; la cola crece hasta $\Theta(m)$, se repite trabajo y, si d y π se sobrescriben al reencolar, se pierden los valores correctos.
- En un dirigido, alcanzable es en la dirección de las flechas: z apunta a s y es inalcanzable. La propiedad $|d[u] - d[v]| \leq 1$ en cada arista es exclusiva del caso no dirigido.
- Un lazo no estorba. En un BFS desde 3 sobre el grafo del ejemplo 4.3, al sacar a 6 este se examina a sí mismo y se encuentra gris, no negro, porque se ennegrece hasta acabar su lista; como solo importa que no sea blanco, no pasa nada.
- Un BFS solo visita lo alcanzable desde s. Para recorrer todo el grafo hace falta un ciclo externo sobre los vértices, como el del algoritmo bipartito o el de DFS.

Ciclos impares y grafos bipartitos

En un grafo no dirigido, el lema 4.15 dice que toda arista une dos niveles iguales o contiguos del árbol BFS. Eso afina la regla de ciclos de la clase. Si una arista fuera del árbol une u y v con $d[u] = d[v] = k$ y x es su ancestro común más bajo, los dos caminos del árbol de x a u y a v más la arista cierran un ciclo de $2(k - d[x]) + 1$ aristas: impar. Si los niveles difieren en 1, el ciclo es par. En el grafo G de la clase, la arista E–F une dos vértices con $d = 2$, y el ciclo A, B, E, F, C, A tiene cinco aristas: G no es bipartito. Esta es exactamente la prueba del algoritmo 3.4 de la sección 3, que colorea por la paridad de d y también resuelve el problema de los luchadores buenos y malos de Cormen (ejercicio 22.2-7) en $\mathcal{O}(n + r)$, con r el número de rivalidades.

Diámetro de un árbol con dos BFS

El **diámetro** de un árbol T es la mayor de todas las distancias, $\max_{u,v} \delta(u,v)$. Calcularlo par por par costaría n BFS. En un árbol bastan dos, gracias a esta propiedad:

Proposición 4.24. *En un árbol, desde cualquier s, el vértice a más lejano es extremo de algún diámetro.*

Demostración. Sea p, q un par que realiza el diámetro y P el camino entre ellos. Sea x el vértice de P más cercano a s; en un árbol todo camino de s a un vértice de P pasa por x. Sea z el último vértice común de los caminos de s a a y de s a q, el punto donde se separan. Hay dos casos.

Si z está en el tramo de s a x, como a es el más lejano, $\delta(s,z) + \delta(z,a) = \delta(s,a) \geq \delta(s,p) = \delta(s,z) + \delta(z,x) + \delta(x,p)$, así que $\delta(z,a) \geq \delta(z,x) + \delta(x,p)$. Entonces

$$\delta(a,q) = \delta(a,z) + \delta(z,x) + \delta(x,q) \geq \delta(x,p) + \delta(x,q) = \delta(p,q),$$

y el par a, q también realiza el diámetro.

Si z está en P después de x, en el tramo hacia q, entonces $\delta(s,z) + \delta(z,a) \geq \delta(s,z) + \delta(z,q)$, o sea $\delta(z,a) \geq \delta(z,q)$, y $\delta(p,a) = \delta(p,z) + \delta(z,a) \geq \delta(p,z) + \delta(z,q) = \delta(p,q)$: el par p, a realiza el diámetro. □

Entonces un BFS desde cualquier s da un extremo a de un diámetro, y un segundo BFS desde a encuentra el otro extremo:

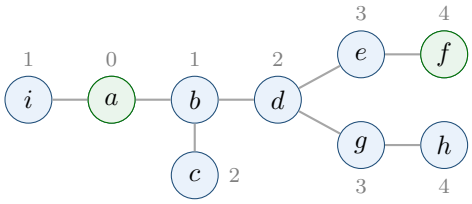
Algoritmo 4.8 Diámetro(T) de un árbol con dos BFS

```
1: procedimiento Diámetro(T)
2:     s ← cualquier vértice de T
3:     BFS(T, s);  a ← un vértice con d[a] máxima
4:     BFS(T, a);  b ← un vértice con d[b] máxima
5:     devolver d[b]                    // el camino sale de Imprimir-Camino(T, a, b)
6: fin procedimiento
```

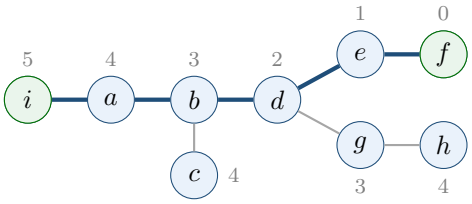
Son dos BFS de $\mathcal{O}(n+m)$ con $m = n-1$, más dos recorridos para hallar el máximo: $\mathcal{O}(n)$.

Un solo BFS no basta, y el truco es exclusivo de árboles: en K_4 con vértices 1 a 4 sin la arista $\{3,4\}$, desde 1 todos están a distancia 1; si se toma 2 como el más lejano, el segundo BFS vuelve a dar 1, pero $\delta(3,4) = 2$. La prueba usó que el camino entre dos vértices es único, y eso solo pasa en árboles.

Ejemplo 4.25 (Diámetro de un árbol)**.** El árbol tiene $V = \{a, \ldots, i\}$ y aristas i–a, a–b, b–c, b–d, d–e, e–f, d–g, g–h. Junto a cada vértice, su d en cada BFS; en verde, la fuente y el más lejano de cada uno.



BFS desde a: el más lejano es f, a 4



BFS desde f: el más lejano es i, a 5

Desde a hay empate entre f y h a distancia 4; cualquiera sirve. Desde f el más lejano es i, a 5: el diámetro es 5 y el camino, en azul, es f, e, d, b, a, i. Desde a el vértice más lejano estaba a 4, así que un solo BFS se habría quedado corto. El procedimiento da 5 desde cualquiera de los nueve vértices iniciales.

Búsqueda en profundidad a fondo

La clase dio DFS-Visitar con visto, un reloj y los tiempos entra y sale, y la corrió sobre G desde A. Le faltan dos piezas que Cormen pone: un ciclo externo que vuelve a arrancar desde cada vértice que siga sin descubrir, de modo que el recorrido cubre todo el grafo y produce un bosque, y los tres colores, que aquí sí cambian lo que se puede deducir. Un vértice es blanco antes de entra$[u]$, gris entre entra$[u]$ y sale$[u]$, y negro después. El reloj t de la clase se llama aquí *tiempo*, porque uno de los ejemplos tiene un vértice t.

Definición 4.26 (Bosque DFS)**.** El subgrafo predecesor de DFS es $G_\pi = (V, E_\pi)$ con $E_\pi = \{(\pi[v], v) : v \in V,\ \pi[v] \neq \text{nulo}\}$. Es un **bosque DFS**, formado por **árboles DFS**, uno por cada raíz que elige el ciclo externo; sus aristas son las **aristas de árbol**.

A diferencia del de BFS, G_π contiene a todo V. Reproduce el árbol de llamadas recursivas: $\pi[v] = u$ si y solo si DFS-Visitar(G, v) se llamó desde el ciclo de u. En cada instante, los vértices grises son exactamente las llamadas activas, del más viejo al más reciente, y forman una cadena de descendientes en el bosque. De ahí sale la llave de casi todo lo que sigue: una arista que llega a un vértice gris llega a un antecesor.

Algoritmo 4.9 DFS(G) con colores y tiempos (Cormen, sección 22.3)

```
1:  procedimiento DFS(G)
2:      para todo u ∈ V hacer
3:          color[u] ← BLANCO,    π[u] ← NULO
4:      fin para
5:      tiempo ← 0
6:      para todo u ∈ V hacer
7:          si color[u] = BLANCO entonces
8:              DFS-Visitar(G, u)                    // u es raíz de un árbol nuevo
9:          fin si
10:     fin para
11: fin procedimiento
```

Algoritmo 4.10 DFS-Visitar(G, u), la visita recursiva de DFS

```
1:  procedimiento DFS-Visitar(G, u)
2:      tiempo ← tiempo + 1,    entra[u] ← tiempo
3:      color[u] ← GRIS
4:      para todo v ∈ Adj[u] hacer                    // explorar la arista (u, v)
5:          si color[v] = BLANCO entonces
6:              π[v] ← u
7:              DFS-Visitar(G, v)
8:          fin si
9:      fin para
10:     color[u] ← NEGRO                              // u terminó
11:     tiempo ← tiempo + 1,    sale[u] ← tiempo
12: fin procedimiento
```

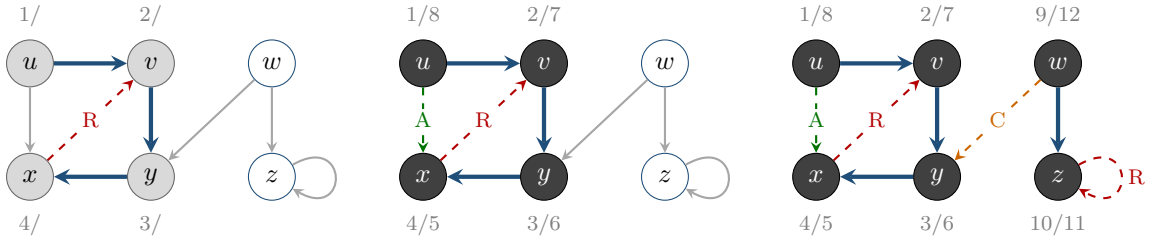
El reloj sube antes de asignar, así que el primer descubrimiento es el 1. Hay exactamente un descubrimiento y una finalización por vértice, de modo que los tiempos son los enteros de 1 a $2n$, todos distintos, y siempre entra$[u] <$ sale$[u]$. Un vértice gris o negro no se vuelve a visitar, pero la arista hacia él sí cuenta como explorada: eso es lo que permite clasificarla más adelante.

Costo: aquí sí es $\Theta(n+m)$. DFS-Visitar se llama exactamente una vez por vértice. A lo más una, porque solo se llama sobre un vértice blanco y lo primero que hace es pintarlo de gris, color del que nunca regresa. Al menos una, porque el ciclo externo llama sobre todo vértice que siga blanco. Fuera de su ciclo, cada llamada hace trabajo $\mathcal{O}(1)$: $\Theta(n)$. En la llamada sobre u, el ciclo da $|Adj[u]|$ vueltas de costo $\mathcal{O}(1)$, sin contar la llamada recursiva, que ya se contó. Sumando con el lema 4.1 sale $\Theta(m)$, y el total es $\Theta(n+m)$ en toda entrada, porque cada vértice y cada entrada de cada lista se tocan. La cuenta no es la de BFS, que contaba entradas a la cola; aquí se cuentan llamadas. Con matriz el total sube a $\Theta(n^2)$. La memoria extra es $\Theta(n)$ más la pila de recursión, que llega a profundidad n cuando el árbol es un camino; en grafos grandes eso desborda la pila del sistema (Python corta en 1000 llamadas por omisión), y es la razón práctica de la versión iterativa del final.

Ejemplo 4.27 (La figura 22.4 de Cormen, paso a paso)**.** $V = \{u, v, w, x, y, z\}$ y las aristas (u,v), (u,x), (v,y), (w,y), (w,z), (x,v), (y,x) y el lazo (z,z), con listas u: v, x; v: y; w: y, z; x: v; y: x; z: z. El ciclo externo toma los vértices en orden alfabético. La columna de grises va del fondo al tope y es literalmente la pila de llamadas.

tiempo	**Evento**	**Grises**	**Arista explorada**	**Color de v, tipo**
1	descubre u (raíz)	u	(u,v)	blanco: árbol
2	descubre v	u, v	(v,y)	blanco: árbol
3	descubre y	u, v, y	(y,x)	blanco: árbol
4	descubre x	u, v, y, x	(x,v)	gris: retroceso
5	termina x	u, v, y		
6	termina y	u, v		
7	termina v	u	(u,x)	negro, $1 < 4$: avance
8	termina u	vacía		
9	descubre w (raíz)	w	(w,y)	negro, $9 > 3$: cruce
			(w,z)	blanco: árbol
10	descubre z	w, z	(z,z)	gris: retroceso
11	termina z	w		
12	termina w	vacía		

Tres instantáneas, en los tiempos 4, 8 y 12; junto a cada vértice, entra/sale. Las aristas grises delgadas aún no se exploran; las punteadas llevan su tipo: R de retroceso, A de avance, C de cruce.



(a) *tiempo* = 4 (b) *tiempo* = 8 (c) *tiempo* = 12

Quedan dos árboles, $\{u, v, y, x\}$ y $\{w, z\}$. El vértice y queda en el primero aunque w también le apunte: cuando w explora (w, y), y ya es negro. El ejemplo de la clase sobre G es una corrida del mismo algoritmo con una sola raíz, porque desde A se alcanza todo.

El teorema del paréntesis

Si se escribe "$(u$" en el instante entra$[u]$ y "$u)$" en sale$[u]$, la sucesión de eventos de una DFS es una expresión de paréntesis bien balanceada. En el ejemplo 4.27 es $(u\ (v\ (y\ (x\ x)\ y)\ v)\ u)\ (w\ (z\ z)\ w)$, y en el de la clase sobre G, $(A\ (B\ (D\ (G\ G)\ D)\ (E\ (F\ (C\ C)\ F)\ E)\ B)\ A)$. La razón es que la recursión es una pila: una llamada que empieza dentro de otra termina antes que ella.

Teorema 4.28 (Del paréntesis; Cormen, teorema 22.7)**.** *En cualquier DFS de un grafo, dirigido o no, para cualesquiera dos vértices u y v se cumple exactamente una de estas tres condiciones:*

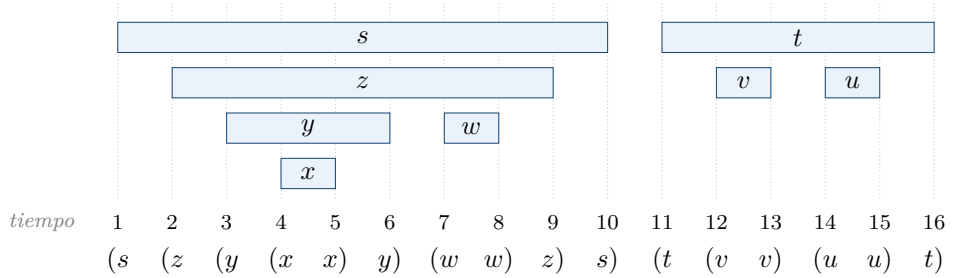
1. *los intervalos $[\text{entra}[u], \text{sale}[u]]$ y $[\text{entra}[v], \text{sale}[v]]$ son disjuntos, y ninguno de los dos vértices desciende del otro en el bosque;*
2. *el intervalo de u está contenido en el de v, y u desciende de v;*
3. *el intervalo de v está contenido en el de u, y v desciende de u.*

Demostración. Supóngase entra$[u]$ < entra$[v]$; el otro caso es simétrico. Si entra$[v]$ < sale$[u]$, v se descubrió mientras u era gris, es decir, durante la llamada sobre u, así que desciende de u. Como v es más reciente, su llamada termina antes de que la búsqueda regrese a u: sale$[v]$ < sale$[u]$, y el intervalo de v queda dentro del de u. Si en cambio sale$[u]$ < entra$[v]$, entonces entra$[u]$ < sale$[u]$ < entra$[v]$ < sale$[v]$: los intervalos son disjuntos, y ninguno se descubrió con el otro gris, así que ninguno desciende del otro. □

Lo que el teorema prohíbe es el traslape parcial: dos intervalos se anidan o se separan, como los paréntesis.

Corolario 4.29 (Anidamiento; Cormen, corolario 22.8)**.** *v es descendiente propio de u si y solo si entra$[u]$ < entra$[v]$ < sale$[v]$ < sale$[u]$.*

Ejemplo 4.30 (La figura 22.5 de Cormen: los paréntesis como intervalos)**.** $V = \{s, t, u, v, w, x, y, z\}$ y trece aristas: (s, z), (s, w), (t, v), (t, u), (u, t), (u, v), (v, s), (v, w), (w, x), (x, z), (y, x), (z, y), (z, w). Cormen no da las listas; estas reproducen sus tiempos: s: z, w; t: v, u; u: t, v; v: s, w; w: x; x: z; y: x; z: y, w, con s primero y t segundo en el ciclo externo. Cada vértice está dibujado como su intervalo, un renglón por nivel de profundidad, y abajo van los paréntesis en el instante en que se escriben.



Un rectángulo debajo de otro y dentro de su sombra es un descendiente: x dentro de y, dentro de z, dentro de s. Dos rectángulos sin traslape no tienen relación: y y w son hermanos, y s y t están en árboles distintos. Nunca hay traslape parcial. Los árboles son s, z, y, x, w, con $\pi[z] = s$, $\pi[y] = \pi[w] = z$ y $\pi[x] = y$, y t, v, u, con $\pi[v] = \pi[u] = t$.

El teorema del camino blanco

El paréntesis dice cómo reconocer a un descendiente a partir de los tiempos. El siguiente teorema dice cómo saber, en el instante en que se descubre u, quiénes van a terminar colgando de él.

Teorema 4.31 (Del camino blanco; Cormen, teorema 22.9)**.** *En un bosque DFS de un grafo dirigido o no, v es descendiente de u si y solo si, en el instante entra$[u]$, existe un camino de u a v formado solo por vértices blancos.*

Demostración. ($\Rightarrow$) Si $v = u$, el camino es solo u, que es blanco justo antes de que se fije entra$[u]$. Si v es descendiente propio, todos los vértices del camino del árbol de u a v descienden de u, así que por el corolario 4.29 se descubren después de entra$[u]$: en ese instante eran blancos.

($\Leftarrow$) Supóngase que en entra$[u]$ hay un camino blanco de u a v, pero v no desciende de u. Sea x el primer vértice del camino que no desciende de u, y w el anterior a él en el camino; w sí desciende de u (puede ser $w = u$), así que sale$[w] \leq$ sale$[u]$. Como x era blanco en entra$[u]$, se descubre después; y como w explora la arista (w, x) antes de terminar, x se descubre a más tardar en ese momento. Entonces

$$\text{entra}[u] < \text{entra}[x] < \text{sale}[w] \leq \text{sale}[u].$$

El intervalo de x empieza dentro del de u; por el teorema 4.28 no puede traslaparse a medias, así que queda contenido, y por el corolario 4.29 x desciende de u. Contradicción. □

La lectura útil: ser descendiente no depende de lo que pase después, sino de qué estaba blanco al descubrir u. Un camino de u a v que pase por un vértice ya gris o negro no obliga a nada.

Clasificación de aristas

Definición 4.32 (Tipos de arista)**.** Respecto del bosque DFS, una arista (u, v) es

1. **de árbol** si está en E_π: v se descubrió al explorarla;
2. **de retroceso** si une a u con un antecesor v en su árbol; los lazos cuentan como de retroceso;
3. **de avance** si no es de árbol y une a u con un descendiente v;
4. **de cruce** en cualquier otro caso: entre vértices del mismo árbol sin relación de ancestría, o entre árboles distintos.

Proposición 4.33 (Clasificación por el color)**.** *Cuando DFS explora (u, v) por primera vez, el tipo lo decide el color de v: blanco, árbol; gris, retroceso; negro, avance si entra$[u] <$ entra$[v]$ y cruce si entra$[u] >$ entra$[v]$.*

Demostración. Blanco: v se descubre justo ahora, desde u, por construcción. Gris: los grises son la cadena de llamadas activas y la exploración sale siempre del más reciente, que es u; cualquier otro gris es antecesor de u, y si $v = u$ es un lazo. Negro con entra$[u] <$ entra$[v]$: v se descubrió y terminó mientras u era gris, así que desciende de u, y no es de árbol porque no estaba blanco. Negro con entra$[v] <$ entra$[u]$: u no puede descender de v, porque entonces v seguiría gris; por el teorema 4.28 los intervalos son disjuntos, sale$[v] <$ entra$[u]$, y no hay relación. □

Para separar avance de cruce se compara entra, no sale: cuando se explora la arista, sale$[u]$ todavía no existe. Para clasificar al vuelo basta cambiar el **si** del ciclo de DFS-Visitar por estos cuatro casos; el costo sigue siendo $\Theta(n + m)$, porque cada arista se clasifica en $\mathcal{O}(1)$.

Con los intervalos ya calculados también se clasifica sin repetir la búsqueda (Cormen, ejercicio 22.3-5).

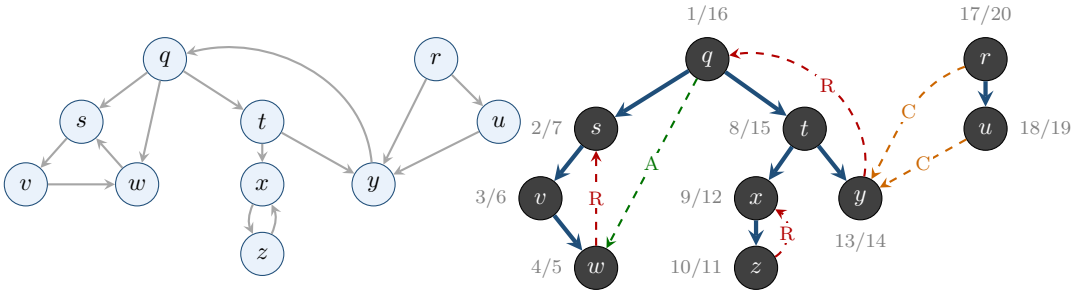
Proposición 4.34 (Clasificación por intervalos)**.** *La arista (u, v) es de árbol o de avance si y solo si entra$[u] <$ entra$[v] <$ sale$[v] <$ sale$[u]$; de retroceso si y solo si entra$[v] \leq$ entra$[u] <$ sale$[u] \leq$ sale$[v]$, con igualdad solo en un lazo; y de cruce si y solo si entra$[v] <$ sale$[v] <$ entra$[u] <$ sale$[u]$. Árbol y avance se separan con π: es de árbol si y solo si $\pi[v] = u$.*

Falta a propósito un cuarto caso, entra$[u] <$ sale$[u] <$ entra$[v]$: ninguna arista sale de un intervalo hacia otro que empieza después de que el primero cerró, porque al explorar (u, v) el vértice v habría estado blanco y se habría vuelto descendiente de u. Por la misma razón nunca hay una arista de un vértice negro a uno blanco: un vértice no termina mientras le quede un vecino sin descubrir. La consecuencia que más se usa es esta: **toda arista que no es de retroceso cumple sale**$[v] <$ **sale**$[u]$. Es la desigualdad que hace funcionar el orden topológico de la sección 5. En el ejemplo 4.30: (x, z) va de $[4, 5]$ hacia $[2, 9]$, que lo contiene, y es de retroceso; (s, w) va de $[1, 10]$ hacia $[7, 8]$, contenido, con $\pi[w] = z \neq s$, y es de avance; (w, x) va de $[7, 8]$ hacia $[4, 5]$, que ya había cerrado, y es de cruce.

Ejemplo 4.35 (Los cuatro tipos; Cormen, figura 22.6 y ejercicio 22.3-2)**.** $V = \{q, r, s, t, u, v, w, x, y, z\}$ y catorce aristas: (q, s), (q, t), (q, w), (r, u), (r, y), (s, v), (t, x), (t, y), (u, y), (v, w), (w, s), (x, z), (y, q), (z, x). Como pide el ejercicio, el ciclo externo y todas las listas van en orden alfabético.

Arista	*tiempo*	**Color de** v	**Tipo**	**Arista**	*tiempo*	**Color de** v	**Tipo**
(q, s)	1	blanco	árbol	(t, y)	12	blanco	árbol
(s, v)	2	blanco	árbol	(y, q)	13	gris	retroceso
(v, w)	3	blanco	árbol	(q, w)	15	negro	avance, $1 < 4$
(w, s)	4	gris	retroceso	(r, u)	17	blanco	árbol
(q, t)	7	blanco	árbol	(u, y)	18	negro	cruce, $18 > 13$
(t, x)	8	blanco	árbol	(r, y)	19	negro	cruce, $17 > 13$
(x, z)	9	blanco	árbol				
(z, x)	10	gris	retroceso				

Tiempos finales: q 1/16, s 2/7, v 3/6, w 4/5, t 8/15, x 9/12, z 10/11, y 13/14, r 17/20, u 18/19. Ocho de árbol, tres de retroceso, una de avance y dos de cruce. A la izquierda el grafo; a la derecha el mismo grafo redibujado como bosque: las de árbol y la de avance bajan, las de retroceso suben y las de cruce van de derecha a izquierda, hacia subárboles que ya terminaron.



(a) el grafo (b) el bosque DFS

Las tres de retroceso cierran tres ciclos: $s \to v \to w \to s$, $x \to z \to x$ y $q \to t \to y \to q$. Para saber que el grafo no es acíclico basta una; es el criterio que la sección 5 demuestra con el teorema del camino blanco.

Grafos no dirigidos: solo árbol y retroceso

En un no dirigido, $\{u, v\}$ aparece en las dos listas y se explora dos veces. Cormen la clasifica con el primero de los cuatro tipos que le aplique, lo que equivale a clasificarla por la dirección en que se explora primero (ejercicio 22.3-6). Con esa regla, dos de los tipos desaparecen.

Teorema 4.36 (Cormen, teorema 22.10)**.** *En una DFS de un grafo no dirigido, toda arista es de árbol o de retroceso.*

Demostración. Sea $\{u, v\}$ una arista con, sin perder generalidad, entra$[u] <$ entra$[v]$. Como v está en la lista de u y era blanco cuando se descubrió u, el teorema del camino blanco lo pone debajo de u: v se descubre y termina mientras u es gris. Si la arista se explora primero de u a v, v seguía blanco en ese momento, porque si ya se hubiera descubierto, la llamada sobre v habría explorado la arista desde su lado antes: es de árbol. Si se explora primero de v a u, u sigue gris: es de retroceso. □

El ejemplo de la clase lo ilustra. La arista $\{A, C\}$ se explora primero desde C, en el tiempo 9, con A gris: es de retroceso. Cuando A la ve en el tiempo 13, C ya es negro y no se reclasifica. Para implementarlo, al explorar $\{u, v\}$ desde u: si v es blanco, árbol; si es gris y $v \neq \pi[u]$, retroceso; si $v = \pi[u]$, es la arista de árbol vista desde abajo; si v es negro, la arista ya se exploró desde v. En los dos últimos casos no se hace nada. Es la misma condición $v \neq \pi[u]$ de la regla de ciclos con BFS de la clase.

Ojito…

Con aristas paralelas la condición $v \neq \pi[u]$ falla. En el multigrafo con dos aristas entre A y B y una entre B y C, DFS desde A baja a B por la primera copia. Desde B, la segunda copia lleva a A, que es gris pero es $\pi[B]$, y se descarta como si fuera la de árbol; cuando A la ve después, B ya es negro y también se descarta. El ciclo A, B, A de dos aristas nunca se reporta. La solución es comparar aristas y no vértices: guardar con cada vértice la arista por la que se llegó a él, y descartar solo esa. Con esa regla la segunda copia sale de retroceso, como debe.

Ojito...

- El resultado depende del orden. En la figura 22.4, si el ciclo externo empieza por w, sale w 1/10, y 2/7, x 3/6, v 4/5, z 8/9, u 11/12, y la de retroceso es (v, y) en lugar de (x, v). Lo que no depende del orden es *si hay* alguna de retroceso, porque eso equivale a tener un ciclo.
- Un bit basta para recorrer, no para clasificar. Si se quita la línea que pinta de negro, entra, sale y π salen idénticos (Cormen, ejercicio 22.3-4), así que el visto de la clase alcanza para el recorrido. Pero sin distinguir gris de negro una de retroceso y una de cruce se ven igual, y la detección de ciclos dirigidos da falsos positivos, como la regla con BFS que la clase advierte.
- Un vértice puede quedar solo en su árbol aunque tenga aristas de entrada y de salida (ejercicio 22.3-11): con (u, v) y (w, u) y el ciclo externo en el orden v, u, w, salen tres árboles de un vértice y dos aristas de cruce.

DFS iterativo con pila

La recursión de DFS llega a profundidad n, y en un grafo grande eso desborda la pila del sistema. La clase menciona que la versión iterativa usa una pila explícita, pero no la escribe, y hay dos formas naturales de escribirla mal. Primero la versión corta, que sirve cuando solo interesan el orden de visita y π.

Algoritmo 4.11 DFS-Pila(G, s): orden de visita y padres

```
1:  procedimiento DFS-Pila(G, s)
2:      para todo v ∈ V hacer
3:          visto[v] ← falso
4:      fin para
5:      P ← pila vacía;  Apilar(P, (s, nulo))
6:      mientras P no está vacía hacer
7:          (u, p) ← Desapilar(P)
8:          si no visto[u] entonces                    // si no, es copia vieja
9:              visto[u] ← verdadero,    π[u] ← p
10:             para todo v ∈ Adj[u], del último al primero hacer
11:                 si no visto[v] entonces
12:                     Apilar(P, (v, u))
13:                 fin si
14:             fin para
15:         fin si
16:     fin mientras
17: fin procedimiento
```

Las dos decisiones que la hacen DFS son justo las dos trampas.

Trampa 1: marcar al apilar. Aquí se marca al *sacar* (línea 9), y un vértice puede quedar varias veces en la pila; las copias viejas se descartan en la línea 8. La tentación es marcar al apilar, como en BFS, para que nadie entre dos veces. Eso ya no es DFS: el vecino se marca cuando lo apila el primero que lo ve, y queda colgando de él aunque la búsqueda llegue después por otro lado. En el triángulo a, b, c produce una estrella con centro a, $\pi[b] = \pi[c] = a$, y la arista $\{b, c\}$ sería de cruce, cosa que el teorema 4.36 prohíbe. En G es más traicionero: el orden de visita sale igual al de la recursión, A, B, D, G, E, F, C, pero C queda con $\pi[C] = A$ en lugar de F, porque A lo marcó al principio. El árbol es otro, y F–C une dos ramas distintas.

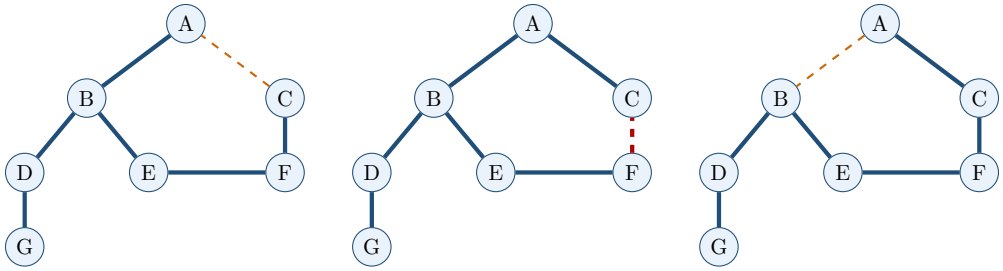
Trampa 2: apilar en el orden de la lista. La pila invierte el orden: lo último que se apila es lo primero que sale. Para visitar los vecinos en el orden de la lista hay que apilarlos del último al primero (línea 10). Si se apilan en el orden de la lista, el resultado es una DFS válida, pero la de las listas al revés: en G sale A, C, F, E, B, D, G, y el árbol ya no es el de la clase.

Además, esta versión no da tiempos de salida, y su pila puede guardar copias: cada apilado corresponde a una entrada de una lista revisada desde un vértice recién visitado, así que hay a lo más $m+1$ apilados en un dirigido y $2m+1$ en uno no dirigido. Cada vértice se visita una vez y recorre su lista una vez: $\mathcal{O}(n+m)$ de tiempo, con $\mathcal{O}(m)$ de memoria para la pila.

Ejemplo 4.37 (DFS-Pila sobre G desde A)**.** Con las listas en orden alfabético. En la pila, C_A es el par (C, A): el vértice C, apilado por A. Cada renglón saca el tope.

Saca	Acción	Pila después, fondo a tope
A_{nulo}	visita A; apila C y B	C_A, B_A
B_A	visita B; apila E y D	C_A, E_B, D_B
D_B	visita D; apila G	C_A, E_B, G_D
G_D	visita G; nada que apilar	C_A, E_B
E_B	visita E; apila F	C_A, F_E
F_E	visita F; apila C	C_A, C_F
C_F	visita C, con $\pi[C] = F$	C_A
C_A	copia vieja: C ya se visitó	vacía

Orden A, B, D, G, E, F, C, y los mismos padres que la recursión. Hubo ocho apilados y una copia vieja, C_A. Las tres versiones, lado a lado:



(a) recursión o DFS-Pila: C–A de retroceso

(b) marcando al apilar: F–C sería de cruce

(c) apilando en orden: otra DFS

Para tener tiempos de salida hay que simular la recursión de verdad (Cormen, ejercicio 22.3-7). La pila guarda solo vértices grises, y cada uno lleva un apuntador sig[u] al siguiente nodo por revisar de su lista, que es justo lo que la recursión guarda sin que se vea en la variable del ciclo. En cada vuelta se mira el tope u sin sacarlo. Si sig[u] no es nulo, se toma ese vecino v y se avanza el apuntador; si v es blanco, recibe $\pi[v] = u$ y su entra[v], se pinta de gris y se apila. Si sig[u] es nulo, u agotó su lista: se desapila, se pinta de negro y recibe su sale[u]. El tope avanza una sola arista por vuelta, y el ciclo externo es el mismo de DFS.

El invariante es que la pila contiene exactamente los vértices grises, del más viejo en el fondo al más reciente en el tope, igual que la pila de llamadas; por eso da los mismos entra, sale y π que la versión recursiva. Sobre G desde A reproduce los tiempos de la clase, de A 1/14 a C 9/10, con la pila llegando a cinco vértices (A, B, E, F, C). Sin sig[u], cada vez que u regresa al tope habría que recorrer su lista desde el principio, y el costo podría subir a $\mathcal{O}(nm)$. Con él, cada vértice entra y sale de la pila una vez y cada vuelta del ciclo avanza un apuntador o saca un vértice: $\Theta(n+m)$, con $\mathcal{O}(n)$ de pila.

Componentes conexas con DFS

En un grafo no dirigido, cada árbol del bosque DFS es exactamente una componente conexa (Cormen, ejercicio 22.3-12). Basta numerar los árboles.

Algoritmo 4.12 Cambio a DFS para numerar las componentes

```
1: k ← 0                                              // antes del ciclo externo
2: para todo u ∈ V hacer
3:     si color[u] = BLANCO entonces
4:         k ← k + 1                                  // árbol nuevo, componente nueva
5:         DFS-Visitar(G, u)
6:     fin si
7: fin para
```

En DFS-Visitar(G, u), junto con pintar a u de gris: $\text{cc}[u] \leftarrow k$.

Por qué funciona. Por inducción sobre los árboles, en el orden en que se crean. Supóngase que los árboles anteriores son componentes completas. La nueva raíz u es blanca, así que no está en ninguna de ellas, y su componente es disjunta de todas: en el instante entra$[u]$ toda la componente de u es blanca. Por el teorema del camino blanco, todo vértice de esa componente termina debajo de u. Y nada más, porque las aristas de árbol son aristas del grafo y no hay aristas hacia otras componentes. El costo es el de una DFS con una asignación más por vértice, $\Theta(n + m)$. Con G de la clase más una arista H–I y un vértice J aislado salen tres árboles y tres componentes: $\{A, \ldots, G\}$, $\{H, I\}$ y $\{J\}$. En un dirigido los árboles DFS no son componentes de nada, y su número depende del orden: las componentes fuertemente conexas necesitan el algoritmo de dos DFS de la sección 5.

5. Aplicaciones de DFS

Complemento: Cormen, cap. 22.4, 22.5 y problemas

Resumen rápido. El DFS de la sección 4 deja en cada vértice dos marcas de tiempo, entra[v] y sale[v], y esta sección muestra que esas marcas no son contabilidad del recorrido: son la estructura del grafo. Con ellas se resuelven, todos en $\Theta(n+m)$, cuatro problemas. El orden topológico de un DAG sale de listar los vértices por tiempo de salida decreciente, o del algoritmo de Kahn, que quita vértices sin predecesores. Las componentes fuertemente conexas de un grafo dirigido salen de dos DFS, el segundo sobre el grafo transpuesto. Los puntos de articulación, los puentes y las componentes biconexas de un grafo no dirigido salen de un solo DFS que calcula low[v]. Y la etiqueta mínima alcanzable desde cada vértice sale de un DFS sobre el transpuesto. Se cierra con el recorrido de Euler dirigido. Cada algoritmo trae su prueba, su costo y un ejemplo trabajado; la ropa del profesor, la figura 22.9 de Cormen y el grafo G de la libreta son los grafos de trabajo.

Lo que se usa del DFS

Todo lo que sigue corre sobre el DFS con colores y tiempos de la sección 4 (algoritmo 4.9): un reloj global t que sube en uno cada vez que un vértice se descubre o se termina, los tiempos entra[u] y sale[u], el padre $\pi[u]$ y el color, que es blanco antes de entra[u], gris entre entra[u] y sale[u] y negro después. En cada instante los vértices grises son exactamente la pila de llamadas activas, una cadena de antecesores en el árbol DFS. Las pruebas de esta sección se apoyan en cuatro hechos de la sección 4:

- **Paréntesis.** Para dos vértices cualesquiera, los intervalos $[\text{entra}[u], \text{sale}[u]]$ y $[\text{entra}[v], \text{sale}[v]]$ son ajenos o uno contiene al otro.
- **Descendientes.** v es descendiente propio de u si y solo si $\text{entra}[u] < \text{entra}[v] < \text{sale}[v] < \text{sale}[u]$.
- **Camino blanco.** v es descendiente de u si y solo si, en el instante entra[u], hay un camino de u a v formado solo por vértices blancos.
- **Clasificación de aristas.** Al explorar (u, v), la arista es de árbol si v es blanco; de retroceso si v es gris (va a un antecesor de u, y los lazos cuentan aquí); de avance si v es negro y $\text{entra}[u] < \text{entra}[v]$; de cruce si v es negro y $\text{entra}[v] < \text{entra}[u]$. En un grafo no dirigido solo hay aristas de árbol y de retroceso.

Hace falta una notación más. Para un conjunto de vértices U se escribe $\text{entra}(U) = \min_{u \in U} \text{entra}[u]$ y $\text{sale}(U) = \max_{u \in U} \text{sale}[u]$: el primer descubrimiento y la última salida dentro de U. Cormen los llama $d(U)$ y $f(U)$. Con $u \rightsquigarrow v$ se indica que hay un camino de u a v; el camino de longitud cero cuenta, así que $u \rightsquigarrow u$ siempre.

Orden topológico

La clase del 22 de octubre afirma que un DAG siempre tiene un orden topológico, el orden en que se pueden cursar las materias del plan de estudios, pero no lo prueba. Aquí se prueba de dos maneras, y cada prueba trae su algoritmo lineal.

Definición 5.1 (Orden topológico)**.** Un **orden topológico** (*topological sort*) de un grafo dirigido $G = (V, E)$ es un orden lineal de todos sus vértices tal que, si $(u, v) \in E$, entonces u aparece antes que v.

Dibujados en fila según ese orden, todas las flechas apuntan a la derecha. El orden no es único: en un grafo sin aristas cualquier permutación sirve. Si el grafo tiene un ciclo no existe, porque cada vértice del ciclo tendría que ir antes que su sucesor y, dando la vuelta, antes que sí mismo. Las aplicaciones

son las de siempre: calendarizar tareas con precedencias, compilar módulos que dependen unos de otros, recalcular las celdas de una hoja de cálculo.

Ciclos y aristas de retroceso

Lema 5.2 (Cormen, lema 22.11)**.** *Un grafo dirigido G es acíclico si y solo si un DFS de G no produce aristas de retroceso.*

Demostración. Si hay una arista de retroceso (u, v), entonces v es antecesor de u en el bosque DFS, o $v = u$ si es un lazo. El camino de árbol de v a u seguido de la arista (u, v) es un ciclo.

Al revés, supóngase que G tiene un ciclo c. Sea v el primer vértice de c que se descubre y sea (u, v) la arista de c que llega a v. En el instante entra$[v]$ los demás vértices de c siguen blancos, así que el tramo de c que va de v a u es un camino blanco. Por el teorema del camino blanco, u queda como descendiente de v. Cuando el DFS explora (u, v), desde dentro de la llamada de u, la llamada de v sigue abierta: v es gris y la arista es de retroceso. □

El lema vale para cualquier DFS, con cualquier orden de los vértices en el ciclo exterior y en las listas. Basta una corrida para decidir si un grafo dirigido tiene ciclo, y la arista de retroceso, junto con los padres, entrega el ciclo mismo: se sube por π desde u hasta llegar a v. Es la versión dirigida de la regla de ciclos con BFS de la clase del 22 de octubre, que en dirigidos da falsos positivos.

El algoritmo por tiempos de salida

La idea cabe en una línea: correr DFS y listar los vértices en orden decreciente de sale. El teorema siguiente es la razón.

Teorema 5.3 (Cormen, teorema 22.12)**.** *Si G es un DAG, en cualquier DFS de G se cumple sale$[v] <$ sale$[u]$ para toda arista (u, v). En consecuencia, los vértices en orden decreciente de sale forman un orden topológico.*

Demostración. Considérese el momento en que el DFS explora (u, v). En ese momento u es gris. Hay tres casos según el color de v.

- v gris: v sería antecesor de u y la arista sería de retroceso, lo que el lema 5.2 descarta en un DAG.
- v blanco: v se vuelve descendiente de u y, por la caracterización de descendientes, sale$[v] <$ sale$[u]$.
- v negro: v ya terminó y sale$[v]$ ya tiene valor, mientras que u todavía no termina. El reloj solo avanza, así que sale$[v] <$ sale$[u]$.

En los dos casos posibles u termina después que v, y queda antes que v en el orden decreciente. □

No hace falta ordenar nada. Cada vértice que termina se inserta al frente de una lista: el primero en terminar queda hasta el final y el último en terminar queda al frente. El algoritmo 5.1 es el DFS del algoritmo 4.9 con dos agregados: esa inserción y la prueba del vértice gris, que Cormen no pone y que convierte al algoritmo en un detector de ciclos.

Algoritmo 5.1 Orden-Topológico(G), con DFS y detección de ciclos

```
1:  procedimiento Orden-Topológico(G)
2:      para todo u ∈ V hacer
3:          color[u] ← blanco,    π[u] ← nulo
4:      fin para
5:      t ← 0;  L ← lista vacía
6:      para todo u ∈ V hacer
7:          si color[u] = blanco entonces
8:              Visitar-Orden(G, u)
9:          fin si
10:     fin para
11:     devolver L
12: fin procedimiento

13: procedimiento Visitar-Orden(G, u)
14:     t ← t + 1,    entra[u] ← t,    color[u] ← gris
15:     para todo v en la lista de adyacencia de u hacer
16:         si color[v] = blanco entonces
17:             π[v] ← u;  Visitar-Orden(G, v)
18:         si no si color[v] = gris entonces                  // arista de retroceso
19:             error: hay un ciclo, no existe orden topológico
20:         fin si
21:     fin para
22:     color[u] ← negro,    t ← t + 1,    sale[u] ← t
23:     insertar u al frente de L
24: fin procedimiento
```

- **Línea 14.** El vértice se pinta de gris al descubrirlo y sigue gris mientras su llamada está en la pila.
- **Líneas 18 y 19.** Una arista hacia un vértice gris es de retroceso y, por el lema 5.2, hay ciclo. Sin estas dos líneas el algoritmo devuelve un orden de todos modos, sin avisar que es inválido.
- **Línea 23.** Insertar al frente equivale a ordenar por sale decreciente sin ordenar.
- Las aristas hacia vértices negros, de avance o de cruce, se ignoran: esos vértices ya terminaron y quedarán a la derecha de u.

Costo. La inicialización cuesta $\Theta(n)$. Visitar-Orden se llama exactamente una vez por vértice, porque solo se llama sobre vértices blancos y lo primero que hace es pintarlos de gris. Cada llamada recorre la lista de su vértice una vez; las listas suman m entradas y cada una cuesta $\mathcal{O}(1)$. Insertar al frente de una lista ligada es $\mathcal{O}(1)$. Total: $\Theta(n+m)$ con listas de adyacencia. Con matriz, encontrar los vecinos de cada vértice cuesta $\Theta(n)$ y el total sube a $\Theta(n^2)$.

Ejemplo 5.4 (La ropa del profesor, con DFS)**.** Es la figura 22.7 de Cormen. Nueve prendas, donde la arista (u, v) significa que u se pone antes que v: calzoncillos → pantalón, calzoncillos → zapatos, pantalón → zapatos, pantalón → cinturón, camisa → cinturón, camisa → corbata, corbata → saco, cinturón → saco y calcetines → zapatos. El reloj está aislado. $n = 9$ y $m = 9$.

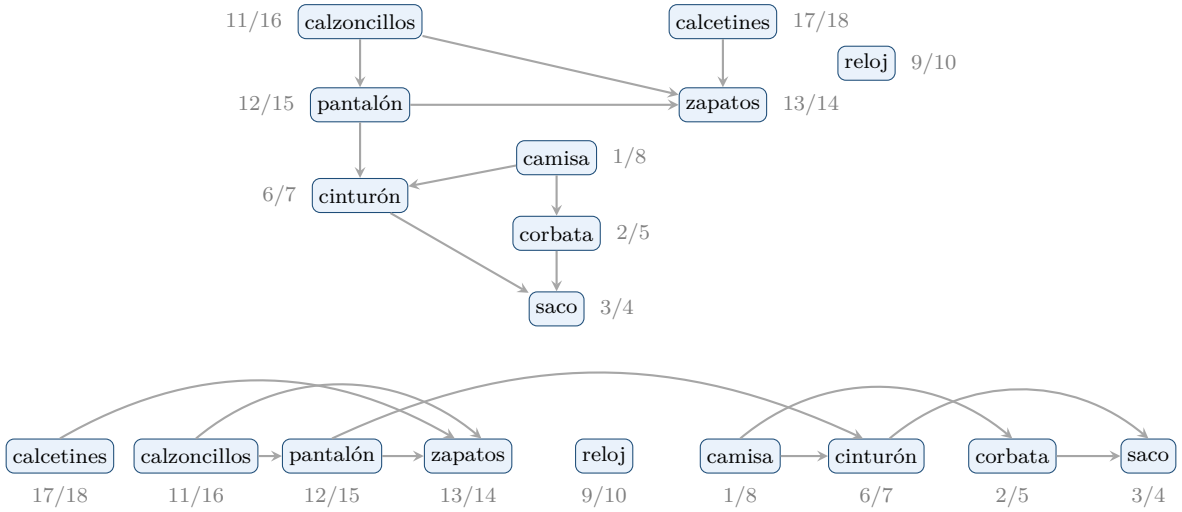
Cormen no dice en qué orden recorre los vértices; este orden reproduce exactamente sus tiempos. Ciclo exterior: camisa, corbata, saco, cinturón, reloj, calzoncillos, pantalón, zapatos, calcetines. En las listas,

camisa tiene a corbata antes que a cinturón, y pantalón tiene a zapatos antes que a cinturón.

- camisa (1) → corbata (2) → saco (3); saco termina (4) y corbata (5). De regreso en camisa se descubre cinturón (6), cuyo único vecino, saco, ya es negro: cinturón termina (7) y camisa (8).
- reloj se descubre y termina sin vecinos: 9 y 10.
- calzoncillos (11) → pantalón (12) → zapatos (13), que termina (14). Cinturón ya es negro, así que pantalón termina (15). Zapatos ya es negro; calzoncillos termina (16).
- calcetines: 17 y 18; su vecino zapatos ya es negro.

Ninguna arista llega a un vértice gris: es un DAG. La lista L crece por el frente:

sale	**Termina**	L **después (el frente a la izquierda)**
4	saco	saco
5	corbata	corbata, saco
7	cinturón	cinturón, corbata, saco
8	camisa	camisa, cinturón, corbata, saco
10	reloj	reloj, camisa, …
14	zapatos	zapatos, reloj, camisa, …
15	pantalón	pantalón, zapatos, reloj, …
16	calzoncillos	calzoncillos, pantalón, zapatos, …
18	calcetines	calcetines, calzoncillos, pantalón, zapatos, reloj, camisa, cinturón, corbata, saco



Arriba, el DAG con entra/sale junto a cada prenda. Abajo, las mismas prendas en orden decreciente de sale: calcetines, calzoncillos, pantalón, zapatos, reloj, camisa, cinturón, corbata, saco. Todas las flechas van a la derecha. El teorema 5.3 se comprueba arista por arista; por ejemplo, en cinturón → saco se tiene $7 > 4$, aunque cuando se exploró esa arista saco ya era negro.

Ojito...

Sin la prueba de gris (líneas 18 y 19) el algoritmo no avisa que hay ciclo: devuelve un orden inválido sin quejarse. Si a la ropa se le agrega la arista saco → camisa, absurda pero ilustrativa, el DFS con el mismo orden la explora cuando camisa sigue gris: es de retroceso, y cierra los ciclos camisa, corbata, saco, camisa y camisa, cinturón, saco, camisa. La versión de Cormen, que no tiene esa prueba, devuelve exactamente el mismo orden de antes, y ese orden pone a camisa antes que saco, en contra de la arista nueva.

El algoritmo de Kahn

La otra forma de construir el orden va de adelante hacia atrás. Un vértice sin aristas de entrada no depende de nada y puede ir primero. Se saca, se borran sus aristas y se repite con lo que queda. La clase del 22 de octubre lo menciona en el recuadro de ciclos con BFS, y Cormen lo deja como ejercicio (22.4-5). En lugar de borrar aristas se lleva un contador por vértice, faltan[v]: cuántos predecesores de v faltan por salir. Empieza en el grado de entrada $\text{gr}^-(v)$ y solo baja.

Algoritmo 5.2 Kahn(G), orden topológico por grados de entrada

```
1:  procedimiento Kahn(G)
2:      faltan[v] ← 0 para todo v ∈ V
3:      para todo u ∈ V hacer
4:          para todo v en la lista de adyacencia de u hacer
5:              faltan[v] ← faltan[v] + 1
6:          fin para
7:      fin para
8:      Q ← cola con todos los v tales que faltan[v] = 0;  L ← lista vacía
9:      mientras Q no está vacía hacer
10:         u ← Desencolar(Q);  agregar u al final de L
11:         para todo v en la lista de adyacencia de u hacer
12:             faltan[v] ← faltan[v] − 1                    // se borra la arista (u, v)
13:             si faltan[v] = 0 entonces
14:                 Encolar(Q, v)
15:             fin si
16:         fin para
17:     fin mientras
18:     si |L| < n entonces
19:         error: los vértices que no están en L están en un ciclo o debajo de uno
20:     fin si
21:     devolver L
22: fin procedimiento
```

- **Líneas 2 a 7.** Calculan los grados de entrada recorriendo todas las listas.
- **Línea 12.** No se borra nada de la estructura; decrementar el contador simula borrar la arista.
- **Líneas 18 y 19.** Si salen menos de n vértices, hay ciclo.

Proposición 5.5 (Correctitud de Kahn)**.** *Si G es un DAG,* Kahn *devuelve un orden topológico de sus n vértices. Si G tiene un ciclo, devuelve menos de n vértices, y ninguno de los vértices del ciclo está entre ellos.*

Demostración. En todo momento, faltan$[v]$ es el número de predecesores de v que no están en L, porque cada vértice que se agrega a L decrementa una vez el contador de cada uno de sus sucesores. Un vértice entra a la cola justo cuando su contador llega a cero, es decir, cuando todos sus predecesores ya están en L. Por lo tanto, si $(u, v) \in E$ y v está en L, u está antes que v.

Falta ver cuántos salen. Supóngase que la cola se vacía y queda un conjunto no vacío R de vértices fuera de L. Ningún $v \in R$ tiene faltan$[v] = 0$, porque habría entrado a la cola y estaría en L; así que todo vértice de R tiene un predecesor en R. Partiendo de cualquier vértice de R y retrocediendo de predecesor en predecesor sin salir de R, en a lo más $|R|$ pasos algún vértice se repite, y eso es un ciclo. En un DAG es imposible, así que $R = \emptyset$ y salen los n vértices.

Si G tiene un ciclo c, ningún vértice de c llega a L: el primero que llegara necesitaría a su predecesor en c ya en L. □

La prueba da una segunda demostración de que todo DAG tiene orden topológico, y de paso un hecho que se usa seguido: todo DAG no vacío tiene al menos un vértice con grado de entrada cero, una **fuente**, y por simetría uno con grado de salida cero, un **sumidero**.

Costo. Calcular los grados recorre todas las listas: $\Theta(n + m)$. Cada vértice entra a la cola a lo más una vez, porque su contador solo baja y pasa por cero una sola vez; son $\mathcal{O}(n)$ operaciones de cola de $\mathcal{O}(1)$. Cada vértice que sale recorre su lista una vez, $\mathcal{O}(m)$ en total. Suma: $\Theta(n + m)$. Si se quiere el orden lexicográficamente menor, la cola se cambia por un montículo de mínimos y el costo sube a $\mathcal{O}(n \log n + m)$.

Ejemplo 5.6 (La ropa del profesor, con Kahn)**.** Con cola FIFO y los empates en orden alfabético. Los grados de entrada iniciales son: zapatos 3; cinturón y saco 2; pantalón y corbata 1; calcetines, calzoncillos, camisa y reloj 0.

Paso	Sale	Contadores que bajan	Cola después
0			calcetines, calzoncillos, camisa, reloj
1	calcetines	zapatos $3 \to 2$	calzoncillos, camisa, reloj
2	calzoncillos	pantalón $1 \to \mathbf{0}$, zapatos $2 \to 1$	camisa, reloj, pantalón
3	camisa	cinturón $2 \to 1$, corbata $1 \to \mathbf{0}$	reloj, pantalón, corbata
4	reloj		pantalón, corbata
5	pantalón	cinturón $1 \to \mathbf{0}$, zapatos $1 \to \mathbf{0}$	corbata, cinturón, zapatos
6	corbata	saco $2 \to 1$	cinturón, zapatos
7	cinturón	saco $1 \to \mathbf{0}$	zapatos, saco
8	zapatos		saco
9	saco		

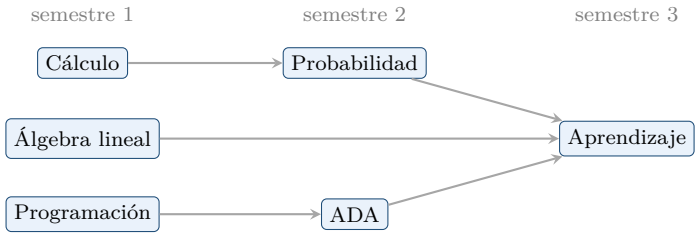
El orden de Kahn es calcetines, calzoncillos, camisa, reloj, pantalón, corbata, cinturón, zapatos, saco. Es distinto del de DFS y también es válido; el grafo tiene 1728 órdenes topológicos, contados enumerando las 9! permutaciones. La diferencia de estilo se nota: Kahn saca primero todo lo que no depende de nada, las cuatro fuentes, mientras que DFS se mete por una cadena completa antes de pasar a otra.

Con la arista saco → camisa agregada, Kahn saca calcetines, calzoncillos, reloj, pantalón y zapatos, y se detiene. Camisa, corbata, cinturón y saco nunca llegan a cero, y son justo los vértices de los dos ciclos.

Los dos algoritmos cuestan lo mismo y resuelven lo mismo, pero no son intercambiables en todo:

	DFS por sale	**Kahn**
Costo	$\Theta(n+m)$	$\Theta(n+m)$
Estructura	recursión o pila	cola y arreglo de contadores
Detecta ciclos	solo con la prueba de gris	siempre: salen menos de n
Si hay ciclo, reporta	una arista de retroceso; con π se reconstruye el ciclo	el conjunto de vértices bloqueados
Construye el orden	de atrás hacia adelante	de adelante hacia atrás
Recursión	hasta n niveles	ninguna
Lo que da de más	los tiempos que usan las componentes fuertes	los niveles: cada ronda es un lote de tareas paralelas

Kahn por niveles. Si en lugar de sacar un vértice a la vez se saca la ronda completa de vértices con contador cero, y solo después se decrementan los contadores, cada ronda es un conjunto de tareas que se pueden hacer en paralelo. El número de rondas es el mínimo posible para terminarlo todo, y es igual al número de vértices del camino más largo del DAG. Con el DAG de materias de la clase del 22 de octubre salen tres semestres:



En la primera ronda salen las tres materias sin requisitos. Al sacarlas, Probabilidad y ADA quedan libres y el contador de Aprendizaje baja de 3 a 2. En la segunda ronda salen esas dos y Aprendizaje llega a cero. El camino más largo, Cálculo, Probabilidad, Aprendizaje, tiene tres vértices: no hay manera de terminar en dos semestres.

Ojito...

Lo que Kahn deja fuera no es solo el ciclo: son los vértices que están en un ciclo *o debajo de uno*, alcanzables desde él. Con $a \to b$, $b \to a$ y $b \to c$, ningún vértice tiene grado de entrada cero y la cola nace vacía; c se queda fuera sin estar en ningún ciclo. Si se quiere el ciclo mismo, Kahn no lo da directo; DFS sí, con la arista de retroceso y los padres. Del lado de DFS, el riesgo es la recursión: en una cadena de 10^6 vértices el DFS recursivo anida 10^6 llamadas y revienta la pila (Python se detiene en 1000 por omisión). Kahn no tiene ese problema; un DFS para grafos grandes se escribe con pila explícita.

Observación. El orden topológico reaparece en la sección 7: relajar las aristas de un DAG en orden topológico da las distancias desde una fuente en $\Theta(n+m)$, incluso con pesos negativos (algoritmo 7.4), que es justo donde Dijkstra falla. Con los pesos negados da el camino más largo, la ruta crítica de un proyecto. Y toda recurrencia de programación dinámica de la libreta se evalúa en un orden topológico del DAG de sus subproblemas.

Componentes fuertemente conexas

En un grafo dirigido, que de u se llegue a v no dice nada sobre si de v se llega a u. Las componentes fuertemente conexas agrupan los vértices que sí se alcanzan en los dos sentidos. Al contraer cada una a un punto queda un DAG, el esqueleto del grafo: muchos algoritmos sobre grafos dirigidos empiezan por ahí, resuelven cada componente por separado y luego combinan siguiendo ese DAG en orden topológico.

Definición 5.7 (Componente fuertemente conexa)**.** Una **componente fuertemente conexa** (*strongly connected component*) de un grafo dirigido $G = (V, E)$ es un conjunto maximal $C \subseteq V$ tal que, para todo $u, v \in C$, se tiene $u \rightsquigarrow v$ y $v \rightsquigarrow u$.

"Se alcanzan mutuamente" es una relación de equivalencia: reflexiva por el camino de longitud cero, simétrica por definición y transitiva pegando caminos. Las componentes son sus clases, así que parten a V y cada vértice está en exactamente una. Un vértice solo forma una componente aunque no tenga lazo. El grafo es **fuertemente conexo** si tiene una sola componente.

Definición 5.8 (Grafo transpuesto y grafo de componentes)**.** El **transpuesto** de $G = (V, E)$ es $G^T = (V, E^T)$, con $E^T = \{(u, v) : (v, u) \in E\}$: las mismas aristas, volteadas. Si $C_1, \ldots, C_k$ son las componentes fuertemente conexas de G, el **grafo de componentes** G^{CFC} tiene un vértice c_i por cada C_i, y la arista (c_i, c_j), con $i \neq j$, si hay alguna arista $(x, y) \in E$ con $x \in C_i$ y $y \in C_j$. Cormen lo escribe G^{SCC}.

El algoritmo de dos DFS

El algoritmo, de Kosaraju y Sharir, corre dos DFS. El primero, sobre G, solo sirve para calcular los tiempos de salida. El segundo corre sobre G^T y toma las raíces en orden decreciente de esos tiempos. Cada árbol del segundo bosque es exactamente una componente.

Algoritmo 5.3 Componentes-Fuertes(G), dos DFS (Kosaraju)

```
1:  procedimiento Componentes-Fuertes(G)
2:      DFS sobre G (algoritmo 4.9);  L ← vértices en sale decreciente
3:      construir G^T;  comp[u] ← 0 para todo u ∈ V;  k ← 0      // 0: sin componente
4:      para todo u en el orden de L hacer                       // sale decreciente
5:          si comp[u] = 0 entonces
6:              k ← k + 1;  Marcar(u, k)
7:          fin si
8:      fin para
9:      devolver comp
10: fin procedimiento

11: procedimiento Marcar(u, k)                                   // DFS sobre G^T
12:     comp[u] ← k
13:     para todo v ∈ Adj^T[u] hacer
14:         si comp[v] = 0, Marcar(v, k)
15:     fin para
16: fin procedimiento
```

- **Línea 2.** Es el orden por sale del algoritmo 5.1, sin la prueba de ciclo (aquí los ciclos son el punto): cada vértice que termina se inserta al frente de L.
- **Línea 3.** Para construir G^T se crea una lista vacía $\text{Adj}^T[u]$ por vértice y, por cada arista (u, v) de G, se agrega u a $\text{Adj}^T[v]$.
- **Líneas 4 a 8.** El ciclo exterior del segundo DFS recorre L, o sea los vértices en sale decreciente. Cada llamada a Marcar desde aquí es la raíz de un árbol nuevo, una componente nueva, y $\text{comp}[u] = k$ dice a cuál pertenece u.

- Marcar es un DFS normal sobre G^T. No necesita gris, porque no importa la clasificación de aristas, solo quién es alcanzable; $\text{comp}[v] = 0$ hace de blanco.

Costo. El primer DFS cuesta $\Theta(n+m)$. Transponer cuesta $\Theta(n)$ para crear las listas vacías y $\Theta(m)$ para llenarlas. En el segundo DFS cada vértice se marca una vez y cada lista de G^T, que suman m, se recorre una vez: $\Theta(n+m)$. El orden por sale no cuesta un ordenamiento, porque la lista L se arma en $\mathcal{O}(1)$ por vértice; y aunque se ordenara, como $\text{sale} \in \{1, \ldots, 2n\}$, bastaría un ordenamiento por conteo en $\Theta(n)$. Total: $\Theta(n+m)$, con memoria extra $\Theta(n+m)$ por las listas de G^T.

Por qué funciona

Primero, dos hechos estructurales. G y G^T tienen exactamente las mismas componentes: voltear todas las aristas voltea todos los caminos, así que u y v se alcanzan mutuamente en G si y solo si se alcanzan mutuamente en G^T. Y entre dos componentes distintas los caminos van en un solo sentido:

Lema 5.9 (Cormen, lema 22.13)**.** *Sean $C \neq C'$ componentes, $u, v \in C$ y $u', v' \in C'$. Si hay un camino $u \rightsquigarrow u'$, no hay camino $v' \rightsquigarrow v$. En consecuencia, G^{CFC} es un DAG.*

Demostración. Si hubiera un camino $v' \rightsquigarrow v$, se tendría $u \rightsquigarrow u' \rightsquigarrow v'$ y $v' \rightsquigarrow v \rightsquigarrow u$, así que u y v' se alcanzarían mutuamente y estarían en la misma componente. Un ciclo en G^{CFC} daría caminos en los dos sentidos entre dos componentes distintas del ciclo, que es justo lo que se acaba de descartar. □

Todo lo demás descansa en un lema sobre los tiempos de salida de las componentes.

Lema 5.10 (Cormen, lema 22.14)**.** *Sean $C \neq C'$ componentes y $(u, v) \in E$ con $u \in C$ y $v \in C'$. Entonces sale$(C) >$ sale(C').*

Demostración. Hay dos casos, según cuál de las dos componentes toca primero el DFS.

Si $\text{entra}(C) < \text{entra}(C')$, sea x el primer vértice de C que se descubre. En el instante $\text{entra}[x]$ todos los vértices de $C \cup C'$ son blancos. Desde x hay un camino blanco a cada vértice de C y, pasando por la arista (u, v), a cada vértice de C'. Por el teorema del camino blanco todos quedan como descendientes de x, así que x es el último en terminar: $\text{sale}[x] = \text{sale}(C) > \text{sale}(C')$.

Si $\text{entra}(C) > \text{entra}(C')$, sea y el primer vértice de C' que se descubre. Por el mismo argumento, todo C' queda debajo de y y termina a más tardar en $\text{sale}[y] = \text{sale}(C')$. Durante el intervalo de y no se descubre ningún vértice de C: lo que se descubre ahí es alcanzable desde y, y por el lema 5.9 ningún vértice de C es alcanzable desde C', porque sí hay camino de C a C'. Como tampoco se descubrió ninguno antes de $\text{entra}[y]$, todo C se descubre después de $\text{sale}(C')$ y, por lo tanto, termina después. □

Corolario 5.11 (Cormen, corolario 22.15)**.** *Si $(u, v) \in E^T$ con $u \in C$, $v \in C'$ y $C \neq C'$, entonces sale$(C) <$ sale(C'). En palabras: en G^T, toda arista entre componentes va de la que terminó antes a la que terminó después.*

Demostración. $(u, v) \in E^T$ significa $(v, u) \in E$; se aplica el lema 5.10 con los papeles invertidos. □

Idea clave

Sea C la componente con sale(C) máximo, la que contiene al vértice que terminó al último en el primer DFS. Por el lema 5.10, ninguna arista de G entra a C desde otra componente, porque esa otra tendría sale mayor: C es una **fuente** de G^{CFC}. En G^T las flechas se voltean y C se vuelve un **sumidero**: un DFS que arranca en C recorre toda C y no puede salir. Ese primer árbol es exactamente C. Después se toma el vértice con sale máximo entre los que quedan; por el corolario 5.11, las aristas de G^T que salen de su componente solo pueden ir a componentes con sale mayor, que ya están marcadas. Cada árbol queda encerrado porque todas sus salidas en G^T llevan a árboles ya construidos. Por eso el orden es *decreciente*.

Teorema 5.12 (Cormen, teorema 22.16)**.** Componentes-Fuertes *calcula correctamente las componentes fuertemente conexas de* G.

Demostración. Por inducción sobre el número de árboles del segundo DFS. La hipótesis es que los primeros k árboles son componentes completas; con $k = 0$ no hay nada que probar. Sea u la raíz del árbol $k + 1$ y C su componente. Ningún vértice de C está marcado, porque los árboles anteriores son componentes completas distintas de C. Por la elección de las raíces, u tiene el mayor sale entre los vértices sin marcar, así que $\text{sale}[u] = \text{sale}(C) > \text{sale}(C')$ para toda componente $C' \neq C$ todavía sin marcar.

Todos los vértices de C se alcanzan desde u en G^T por caminos dentro de C, sin marcar, así que quedan en el árbol de u. Por otro lado, por el corolario 5.11, toda arista de G^T que sale de C llega a una componente con sale mayor que sale(C), que entonces no puede estar sin marcar: ya está marcada, y Marcar no entra ahí. Ningún vértice fuera de C entra al árbol, y el árbol es exactamente C. □

Hay otra forma de leer el resultado: el segundo DFS visita las componentes en orden topológico de G^{CFC}, de fuente a sumidero. Las componentes salen ya ordenadas.

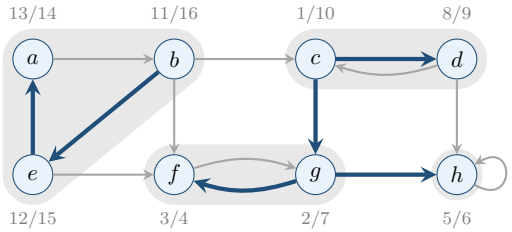
Ejemplo 5.13 (La figura 22.9 de Cormen)**.** Se usan las letras minúsculas de Cormen para no confundir con el grafo G de la libreta. $V = \{a, b, c, d, e, f, g, h\}$, $n = 8$ y $m = 14$:

$$E = \{(a,b),(b,c),(b,e),(b,f),(c,d),(c,g),(d,c),(d,h),\\ (e,a),(e,f),(f,g),(g,f),(g,h),(h,h)\}.$$

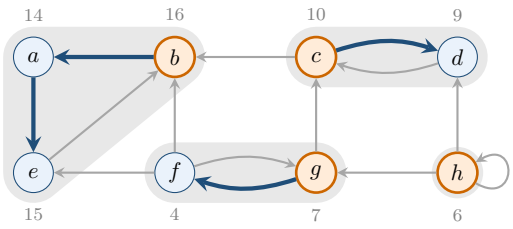
Primer DFS. Para reproducir los tiempos de la figura, el ciclo exterior empieza en c y el siguiente blanco es b; en la lista de c va g antes que d, y en la de g, f antes que h.

- c (1) $\to$ g (2) $\to$ f (3); $f \to g$ es de retroceso y f termina (4). $g \to h$ (5); el lazo $h \to h$ es de retroceso, h termina (6) y g termina (7).
- $c \to d$ (8); $d \to c$ es de retroceso y $d \to h$ de cruce; d termina (9) y c termina (10).
- b (11): $b \to c$ es de cruce; $b \to e$ (12) $\to$ a (13); $a \to b$ es de retroceso y a termina (14); $e \to f$ es de cruce y e termina (15); $b \to f$ es de cruce y b termina (16).

En sale decreciente, $L = b\,(16), e\,(15), a\,(14), c\,(10), d\,(9), g\,(7), h\,(6), f\,(4)$.



(a) G y el primer DFS



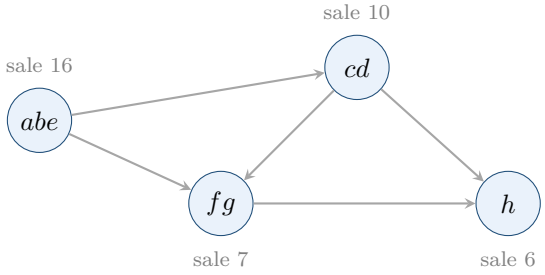
(b) G^T y el segundo DFS

En (a), las aristas de árbol del primer DFS van en azul grueso, las componentes sombreadas y entra/sale junto a cada vértice. En (b), G^T con el bosque del segundo DFS que se describe enseguida; las raíces b, c, g y h resaltadas, y junto a cada vértice su sale del primer DFS.

Segundo DFS, sobre G^T y en el orden de L. Las listas de G^T son a: e; b: a; c: b, d; d: c; e: b; f: b, e, g; g: c, f; h: d, g, h.

Árbol	**Raíz (sale)**	**Vértices**	**Aristas de G^T que topan con árboles anteriores**
1	b (16)	b, a, e	ninguna: en G^T nada sale de $\{a, b, e\}$
2	c (10)	c, d	$c \to b$
3	g (7)	g, f	$g \to c$, $f \to b$, $f \to e$
4	h (6)	h	$h \to d$, $h \to g$

e, a, d y f aparecen en L, pero cuando les toca ya están marcados. Las componentes son $\{a, b, e\}$, $\{c, d\}$, $\{f, g\}$ y $\{h\}$. La última columna es el corolario 5.11 en acción: cada arista de G^T que sale de un árbol choca con un árbol anterior, nunca con uno futuro. Al contraer cada componente queda el grafo de componentes:



(c) grafo de componentes G^{CFC}

En (c), el grafo de componentes con el sale de cada una: en cada arista el sale baja, que es el lema 5.10. Las componentes salieron en el orden abe, cd, fg, h, que es un orden topológico de G^{CFC}.

Ojito...

El primer DFS *no* respeta las componentes. En la figura (a), el árbol que empieza en c se lleva $\{c, d\}$, $\{f, g\}$ y $\{h\}$ juntos. Del primer DFS solo sirven los tiempos de salida; sus árboles no significan nada.
Las dos piezas del segundo DFS son necesarias, el transpuesto y el orden decreciente de sale, y los tiempos son siempre los del *primer* DFS. En el mismo grafo, quitar cualquiera de las dos falla:

- sobre G, sin transponer, en sale decreciente: desde b se alcanza todo y sale un solo árbol con los ocho vértices;
- sobre G^T en sale creciente: desde f salen $\{f, b, a, e, g, c, d\}$ y luego $\{h\}$;
- sobre G en sale creciente, la propuesta de Bacon: $\{f, g, h\}$, $\{c, d\}$, $\{a, b, e\}$, que junta a $\{f, g\}$ con $\{h\}$.

Observación. La tabla BFS contra DFS de la clase del 22 de octubre dice que DFS sirve para "componentes". En un grafo no dirigido, las componentes conexas son simplemente los árboles del bosque DFS. En un dirigido no: los árboles de un DFS mezclan componentes, como se acaba de ver, y hace falta el algoritmo de dos DFS. El grafo dirigido que la clase usa en Floyd es una sola componente fuerte, porque $1 \to 2 \to 3 \to 4 \to 1$ es un circuito que pasa por los cuatro vértices; su grafo de componentes es un solo punto.

Observación. El algoritmo de Tarjan encuentra las mismas componentes con un solo DFS, sin construir G^T, usando una pila de vértices y un valor muy parecido al low$[v]$ que aparece más abajo. Kosaraju es más fácil de explicar y de probar; Tarjan hace un solo recorrido. Una aplicación clásica de las componentes fuertes es 2-SAT: una fórmula en forma normal conjuntiva con dos literales por cláusula es satisfacible si y solo si ninguna variable queda en la misma componente que su negación en el grafo de implicaciones.

Puntos de articulación, puentes y componentes biconexas

Esta parte es sobre grafos *no dirigidos*. La sección 2 define los dos conceptos de fragilidad de un grafo conexo. Un **punto de articulación** (*articulation point*; allá, vértice de corte) es un vértice cuya eliminación, junto con sus aristas, desconecta el grafo. Un **puente** (*bridge*) es una arista cuya eliminación lo desconecta. Allá también se prueba que una arista es puente si y solo si no está en ningún ciclo. En una red de computadoras, un punto de articulación es un servidor cuya caída parte la red en dos. Decidirlo con la definición cuesta un recorrido por cada vértice, $\mathcal{O}(n(n+m))$. Aquí se hace todo en un solo DFS (Cormen, problema 22-2).

Definición 5.14 (Componente biconexa)**.** Una **componente biconexa** (*biconnected component*) de un grafo no dirigido es un conjunto maximal de aristas tal que dos aristas cualesquiera del conjunto están en un ciclo común.

La relación "$e = e'$, o e y e' están en un ciclo común" es de equivalencia sobre las aristas. Lo delicado es la transitividad, que se prueba recombinando dos ciclos que comparten una arista (Cormen, problema 22-2g). Sus clases son las componentes biconexas, así que parten a E.

Observación (Convención para los puentes)*.* Un puente no está en ningún ciclo, así que su clase es él solo. En esta libreta **cada puente es una componente biconexa de una sola arista**. Es la convención de la teoría de grafos (los *bloques*), cumple la definición de arriba por vacuidad y es lo que produce el algoritmo de la pila: con ella las componentes biconexas parten a todas las aristas, sin excepción, y los puntos de articulación son exactamente los vértices que tocan dos o más componentes. Cormen, en la figura 22.10 y en el inciso (g) del problema, deja los puentes fuera de toda componente biconexa.

Para obtener su versión basta descartar las componentes de una sola arista, que son exactamente los puentes.

Criterios con el árbol DFS

Sea G no dirigido y conexo, y sea T el árbol de un DFS desde una raíz r. Todo lo que sigue depende de un hecho de la sección 4: en un no dirigido no hay aristas de avance ni de cruce, así que toda arista que no es del árbol une a un vértice con un antecesor suyo. El **subárbol** de un vértice v son v y sus descendientes en T.

Proposición 5.15 (Regla de la raíz)**.** *La raíz r es punto de articulación si y solo si tiene al menos dos hijos en T.*

Demostración. Si r tiene dos hijos s_1 y s_2, toda arista que sale del subárbol de s_1 llega a un antecesor de s_1 fuera del subárbol, y el único es r. Sin r, ninguna arista une el subárbol de s_1 con el de s_2, y el grafo se desconecta. Si r tiene un solo hijo s, el subárbol de s contiene a todos los demás vértices, porque el DFS desde r alcanza todo el grafo conexo, y sigue conexo por sus aristas de árbol al quitar r. □

Proposición 5.16 (Vértices que no son la raíz)**.** *Un vértice $v \neq r$ es punto de articulación si y solo si tiene un hijo s tal que ninguna arista de retroceso sale del subárbol de s hacia un antecesor propio de v.*

Demostración. Si existe ese hijo s, las aristas que salen del subárbol de s llegan a antecesores de s fuera del subárbol, o sea a v o a antecesores propios de v. Por hipótesis ninguna llega a estos últimos, así que todas llegan a v. Al quitar v, el subárbol de s queda separado de r, que existe porque $v \neq r$.

Si cada hijo s de v tiene una arista de retroceso desde su subárbol hasta un antecesor propio de v, al quitar v queda la parte de T fuera del subárbol de v, que sigue conexa por aristas de árbol y contiene a r. Cada subárbol de un hijo sigue conexo por dentro y queda colgado de esa parte por su arista de retroceso. El grafo sigue conexo. □

Revisar si desde cada subárbol hay una arista que escapa por encima de v se vuelve una comparación de números con el valor siguiente.

Definición 5.17 (El valor low)**.** Para cada vértice v, low[v] es el mínimo entre entra[v] y los entra[w] de todas las aristas de retroceso $\{x, w\}$ con x en el subárbol de v y w antecesor de x. Es el descubrimiento más antiguo al que se puede saltar desde el subárbol de v con una sola arista de retroceso. Cormen lo escribe $v.low$.

Como el subárbol de v es v junto con los subárboles de sus hijos,

$$\text{low}[v] = \min\Big(\text{entra}[v],\ \min_{\{v,w\} \text{ de retroceso}} \text{entra}[w],\ \min_{s \text{ hijo de } v} \text{low}[s]\Big),$$

y se calcula en postorden dentro del mismo DFS: al regresar de cada hijo, su low ya es definitivo.

Corolario 5.18 (Criterio de articulación con low)**.** *Un vértice $v \neq r$ es punto de articulación si y solo si tiene un hijo s con low[s] $\geq$ entra[v].*

Demostración. Los antecesores propios de v tienen entra menor que entra[v]. Una arista de retroceso desde el subárbol de s llega a un antecesor de su extremo de abajo, y ese antecesor está dentro del subárbol de s (con entra mayor que entra[v]), es v, o es un antecesor propio de v. Por lo tanto, alguna llega a un antecesor propio de v si y solo si low[s] $<$ entra[v]. Se aplica la proposición 5.16. □

Proposición 5.19 (Criterio de puente)**.** *Una arista de retroceso nunca es puente. Una arista de árbol* $\{u, v\}$*, con* $u = \pi[v]$*, es puente si y solo si* $low[v] > entra[u]$*, lo que equivale a* $low[v] = entra[v]$*.*

Demostración. Una arista de retroceso cierra un ciclo con el camino del árbol entre sus extremos, y un puente no está en ningún ciclo. Para la arista de árbol $\{u, v\}$: si una arista de retroceso $\{x, w\}$ sale del subárbol de v hacia u o más arriba, el camino de árbol de w a u, la arista $\{u, v\}$, el camino de árbol de v a x y la arista $\{x, w\}$ forman un ciclo, y $\{u, v\}$ no es puente. Si ninguna sale, la única arista que une el subárbol de v con el resto es $\{u, v\}$, y quitarla lo separa. "Alguna sale hacia u o más arriba" es exactamente $\text{low}[v] \leq \text{entra}[u]$.

La equivalencia final se debe a que $\text{low}[v]$ es el entra de un antecesor de algún vértice del subárbol: o está en el subárbol, con entra $\geq \text{entra}[v]$, o es u o algo más arriba, con entra $\leq \text{entra}[u]$. No hay valores intermedios. □

Las componentes biconexas salen en el mismo recorrido con una pila de aristas. Cada arista se apila la primera vez que se explora. Cuando, al regresar del hijo v a u, se cumple $\text{low}[v] \geq \text{entra}[u]$, nada del subárbol de v salta por encima de u: las aristas apiladas desde $\{u, v\}$ hasta el tope son las del subárbol de v que todavía no tienen componente, junto con las de retroceso que llegan a u, y forman una componente completa. Se desapilan juntas.

El algoritmo

El algoritmo 5.4 junta los tres criterios en un solo DFS. No necesita colores: $\text{entra}[v] = 0$ indica que v no se ha visto, y en un no dirigido no hace falta distinguir gris de negro. El reloj avanza también al terminar, como en el algoritmo 4.9, para que los tiempos coincidan con los de la libreta. Cormen solo pide los criterios y la idea en $\mathcal{O}(m)$; el pseudocódigo es de esta libreta.

Algoritmo 5.4 Articulaciones(G): puntos de articulación, puentes y componentes biconexas

```
1:  procedimiento Articulaciones(G)                                  // G no dirigido
2:      entra[u] ← 0,  hijos[u] ← 0,  π[u] ← nulo, para todo u ∈ V
3:      t ← 0;  P ← pila de aristas vacía
4:      para todo r ∈ V hacer
5:          si entra[r] = 0 entonces                                 // r es raíz de un árbol nuevo
6:              Visitar-Low(r)
7:              si hijos[r] ≥ 2, marcar r como punto de articulación  // regla de la raíz
8:          fin si
9:      fin para
10: fin procedimiento

11: procedimiento Visitar-Low(u)
12:     t ← t + 1,    entra[u] ← t,    low[u] ← t
13:     para todo v vecino de u hacer
14:         si entra[v] = 0 entonces                                 // arista de árbol
15:             π[v] ← u,    hijos[u] ← hijos[u] + 1;  apilar {u, v} en P
16:             Visitar-Low(v)
17:             low[u] ← mín(low[u], low[v])
18:             si low[v] ≥ entra[u] entonces                        // nada debajo de v salta arriba de u
19:                 si π[u] ≠ nulo, marcar u como punto de articulación
20:                 desapilar de P hasta {u, v} inclusive: es una componente biconexa
21:             fin si
22:             si low[v] > entra[u], {u, v} es puente
23:         si no si v ≠ π[u] y entra[v] < entra[u] entonces          // retroceso
24:             apilar {u, v} en P
25:             low[u] ← mín(low[u], entra[v])
26:         fin si
27:     fin para
28:     t ← t + 1,    sale[u] ← t
29: fin procedimiento
```

- **Líneas 14 a 22.** Arista de árbol. Al regresar de v ya se conoce low[v], y la línea 17 lo propaga a u.
- **Línea 18.** low[v] ≥ entra[u] dice que nada del subárbol de v salta por encima de u. Si u no es raíz, eso lo hace articulación (corolario 5.18). En la raíz la condición se cumple siempre, porque nada está por encima de ella; por eso la raíz usa la regla de los dos hijos (línea 7). Las componentes biconexas sí se desapilan también en la raíz.
- **Línea 22.** Desigualdad estricta. Con igualdad, el subárbol de v regresa justo a u por otra arista, así que $\{u, v\}$ está en un ciclo.
- **Línea 23.** $v \neq \pi[u]$ excluye la arista por la que se llegó, que vista desde u no es de retroceso. La condición entra[v] < entra[u] hace que cada arista de retroceso se apile una sola vez, desde el extremo de abajo: cuando el antecesor la ve desde arriba, v ya es un descendiente terminado con entra mayor.
- **Línea 25.** Se usa entra[v], no low[v]. Con low[v] el algoritmo pierde articulaciones, como se ve en el ejemplo siguiente.

Costo. Es un DFS sobre un grafo no dirigido: cada vértice se visita una vez y cada lista se recorre una vez, y las listas suman $2m$ entradas. Cada arista se apila exactamente una vez, las de árbol al descubrirlas y las de retroceso solo desde abajo, y se desapila una vez. Total: $\Theta(n+m)$, que es $\mathcal{O}(m)$ en un grafo conexo porque ahí $n \leq m+1$.

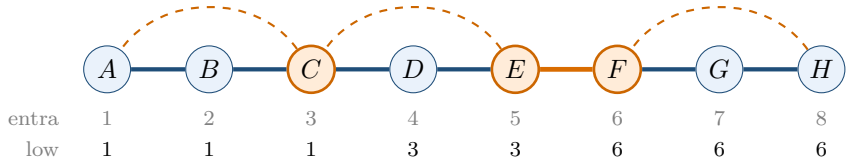
Ejemplo 5.20 (Un moño con cola)**.** El grafo H está hecho para que aparezcan todos los casos. $V = \{A, \ldots, H\}$, $n = 8$, $m = 10$:

$$E = \{\{A,B\},\{A,C\},\{B,C\},\{C,D\},\{C,E\},\{D,E\},\{E,F\},\{F,G\},\{F,H\},\{G,H\}\}.$$

Son dos triángulos que comparten el vértice C, un moño, más un puente $\{E,F\}$ y un tercer triángulo colgado de F.

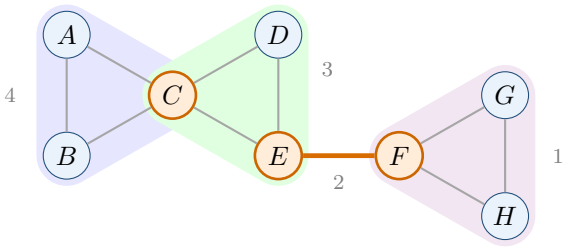
El DFS empieza en A, con los vecinos en orden alfabético. El árbol resulta ser un camino, $A, B, C, \ldots, H$, así que entra va de 1 a 8. Las aristas de retroceso son $\{C,A\}$, $\{E,C\}$ y $\{H,F\}$. La tabla está ordenada por entra para seguir el árbol, pero se llena en postorden, de H hacia arriba:

u	entra	π	**Retroceso**	**Prueba con el hijo** s	low
A	1			raíz con un solo hijo: no	1
B	2	A		low$[C] = 1 < 2$: no	1
C	3	B	a A, entra 1	low$[D] = 3 \geq 3$: **sí**	1
D	4	C		low$[E] = 3 < 4$: no	3
E	5	D	a C, entra 3	low$[F] = 6 \geq 5$: **sí**, y $6 > 5$: puente	3
F	6	E		low$[G] = 6 \geq 6$: **sí**	6
G	7	F		low$[H] = 6 < 7$: no	6
H	8	G	a F, entra 6	sin hijos: no	6



El árbol DFS tendido en línea, con las aristas de retroceso punteadas y las filas de entra y low. Resultado: puntos de articulación C, E y F, resaltados, y un solo puente, $\{E,F\}$, en naranja. C es el caso fino: low$[D]$ = entra$[C] = 3$, así que C es articulación pero $\{C,D\}$ no es puente, porque la arista $\{E,C\}$ regresa justo a C.

Las componentes biconexas salen de la pila en este orden: $\{FG, GH, HF\}$ al regresar de G a F; $\{EF\}$, el puente, al regresar de F a E; $\{CD, DE, EC\}$ al regresar de D a C; y $\{AB, BC, CA\}$ al regresar de B a A.



Las cuatro componentes, numeradas en el orden en que salen de la pila; la 2 es el puente. Las articulaciones son justo los vértices que tocan dos componentes. Todo coincide con la fuerza bruta: quitar cada vértice y cada arista y revisar si lo que queda sigue conexo.

Si el DFS arrancara en C, entraría en juego la regla de la raíz: $C \to A \to B$, y $\{B, C\}$ es de retroceso; de vuelta en C, B ya está visto y el recorrido sigue $C \to D \to E \to F \to G \to H$. La raíz C queda con dos hijos, A y D, y es articulación por la proposición 5.15. El resultado es el mismo: C, E y F, con el puente $\{E, F\}$.

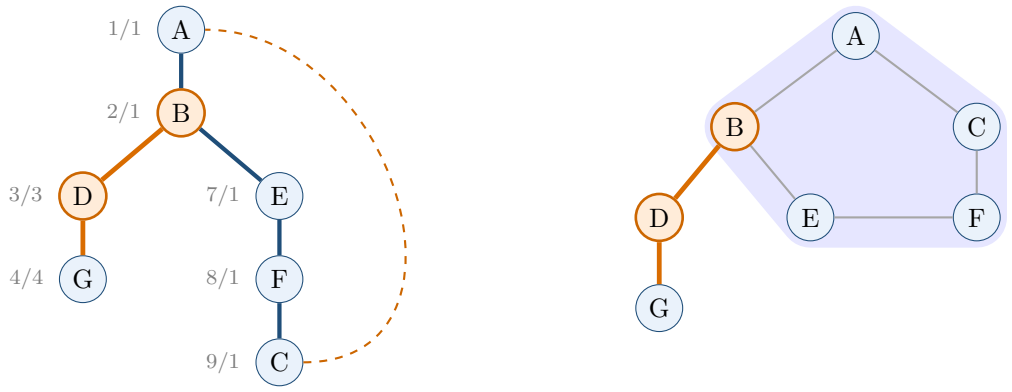
Ojito...

En la línea 25, la arista de retroceso aporta entra$[v]$, no low$[v]$. Es un error muy común, y en H se nota: cuando E ve su arista de retroceso hacia C, low$[C]$ ya vale 1 por la arista $\{C, A\}$, aunque C todavía no termina. Con low$[C]$, los valores de E y de D bajan a 1, la prueba en C da $1 < 3$ y el algoritmo *pierde la articulación* C. El low de un antecesor gris todavía no es definitivo y, peor, describe saltos que no pasan por la arista en cuestión. Para puentes el error no se nota; para articulaciones sí: en 3000 grafos conexos aleatorios de 3 a 9 vértices, la variante errónea cambió las articulaciones en 25 y los puentes en ninguno.

Ejemplo 5.21 (El grafo G de la libreta)**.** La clase del 22 de octubre ya hizo un DFS sobre G desde A, con los vecinos en orden alfabético, y obtuvo A 1/14, B 2/13, D 3/6, G 4/5, E 7/12, F 8/11 y C 9/10, con una sola arista de retroceso, $\{C, A\}$. Con esa misma traza:

- low$[C]$ = entra$[A] = 1$, y ese 1 sube por F, E, B y A: todos tienen low $= 1$.
- G no tiene aristas de retroceso, así que low$[G]$ = entra$[G] = 4$, y low$[D] = \min(3, 4) = 3$.
- B es articulación por su hijo D: low$[D] = 3 \geq$ entra$[B] = 2$. D lo es por su hijo G: $4 \geq 3$. Los demás no: E y F tienen hijos con low $= 1$, y la raíz A tiene un solo hijo.
- Puentes: $\{B, D\}$, porque low$[D] = 3 > 2$, y $\{D, G\}$, porque $4 > 3$.

Las componentes biconexas salen en el orden $\{DG\}$, $\{BD\}$ y $\{AB, BE, EF, FC, CA\}$: los dos puentes y el ciclo A, B, E, F, C, A que la clase encontró con BFS.



A la izquierda, el árbol DFS de la clase con entra/low junto a cada vértice y la arista de retroceso punteada. A la derecha, G con sus tres componentes biconexas: el ciclo sombreado y los dos puentes en naranja. Los puntos de articulación, resaltados, son los vértices donde se tocan.

Ojito...

Tres detalles más que rompen el algoritmo si se olvidan.

- **La arista al padre.** Si no se excluye la arista por la que se llegó a u, siempre $\text{low}[u] \leq \text{entra}[\pi[u]]$ y no se encuentra ningún puente. En un multigrafo hay que excluir *esa arista*, por identificador, no al vértice padre: dos aristas paralelas entre u y v forman un ciclo y ninguna es puente.
- **La raíz.** La prueba $\text{low}[s] \geq \text{entra}[r]$ se cumple siempre en la raíz. Aplicarla ahí declara articulación a toda raíz; por eso la raíz usa la regla de los dos hijos.
- **Grafos no conexos.** El ciclo exterior de Articulaciones arranca un DFS en cada componente conexa, y cada raíz usa su propia regla. En ese caso "desconecta" se lee como "aumenta el número de componentes conexas".

Recorrido de Euler en grafos dirigidos

La sección 2 trata el caso no dirigido: un multigrafo conexo tiene circuito euleriano si y solo si todos sus grados son pares, y el algoritmo de Hierholzer lo construye en $\Theta(n+m)$. El problema 22-3 de Cormen pide la versión dirigida. Un **recorrido de Euler** (*Euler tour*) de un grafo dirigido es un circuito que usa cada arista exactamente una vez. Puede repetir vértices, así que no es un ciclo en el sentido de esta libreta, aunque Cormen lo llame así.

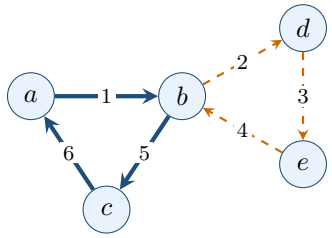
Teorema 5.22 (Cormen, problema 22-3)**.** *Un grafo dirigido fuertemente conexo tiene recorrido de Euler si y solo si $gr^-(v) = gr^+(v)$ para todo vértice v.*

Es el teorema 2.31 de la sección 2 en la forma de Cormen. Allá basta la conexidad débil, porque con los grados balanceados un grafo débilmente conexo resulta fuertemente conexo.

Idea de la prueba. Cada paso del recorrido por v entra por una arista y sale por otra, y cada arista se usa una vez: los grados quedan balanceados. Al revés, se camina desde s por aristas no usadas mientras se pueda. Al llegar a un $v \neq s$ se ha entrado a v una vez más de las que se ha salido, y como sus grados están balanceados le queda una salida libre: la caminata solo puede atorarse en s, y al hacerlo cierra un circuito. Si sobran aristas, por conexidad alguna sale de un vértice x del circuito; en lo que sobra los grados siguen balanceados, así que desde x sale otro circuito, que se empalma en x. Es el argumento del caso no dirigido con "grado par" cambiado por "entra tanto como sale". □

Hierholzer funciona igual siguiendo solo las aristas de salida: una pila guarda el camino actual, se avanza mientras el tope tenga aristas sin usar, y cuando se atora se desapila e inserta al frente del recorrido. Cada arista se usa una vez: $\Theta(n+m)$.

Ejemplo 5.23 (Un empalme)**.** Con las aristas $a \to b$, $b \to c$, $c \to a$, $b \to d$, $d \to e$ y $e \to b$, el vértice b tiene grados de entrada y salida $(2,2)$ y los demás $(1,1)$, y el grafo es fuertemente conexo. En la lista de b va c antes que d, para que el primer intento se atore antes de usarlo todo.



Desde a, la pila crece a, b, c, a y se atora en a sin haber usado $b \to d$. Al desapilar (a, luego c) se regresa a b, que todavía tiene aristas, y desde ahí se hace el desvío b, d, e, b. Luego todo se atora y se desapila. El recorrido final es a, b, d, e, b, c, a: el desvío quedó empalmado en b. En azul, el primer circuito que se encontró; en naranja punteado, el desvío; los números dan la posición de cada arista en el recorrido final.

Ojito...

Sin conexidad fuerte el balance no basta: dos ciclos ajenos están balanceados y no tienen recorrido de Euler. La condición general es el balance más que todas las aristas estén en una misma componente fuertemente conexa; los vértices aislados no importan. Y el grafo dirigido de Floyd de la clase del 22 de octubre, aunque es fuertemente conexo, no tiene recorrido de Euler: el vértice 1 tiene grado de entrada 3 y de salida 2.

Alcanzabilidad mínima

El problema 22-4 de Cormen es un ejercicio sobre la dirección de las aristas. Cada vértice u de un grafo dirigido tiene una etiqueta distinta $L(u) \in \{1, \ldots, n\}$, y se quiere, para cada u, el vértice mín$[u]$ de etiqueta mínima entre los alcanzables desde u, incluido u. Correr un recorrido desde cada vértice costaría $\mathcal{O}(n(n+m))$.

La clave es que $u \rightsquigarrow v$ en G si y solo si $v \rightsquigarrow u$ en G^T. Se procesan los vértices en orden *creciente* de etiqueta, y desde cada v todavía sin asignar se corre un DFS en G^T que asigna mín$[u] = v$ a todo vértice sin asignar que alcance.

Algoritmo 5.5 Mínimos-Alcanzables(G, L), problema 22-4

```
1:  procedimiento Mínimos-Alcanzables(G, L)
2:      construir G^T
3:      para todo u ∈ V hacer
4:          vert[L(u)] ← u,    mín[u] ← nulo          // ordena por L en Θ(n)
5:      fin para
6:      para i ← 1 hasta n hacer
7:          v ← vert[i];  si mín[v] = nulo, Asignar(v, v)
8:      fin para
9:      devolver mín
10: fin procedimiento

11: procedimiento Asignar(u, v)                        // DFS sobre G^T
12:     mín[u] ← v
13:     para todo w ∈ Adj^T[u] hacer
14:         si mín[w] = nulo, Asignar(w, v)
15:     fin para
16: fin procedimiento
```

Por qué funciona. Si u sigue sin asignar cuando le toca a v, ningún vértice de etiqueta menor es alcanzable desde u; si además $u \rightsquigarrow v$, entonces v es su mínimo. Detener el DFS en los vértices ya asignados no pierde nada: si w fue asignado por v', con $L(v') < L(v)$, todo lo que alcanza a w alcanza a v' y ya tiene un mínimo de etiqueta a lo más $L(v')$. Por la misma razón, el camino $u \rightsquigarrow v$ de un u sin asignar pasa solo por vértices sin asignar, y el DFS lo encuentra.

Costo. G^T cuesta $\Theta(n+m)$. Invertir L cuesta $\Theta(n)$, porque es una biyección con $\{1,\dots,n\}$ y no hace falta comparar. Asignar se llama una vez por vértice y cada lista de G^T se recorre una vez. Total: $\Theta(n+m)$.

Ejemplo 5.24 (Mínimos alcanzables)**.** Con las aristas $p \to q$, $q \to p$, $q \to r$, $r \to s$, $s \to r$, $t \to s$, $t \to u$ y las etiquetas $L(p)=1$, $L(s)=2$, $L(u)=3$, $L(r)=4$, $L(q)=5$, $L(t)=6$:

L	**Vértice**	**¿Asignado?**	**El DFS en G^T asigna**
1	p	no	p, q
2	s	no	s, r, t (de r se llegaría a q, ya asignado)
3	u	no	u (su vecino en G^T, t, ya está asignado)
$4, 5, 6$	r, q, t	sí	nada

Resultado: mín$[p]$ = mín$[q] = p$; mín$[r]$ = mín$[s]$ = mín$[t] = s$; mín$[u] = u$. El caso interesante es t: alcanza a s, r y u, con etiquetas 2, 4 y 3, y su mínimo es s aunque u esté a un solo paso.

Ojito...

Aquí importa la dirección dos veces. El DFS va sobre G^T, porque se busca quién *alcanza* a v, y en orden *creciente* de etiqueta. Sobre G se calcularía otra cosa: el mínimo de los vértices que alcanzan a cada uno.

Resumen

Problema	**Idea**	**Costo**
Orden topológico	DFS; vértices en sale decreciente; un vértice gris delata un ciclo	$\Theta(n+m)$
Orden topológico	Kahn: sacar vértices con grado de entrada cero; si salen menos de n, hay ciclo	$\Theta(n+m)$
Componentes fuertes	DFS en G para los tiempos; DFS en G^T con raíces en sale decreciente	$\Theta(n+m)$
Articulaciones, puentes y biconexas	un DFS con low; articulación si un hijo cumple low$[s] \geq$ entra$[v]$, puente si low$[v] >$ entra$[u]$; pila de aristas para las componentes	$\Theta(n+m)$
Recorrido de Euler dirigido	fuertemente conexo y balanceado; Hierholzer con aristas de salida	$\Theta(n+m)$
Mínimo alcanzable	DFS en G^T en orden creciente de etiqueta	$\Theta(n+m)$

6. Árboles de expansión mínima

Complemento: Cormen, cap. 23

Resumen rápido. La clase del 22 de octubre presentó Kruskal con un ejemplo y dejó a Prim en una observación. Esta sección da la teoría que los sostiene y las dos implementaciones completas. Todo descansa en un teorema, la propiedad del corte: si se parten los vértices en dos lados sin separar ninguna arista ya elegida, la arista más ligera que cruza de un lado al otro se puede agregar sin perder la optimalidad. De ahí salen un método genérico, la propiedad del ciclo y la unicidad del árbol cuando los pesos son distintos. Kruskal y Prim son dos maneras de escoger el corte. Kruskal usa conjuntos disjuntos y cuesta $\mathcal{O}(m \log n)$, dominado por el ordenamiento; Prim usa una cola de prioridad y cuesta $\mathcal{O}(m \log n)$ con montículo binario, $\mathcal{O}(m + n \log n)$ con montículo de Fibonacci y $\Theta(n^2)$ con un arreglo. Cierra la sección Prim contra Dijkstra sobre el grafo pesado de la libreta.

El problema y el método genérico

La clase del 22 de octubre (sección 1) definió el problema: en un grafo $G = (V, E)$ no dirigido y conexo, con un peso $w : E \to \mathbb{R}$ en cada arista, se busca un conjunto $T \subseteq E$ acíclico que conecte todos los vértices y minimice

$$w(T) = \sum_{\{u,v\} \in T} w(u, v).$$

Un conjunto así es un árbol y tiene $n - 1$ aristas, igual que cualquier otro árbol de expansión. Lo que se minimiza no es el número de aristas, que no cambia, sino su peso: el nombre completo es **árbol de expansión de peso mínimo**, y se abrevia MST por *minimum spanning tree*. Los pesos son reales cualesquiera, negativos incluidos; a diferencia de Dijkstra, aquí nada depende de su signo.

Dos convenciones para esta sección. Las letras e y f denotan aristas, y $T - f + e$ abrevia $(T \setminus \{f\}) \cup \{e\}$, para no apilar llaves dentro de llaves. Los hechos sobre árboles que usan las pruebas son las caracterizaciones de la sección 3 (Cormen, teorema B.2); el lema 6.4 reúne los tres que hacen falta.

Aristas seguras

Kruskal y Prim son voraces y agregan una arista a la vez. Lo que tienen en común es un invariante sobre el conjunto A de aristas elegidas hasta el momento: *antes de cada iteración, A es subconjunto de algún árbol de expansión mínima.*

Definición 6.1 (Arista segura)**.** Sea $A \subseteq E$ un subconjunto de algún MST de G. Una arista $e \notin A$ es **segura** para A (*safe edge*) si $A \cup \{e\}$ también es subconjunto de algún MST.

Con esa definición el algoritmo cabe en cuatro líneas, aunque todavía no se pueda ejecutar:

Algoritmo 6.1 MST-Genérico(G, w)

```
1: procedimiento MST-Genérico(G, w)
2:     A ← ∅
3:     mientras A no forma un árbol de expansión hacer
4:         encontrar una arista e segura para A
5:         A ← A ∪ {e}
6:     fin mientras
7:     devolver A
8: fin procedimiento
```

La línea 4 no dice cómo encontrar la arista, y es justo donde Kruskal y Prim se separan. Escribir

primero el método genérico tiene una ventaja: la corrección se prueba una sola vez, aquí, y a cada algoritmo concreto solo le queda mostrar que la arista que escoge es segura.

Por qué funciona. Al inicio $A = \emptyset$, que está contenido en cualquier MST. Cada iteración agrega una arista segura, que por definición conserva el invariante. Al salir del ciclo A es un árbol de expansión contenido en un MST T; los dos tienen $n-1$ aristas, así que $A = T$. El ciclo termina porque cada vuelta agrega una arista a un conjunto que, al vivir dentro de un árbol, no pasa de $n-1$ aristas. Falta ver que la línea 4 siempre tiene algo que encontrar: el invariante da un MST T con $A \subseteq T$, y como A todavía no es árbol de expansión, la contención es estricta. Cualquier arista de $T \setminus A$ es segura, porque A con ella sigue dentro de T.

Ese argumento muestra que la arista existe, pero la encuentra dentro de un MST que nadie conoce. Reconocer una arista segura sin conocer el árbol es el trabajo del teorema central de la sección.

Cortes y la propiedad del corte

Definición 6.2 (Corte, cruce, arista ligera)**.** Un **corte** $(S, V-S)$ de G (*cut*) es una partición de V en dos conjuntos. La arista $\{u,v\}$ **cruza** el corte (*crosses*) si uno de sus extremos está en S y el otro en $V-S$. El corte **respeta** un conjunto de aristas A (*respects*) si ninguna arista de A lo cruza. Una **arista ligera** que cruza el corte (*light edge*) es una de peso mínimo entre las que lo cruzan.

Con empates puede haber varias aristas ligeras para el mismo corte, y el teorema vale para cualquiera de ellas. "Ligera" siempre se dice respecto a un corte; no tiene nada que ver con las aristas más baratas del grafo, como muestra el ejemplo.

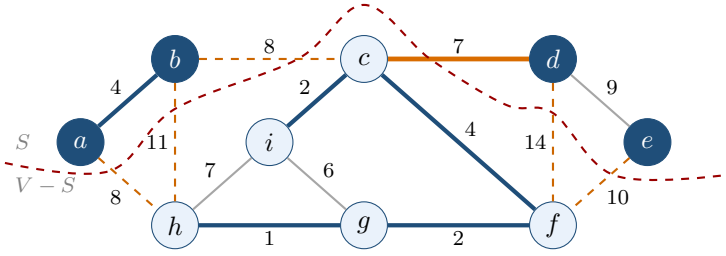
Ejemplo 6.3 (Un corte del grafo de Cormen)**.** Este ejemplo y casi todos los que siguen usan el grafo de la figura 23.1 de Cormen, con $n = 9$ vértices y $m = 14$ aristas. Los vértices van en minúscula para no confundirlos con los del grafo G de la libreta:

$$\{a,b\}\ 4,\ \{a,h\}\ 8,\ \{b,c\}\ 8,\ \{b,h\}\ 11,\ \{c,d\}\ 7,\ \{c,f\}\ 4,\ \{c,i\}\ 2,$$
$$\{d,e\}\ 9,\ \{d,f\}\ 14,\ \{e,f\}\ 10,\ \{f,g\}\ 2,\ \{g,h\}\ 1,\ \{g,i\}\ 6,\ \{h,i\}\ 7.$$

Se toma $S = \{a,b,d,e\}$, en relleno oscuro, y $A = \{\{a,b\},\{c,i\},\{c,f\},\{g,h\},\{f,g\}\}$, en azul grueso. Ninguna arista de A tiene un extremo de cada lado, así que el corte $(S, V-S)$ respeta A. Las que cruzan, punteadas, son seis:

Arista	$\{c,d\}$	$\{a,h\}$	$\{b,c\}$	$\{e,f\}$	$\{b,h\}$	$\{d,f\}$
Peso	**7**	8	8	10	11	14
Extremo en S	d	a	b	e	b	d

La arista ligera es $\{c,d\}$, con peso 7, y es la única. El teorema de abajo garantiza que $A \cup \{\{c,d\}\}$ cabe en algún árbol de expansión mínima, aunque el corte no tenga nada que ver con cómo se construyó A. Y $\{c,d\}$ es ligera aunque el grafo tenga seis aristas más baratas que ella: ninguna de esas seis cruza este corte.



Las tres pruebas que siguen manipulan árboles de la misma manera: quitan una arista, agregan otra y comprueban que lo que queda sigue siendo árbol. Conviene tener a la mano por qué.

Lema 6.4 (Tres hechos sobre árboles de expansión)**.** *Sea T un árbol de expansión de G.*

(a) Entre cada par de vértices hay un único camino dentro de T.

(b) Quitar una arista $f = \{x, y\}$ de T deja exactamente dos componentes, una con x y otra con y. Ellas definen un corte, y f es la única arista de T que lo cruza.

(c) Si $e = \{u, v\} \notin T$ y f es cualquier arista del camino de u a v en T, entonces $T - f + e$ es otra vez un árbol de expansión.

Demostración. (a) y (b) son caracterizaciones de árbol de la sección 3; la última frase de (b) se debe a que las demás aristas de T quedan dentro de alguna de las dos componentes. Para (c): por (b), $T - f$ tiene dos componentes. Los vértices u y v quedan en componentes distintas, porque si siguieran conectados en $T - f$ habría en T un segundo camino de u a v que no usa f, contra (a). La arista e une esas dos componentes, así que $T - f + e$ es conexo, y tiene $n - 1$ aristas: es un árbol. □

Teorema 6.5 (Propiedad del corte; Cormen, teorema 23.1)**.** *Sea $G = (V, E)$ un grafo no dirigido y conexo con pesos $w : E \to \mathbb{R}$. Sea $A \subseteq E$ un subconjunto de algún árbol de expansión mínima de G, sea $(S, V - S)$ un corte que respeta A y sea $e = \{u, v\}$ una arista ligera que cruza $(S, V - S)$. Entonces e es segura para A.*

Demostración. Sea T un MST con $A \subseteq T$. Si $e \in T$, entonces $A \cup \{e\} \subseteq T$ y no hay nada que probar. Supongamos que $e \notin T$; vamos a fabricar otro MST que sí la contenga.

Una arista de T que cruza. Sin perder generalidad $u \in S$ y $v \in V - S$. Sea p el camino de u a v dentro de T. Empieza en S y termina fuera de S, así que al recorrerlo desde u hay una primera arista $f = \{x, y\}$ con $x \in S$ y $y \in V - S$. Esa arista cruza el corte. No está en A, porque el corte respeta A. Y es distinta de e, porque $f \in T$ y $e \notin T$.

El intercambio. Por el lema 6.4(c), $T' = T - f + e$ es un árbol de expansión, y pesa

$$w(T') = w(T) - w(f) + w(e).$$

Las dos aristas e y f cruzan el corte y e es ligera, así que $w(e) \leq w(f)$ y por lo tanto $w(T') \leq w(T)$.

Los dos casos. Si $w(e) < w(f)$, entonces $w(T') < w(T)$, lo que es imposible porque T es mínimo: este caso no ocurre. Queda el empate, $w(e) = w(f)$. Entonces $w(T') = w(T)$ y T' es otro MST: el intercambio no mejora, empata.

T' contiene lo que hace falta. Como $A \subseteq T$ y $f \notin A$, se tiene $A \subseteq T - f \subseteq T'$; además $e \in T'$. Entonces $A \cup \{e\} \subseteq T'$, con T' un MST, y e es segura. □

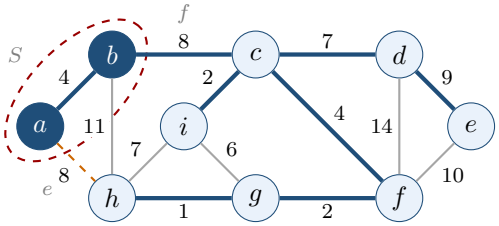
El tercer paso dice algo más de lo que pide el teorema. Si e no está en un MST T que contiene a A, es porque hay otra arista de cruce empatada con ella. En particular, cuando la arista ligera de un corte que respeta A es *única*, está en todo MST que contenga a A. Esa observación es la semilla de la unicidad, más adelante.

Ejemplo 6.6 (El intercambio de la prueba)**.** Mismo grafo y mismo A, ahora con el corte $S = \{a, b\}$, que también respeta A. Cruzan $\{a, h\}$ con 8, $\{b, c\}$ con 8 y $\{b, h\}$ con 11: hay **dos** aristas ligeras. Sea

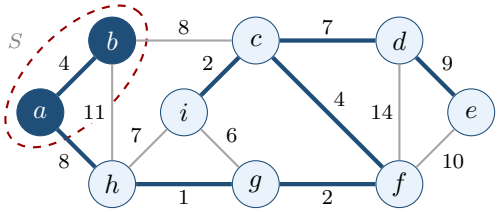
T el árbol que construye Prim desde a (ejemplo 6.16), de peso 37, que contiene a A y a $\{b, c\}$ pero no a $e = \{a, h\}$.

1. El camino p de a a h dentro de T es a, b, c, f, g, h. Con e cierra un ciclo.
2. La primera arista de p que sale de S es $f = \{b, c\}$; no está en A, porque el corte respeta A.
3. $T' = T - f + e$ es otro árbol de expansión, de peso $37 - 8 + 8 = 37$.

El peso no bajó: empató, porque $w(a, h) = w(b, c)$. Es el caso del empate de la prueba, y T' resulta ser justamente el árbol que devuelve Kruskal (ejemplo 6.14). Una enumeración de los 662 árboles de expansión del grafo confirma que solo esos dos pesan 37.



T: árbol de Prim, peso 37



$T' = T - f + e$: árbol de Kruskal, peso 37

El teorema deja escoger el corte con total libertad, siempre que respete A. Kruskal y Prim usan el mismo caso particular: el corte que separa uno de los árboles ya construidos del resto del grafo.

Corolario 6.7 (Cormen, corolario 23.2)**.** *Sean G y A como en el teorema 6.5 y sea $C = (V_C, E_C)$ un componente del bosque $G_A = (V, A)$. Si e es una arista ligera entre las que unen C con otro componente de G_A, entonces e es segura para A.*

Demostración. El corte $(V_C, V - V_C)$ respeta A, porque ninguna arista de A sale de un componente de G_A. Las aristas que cruzan ese corte son exactamente las que unen C con otro componente, así que e es ligera para él. El teorema hace el resto. □

Del teorema salen también tres datos de la forma del método genérico. A siempre es acíclico, porque vive dentro de un árbol, así que $G_A = (V, A)$ es un bosque; al inicio tiene n árboles de un vértice. Toda arista segura une dos componentes distintos de G_A: una arista con los dos extremos en el mismo componente cerraría un ciclo con A, y ningún MST contiene un ciclo. Y como cada arista segura junta dos árboles en uno, el ciclo del algoritmo 6.1 da exactamente $n - 1$ vueltas.

Ojito...

Segura no es lo mismo que "está en algún MST". El invariante pide que $A \cup \{e\}$ quepa entero en *un mismo* MST. En el grafo de Cormen, $\{a, h\}$ está en un MST y $\{b, c\}$ en el otro. Si A son las siete aristas comunes a los dos MST más $\{a, h\}$, la arista $\{b, c\}$ ya no es segura, aunque pertenezca a un MST: con A cierra el ciclo a, b, c, f, g, h, a. Por eso el teorema exige que el corte respete A; es lo que garantiza que las decisiones de pasos distintos apunten al mismo árbol. Y por eso el argumento informal "la arista más ligera que cruza un corte está en algún MST" no alcanza para encadenar pasos voraces.

Observación. La recíproca del teorema es falsa (Cormen, ejercicio 23.1-2): una arista segura no tiene que ser ligera para el corte que se escogió. En el camino a, b, c con $w(a, b) = 1$ y $w(b, c) = 2$, con $A = \emptyset$ y el corte $(\{a, c\}, \{b\})$, cruzan las dos aristas. $\{b, c\}$ es segura, porque el grafo es su propio MST, y no

es ligera. Lo que sí es cierto (ejercicio 23.1-3) es que toda arista de un MST es ligera para *algún* corte: el que deja el árbol al quitarle esa arista.

La propiedad del ciclo y la unicidad

El teorema del corte dice qué aristas se pueden meter. Su gemelo dice cuáles se pueden dejar fuera.

Lema 6.8 (Propiedad del ciclo; Cormen, ejercicio 23.1-5)**.** *Sea e una arista de peso máximo en algún ciclo K de G. Entonces hay un MST de G que no contiene a e. Si además e es la única de peso máximo en K, ningún MST contiene a e.*

Demostración. Si ningún MST contiene a e no hay nada que probar. Sea entonces T un MST con $e = \{u, v\} \in T$. Por el lema 6.4(b), quitar e de T deja dos componentes, con u y v en lados distintos, que definen un corte cruzado por e y por ninguna otra arista de T. El resto del ciclo K es un camino de u a v: empieza de un lado y termina del otro, así que alguna arista $f \neq e$ de K cruza ese corte, y $f \notin T$. Entonces $T - e + f$ vuelve a conectar las dos componentes; es un árbol de expansión de peso $w(T) - w(e) + w(f) \leq w(T)$, porque e es máxima en K. Es un MST sin e. Si e es la única máxima, $w(f) < w(e)$ y el árbol nuevo pesa menos que T, lo que contradice que T sea mínimo; así que ningún MST contenía a e. □

Ojito...

La versión que circula, "la arista más pesada de un ciclo no está en ningún MST", es falsa con empates. En el grafo de Cormen el ciclo a, b, c, i, h, a tiene pesos $4, 8, 2, 7, 8$. Sus dos aristas de peso máximo, $\{b, c\}$ y $\{a, h\}$, están cada una en uno de los dos MST. La versión fuerte pide un máximo estricto: $\{d, f\}$, con 14, es la única máxima del ciclo c, d, f, c (pesos $7, 14, 4$) y, en efecto, no está en ningún MST.

Teorema 6.9 (Unicidad; Cormen, ejercicio 23.1-6 y problema 23-1a)**.** *Si todo corte de G tiene una sola arista ligera, en particular si todos los pesos son distintos, el MST de G es único.*

Demostración. Supongamos que hay dos MST distintos, T y T'. Como tienen el mismo número de aristas, existe $e = \{u, v\} \in T \setminus T'$. Quitar e de T define un corte, y e es ligera para él: si otra arista g que lo cruza pesara menos, $T - e + g$ sería un árbol de expansión más barato que T. Por hipótesis e es la *única* ligera de ese corte. Ahora se agrega e a T': cierra un ciclo con el camino de u a v en T', y ese camino, que va de un lado del corte al otro, contiene alguna arista $f \in T'$ que cruza. Como $f \neq e$ y la ligera es única, $w(f) > w(e)$. Por el lema 6.4(c), $T' - f + e$ es un árbol de expansión que pesa menos que T', contradicción. □

La recíproca es falsa. En el triángulo con $w(x, y) = w(y, z) = 1$ y $w(x, z) = 2$ el MST $\{\{x, y\}, \{y, z\}\}$ es único, pero el corte $(\{y\}, \{x, z\})$ tiene dos aristas ligeras. El grafo de Cormen ilustra el otro lado: tiene pesos repetidos y dos MST, que difieren en $\{a, h\}$ contra $\{b, c\}$. Aun así comparten la lista ordenada de pesos, $1, 2, 2, 4, 4, 7, 8, 9$; todos los MST de un grafo la comparten (Cormen, ejercicio 23.1-8).

Observación. Kruskal y Prim nunca suman pesos: solo los comparan. Por eso los pesos negativos no estorban. Por la misma razón, sumar una constante c a todos los pesos no cambia cuáles árboles son mínimos, porque todo árbol de expansión tiene $n - 1$ aristas y todos suben $(n - 1)c$. En caminos más cortos esa suma sí cambia la respuesta, porque dos caminos pueden tener distinto número de aristas. Un árbol de expansión de peso *máximo* se obtiene negando los pesos.

Conjuntos disjuntos

Kruskal pregunta una y otra vez si dos vértices están en el mismo árbol del bosque G_A, mientras los árboles se van fusionando. La clase del 22 de octubre nombró las tres operaciones que necesita: Crear-Conjunto(x) forma el conjunto $\{x\}$, Encontrar(x) devuelve un representante del conjunto de x y Unir(x, y) junta los conjuntos de x y de y. El costo de Kruskal depende de cómo se implementen (Cormen, cap. 21).

La implementación rápida guarda cada conjunto como un árbol con raíz, en el que cada nodo apunta a su padre $p[x]$. La raíz es el representante y se apunta a sí misma. Encontrar sube hasta la raíz, y Unir cuelga una raíz de la otra. Sin cuidado se forman cadenas de largo n y cada Encontrar cuesta $\mathcal{O}(n)$. Dos heurísticas lo evitan:

- **Unión por rango.** Cada raíz guarda un rango, una cota superior de la altura de su árbol. Al unir, la raíz de rango menor cuelga de la de rango mayor; si empatan, se escoge una y su rango sube en uno.
- **Compresión de caminos.** Encontrar, al regresar de la recursión, deja a cada nodo del camino apuntando directo a la raíz. El trabajo de subir una vez ahorra las subidas siguientes.

Algoritmo 6.2 Conjuntos disjuntos con unión por rango y compresión de caminos

```
1:  procedimiento Crear-Conjunto(x)
2:      p[x] ← x,    rango[x] ← 0
3:  fin procedimiento
4:  procedimiento Encontrar(x)
5:      si x ≠ p[x] entonces
6:          p[x] ← Encontrar(p[x])                    // compresión de caminos
7:      fin si
8:      devolver p[x]
9:  fin procedimiento
10: procedimiento Unir(x, y)
11:     a ← Encontrar(x),    b ← Encontrar(y)
12:     si rango[a] < rango[b] entonces
13:         p[a] ← b                                  // el árbol bajo cuelga del alto
14:     si no
15:         p[b] ← a
16:         si rango[a] = rango[b] entonces
17:             rango[a] ← rango[a] + 1
18:         fin si
19:     fin si
20: fin procedimiento
```

Unir recibe vértices cualesquiera, no representantes, y busca las raíces por su cuenta. La unión por rango sola ya da una cota logarítmica, con una prueba de tres líneas.

Lema 6.10 (Unión por rango)**.** *Con unión por rango y sin compresión de caminos, un árbol cuya raíz tiene rango r tiene altura r y al menos 2^r nodos. En consecuencia la altura nunca pasa de $\log_2 n$ y* Encontrar *cuesta $\mathcal{O}(\log n)$.*

Demostración. Por inducción sobre las uniones. Un conjunto recién creado tiene rango 0, altura 0 y

un nodo. Al unir raíces a y b de rangos distintos, digamos rango$[b] <$ rango$[a]$, el árbol de b queda un nivel abajo de a, con altura rango$[b] + 1 \leq$ rango$[a]$: la altura y el rango de a no cambian y sus nodos aumentan. Si los dos rangos valen r, el árbol nuevo tiene altura $r + 1$, que es el rango nuevo, y al menos $2^r + 2^r = 2^{r+1}$ nodos. Como ningún árbol tiene más de n nodos, $2^r \leq n$. □

Con las dos heurísticas juntas la cota baja a casi constante por operación. La prueba ocupa la sección 21.4 de Cormen y no se repite aquí; basta el enunciado.

Teorema 6.11 (Cormen, teorema 21.14)**.** *Con unión por rango y compresión de caminos, una sucesión de k operaciones, de las cuales n son* Crear-Conjunto*, cuesta $\mathcal{O}(k\,\alpha(n))$ en el peor caso.*

Cormen llama m al número de operaciones; aquí es k, porque m ya es el número de aristas. La función α es la inversa de la función de Ackermann y crece tan despacio que $\alpha(n) \leq 4$ para todo $n \leq 10^{80}$, más que el número estimado de átomos del universo observable. En la práctica es una constante, pero no lo es en teoría: $\alpha(n) \to \infty$. El ejemplo 6.13, en la parte de Kruskal, sigue la estructura durante una corrida completa.

Kruskal

Kruskal toma, entre todas las aristas que unen dos árboles distintos del bosque G_A, la de menor peso, y para eso basta recorrerlas en orden de peso. Cada conjunto disjunto guarda los vértices de un árbol del bosque. El pseudocódigo es el de la clase, dentro de un procedimiento para que los números de línea coincidan con los que se citan abajo.

Algoritmo 6.3 MST-Kruskal(G, w)

```
1:  procedimiento MST-Kruskal(G, w)
2:      A ← ∅
3:      para todo v ∈ V hacer
4:          Crear-Conjunto(v)
5:      fin para
6:      ordenar E por peso, de menor a mayor
7:      para todo {u, v} ∈ E, en ese orden hacer
8:          si Encontrar(u) ≠ Encontrar(v) entonces
9:              A ← A ∪ {{u, v}}                    // une dos árboles distintos
10:             Unir(u, v)
11:         fin si
12:     fin para
13:     devolver A
14: fin procedimiento
```

En la línea 6 el orden es no decreciente y los empates se rompen como sea; esa decisión cambia qué árbol sale, no su peso. La línea 8 es la que decide, y su justificación es el corolario.

Proposición 6.12 (Corrección de Kruskal)**.** *Si G es conexo, MST-Kruskal devuelve un árbol de expansión mínima.*

Demostración. Los conjuntos disjuntos son siempre los vértices de los árboles de G_A: empiezan como n conjuntos de un vértice, que son los árboles de $G_\emptyset$, y la línea 10 junta dos conjuntos justo cuando la 9 junta dos árboles. Basta ver que cada arista aceptada es segura; el método genérico hace el resto.

Sea $e = \{u, v\}$ la arista examinada. Si u y v están en el mismo conjunto, ya hay un camino entre ellos dentro de A y e cerraría un ciclo; se rechaza. Si están en conjuntos distintos, sea C el árbol de u. Toda arista g que une C con otro árbol pesa al menos $w(e)$. Las que vienen después de e en el orden pesan $\geq w(e)$. Las que vinieron antes ya se examinaron, y en ese momento o se aceptaron o se rechazaron por tener los extremos juntos; en los dos casos sus extremos quedaron en un mismo árbol, y como los árboles solo se fusionan, siguen juntos y g no sale de C. Entonces e es ligera entre las aristas que unen C con otro árbol, y el corolario 6.7 la hace segura.

Al final A es un árbol de expansión. Si G_A tuviera dos árboles, alguna arista de G los uniría, porque G es conexo. Esa arista se examinó, y sus extremos ya estaban entonces en árboles distintos (lo están al final y los árboles solo se fusionan), así que se habría aceptado. Contradicción. □

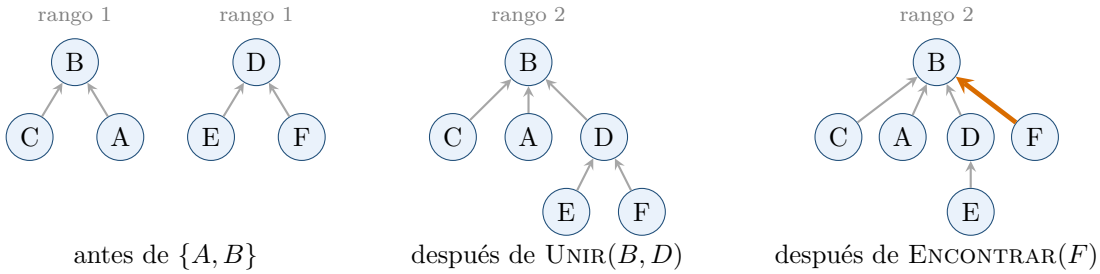
Costo. Con la estructura del algoritmo 6.2:

1. Líneas 2 a 5: n llamadas a Crear-Conjunto, $\mathcal{O}(n)$.
2. Línea 6: un ordenamiento por comparaciones (mergesort, heapsort), $\mathcal{O}(m \log m)$.
3. Líneas 7 a 12: $2m$ llamadas a Encontrar y $n-1$ a Unir, una por arista aceptada. Junto con las n Crear-Conjunto son $k = n + 2m + (n-1)$ operaciones, y por el teorema 6.11 cuestan $\mathcal{O}((n+m)\,\alpha(n))$. Como G es conexo, $m \geq n-1$, y esto es $\mathcal{O}(m\,\alpha(n))$.
4. Total: $\mathcal{O}(m \log m + m\,\alpha(n))$. Como $\alpha(n) = \mathcal{O}(\log n)$ y $n \leq m+1$, es $\mathcal{O}(m \log m)$. Y como $m < n^2$, $\log m < 2 \log n$, así que Kruskal cuesta $\mathcal{O}(m \log n)$.

El cuello de botella es el ordenamiento, no la estructura. De hecho, para la cota $\mathcal{O}(m \log n)$ basta la unión por rango sola: por el lema 6.10 las $\mathcal{O}(m)$ operaciones cuestan $\mathcal{O}(m \log n)$, que el ordenamiento absorbe. La compresión de caminos importa cuando ordenar es barato: si las aristas ya vienen ordenadas, o si los pesos son enteros en un rango chico (de 1 a n, o de 1 a una constante), counting sort las ordena en $\mathcal{O}(n+m)$ y Kruskal baja a $\mathcal{O}(m\,\alpha(n))$, casi lineal (Cormen, ejercicio 23.2-4). Al revés, sin conjuntos disjuntos, revisando con BFS dentro de A si u y v ya están conectados, cada prueba cuesta $\mathcal{O}(n)$ y el total sube a $\mathcal{O}(mn)$.

La clase del 22 de octubre ya corrió Kruskal sobre el grafo pesado de la libreta. Vale la pena repetir esa corrida mirando dentro de la estructura de conjuntos.

Ejemplo 6.13 (El bosque de conjuntos en el Kruskal de la clase)**.** Las operaciones del ejemplo de Kruskal de la clase sobre el grafo pesado, con las aristas en el mismo orden. Unir(B, C) encuentra rangos iguales: C cuelga de B y rango$[B] = 1$. En Unir(A, C), Encontrar$(C) = B$ y rango$[A] = 0 < 1$, así que A cuelga de B. Del otro lado, E cuelga de D, que sube a rango 1, y luego F cuelga de D. La arista $\{A, B\}$ se rechaza porque Encontrar(A) = Encontrar$(B) = B$. En Unir(B, D) empatan los rangos en 1: D cuelga de B, que sube a rango 2. El pseudocódigo sigue revisando aristas, y al examinar $\{D, F\}$ la llamada Encontrar(F) sube por F, D, B y, al regresar, deja a F colgado directo de B (en naranja). Las flechas apuntan al padre.



antes de $\{A, B\}$ después de Unir(B, D) después de Encontrar(F)

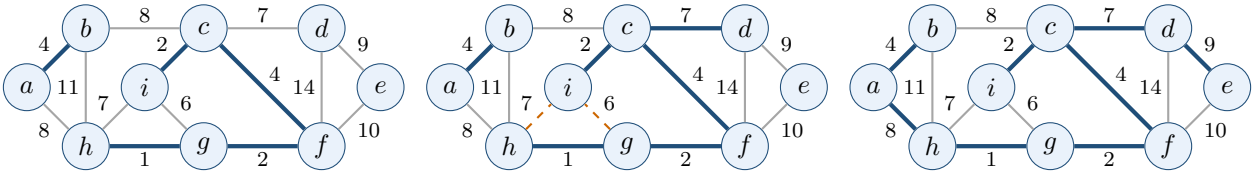
El rango de B se queda en 2 aunque la altura real haya bajado: después de una compresión el rango es solo una cota. Al examinar $\{C, E\}$ le pasa lo mismo a E, y al terminar los otros cinco vértices cuelgan directo de B.

El grafo de Cormen tiene más aristas y, sobre todo, un empate que decide el árbol.

Ejemplo 6.14 (Kruskal sobre el grafo de Cormen)**.** Las 14 aristas en orden de peso, con los empates en el orden de la figura 23.4 de Cormen. La última columna es el bosque G_A después del paso; en los rechazos no cambia.

Paso	Arista	Peso	¿Entra?	Árboles del bosque después
1	$\{h, g\}$	1	sí	$\{a\}$ $\{b\}$ $\{c\}$ $\{d\}$ $\{e\}$ $\{f\}$ $\{g, h\}$ $\{i\}$
2	$\{c, i\}$	2	sí	$\{a\}$ $\{b\}$ $\{c, i\}$ $\{d\}$ $\{e\}$ $\{f\}$ $\{g, h\}$
3	$\{g, f\}$	2	sí	$\{a\}$ $\{b\}$ $\{c, i\}$ $\{d\}$ $\{e\}$ $\{f, g, h\}$
4	$\{a, b\}$	4	sí	$\{a, b\}$ $\{c, i\}$ $\{d\}$ $\{e\}$ $\{f, g, h\}$
5	$\{c, f\}$	4	sí	$\{a, b\}$ $\{c, f, g, h, i\}$ $\{d\}$ $\{e\}$
6	$\{i, g\}$	6	no	cerraría el ciclo i, c, f, g, i
7	$\{c, d\}$	7	sí	$\{a, b\}$ $\{c, d, f, g, h, i\}$ $\{e\}$
8	$\{i, h\}$	7	no	cerraría el ciclo i, c, f, g, h, i
9	$\{a, h\}$	8	sí	$\{a, b, c, d, f, g, h, i\}$ $\{e\}$
10	$\{b, c\}$	8	no	cerraría el ciclo b, a, h, g, f, c, b
11	$\{d, e\}$	9	sí	$\{a, b, c, d, e, f, g, h, i\}$
12–14	$\{e, f\}$, $\{b, h\}$, $\{d, f\}$		no	ya hay $n - 1 = 8$ aristas

Peso total $1 + 2 + 2 + 4 + 4 + 7 + 8 + 9 = 37$. En el paso 9 el empate entre $\{a, h\}$ y $\{b, c\}$, las dos de peso 8, se rompió a favor de $\{a, h\}$ solo porque aparece antes en la lista ordenada. Fueron 28 llamadas a Encontrar en la línea 8 y 8 a Unir. Las tres instantáneas: después del paso 5, después del 8 (en naranja los dos rechazos) y al final.



pasos 1 a 5 pasos 6 a 8 final: peso 37

Observación. El pseudocódigo de Cormen recorre las m aristas aunque ya tenga $n - 1$; en el ejemplo, las tres últimas solo se examinan para rechazarlas. Cortar al llegar a $n - 1$ ahorra llamadas a Encontrar, pero no el ordenamiento, que domina. Si G no es conexo, Kruskal no se queja: devuelve un **bosque** de expansión mínima, con $n - c$ aristas si G tiene c componentes. Y detenido cuando quedan k árboles, es exactamente el agrupamiento jerárquico de enlace simple con k grupos.

Prim

Prim hace crecer un solo árbol desde una raíz r hasta cubrir V. En cada paso agrega la arista más ligera que une el árbol con un vértice de afuera: es el corolario 6.7 con C igual al árbol que se está construyendo. Para escoger esa arista rápido, los vértices de afuera viven en una cola de prioridad Q con la llave llave[v]: el peso de la arista más barata que une a v con el árbol, o ∞ si no hay ninguna (Cormen escribe $v.key$). El padre $\pi[v]$ es el extremo de esa arista dentro del árbol. Es la misma organización que Dijkstra, con otra llave.

Algoritmo 6.4 MST-Prim(G, w, r)

```
1:  procedimiento MST-Prim(G, w, r)
2:      para todo u ∈ V hacer
3:          llave[u] ← ∞,    π[u] ← nulo
4:      fin para
5:      llave[r] ← 0
6:      Q ← montículo de mínimos con todo V, ordenado por llave
7:      mientras Q ≠ ∅ hacer
8:          u ← Extraer-Min(Q)                    // entra u con la arista {u, π[u]}
9:          para todo v adyacente a u hacer
10:             si v ∈ Q y w(u, v) < llave[v] entonces
11:                 π[v] ← u
12:                 llave[v] ← w(u, v)            // Decrementar-Llave
13:             fin si
14:         fin para
15:     fin mientras
16:     devolver {{v, π[v]} : v ∈ V − {r}}
17: fin procedimiento
```

Las líneas no obvias:

- **Líneas 2 a 6.** Todas las llaves en ∞ salvo la de la raíz, que vale 0 para que salga primero. En la primera vuelta $u = r$ y no entra ninguna arista; r es el único vértice con π nulo al final.
- **Línea 8.** Los vértices del árbol son los de $V - Q$. El que sale tiene la arista más barata hacia el árbol, así que $\{u, \pi[u]\}$ es una arista ligera que cruza el corte $(V - Q, Q)$; entra al árbol de manera implícita.
- **Línea 10.** La prueba $v \in Q$ no es decorativa: sin ella el resultado deja de ser un árbol, como muestra el ejemplo 6.17. Se implementa en $\mathcal{O}(1)$ con un bit por vértice que se apaga al extraerlo.
- **Línea 12.** Es un Decrementar-Llave sobre el montículo. Para hacerlo en $\mathcal{O}(\log n)$ el montículo tiene que saber en qué posición del arreglo está v; se guarda un índice por vértice que se actualiza en cada intercambio.

Proposición 6.15 (Corrección de Prim)**.** *Si G es conexo, MST-Prim devuelve un árbol de expansión mínima.*

Demostración. Sea $A = \{\{v, \pi[v]\} : v \in V - Q - \{r\}\}$, las aristas con las que entraron los vértices que ya salieron de Q. Antes de cada vuelta del **mientras**, salvo la primera, se cumple:

(i) A es subconjunto de algún MST, y sus aristas tienen los dos extremos en $V - Q$;

(ii) para todo $v \in Q$, llave$[v]$ es el peso mínimo de una arista entre v y $V - Q$, y $\pi[v]$ es el otro extremo de esa arista (∞ y nulo si no hay ninguna).

Después de la primera vuelta $V - Q = \{r\}$, $A = \emptyset$ y las líneas 9 a 13 pusieron llave$[v] = w(r, v)$ en los vecinos de r: las dos partes valen.

Para (ii): cuando u pasa de Q a $V - Q$, el nuevo mínimo de un $v \in Q$ vecino de u es el menor entre llave$[v]$ y $w(u, v)$, y el de un v no vecino no cambia. Las líneas 9 a 13 calculan exactamente eso, y gracias a la prueba $v \in Q$ no tocan a nadie más. Para (i): el u que sale tiene la llave mínima de Q,

que por (ii) es el peso de la arista más ligera que cruza el corte $(V - Q, Q)$, y esa arista es $\{u, \pi[u]\}$. La llave es finita porque G es conexo y los dos lados del corte son no vacíos. El corte respeta A por la segunda mitad de (i), así que el teorema 6.5 hace segura a $\{u, \pi[u]\}$. Sus dos extremos quedan en $V - Q$, y $\pi[u]$ ya no vuelve a cambiar, otra vez por la prueba $v \in Q$; el conjunto A de la fórmula es entonces el que hace crecer el método genérico.

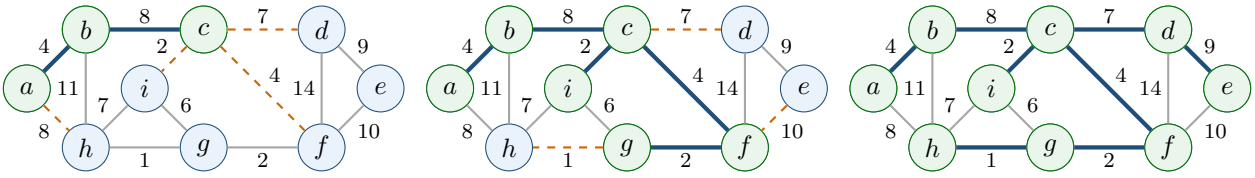
Al terminar, $Q = \emptyset$ y A tiene $n - 1$ aristas dentro de un MST: es ese MST. □

Cormen enuncia un invariante parecido, en tres partes, pero no lo prueba en detalle. La prueba deja ver dónde trabaja la línea 10: en las dos partes del invariante.

Ejemplo 6.16 (Prim sobre el grafo de Cormen, desde a)**.** Cada renglón muestra las llaves después de procesar el vértice que sale; en negritas, los que ya están en el árbol, cuya llave ya no cambia y es el peso de la arista con la que entraron. Los empates en la llave mínima se rompen por orden alfabético, como en la figura 23.5 de Cormen.

Paso	**Sale**	a	b	c	d	e	f	g	h	i	**Cambios**
0		0	∞	∞	∞	∞	∞	∞	∞	∞	
1	a	**0**	4	∞	∞	∞	∞	∞	8	∞	b, h desde a
2	b	**0**	**4**	8	∞	∞	∞	∞	8	∞	c desde b; h: $11 \not< 8$
3	c	**0**	**4**	**8**	7	∞	4	∞	8	2	empate c, h en 8; d, f, i desde c
4	i	**0**	**4**	**8**	7	∞	4	6	7	**2**	g desde i; h: $7 < 8$
5	f	**0**	**4**	**8**	7	10	**4**	2	7	**2**	e desde f; g: $2 < 6$
6	g	**0**	**4**	**8**	7	10	**4**	**2**	1	**2**	h: $1 < 7$
7	h	**0**	**4**	**8**	7	10	**4**	**2**	**1**	**2**	nada
8	d	**0**	**4**	**8**	**7**	9	**4**	**2**	**1**	**2**	e: $9 < 10$
9	e	**0**	**4**	**8**	**7**	**9**	**4**	**2**	**1**	**2**	

Orden de entrada: $a, b, c, i, f, g, h, d, e$. Los padres quedan $\pi[b] = a$, $\pi[c] = b$, $\pi[i] = c$, $\pi[f] = c$, $\pi[g] = f$, $\pi[h] = g$, $\pi[d] = c$, $\pi[e] = d$, y el peso es la suma de las llaves finales, $0+4+8+7+9+4+2+1+2 = 37$. La llave de h cambió tres veces, $\infty \to 8 \to 7 \to 1$: desde a, desde i y desde g. En las instantáneas, en verde los vértices del árbol y en naranja, para cada vértice de afuera con llave finita, la arista $\{v, \pi[v]\}$ que le da esa llave: la más corta de ellas es la que entra en el paso siguiente.



Kruskal y Prim llegan a 37 por caminos distintos: Kruskal tomó $\{g, h\}$ primero y $\{d, e\}$ al último; Prim tomó $\{a, b\}$ primero. Difieren en una arista. Kruskal se quedó con $\{a, h\}$ porque aparecía antes en la lista ordenada; Prim con $\{b, c\}$ porque en el paso 3 la llave de c empató con la de h en 8 y el desempate alfabético sacó a c. Con cualquiera de las nueve raíces Prim llega a 37.

La prueba $v \in Q$

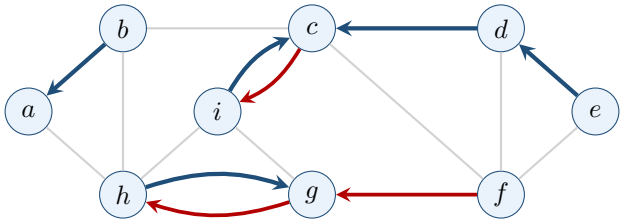
Prim se suele describir como Dijkstra con la llave $w(u, v)$ en lugar de $d[u] + w(u, v)$. Es cierto, con una diferencia que se paga cara: en Prim la condición $v \in Q$ de la línea 10 es indispensable.

Ejemplo 6.17 (Prim sin la prueba $v \in Q$)**.** Se corre MST-Prim sobre el grafo de Cormen desde a, borrando "$v \in Q$ **y**" de la línea 10. Los pasos 1 a 3 son idénticos a los del ejemplo 6.16. En el paso 4 sale

i, con llave 2, y revisa a su vecino c, que ya está en el árbol con llave 8 y padre b. Como $w(i,c) = 2 < 8$, la línea 11 hace $\pi[c] = i$, mientras que $\pi[i] = c$. Pasa lo mismo dos veces más:

Paso	Sale	Vértice ya en el árbol	Padre	Llave	Ahora
4	i	c	$b \to i$	$8 \to 2$	$\pi[c] = i$, $\pi[i] = c$
6	g	f	$c \to g$	$4 \to 2$	$\pi[f] = g$
7	h	g	$f \to h$	$2 \to 1$	$\pi[g] = h$, $\pi[h] = g$

El orden de extracción no cambia, porque Extraer-Min solo mira las llaves de Q, y las que se corrompen son de vértices que ya salieron. Lo que se rompe es lo que devuelve la línea 16: los pares $\{v, \pi[v]\}$ son $\{b,a\}$, $\{c,i\}$, $\{i,c\}$, $\{d,c\}$, $\{e,d\}$, $\{f,g\}$, $\{g,h\}$, $\{h,g\}$. Ocho pares pero solo seis aristas distintas, repartidas en tres componentes, $\{a,b\}$, $\{c,d,e,i\}$ y $\{f,g,h\}$. No es un árbol. La suma de las llaves finales da 28, menos que el óptimo de 37, y eso solo ya delata el problema: ningún árbol de expansión pesa menos que un MST. En la figura, cada flecha va de v a $\pi[v]$; en rojo, los tres padres reescritos.



La arista $\{u, \pi[u]\}$ se elige en el momento en que u sale de Q, y desde ese momento $\pi[u]$ debe quedarse quieto. La línea 10 es la que lo protege.

En Dijkstra la misma prueba se puede omitir sin daño: con pesos no negativos, todo v que ya salió tiene $d[v] \le d[u]$, así que $d[u] + w(u,v) \ge d[v]$ y la relajación nunca dispara sobre él. Por eso el Dijkstra de la clase no la escribe. En Prim la comparación $w(u,v) < \text{llave}[v]$ no guarda ninguna relación con el orden de salida: $w(i,c) = 2$ es menor que la llave 8 con la que c entró al árbol.

Cuánto cuesta Prim

El costo depende de la cola de prioridad. Con un **montículo binario de mínimos**, el de la clase del 17 de septiembre en su versión de mínimos:

1. Líneas 2 a 6: inicializar es $\mathcal{O}(n)$, y construir el montículo también, con el mismo análisis que Build-Max-Heap.
2. El **mientras** da n vueltas, una por vértice, y cada Extraer-Min cuesta $\mathcal{O}(\log n)$: $\mathcal{O}(n \log n)$.
3. El **para todo** de la línea 9 recorre la lista de adyacencia de cada vértice una vez, $2m$ iteraciones en total. Cada una hace la prueba de la línea 10 en $\mathcal{O}(1)$ y, a veces, un Decrementar-Llave de $\mathcal{O}(\log n)$.
4. Hay a lo más m decrementos: una arista solo puede bajar una llave cuando sale el primero de sus extremos, porque cuando sale el segundo el otro ya no está en Q. En el grafo de Cormen hubo 12 decrementos para $m = 14$; en el pesado de la libreta, 9 para $m = 10$.
5. Total: $\mathcal{O}(n + n \log n + m + m \log n) = \mathcal{O}((n+m) \log n)$, que con G conexo es $\mathcal{O}(m \log n)$. Asintóticamente igual que Kruskal.

Con un **montículo de Fibonacci** (Cormen, cap. 19), Extraer-Min cuesta $\mathcal{O}(\log n)$ amortizado y Decrementar-Llave $\mathcal{O}(1)$ amortizado. "Amortizado" significa que la suma de los costos de toda la

sucesión está acotada por la suma de esas cotas, aunque una operación suelta cueste más. Los términos quedan $n \cdot \mathcal{O}(\log n)$ por las extracciones y $m \cdot \mathcal{O}(1)$ por los decrementos: $\mathcal{O}(m + n \log n)$. Lo que se ganó es que el término que multiplica a m ya no lleva $\log n$.

Con un **arreglo** (Cormen, ejercicio 23.2-2), conveniente cuando el grafo viene como matriz de adyacencia, la llave se guarda en un arreglo indexado por vértice. Buscar el mínimo es un barrido de $\Theta(n)$ y decrementar es una asignación de $\mathcal{O}(1)$. Con la matriz, revisar los vecinos de u es otro barrido de $\Theta(n)$. Son n vueltas de $\Theta(n)$: $\Theta(n^2)$, sin importar m. La prueba $v \in Q$ se vuelve un arreglo booleano dentro$[v]$, y por la misma razón no se puede quitar. La tabla 1 reúne las cuatro opciones (Cormen, ejercicio 23.2-3).

Implementación	**Costo**	**Disperso,** $m = \Theta(n)$	**Denso,** $m = \Theta(n^2)$
Kruskal, conjuntos disjuntos	$\mathcal{O}(m \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2 \log n)$
Prim, montículo binario	$\mathcal{O}(m \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2 \log n)$
Prim, montículo de Fibonacci	$\mathcal{O}(m + n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$
Prim, arreglo o matriz	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$

Tabla 1: Costo de los algoritmos de árbol de expansión mínima según la densidad del grafo.

La tabla se lee así. Fibonacci le gana asintóticamente al binario exactamente cuando $m = \omega(n)$: en ese caso $m \log n$ domina tanto a m como a $n \log n$. Cuando $m = \Omega(n \log n)$, Prim con Fibonacci ya es $\mathcal{O}(m)$, lineal. En grafos densos el arreglo empata con Fibonacci y tiene constantes mucho menores, así que es la opción práctica. En grafos dispersos todos dan $\mathcal{O}(n \log n)$ y deciden las constantes: Kruskal es el más simple de programar y es casi lineal si los pesos se ordenan en tiempo lineal; Prim con montículo binario es lo usual con listas de adyacencia. Para dar una idea de tamaños, con $n = 1000$ y sin constantes: en un grafo disperso con $m = 3000$, $m \log_2 n \approx 3 \cdot 10^4$ contra $n^2 = 10^6$, y gana el montículo; en el grafo completo, con $m = 499\,500$, $m \log_2 n \approx 5 \cdot 10^6$ contra $n^2 = 10^6$, y gana el arreglo.

Ojito...

El montículo de la biblioteca no trae Decrementar-Llave. Esto no viene en Cormen, pero es lo primero que aparece al programarlo: `heapq` de Python y `priority_queue` de C++ no decrementan. La salida usual es la versión perezosa. En lugar de decrementar se inserta otra copia (llave, v), y al extraer se descartan los v que ya están en el árbol; ese descarte es la prueba $v \in Q$ con otro disfraz. El montículo llega a tener $\mathcal{O}(m)$ entradas y el costo es $\mathcal{O}(m \log m) = \mathcal{O}(m \log n)$, la misma cota.

Ojito...

Grafos desconexos y dirigidos. Si G no es conexo, Prim desde r solo cubre el componente de r: los demás vértices salen de Q con llave ∞ y padre nulo, y el pseudocódigo no lo detecta. Basta revisar si la llave extraída es ∞. Con grafos dirigidos el capítulo no aplica, porque el corte y el ciclo usan que la arista sirve en los dos sentidos. Con raíz r y arcos $r \to a$ de peso 2, $r \to b$ de 3, $a \to b$ de 3 y $b \to a$ de 1, tomar siempre el arco más barato que sale del árbol da $r \to a$ y luego un arco de peso 3 hacia b, total 5; la arborescencia óptima es $r \to b$, $b \to a$, con peso 4. El problema dirigido tiene su propio algoritmo (Chu-Liu y Edmonds), fuera de este curso.

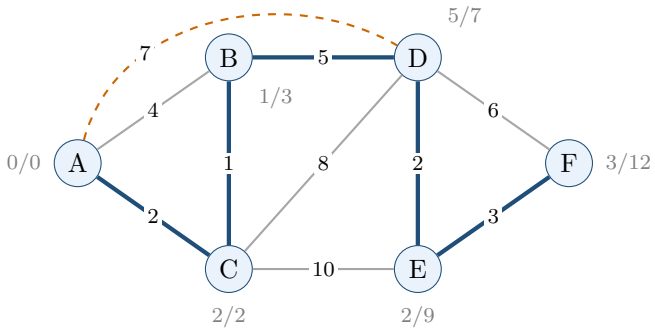
Prim contra Dijkstra

La clase del 22 de octubre comparó el árbol de Dijkstra desde A (peso 15) con el MST (peso 13) en el grafo pesado. Corriendo los dos algoritmos lado a lado se ve en qué paso se separan.

Ejemplo 6.18 (Prim contra Dijkstra en el grafo pesado de la libreta)**.** Desde A, con el grafo pesado de la clase. En cada celda, llave de Prim / d de Dijkstra; en negritas, los vértices ya fijados. A tiene 0/0 todo el tiempo.

Paso	**Sale**	B	C	D	E	F
1	A	4/4	2/2	7/7	∞/∞	∞/∞
2	C	1/3	**2/2**	7/7	10/12	∞/∞
3	B	**1/3**	**2/2**	5/7	10/12	∞/∞
4	D	**1/3**	**2/2**	**5/7**	2/9	6/13
5	E	**1/3**	**2/2**	**5/7**	**2/9**	3/12
6	F	**1/3**	**2/2**	**5/7**	**2/9**	**3/12**

Los dos sacan los vértices en el mismo orden, A, C, B, D, E, F, y aun así construyen árboles distintos. La diferencia está en el paso 3: al salir B, Prim compara $w(B, D) = 5$ contra la llave 7 y cambia el padre de D a B; Dijkstra compara $d[B] + w(B, D) = 8$ contra 7 y lo deja colgado de A. Prim termina con el MST de peso $0 + 2 + 1 + 5 + 2 + 3 = 13$, el mismo que encontró Kruskal; Dijkstra, con el árbol de caminos más cortos de peso 15. En azul el árbol de Prim, punteada la arista que Dijkstra usa en lugar de $\{B, D\}$, y junto a cada vértice su llave final y su d final.



Prim minimiza el peso total y Dijkstra la distancia desde la raíz, así que sus árboles no tienen por qué parecerse aunque el código sí. Otra diferencia: el árbol de Dijkstra depende de la raíz por definición; el peso del de Prim no (en el grafo de Cormen sale 37 desde cualquiera de los nueve vértices). Y un dato curioso: el árbol de Dijkstra desde A es uno de los dos *segundos mejores* árboles de expansión de este grafo, con peso 15.

7. Caminos más cortos desde una fuente *Complemento: Cormen, cap. 24.1 a 24.4*

Resumen rápido. El problema: un grafo dirigido con pesos reales en las aristas, que pueden ser negativos, y un vértice fuente s; se quiere el peso $\delta(s, v)$ de un camino más corto de s a cada vértice, y el camino. La sección arranca con los fundamentos: qué vale δ cuando hay ciclos de peso negativo, por qué un camino más corto se puede tomar sin ciclos y cómo se guardan todos los caminos en un árbol. Sigue la herramienta común a todos los algoritmos, la relajación, con las seis propiedades que la gobiernan. Sobre ella se montan tres algoritmos que solo difieren en el orden en que relajan: Bellman-Ford, para pesos cualesquiera, que detecta los ciclos negativos en $\Theta(nm)$; el de los grafos acíclicos, que relaja en orden topológico en $\Theta(n + m)$ y de paso calcula rutas críticas; y Dijkstra, con su prueba completa y su costo según la cola de prioridad. Al final, los sistemas de restricciones de diferencia muestran que Bellman-Ford también resuelve una familia de programas lineales.

El problema

La clase del 22 de octubre resolvió dos casos particulares del problema de esta sección: BFS, cuando todas las aristas valen lo mismo, y Dijkstra, cuando ningún peso es negativo. Aquí va el caso general, con pesos reales cualesquiera. En toda la sección $G = (V, E)$ es un grafo dirigido guardado en listas de adyacencia, y cada entrada de la lista trae el peso de su arista, así que leer $w(u, v)$ cuesta $\mathcal{O}(1)$. El peso es cualquier cantidad que se acumule sumando a lo largo de un recorrido: distancia, tiempo, costo, pérdida.

Definición 7.1 (Peso de una caminata y peso de camino más corto)**.** El **peso** de una caminata $p = \langle v_0, v_1, \ldots, v_k \rangle$ es la suma de los pesos de sus aristas,

$$w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i).$$

El **peso de camino más corto** (*shortest-path weight*) de u a v es

$$\delta(u, v) = \begin{cases} \text{ínf}\,\{w(p) : p \text{ es una caminata de } u \text{ a } v\} & \text{si hay alguna,} \\ \infty & \text{si no.} \end{cases}$$

Un **camino más corto** de u a v es un camino p de u a v con $w(p) = \delta(u, v)$. No tiene por qué ser único.

Observación. Cormen llama *path* a lo que aquí es una caminata, que puede repetir vértices, y *simple path* a lo que aquí es un camino. El ínfimo se toma sobre caminatas a propósito: si una caminata de u a v puede pasar por un ciclo de peso negativo, las que le dan más vueltas pesan cada vez menos, y eso tiene que reflejarse en δ. Cuando no hay ciclos negativos de por medio, el ínfimo lo alcanza un camino (proposición 7.4) y las dos lecturas coinciden.

Con la fuente única se resuelven las otras variantes del problema:

- **Fuente única** (*single-source*): de un vértice s fijo a todos los demás. Es el problema de esta sección.
- **Destino único**: de todos los vértices a un destino t fijo. Se invierte cada arista y se resuelve fuente única desde t.
- **Par único**: de u a v. Se resuelve fuente única desde u. No se conoce nada asintóticamente mejor en el peor caso: saber el destino no ayuda.

- **Todos los pares**: basta correr fuente única desde cada vértice, pero hay métodos mejores (sección 9).

Subestructura óptima

Lema 7.2 (Los subcaminos de un camino más corto son caminos más cortos; Cormen, lema 24.1)**.** *Sea $p = \langle v_0, v_1, \ldots, v_k \rangle$ un camino más corto de v_0 a v_k. Para todo $0 \le i \le j \le k$, el subcamino $p_{ij} = \langle v_i, v_{i+1}, \ldots, v_j \rangle$ es un camino más corto de v_i a v_j.*

Demostración. Se parte p en tres tramos, $v_0 \overset{p_{0i}}{\rightsquigarrow} v_i \overset{p_{ij}}{\rightsquigarrow} v_j \overset{p_{jk}}{\rightsquigarrow} v_k$, de modo que $w(p) = w(p_{0i}) + w(p_{ij}) + w(p_{jk})$. Si hubiera una caminata p' de v_i a v_j con $w(p') < w(p_{ij})$, la caminata que recorre p_{0i}, luego p' y luego p_{jk} iría de v_0 a v_k con peso

$$w(p_{0i}) + w(p') + w(p_{jk}) < w(p) = \delta(v_0, v_k),$$

imposible porque δ es el ínfimo sobre todas las caminatas. Así que ninguna caminata de v_i a v_j pesa menos que p_{ij}, y $w(p_{ij}) = \delta(v_i, v_j)$. □

Es la **subestructura óptima** de la programación dinámica: una solución óptima contiene soluciones óptimas de sus subproblemas. Cormen apoya en este lema tanto a Dijkstra, que es voraz, como a Floyd-Warshall, que es programación dinámica.

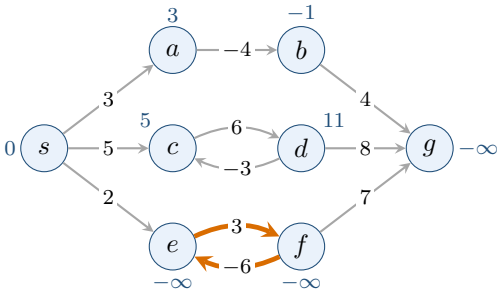
Pesos negativos y ciclos negativos

Una arista de peso negativo no rompe nada por sí sola. Lo que rompe la definición es un **ciclo de peso negativo** (*negative-weight cycle*) en alguna caminata de s a v: cada vuelta adicional baja el peso, el ínfimo es $\delta(s, v) = -\infty$ y ningún camino es el más corto. Si v no es alcanzable desde s, en cambio, $\delta(s, v) = \infty$, aunque v esté en un ciclo negativo. Para que las cuentas valgan con infinitos se fija la aritmética

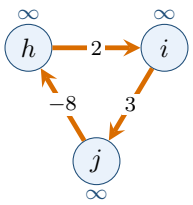
$$a + \infty = \infty \quad (a \neq -\infty), \qquad a + (-\infty) = -\infty \quad (a \neq \infty).$$

La segunda regla solo hace falta en las pruebas, cuando hay ciclos negativos.

Ejemplo 7.3 (Los tres comportamientos; Cormen, figura 24.1)**.** En la figura, junto a cada vértice está $\delta(s, \cdot)$, y en naranja los dos ciclos de peso negativo.



ciclo negativo alcanzable: $3 - 6 = -3$



ciclo negativo no alcanzable

- A a y a b se llega de una sola forma: $\delta(s, a) = 3$ y $\delta(s, b) = 3 - 4 = -1$. Una arista negativa sola no estorba.
- A c se llega por $\langle s, c \rangle$, $\langle s, c, d, c \rangle$, $\langle s, c, d, c, d, c \rangle$, y así sin fin. El ciclo $\langle c, d, c \rangle$ pesa $6 - 3 = 3 > 0$, así que cada vuelta empeora: $\delta(s, c) = 5$ y $\delta(s, d) = 11$.

- El ciclo $\langle e, f, e\rangle$ pesa $3 - 6 = -3$ y es alcanzable: $\delta(s, e) = \delta(s, f) = -\infty$. Como g es alcanzable desde f, también $\delta(s, g) = -\infty$, aunque el camino s, a, b, g pese 3.
- h, i y j forman un ciclo de peso $2 + 3 - 8 = -3$, pero desde s no se llega a ninguno de ellos: los tres tienen $\delta = \infty$. Un ciclo negativo solo contamina lo que está a la vez al alcance de la fuente y río abajo de él.

Un camino más corto no tiene ciclos

Proposición 7.4. *Si desde s no se alcanza ningún ciclo de peso negativo, para todo v alcanzable desde s el valor $\delta(s, v)$ es un número real y lo alcanza un camino, que tiene a lo más $n - 1$ aristas.*

Demostración. Sea $q = \langle v_0, v_1, \ldots, v_k\rangle$ una caminata de $s = v_0$ a $v = v_k$ que repite algún vértice. Entre las parejas de posiciones $i < j$ con $v_i = v_j$ se toma una con $j - i$ mínimo. El tramo $c = \langle v_i, \ldots, v_j\rangle$ no repite vértices salvo sus extremos, porque otra repetición dentro de él estaría más cerca; es un ciclo, y es alcanzable desde s, así que $w(c) \geq 0$. Quitarlo deja la caminata $q' = \langle v_0, \ldots, v_i, v_{j+1}, \ldots, v_k\rangle$ de s a v, con menos aristas y $w(q') = w(q) - w(c) \leq w(q)$. Repitiendo se llega a un camino p con $w(p) \leq w(q)$. Entonces el ínfimo sobre caminatas es el mínimo sobre caminos, y como hay una cantidad finita de caminos, ese mínimo existe y es un número real. Un camino no repite vértices, así que tiene a lo más n vértices y $n - 1$ aristas. □

La prueba dice qué pasa con cada tipo de ciclo. Uno de peso positivo nunca está en un camino más corto, porque quitarlo baja el peso. Uno de peso cero se puede quitar sin cambiar nada. Uno de peso negativo hace que no haya camino más corto. Por eso basta buscar entre caminos, y ese $n - 1$ es exactamente el número de pasadas de Bellman-Ford.

Cómo se guardan los caminos

Igual que BFS, los algoritmos de esta sección no guardan los caminos completos: cada vértice guarda un **predecesor** $\pi[v]$, que es otro vértice o nulo. Al terminar, la cadena $v, \pi[v], \pi[\pi[v]], \ldots$ recorre hacia atrás un camino más corto de s a v, y se imprime como en BFS (sección 4). A media ejecución los π no tienen por qué describir caminos más cortos, y con un ciclo negativo ni siquiera tienen que formar un árbol (ejemplo 7.17).

Definición 7.5 (Subgrafo predecesor)**.** El **subgrafo predecesor** (*predecessor subgraph*) es $G_\pi = (V_\pi, E_\pi)$, con

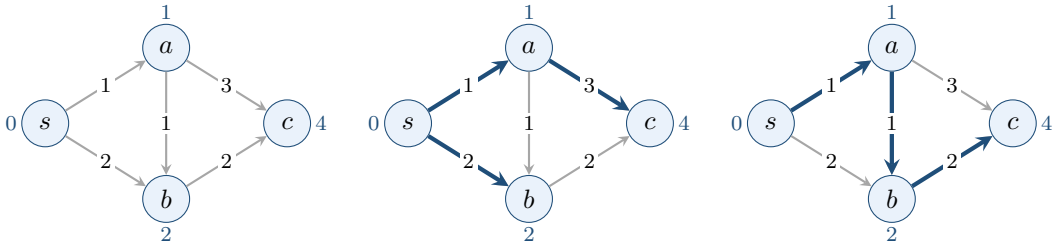
$$V_\pi = \{v \in V : \pi[v] \neq \text{nulo}\} \cup \{s\}\,, \qquad E_\pi = \{(\pi[v], v) : v \in V_\pi - \{s\}\}\,.$$

Definición 7.6 (Árbol de caminos más cortos)**.** Si desde s no se alcanza ningún ciclo negativo, un **árbol de caminos más cortos** (*shortest-paths tree*) con raíz s es un subgrafo $G' = (V', E')$, con $V' \subseteq V$ y $E' \subseteq E$, tal que

1. V' es el conjunto de vértices alcanzables desde s;
2. G' es un árbol con raíz s;
3. para todo $v \in V'$, el único camino de s a v en G' es un camino más corto de s a v en G.

Es el árbol BFS con pesos en lugar de número de aristas, y es el que la clase del 22 de octubre dibujó en azul en el ejemplo de Dijkstra. No es único: cuando dos caminos empatan, cualquiera de los dos sirve.

Ejemplo 7.7 (Dos árboles para las mismas distancias)**.** En este grafo $\delta(s,\cdot) = 0, 1, 2, 4$ para s, a, b, c. A b se llega con peso 2 directo o pasando por a, y a c con peso 4 por a o por b.



(a) el grafo (b) un árbol (c) otro árbol

Los dos son árboles de caminos más cortos válidos (Cormen muestra otro caso en su figura 24.2). Un mismo algoritmo puede dar uno u otro según el orden en que revise las aristas: Bellman-Ford, que viene más adelante, da el de (b) con el orden $(s,a),(s,b),(a,b),(a,c),(b,c)$ y el de (c) con $(s,a),(a,b),(b,c),(s,b),(a,c)$.

Relajación

Todos los algoritmos de la sección comparten el mismo esqueleto. Cada vértice guarda una **estimación** $d[v]$ del peso de camino más corto, que siempre es una cota superior de $\delta(s,v)$, y un predecesor $\pi[v]$. Se arranca con $d[s] = 0$ e ∞ en los demás (Initialize-Single-Source en Cormen):

Algoritmo 7.1 Inicializar-Fuente(G, s)

```
1: procedimiento Inicializar-Fuente(G, s)
2:     para todo v ∈ V hacer
3:         d[v] ← ∞,    π[v] ← nulo
4:     fin para
5:     d[s] ← 0
6: fin procedimiento
```

Después, la única operación que cambia d y π es **relajar** una arista (u,v): si llegar a v pasando por u es más barato que lo que se tiene, se actualizan $d[v]$ y $\pi[v]$ (Relax en Cormen).

Algoritmo 7.2 Relajar(u, v, w)

```
1: procedimiento Relajar(u, v, w)
2:     si d[v] > d[u] + w(u, v) entonces
3:         d[v] ← d[u] + w(u, v),    π[v] ← u
4:     fin si
5: fin procedimiento
```

Inicializar-Fuente cuesta $\Theta(n)$ y Relajar $\mathcal{O}(1)$. Es la misma relajación que la clase del 22 de octubre escribió en línea dentro de Dijkstra. El nombre parece al revés, porque la operación aprieta una cota, y la razón es histórica: las distancias finales cumplen $\delta(s,v) \le \delta(s,u) + w(u,v)$, relajar es forzar la restricción $d[v] \le d[u] + w(u,v)$, y una restricción que ya se cumple está "relajada".

Los algoritmos difieren solo en cuántas veces relajan cada arista y en qué orden. Dijkstra y el algoritmo para grafos acíclicos relajan cada arista exactamente una vez; Bellman-Ford, $n-1$ veces. Que eso baste

lo explican seis propiedades.

Las seis propiedades de la relajación

La primera es un hecho sobre δ. Las otras cinco hablan de cualquier sucesión de relajaciones, y todas suponen lo mismo: el grafo se inicializó con Inicializar-Fuente(G, s) y desde entonces d y π solo cambian por llamadas a Relajar. Aquí van los enunciados con su intuición; las pruebas completas están en la sección 8.

Proposición 7.8 (Desigualdad del triángulo; Cormen, lema 24.10)**.** *Para toda arista $(u, v) \in E$, $\delta(s, v) \leq \delta(s, u) + w(u, v)$.*

Intuición. Ir a u de la mejor manera y luego tomar (u, v) es una forma de llegar a v, y la mejor forma no cuesta más que una forma cualquiera. Vale con infinitos: si u no es alcanzable, el lado derecho es ∞; si $\delta(s, u) = -\infty$, el ciclo negativo que llega a u también llega a v y $\delta(s, v) = -\infty$.

Proposición 7.9 (Cota superior; Cormen, lema 24.11)**.** *En todo momento $d[v] \geq \delta(s, v)$ para todo $v \in V$, y en cuanto $d[v]$ alcanza $\delta(s, v)$ ya no cambia.*

Intuición. Cada valor finito de $d[v]$ es el peso de una caminata de verdad de s a v, la que fueron armando las relajaciones, y ninguna caminata pesa menos que el ínfimo. Como relajar solo baja valores, cuando $d[v]$ toca el piso se queda ahí.

Proposición 7.10 (Sin camino; Cormen, corolario 24.12)**.** *Si no hay camino de s a v, en todo momento $d[v] = \delta(s, v) = \infty$.*

Intuición. Es la cota superior con $\delta(s, v) = \infty$: ninguna relajación baja un vértice al que no llega nada.

Proposición 7.11 (Convergencia; Cormen, lema 24.14)**.** *Si $s \rightsquigarrow u \to v$ es un camino más corto y $d[u] = \delta(s, u)$ en algún momento antes de relajar (u, v), entonces $d[v] = \delta(s, v)$ en todo momento después de esa relajación.*

Intuición. Al relajar queda $d[v] \leq d[u] + w(u, v) = \delta(s, u) + w(u, v) = \delta(s, v)$; la última igualdad es el lema 7.2 aplicado al camino más corto. La cota superior impide bajar más. La distancia correcta de u se hereda a v por la última arista del camino.

Proposición 7.12 (Relajación de caminos; Cormen, lema 24.15)**.** *Sea $p = \langle v_0, v_1, \ldots, v_k \rangle$ un camino más corto de $s = v_0$ a v_k. Si sus aristas se relajan en el orden $(v_0, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k)$, entonces $d[v_k] = \delta(s, v_k)$ desde la última de esas relajaciones en adelante. Vale sin importar qué otras relajaciones se intercalen entre ellas.*

Intuición. Inducción con la convergencia. Al principio $d[v_0] = 0 = \delta(s, s)$. Al relajar (v_{i-1}, v_i), v_{i-1} ya es correcto, así que v_i queda correcto, y lo que se haga en medio no lo estropea, porque un valor correcto ya no se mueve. Esta es la propiedad que hace funcionar a todos los algoritmos de la sección: no hace falta saber cuál es el camino más corto, basta garantizar que sus aristas se relajen en orden, aunque sea revueltas entre muchas otras.

Proposición 7.13 (Subgrafo predecesor; Cormen, lema 24.17)**.** *Si desde s no se alcanza ningún ciclo negativo y en algún momento $d[v] = \delta(s, v)$ para todo $v \in V$, entonces G_π es un árbol de caminos más cortos con raíz s.*

Intuición. Sin ciclos negativos, G_π es siempre un árbol con raíz s: una relajación no puede cerrar un ciclo de predecesores. Al final, además, cada arista del árbol queda **tensa**, $d[v] = d[\pi[v]] + w(\pi[v], v)$. Cuando se fijó $\pi[v] = u$ quedó $d[v] = d[u] + w(u, v)$; desde entonces $d[v]$ no cambió, porque si cambiara

cambiaría $\pi[v]$, y $d[u]$ solo pudo bajar, así que $d[v] \geq d[u] + w(u, v)$. La desigualdad del triángulo da la otra mitad. Sumando las igualdades a lo largo del camino del árbol, los d se cancelan en telescopio y el camino pesa $d[v] - d[s] = \delta(s, v)$.

Idea clave

Un algoritmo de caminos más cortos desde una fuente es una política para escoger qué arista relajar. Si la política garantiza que las aristas de cada camino más corto se relajen en orden, la relajación de caminos da las distancias y la propiedad del subgrafo predecesor da el árbol. Bellman-Ford lo garantiza a fuerza de repetir, el algoritmo para grafos acíclicos con el orden topológico, y Dijkstra con la cola de prioridad.

Bellman-Ford

Bellman-Ford resuelve el caso general: pesos reales cualesquiera, con la posibilidad de que haya ciclos negativos. Devuelve un booleano. Si desde s no se alcanza ningún ciclo negativo devuelve verdadero, y entonces $d[v] = \delta(s, v)$ para todo v y G_π es un árbol de caminos más cortos. Si se alcanza alguno devuelve falso, porque el problema no tiene solución. La idea cabe en una línea: relajar todas las aristas, $n - 1$ veces, y luego revisar.

Algoritmo 7.3 Bellman-Ford(G, w, s)

```
1:  procedimiento BELLMAN-FORD(G, w, s)
2:      INICIALIZAR-FUENTE(G, s)
3:      para i ← 1 hasta n − 1 hacer                    // n − 1 pasadas
4:          para todo (u, v) ∈ E hacer
5:              RELAJAR(u, v, w)
6:          fin para
7:      fin para
8:      para todo (u, v) ∈ E hacer                      // revisión
9:          si d[v] > d[u] + w(u, v) entonces
10:             devolver falso                          // hay un ciclo negativo alcanzable
11:         fin si
12:     fin para
13:     devolver verdadero
14: fin procedimiento
```

- **Línea 3, el $n - 1$.** Por la proposición 7.4, un camino más corto tiene a lo más $n - 1$ aristas, y cada pasada avanza al menos una arista de cada camino más corto. Es una cota de peor caso: cada relajación usa los valores que ya bajaron en la misma pasada, así que con un orden favorable una mejora recorre varias aristas de una vez.
- **Línea 4.** El orden de las aristas es arbitrario. No afecta el resultado, solo cuántas pasadas hacen algo de verdad.
- **Líneas 8 a 12.** Una pasada más que solo pregunta. Sin ciclos negativos todo d ya es exacto y ninguna arista se puede relajar; si alguna todavía se puede, hay un ciclo negativo alcanzable desde s.

Lema 7.14 (Cormen, lema 24.2)**.** *Si desde s no se alcanza ningún ciclo negativo, al terminar las $n - 1$ pasadas de las líneas 3 a 7 se tiene $d[v] = \delta(s, v)$ para todo v alcanzable desde s.*

Demostración. Sea v alcanzable y sea $p = \langle v_0, v_1, \ldots, v_k \rangle$ un camino más corto de $s = v_0$ a $v = v_k$; existe y tiene $k \leq n-1$ aristas por la proposición 7.4. Cada pasada relaja todas las aristas, así que la pasada i relaja, entre otras, a (v_{i-1}, v_i). Las aristas de p quedan relajadas en orden, una en cada una de las pasadas $1, \ldots, k$, y la relajación de caminos (proposición 7.12) da $d[v] = \delta(s, v)$. □

Teorema 7.15 (Corrección de Bellman-Ford; Cormen, teorema 24.4)**.** *Si desde s no se alcanza ningún ciclo negativo, Bellman-Ford(G, w, s) devuelve verdadero, $d[v] = \delta(s, v)$ para todo $v \in V$ y G_π es un árbol de caminos más cortos con raíz s. Si desde s se alcanza un ciclo negativo, devuelve falso.*

Demostración. Sin ciclos negativos alcanzables. Para los vértices alcanzables, $d[v] = \delta(s, v)$ es el lema 7.14; para los demás, la propiedad sin camino. Con todas las distancias correctas, la propiedad del subgrafo predecesor da el árbol. Falta ver que la revisión no falla: para toda arista (u, v), por la desigualdad del triángulo,

$$d[v] = \delta(s, v) \leq \delta(s, u) + w(u, v) = d[u] + w(u, v),$$

así que la condición de la línea 9 nunca se cumple y el algoritmo devuelve verdadero.

Con un ciclo negativo alcanzable. Sea $c = \langle v_0, v_1, \ldots, v_k \rangle$, con $v_0 = v_k$, un ciclo alcanzable desde s con

$$\sum_{i=1}^{k} w(v_{i-1}, v_i) < 0.$$

Supongamos que el algoritmo devuelve verdadero. Entonces ninguna arista del ciclo cumplió la condición de la línea 9: $d[v_i] \leq d[v_{i-1}] + w(v_{i-1}, v_i)$ para $i = 1, \ldots, k$. Sumando las k desigualdades,

$$\sum_{i=1}^{k} d[v_i] \leq \sum_{i=1}^{k} d[v_{i-1}] + \sum_{i=1}^{k} w(v_{i-1}, v_i).$$

Como $v_0 = v_k$, cada vértice del ciclo aparece exactamente una vez en cada una de las dos sumas de d, así que son iguales. Además son finitas. Cada v_i es alcanzable por un camino de a lo más $n-1$ aristas, la pasada j relaja la j-ésima arista de ese camino y d solo baja, así que tras las $n-1$ pasadas $d[v_i] < \infty$; y ningún d vale $-\infty$, porque solo toma valores que son pesos de caminatas. Restando las dos sumas iguales queda $0 \leq \sum_{i=1}^{k} w(v_{i-1}, v_i)$, que contradice que el ciclo es negativo. □

La segunda mitad es el corazón de la detección. Un ciclo negativo no puede tener satisfechas las desigualdades de todas sus aristas: al sumarlas alrededor del ciclo los d se cancelan y queda $0 \leq w(c)$. Alguna tiene que fallar, y esa es la que la revisión encuentra. El ejemplo 7.17 lo muestra con números.

Costo. La línea 2 cuesta $\Theta(n)$. Las líneas 3 a 7 hacen $n-1$ pasadas, y cada pasada recorre las m aristas con una relajación de $\mathcal{O}(1)$: $(n-1) \cdot \Theta(m) = \Theta(nm)$. La revisión recorre las aristas otra vez, $\mathcal{O}(m)$. En total, $\Theta(nm)$, con $\Theta(n)$ de memoria extra para d y π. Dos precisiones sobre la estructura de datos:

- "Para toda arista" con listas de adyacencia es "para cada u, para cada v en su lista", que cuesta $\Theta(n+m)$ por pasada y no $\Theta(m)$. El total es $\Theta(n(n+m))$, que coincide con $\Theta(nm)$ cuando $m = \Omega(n)$, por ejemplo cuando todo vértice es alcanzable desde s y por tanto $m \geq n-1$. Con una lista plana de aristas cada pasada sí cuesta $\Theta(m)$.
- Con matriz de adyacencia recorrer las aristas cuesta $\Theta(n^2)$ por pasada y el total es $\Theta(n^3)$. En un grafo denso, $m = \Theta(n^2)$, Bellman-Ford cuesta $\Theta(n^3)$ aun con listas: lo mismo que Floyd para *todos* los pares.

Ejemplo 7.16 (Bellman-Ford sobre la figura 24.4 de Cormen)**.** El grafo tiene $V = \{s, t, x, y, z\}$, fuente s y diez aristas:

$$\begin{array}{lllll} (s,t):6 & (s,y):7 & (t,x):5 & (t,y):8 & (t,z):-4 \\ (x,t):-2 & (y,x):-3 & (y,z):9 & (z,s):2 & (z,x):7 \end{array}$$

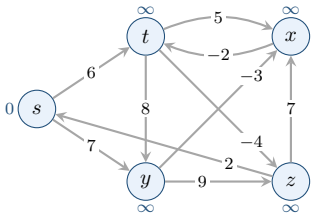
Cada pasada relaja las aristas en el orden de Cormen,

$$(t,x),(t,y),(t,z),(x,t),(y,x),(y,z),(z,x),(z,s),(s,t),(s,y).$$

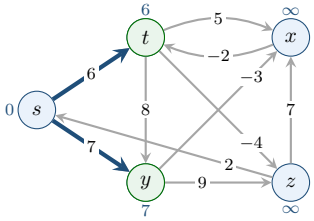
Las aristas de s van al final a propósito: es un orden malo, que obliga a usar las $n-1=4$ pasadas. Cada celda es $d[v]$ con $\pi[v]$ entre paréntesis, al terminar la pasada.

Pasada	t	x	y	z	**Relaciones que cambian algo**
inicial	∞	∞	∞	∞	
1	6 (s)	∞	7 (s)	∞	(s,t): $\infty \to 6$; (s,y): $\infty \to 7$
2	6 (s)	4 (y)	7 (s)	2 (t)	(t,x): $\infty \to 11$; (t,z): $\infty \to 2$; (y,x): $11 \to 4$
3	2 (x)	4 (y)	7 (s)	2 (t)	(x,t): $6 \to 2$
4	2 (x)	4 (y)	7 (s)	-2 (t)	(t,z): $2 \to -2$

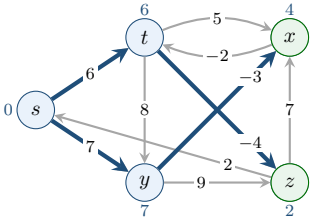
La misma ejecución en dibujos. En verde, los vértices cuya d bajó en esa pasada; en azul, las aristas $(\pi[v], v)$.



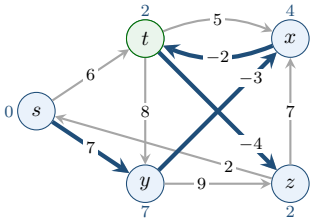
(a) inicial



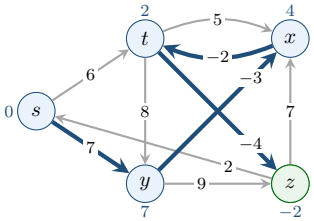
(b) tras la pasada 1



(c) tras la pasada 2



(d) tras la pasada 3



(e) tras la pasada 4

- **Pasada 1.** Solo cambian t e y. Las ocho primeras aristas salen de vértices con $d = \infty$, y la regla $\infty + w = \infty$ hace que no muevan nada.
- **Pasada 2.** (t,x) baja x a 11 y, más adelante en la misma pasada, (y,x) lo baja a 4. (t,z) pone z en 2, que todavía no es su valor final.
- **Pasada 3.** (x,t) da $4-2=2<6$: t tenía 6 por la arista directa y mejora por s,y,x,t, un camino de tres aristas.
- **Pasada 4.** (t,z) da $2-4=-2<2$, y z se corrige por segunda vez.
- **Revisión.** Ninguna arista se puede relajar y el algoritmo devuelve verdadero. Las aristas del árbol quedan tensas, como $d[x] = 4 = d[y] + w(y,x)$; las demás, con holgura, como (z,x): $4 \le -2 + 7$.

El árbol final es un solo camino, $s \to y \to x \to t \to z$, con $n-1=4$ aristas: el peor caso de la cota, y con el orden de Cormen cada pasada avanza exactamente una arista de él. Con las aristas de s al

principio de la lista, la pasada 1 ya deja $t = 6$, $x = 4$, $y = 7$ y $z = 2$, porque (s, y) y luego (y, x) caen en la misma pasada; converge en tres pasadas y la cuarta no cambia nada. El orden cambia cuántas pasadas sirven, no el resultado: $\delta(s, \cdot) = 0, 2, 4, 7, -2$ para s, t, x, y, z.

Un detalle más: el ciclo s, y, x, t, z, s pesa $7 - 3 - 2 - 4 + 2 = 0$. Es un ciclo de peso cero, de los que no estorban, y por eso la arista (z, s) también queda tensa, $d[s] = 0 = d[z] + 2$, sin estar en el árbol.

Ejemplo 7.17 (El mismo grafo con un ciclo negativo)**.** Se cambia un solo peso, $w(z, x)$, de 7 a 4 (Cormen, ejercicio 24.1-1). Aparece el ciclo $z \to x \to t \to z$ de peso $4 - 2 - 4 = -2$, alcanzable desde s por (s, t). Como además hay arista (z, s), desde el ciclo se llega a todos los vértices, incluida la fuente: $\delta(s, v) = -\infty$ para todo v, también para $v = s$.

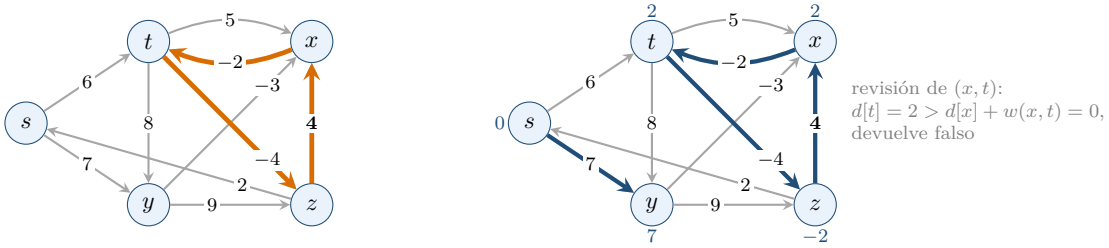
Las pasadas 1 a 3 son idénticas a las del ejemplo anterior: mientras $d[z] \geq 0$, la arista (z, x) no mejora a x, valga 7 o 4. La diferencia sale en la pasada 4.

Pasada	s	t	x	y	z	**Qué pasa**
3	0	2 (x)	4 (y)	7 (s)	2 (t)	idéntico al grafo original
4	0	2 (x)	2 (z)	7 (s)	-2 (t)	(t, z): $2 \to -2$; (z, x): $4 \to 2$
Revisión: (x, t) da $d[t] = 2 > d[x] + w(x, t) = 2 - 2 = 0$, así que devuelve falso.						
5	0	0	2	7	-2	si se siguiera relajando
6	-2	0	0	5	-4	hasta s baja
8	-4	-2	-2	3	-6	cada dos pasadas, 2 menos

En la pasada 4, (t, z) baja z a -2 y unas aristas después (z, x) da $-2 + 4 = 2 < 4$; con el peso original daba $-2 + 7 = 5$ y no cambiaba nada. En la revisión, las tres desigualdades del ciclo quedan así:

- (z, x): $d[x] = 2 \leq d[z] + 4 = 2$, se cumple;
- (x, t): $d[t] = 2 \leq d[x] - 2 = 0$, **falla**;
- (t, z): $d[z] = -2 \leq d[t] - 4 = -2$, se cumple.

Es la prueba del teorema 7.15 con números: la suma de los d del ciclo es $2 + 2 - 2 = 2$, y si las tres se cumplieran quedaría $2 \leq 2 + (-2)$. Alguna tiene que fallar, y falla (x, t).



(a) el ciclo z, x, t, z pesa -2 (b) tras la pasada 4

Dos cosas se ven en (b). La primera: G_π ya no es un árbol. $\pi[t] = x$, $\pi[z] = t$ y $\pi[x] = z$ forman el ciclo $x \to t \to z \to x$, exactamente el ciclo negativo, y t ya no cuelga de s. La segunda está en las últimas filas de la tabla: si se siguiera relajando, los valores bajarían sin fin, 2 cada dos pasadas, el peso del ciclo, y hasta $d[s]$ deja de valer 0. Por eso el algoritmo se detiene en $n - 1$ pasadas y pregunta, en lugar de esperar a que las d se estabilicen.

Ojito...

El infinito no es un entero grande. En código es tentador representar ∞ con un centinela como 10^9. Con pesos negativos, $10^9 + (-8) < 10^9$, y un vértice no alcanzable "mejora" a otro no alcanzable. En un grafo con s aislado y el ciclo h, i, j de la figura 24.1, Bellman-Ford sin protección devuelve falso, aunque ningún ciclo negativo sea alcanzable. La protección es relajar solo si $d[u] \neq \infty$, que es lo que la regla $\infty + a = \infty$ dice en papel. Si el centinela es el entero más grande del lenguaje, además, la suma se desborda.

Ojito...

Qué detecta y qué no. Bellman-Ford solo ve ciclos negativos alcanzables desde s. Uno al que no se llega no afecta nada y sus vértices se quedan en ∞, que es correcto. Para saber si el grafo tiene *algún* ciclo negativo se agrega un vértice nuevo con aristas de peso 0 hacia todos y se corre desde él; equivale a arrancar con todas las d en 0, y es el mismo truco de las restricciones de diferencia, al final de la sección.

Cuando devuelve falso, las d no significan nada. Son números a media caída, G_π puede tener ciclos y ni siquiera $d[s] = 0$ es confiable, como en el ejemplo anterior.

Grafos no dirigidos. Una arista $\{u, v\}$ es un par de arcos, y si su peso es negativo forman el ciclo u, v, u de peso $2w(u, v) < 0$. Bellman-Ford devuelve falso en cuanto hay una arista negativa alcanzable. Con pesos no negativos funciona, pero ahí Dijkstra es más barato.

Observación. Bellman-Ford es programación dinámica sobre el número de aristas. Si $d^{(i)}[v]$ es el menor peso de una caminata de s a v con a lo más i aristas,

$$d^{(i)}[v] = \min\Big(d^{(i-1)}[v],\ \min_{(u,v)\in E}\big(d^{(i-1)}[u] + w(u,v)\big)\Big),$$

y la pasada i calcula esa capa, solo que en el lugar: como usa valores ya actualizados, tras la pasada i queda $d[v] \leq d^{(i)}[v]$, que es todo lo que la prueba necesita. Para todos los pares, la misma recurrencia da el método por multiplicación de matrices (sección 9). Cormen no lo presenta así en este capítulo; solo clasifica a Dijkstra como voraz y a Floyd-Warshall como programación dinámica.

Caminos más cortos en un grafo acíclico

En un grafo dirigido acíclico no hace falta repetir pasadas. Si los vértices se procesan en orden topológico, las aristas de cualquier camino aparecen en orden, y una sola pasada cumple la hipótesis de la relajación de caminos. Los pesos negativos están permitidos: sin ciclos no hay ciclos negativos, y $\delta(s, v)$ siempre está bien definido. El orden topológico se obtiene con DFS, listando los vértices por tiempo de salida sale$[u]$ decreciente (sección 5). En Cormen el procedimiento se llama DAG-Shortest-Paths.

Algoritmo 7.4 Caminos-DAG(G, w, s)

```
1: procedimiento Caminos-DAG(G, w, s)
2:     ordenar topológicamente los vértices de G
3:     Inicializar-Fuente(G, s)
4:     para todo u ∈ V, en orden topológico hacer
5:         para todo v en la lista de adyacencia de u hacer
6:             Relajar(u, v, w)
7:         fin para
8:     fin para
9: fin procedimiento
```

Cuando le toca a u, todos los vértices con aristas hacia u ya se procesaron, así que $d[u]$ ya es definitiva y sus aristas de salida se relajan con el valor bueno. Es la misma idea que Dijkstra, fijar un vértice y relajar lo que sale de él, solo que el orden se conoce de antemano y no hace falta cola de prioridad.

Teorema 7.18 (Cormen, teorema 24.5)**.** *Si G es dirigido, pesado y acíclico, al terminar* Caminos-DAG(G, w, s) *se tiene $d[v] = \delta(s, v)$ para todo $v \in V$, y G_π es un árbol de caminos más cortos con raíz s.*

Demostración. Si v no es alcanzable desde s, la propiedad sin camino da $d[v] = \infty = \delta(s, v)$. Si lo es, como no hay ciclos, $\delta(s, v)$ es un número real y lo alcanza un camino $p = \langle v_0, v_1, \ldots, v_k \rangle$ con $v_0 = s$ y $v_k = v$ (proposición 7.4). En el orden topológico cada v_{i-1} va antes que v_i, porque (v_{i-1}, v_i) es una arista, así que la línea 4 procesa $v_0, v_1, \ldots, v_{k-1}$ en ese orden y las aristas de p se relajan en el orden $(v_0, v_1), \ldots, (v_{k-1}, v_k)$. La relajación de caminos da $d[v] = \delta(s, v)$. El árbol sale de la propiedad del subgrafo predecesor. □

Costo. El orden topológico con DFS cuesta $\Theta(n + m)$, e Inicializar-Fuente, $\Theta(n)$. El ciclo de la línea 4 da n vueltas y el de la línea 5 recorre la lista de u. Por análisis agregado, como en BFS, las listas suman m entradas, así que en total se hacen exactamente m relajaciones de $\mathcal{O}(1)$. Total: $\Theta(n + m)$, lineal en el tamaño de las listas. Con matriz de adyacencia sería $\Theta(n^2)$.

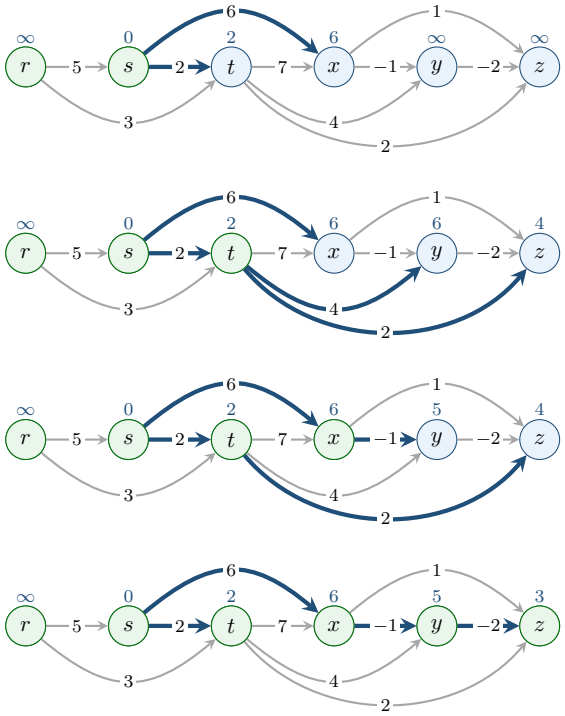
Ejemplo 7.19 (Caminos en el DAG de la figura 24.5 de Cormen)**.** El grafo tiene $V = \{r, s, t, x, y, z\}$, ya en orden topológico (todas las aristas van de izquierda a derecha), fuente s y diez aristas:

$$\begin{array}{lllll} (r,s):5 & (r,t):3 & (s,t):2 & (s,x):6 & (t,x):7 \\ (t,y):4 & (t,z):2 & (x,y):-1 & (x,z):1 & (y,z):-2 \end{array}$$

DFS desde r, con los vecinos en orden alfabético, da justamente el orden r, s, t, x, y, z por sale$[u]$ decreciente. Una fila por vértice procesado, con las d al terminar de relajar sus aristas:

u	r	s	t	x	y	z	**Relajaciones**
inicial	∞	0	∞	∞	∞	∞	
r	∞	0	∞	∞	∞	∞	$d[r] = \infty$: (r,s) y (r,t) no cambian nada
s	∞	0	2	6	∞	∞	t: $\infty \to 2$; x: $\infty \to 6$
t	∞	0	2	6	6	4	x: $2 + 7 = 9$, no; y: $\infty \to 6$; z: $\infty \to 4$
x	∞	0	2	6	5	4	y: $6 - 1 = 5 < 6$; z: $6 + 1 = 7$, no
y	∞	0	2	6	5	3	z: $5 - 2 = 3 < 4$
z	∞	0	2	6	5	3	sin aristas de salida

En dibujos, en verde los vértices ya procesados y en azul las aristas $(\pi[v], v)$:



(a) tras $u = r$ y $u = s$

(b) tras $u = t$

(c) tras $u = x$

(d) tras $u = y$ y $u = z$: final

- r va antes que s y no es alcanzable: sus aristas se relajan, pero con $d[r] = \infty$ no cambian nada.
- A x, la arista directa (s, x) con 6 le gana a s, t, x con 9.
- y baja a 6 por t y luego a 5 por x, gracias a la arista de peso -1; z baja a 4 y luego a 3. Los pesos negativos no estorban: cuando le toca a cada vértice, ya le llegó todo lo que le podía llegar.
- Se hicieron exactamente $10 = m$ relajaciones, una por arista.

Resultado: $\delta(s, \cdot) = \infty, 0, 2, 6, 5, 3$ para r, s, t, x, y, z, con $\pi[t] = \pi[x] = s$, $\pi[y] = x$ y $\pi[z] = y$.

Ruta crítica

En un **diagrama PERT** (*program evaluation and review technique*) las aristas son trabajos y sus pesos, la duración de cada uno; los vértices son eventos. Si (u, v) entra a v y (v, x) sale de v, el trabajo (u, v) tiene que terminar antes de que empiece (v, x). Una **ruta crítica** (*critical path*) es un camino *más largo* del DAG: la sucesión de trabajos que más tarda, y por tanto una cota inferior de la duración del proyecto aunque todo lo demás se haga en paralelo. Hay dos formas de calcularla, las dos en $\Theta(n + m)$:

- negar todos los pesos y correr Caminos-DAG: el camino más corto del grafo negado es el más largo del original;
- correr Caminos-DAG cambiando ∞ por $-\infty$ en Inicializar-Fuente y $>$ por $<$ en Relajar.

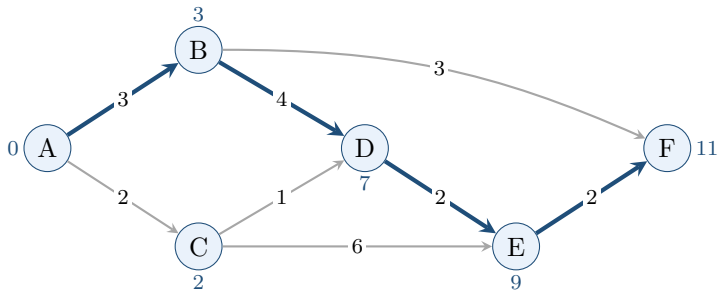
Funciona porque en un DAG negar los pesos no crea ciclos negativos. En un grafo con ciclos el camino más largo es NP-difícil: negar los pesos de un ciclo positivo da uno negativo, y $\delta = -\infty$ deja de decir algo sobre caminos sin repetir vértices.

Ejemplo 7.20 (Ruta crítica de un proyecto)**.** Un proyecto de investigación con seis eventos, de A (inicio) a F (fin). Cada arista es una tarea con su duración en días. A la derecha, la traza de Caminos-DAG con los pesos *negados*, en el orden topológico A, B, C, D, E, F.

Arista	Tarea	Días
$A \to B$	leer la literatura	3
$A \to C$	conseguir los datos	2
$B \to D$	implementar el modelo	4
$C \to D$	limpiar los datos	1
$C \to E$	etiquetar un conjunto de prueba	6
$D \to E$	entrenar	2
$B \to F$	escribir el estado del arte	3
$E \to F$	evaluar y redactar resultados	2

u	A	B	C	D	E	F
A	0	-3	-2	∞	∞	∞
B	0	-3	-2	-7	∞	-6
C	0	-3	-2	-7	-8	-6
D	0	-3	-2	-7	-9	-6
E	0	-3	-2	-7	-9	-11
F	0	-3	-2	-7	-9	-11

Al cambiar el signo, $-d$ es el día más temprano en que puede ocurrir cada evento: D no antes del día 7 (hay que leer e implementar, $3 + 4$; los datos limpios están desde el día 3), E no antes del día 9, y el proyecto no termina antes del día 11.



La ruta crítica es A, B, D, E, F: $3 + 4 + 2 + 2 = 11$ días. Retrasar cualquiera de esas cuatro tareas retrasa el proyecto. El camino A, C, E, F dura 10, casi crítico: etiquetar puede atrasarse un día sin consecuencias. Lo que *no* sirve es el camino más corto, A, B, F con 6: no dice nada del proyecto, porque todas las tareas se tienen que hacer. Las cuatro rutas de A a F duran 11, 6, 7 y 10 días.

Ojito...

El DAG tiene que ser DAG. Si el grafo tiene un ciclo no hay orden topológico, pero el orden por sale[u] decreciente se calcula igual sin quejarse, a menos que se revisen las aristas de retroceso, y Caminos-DAG devuelve basura sin avisar. A diferencia de Bellman-Ford, no tiene revisión final.

Del otro lado, dos ahorros válidos. Los vértices que van antes de s en el orden no son alcanzables, porque todo camino desde s va hacia adelante, así que el ciclo de la línea 4 puede arrancar en s. Y el último vértice del orden no tiene aristas de salida, así que tampoco hace falta procesarlo (Cormen, ejercicio 24.2-2).

Dijkstra, versión de Cormen

La clase del 22 de octubre presentó Dijkstra con un ejemplo completo sobre el grafo pesado (sección 1), que sigue valiendo tal cual: un grafo no dirigido se trata como un par de arcos opuestos por arista, y la traza es la misma con el pseudocódigo de abajo. Aquí se reescribe con Inicializar-Fuente y Relajar, como lo hace Cormen, para que encaje en el esqueleto común y se pueda probar con las seis propiedades. La hipótesis es la de siempre: $w(u, v) \geq 0$ para toda arista.

Algoritmo 7.5 Dijkstra(G, w, s), versión de Cormen

```
1:  procedimiento DIJKSTRA(G, w, s)
2:      INICIALIZAR-FUENTE(G, s)
3:      S ← ∅
4:      Q ← V                                   // cola de mínimos con llave d
5:      mientras Q ≠ ∅ hacer
6:          u ← EXTRAER-MIN(Q)
7:          S ← S ∪ {u}                         // aquí d[u] = δ(s, u)
8:          para todo v en la lista de adyacencia de u hacer
9:              RELAJAR(u, v, w)                // si baja d[v]: DECREMENTAR-LLAVE
10:         fin para
11:     fin mientras
12: fin procedimiento
```

- **Línea 4.** $Q \leftarrow V$ inserta los n vértices de una vez. Nunca se inserta nada después y cada vértice sale una sola vez, así que el **mientras** da exactamente n vueltas. La primera vez sale s, el único con d finita.
- **Línea 7.** El algoritmo no necesita S, que podría leer como $V - Q$; está para la prueba. El comentario marca el momento que el teorema de abajo garantiza.
- **Línea 9.** Relajar puede bajar la llave de v en Q, así que en una implementación con montículo cada relajación exitosa es un Decrementar-Llave. Con $w \geq 0$ esa relajación nunca le pega a un vértice de S: si $v \in S$, por el teorema y la desigualdad del triángulo, $d[v] = \delta(s, v) \leq \delta(s, u) + w(u, v) = d[u] + w(u, v)$, y la condición de Relajar falla. El decremento siempre actúa sobre un vértice que sigue en el montículo. Con pesos negativos esto deja de ser cierto (ejemplo 7.23).

Por qué funciona

La clase del 22 de octubre dio el argumento en corto: cuando u sale con la d mínima, el *primer* vértice no fijado y de un camino más corto hacia u ya tiene su d exacta, porque su antecesor ya fijado relajó la arista hacia él, y el tramo de y a u no resta. Esa pieza es la que hace funcionar todo: sin ella, saber que $d[y] \geq d[u]$ no dice nada del costo real de pasar por y. Aquí va la prueba completa, apoyada en las propiedades de la relajación.

Teorema 7.21 (Corrección de Dijkstra; Cormen, teorema 24.6)**.** *Si $G = (V, E)$ es dirigido, $w(u, v) \geq 0$ para toda arista y $s \in V$, Dijkstra(G, w, s) termina con $d[u] = \delta(s, u)$ para todo $u \in V$.*

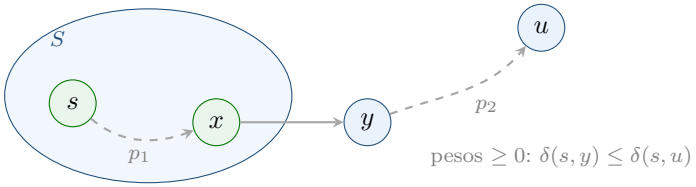
Demostración. Se prueba el invariante: *al inicio de cada vuelta del **mientras**, $d[v] = \delta(s, v)$ para todo $v \in S$.*

Inicio. $S = \emptyset$ y no hay nada que probar.

Mantenimiento. Basta ver que el vértice u que entra a S en la línea 7 ya trae $d[u] = \delta(s, u)$; después no cambia, por la cota superior. Supongamos lo contrario, y sea u el primer vértice que entra a S con $d[u] \neq \delta(s, u)$.

- $u \neq s$. El primero en salir es s, con $d[s] = 0 = \delta(s, s)$; como no hay pesos negativos, ninguna caminata cerrada pesa menos que 0. En particular, justo antes de agregar u, $s \in S$.
- Hay un camino de s a u. Si no, la propiedad sin camino daría $d[u] = \infty = \delta(s, u)$.

- Sea p un camino más corto de s a u; existe por la proposición 7.4, porque sin pesos negativos no hay ciclos negativos. p empieza en $s \in S$ y termina en $u \notin S$. Sea y el primer vértice de p que no está en S y $x \in S$ el anterior a él en p. Puede ser $x = s$ o $y = u$. El camino se parte en $s \overset{p_1}{\rightsquigarrow} x \to y \overset{p_2}{\rightsquigarrow} u$.



Primero, $d[y] = \delta(s, y)$ en el momento en que u entra a S. En efecto, x entró a S antes que u, así que por la elección de u entró con $d[x] = \delta(s, x)$, y en esa vuelta se relajó (x, y). Como $s \overset{p_1}{\rightsquigarrow} x \to y$ es un camino más corto (lema 7.2), la convergencia da $d[y] = \delta(s, y)$ desde entonces.

Segundo, y va antes que u en p y p_2 no tiene pesos negativos, así que $\delta(s, y) = w(p_1) + w(x, y) \leq w(p) = \delta(s, u)$. Con la cota superior,

$$d[y] = \delta(s, y) \leq \delta(s, u) \leq d[u].$$

Pero cuando se eligió u en la línea 6, y también estaba en Q, así que $d[u] \leq d[y]$. Todas las desigualdades son igualdades, y $d[u] = \delta(s, u)$, lo que contradice la elección de u.

Terminación. El ciclo acaba con $Q = \emptyset$, y como $Q = V - S$ al inicio de cada vuelta, $S = V$: todos los vértices tienen $d[v] = \delta(s, v)$. □

Corolario 7.22 (Cormen, corolario 24.7)**.** *Con las mismas hipótesis, al terminar G_π es un árbol de caminos más cortos con raíz s.*

Demostración. Inmediato del teorema 7.21 y la propiedad del subgrafo predecesor; con $w \geq 0$ no hay ciclos negativos. □

La prueba usa la hipótesis $w \geq 0$ en un solo lugar, la desigualdad $\delta(s, y) \leq \delta(s, u)$. Si el tramo p_2 resta, y puede estar más lejos que u aunque vaya antes en el camino, y el argumento se cae justo ahí.

Costo según la cola de prioridad

Fuera de las operaciones de la cola, todo cuesta $\mathcal{O}(1)$ por vértice o por arista. Las operaciones se cuentan así:

- **Insertar**: n, todas en la línea 4.
- **Extraer-Min**: una por vuelta del **mientras**, n en total.
- **Decrementar-Llave**: a lo más una por relajación, y aquí entra el análisis agregado. Cada vértice entra a S una sola vez, así que la lista de adyacencia de cada u se recorre una sola vez, en la vuelta en que sale u. Las listas suman m entradas: en toda la ejecución hay exactamente m relajaciones y a lo más m decrementos. Contar por vuelta daría n vueltas con hasta $n - 1$ decrementos cada una, $\mathcal{O}(n^2)$, una cota mucho peor en un grafo disperso.

El total es $T = T_{\text{construir}} + n \cdot T_{\text{extraer}} + m \cdot T_{\text{decrementar}}$, y todo depende de la cola:

Cola	Insertar (n veces)	Extraer-Min (n veces)	Decrementar ($\leq m$ veces)	Total
arreglo por vértice	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n^2+m)=\mathcal{O}(n^2)$
montículo binario	$\mathcal{O}(n)$ en total	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}((n+m)\log n)$
montículo de Fibonacci	$\mathcal{O}(1)$	$\mathcal{O}(\log n)$ amort.	$\mathcal{O}(1)$ amort.	$\mathcal{O}(n\log n+m)$

- **Arreglo indexado por vértice.** $d[v]$ vive en la casilla v, con una marca de si sigue en Q. Insertar y decrementar son asignaciones; extraer el mínimo obliga a recorrer el arreglo, $\Theta(n)$. Total $\mathcal{O}(n^2+m)=\mathcal{O}(n^2)$, porque $m\leq n^2$.
- **Montículo binario.** Construirlo cuesta $\mathcal{O}(n)$, como en la clase del 17 de septiembre; extraer y decrementar, $\mathcal{O}(\log n)$ cada una. Para decrementar hay que saber dónde está v dentro del montículo: cada vértice guarda su posición y el montículo la actualiza en cada intercambio. Total $\mathcal{O}((n+m)\log n)$, que es $\mathcal{O}(m\log n)$ si todos los vértices son alcanzables desde s, porque entonces $m\geq n-1$.
- **Montículo de Fibonacci.** Extraer cuesta $\mathcal{O}(\log n)$ amortizado y decrementar $\mathcal{O}(1)$ amortizado. Total $\mathcal{O}(n\log n+m)$. Cormen cuenta que se inventaron justo por esto: Dijkstra hace muchos más decrementos que extracciones, y abaratar el decremento sin encarecer la extracción mejora el total.

El umbral de densidad. El montículo binario le gana al arreglo cuando $(n+m)\log n=o(n^2)$, o sea cuando $m=o(n^2/\log n)$. En un grafo denso, $m=\Theta(n^2)$, el binario cuesta $\Theta(n^2\log n)$ y el arreglo gana. Fibonacci nunca pierde asintóticamente: en denso empata con el arreglo, en muy disperso ($m=\mathcal{O}(n)$) empata con el binario, y en medio gana. Con $n=10^4$, sin constantes:

m	arreglo, n^2	binario, $(n+m)\log_2 n$	Fibonacci, $n\log_2 n+m$
$5\cdot10^4$ (disperso)	$1.0\cdot10^8$	$8.0\cdot10^5$	$1.8\cdot10^5$
$10^6=n^{1.5}$	$1.0\cdot10^8$	$1.3\cdot10^7$	$1.1\cdot10^6$
$5\cdot10^7$ (denso)	$1.0\cdot10^8$	$6.6\cdot10^8$	$5.0\cdot10^7$

Aquí el umbral $n^2/\log_2 n$ queda en unos $7.5\cdot10^6$ aristas. En la práctica casi siempre se usa el montículo binario: los grafos reales suelen ser dispersos y las constantes del de Fibonacci son grandes.

Parecidos con BFS y con Prim

BFS. S es el conjunto de vértices negros de BFS: en los dos casos, lo que está ahí ya tiene su distancia final. Con todos los pesos iguales a 1, la cola de prioridad se comporta como la cola de BFS y los vértices salen por capas de distancia. Cormen lo hace preciso en el ejercicio 24.3-7: si cada arista de peso entero w se cambia por una cadena de w aristas de peso 1, BFS sobre el grafo nuevo pinta de negro los vértices originales en el mismo orden en que Dijkstra los extrae, salvo empates. Dijkstra es BFS con una cola que sabe de pesos.

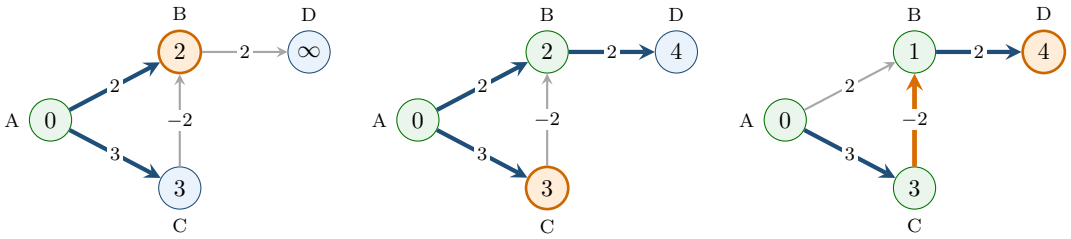
Prim. Los dos hacen crecer un conjunto, el árbol en Prim y S en Dijkstra, usan una cola de prioridad de mínimos para elegir el vértice más ligero fuera de él, lo agregan y ajustan las llaves de sus vecinos. Solo cambia la llave: $d[u]+w(u,v)$ en Dijkstra, $w(u,v)$ en Prim (sección 6). Por eso cuestan lo mismo con cada cola. Pero resuelven problemas distintos, y sus árboles difieren, como mostró la clase del 22 de octubre al comparar el árbol de Dijkstra con el de Kruskal.

Con una arista negativa

Ejemplo 7.23 (Dijkstra falla con una arista negativa)**.** Grafo dirigido con $A \to B$ de peso 2, $A \to C$ de peso 3, $C \to B$ de peso -2 y $B \to D$ de peso 2, fuente A. Las distancias reales son $\delta(A, B) = 1$ por A, C, B y $\delta(A, D) = 3$ por A, C, B, D. Así corre el pseudocódigo de arriba; en negritas, los vértices de S.

Iter.	**Sale**	A	B	C	D
0		0	∞	∞	∞
1	A	**0**	2	3	∞
2	B	**0**	**2**	3	4
3	C	**0**	**1**	**3**	4
4	D	**0**	**1**	**3**	**4**
real		0	1	3	3

B sale en la iteración 2 con $d = 2$ y relaja $B \to D$: D queda en 4. En la iteración 3 sale C y la relajación $C \to B$ baja $d[B]$ a $3 - 2 = 1$, pero B ya está en S y nadie vuelve a relajar $B \to D$. D sale con 4 y la respuesta es incorrecta. En la prueba del teorema, cuando sale B el camino más corto hacia él es A, C, B; ahí $y = C$, con $d[C] = \delta(A, C) = 3$, pero $\delta(A, B) = 1 < 3$ porque $p_2 = (C, B)$ pesa -2.



(a) sale A (b) sale B con $d = 2$ (c) sale C: B baja, D no

Dentro de cada vértice, su d. Verde: en S. Naranja: el mínimo de Q, que sale en la siguiente iteración. Azul: las aristas $(\pi[v], v)$. La arista resaltada en (c) es la relajación tardía: el árbol de predecesores ya es el correcto, A, C, B, D, pero $d[D] = 4 > d[B] + w(B, D) = 3$, porque el valor de D se calculó con el $d[B]$ viejo.

Ojito...

Por qué hace falta D. Con solo A, B y C, el pseudocódigo de Dijkstra que relaja todos los vecinos sin preguntar si ya salieron (el de la clase del 22 de octubre) termina con $d[B] = 1$, que es correcto: la relajación tardía de $C \to B$ sí baja $d[B]$, aunque B ya esté en S. Lo que falla es la afirmación de que $d[B]$ quedó fija al salir y, con montículo, el Decrementar-Llave sobre un vértice que ya no está en él. El error llega a la salida en cuanto algo depende de B, y para eso está D. Muchas implementaciones con montículo se saltan los vecinos que ya están en S; esa variante ni siquiera corrige a B y termina con $d = 0, 2, 3, 4$ para A, B, C, D. En ningún caso hay garantía.

Ojito...

No siempre falla. Si las únicas aristas negativas salen de s y no hay ciclos negativos, Dijkstra es correcto (Cormen, ejercicio 24.3-10): en la prueba, p_2 va de $y \neq s$ a u por un camino que no vuelve a pasar por s, así que no tiene aristas negativas. Con pesos negativos se pierde la garantía, no necesariamente la respuesta.

Sumar una constante no arregla nada. Es el arreglo que se le ocurre a todos: sumar a cada peso lo suficiente para que ninguno sea negativo. Falla porque castiga más a los caminos con más aristas. En el ejemplo, sumando 2, $A \to B$ pesa 4 y A, C, B pesa $5 + 0 = 5$, y el camino más corto a B cambia de A, C, B a la arista directa. La forma correcta de volver no negativos los pesos es la repesada de Johnson (sección 9), que calcula sus potenciales con Bellman-Ford.

Montículo sin decremento. Bibliotecas como `heapq` de Python no tienen Decrementar-Llave. Lo usual es meter un par $(d[v], v)$ nuevo cada vez que baja $d[v]$ e ignorarlo al sacarlo si v ya está en S. El montículo llega a tener hasta m entradas y el costo es $\mathcal{O}(m \log m) = \mathcal{O}(m \log n)$, porque $m \leq n^2$.

Restricciones de diferencia

Un **programa lineal** pide un vector x que maximice una función lineal $c^T x$ sujeto a un sistema de desigualdades $Ax \leq b$. A veces no hay función objetivo y solo interesa la **factibilidad**: encontrar algún x con $Ax \leq b$ o decidir que no existe. Hay algoritmos polinomiales para programación lineal en general, pero muchos casos particulares se resuelven más rápido con algoritmos de grafos. Este es uno.

Definición 7.24 (Sistema de restricciones de diferencia)**.** Un **sistema de restricciones de diferencia** (*system of difference constraints*) es un programa lineal $Ax \leq b$ en el que cada renglón de A tiene un 1, un -1 y ceros. Con incógnitas $x_1, \ldots, x_p$ y r restricciones, el renglón k dice

$$x_j - x_i \leq b_k, \qquad 1 \leq i, j \leq p,\ i \neq j,\ 1 \leq k \leq r.$$

Cormen llama n al número de incógnitas y m al de restricciones. Aquí son p y r, porque n y m se quedan para los vértices y las aristas del grafo que se va a construir, que tendrá $n = p + 1$ y $m = p + r$.

La aplicación natural es la calendarización. Si x_i es el instante en que empieza la tarea i, "la tarea j empieza a lo más b unidades después de la i" es $x_j - x_i \leq b$, y "la tarea j empieza al menos c unidades después de la i" es $x_j \geq x_i + c$, o sea $x_i - x_j \leq -c$. Estas últimas, con lado derecho negativo, son las que van a obligar a usar Bellman-Ford.

Lema 7.25 (Traslación; Cormen, lema 24.8)**.** *Si $x = (x_1, \ldots, x_p)$ es solución de un sistema de restricciones de diferencia y a es cualquier constante, $x + a = (x_1 + a, \ldots, x_p + a)$ también lo es.*

Demostración. $(x_j + a) - (x_i + a) = x_j - x_i$. □

Un sistema con solución tiene entonces una infinidad, y siempre se puede fijar una incógnita en el valor que se quiera. En calendarización: correr todo el calendario no cambia nada.

La observación clave es de forma. La restricción $x_j - x_i \leq b_k$ se lee $x_j \leq x_i + b_k$, que es la desigualdad del triángulo $\delta(s, v_j) \leq \delta(s, v_i) + w(v_i, v_j)$ con $w(v_i, v_j) = b_k$. Si las incógnitas fueran distancias desde una fuente común en un grafo con esas aristas, cumplirían todas las restricciones solas.

Definición 7.26 (Grafo de restricciones)**.** El **grafo de restricciones** (*constraint graph*) del sistema es el grafo dirigido y pesado con vértices $V = \{v_0, v_1, \ldots, v_p\}$, uno por incógnita y uno extra v_0, y aristas

- (v_i, v_j) con peso b_k por cada restricción $x_j - x_i \leq b_k$;
- (v_0, v_i) con peso 0, para $i = 1, \ldots, p$.

La arista va de la variable que resta a la que suma. El vértice v_0 es la fuente común: llega a todos, así que ningún $\delta(v_0, v_i)$ es ∞, y cualquier ciclo negativo del grafo es alcanzable desde él.

Teorema 7.27 (Cormen, teorema 24.9)**.** *Sea G el grafo de restricciones de un sistema de restricciones de diferencia. Si G no tiene ciclos de peso negativo,*

$$x = (\delta(v_0, v_1), \delta(v_0, v_2), \ldots, \delta(v_0, v_p))$$

es una solución. Si G tiene un ciclo de peso negativo, el sistema no tiene solución.

Demostración. Sin ciclos negativos. Todo v_i es alcanzable desde v_0 por la arista (v_0, v_i), y no hay ciclos negativos, así que cada $\delta(v_0, v_i)$ es un número real (proposición 7.4). Para cada restricción $x_j - x_i \leq b_k$ hay una arista (v_i, v_j) de peso b_k, y la desigualdad del triángulo dice

$$\delta(v_0, v_j) \leq \delta(v_0, v_i) + b_k,$$

que con $x_i = \delta(v_0, v_i)$ es exactamente $x_j - x_i \leq b_k$.

Con un ciclo negativo. Sea $c = \langle u_0, u_1, \ldots, u_q \rangle$, con $u_0 = u_q$, un ciclo de peso negativo. No pasa por v_0, porque a v_0 no le entra ninguna arista, así que cada u_l es el vértice de una incógnita, que se escribe $x(u_l)$. Cada arista (u_{l-1}, u_l) del ciclo viene de una restricción

$$x(u_l) - x(u_{l-1}) \leq w(u_{l-1}, u_l), \qquad l = 1, \ldots, q.$$

Si x fuera solución, cumpliría las q. Al sumarlas, a la izquierda cada incógnita del ciclo aparece una vez sumando y una vez restando, y queda $x(u_q) - x(u_0) = 0$; a la derecha queda $w(c)$. Entonces $0 \leq w(c) < 0$, imposible. □

Para resolver el sistema basta construir el grafo y correr Bellman-Ford desde v_0. Dijkstra no sirve, porque los b_k pueden ser negativos. Como v_0 alcanza a todos, el teorema 7.15 dice que Bellman-Ford devuelve falso exactamente cuando el grafo tiene un ciclo negativo; si devuelve verdadero, $d[v_i] = \delta(v_0, v_i)$. Cada restricción se da como una terna (i, j, b), que significa $x_j - x_i \leq b$.

Algoritmo 7.6 Resolver-Restricciones(p, R)

```
1:  procedimiento Resolver-Restricciones(p, R)
2:      V ← {v_0, v_1, ..., v_p},    E ← ∅
3:      para i ← 1 hasta p hacer
4:          agregar (v_0, v_i) a E con peso 0
5:      fin para
6:      para todo (i, j, b) ∈ R hacer
7:          agregar (v_i, v_j) a E con peso b
8:      fin para
9:      si Bellman-Ford(G, w, v_0) entonces
10:         devolver (d[v_1], d[v_2], ..., d[v_p])
11:     si no
12:         devolver no hay solución                    // hay un ciclo negativo
13:     fin si
14: fin procedimiento
```

Costo. El grafo tiene $n = p+1$ vértices y $m = p+r$ aristas, y construirlo cuesta $\Theta(p+r)$. Bellman-Ford cuesta $\Theta(nm) = \Theta((p+1)(p+r)) = \Theta(p^2+pr)$. El término p^2 lo ponen las p aristas de v_0, que se relajan en cada una de las p pasadas sin aportar nada después de la primera; una variante que no construye v_0 lo quita y cuesta $\mathcal{O}(p+pr)$.

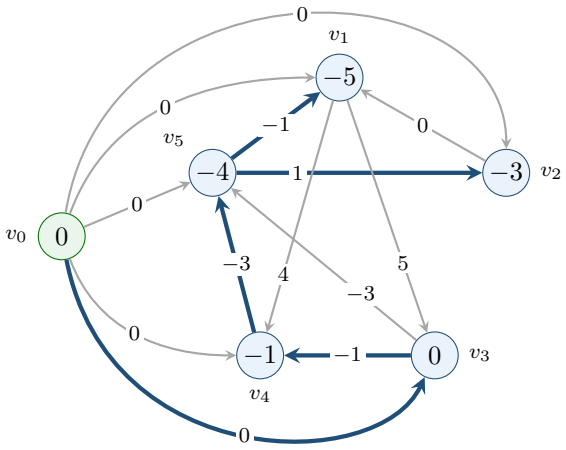
Ejemplo 7.28 (El sistema de la figura 24.8 de Cormen)**.** Cinco incógnitas y ocho restricciones:

$$\begin{array}{llll} x_1 - x_2 \le 0 & x_1 - x_5 \le -1 & x_2 - x_5 \le 1 & x_3 - x_1 \le 5 \\ x_4 - x_1 \le 4 & x_4 - x_3 \le -1 & x_5 - x_3 \le -3 & x_5 - x_4 \le -3 \end{array}$$

Bellman-Ford desde v_0, relajando primero las cinco aristas de v_0 y luego las ocho restricciones en el orden de arriba:

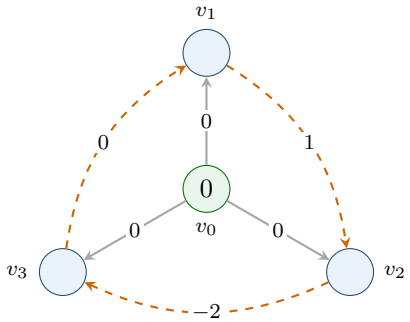
	$d[v_1]$	$d[v_2]$	$d[v_3]$	$d[v_4]$	$d[v_5]$
inicio	∞	∞	∞	∞	∞
pasada 1	-1	0	0	-1	-4
pasada 2	-5	-3	0	-1	-4
pasadas 3 a 5			sin cambios		

Al final ninguna arista se puede relajar, así que Bellman-Ford devuelve verdadero, y la solución es $x = (-5, -3, 0, -1, -4)$.



Dentro de cada vértice, $\delta(v_0, v_i)$. En azul, el árbol de caminos más cortos: $v_0 \to v_3 \to v_4 \to v_5$, que luego se abre hacia v_1 y v_2. Las aristas del árbol son restricciones justas, que se cumplen con igualdad; solo $x_1 - x_2 \le 0$ y $x_5 - x_3 \le -3$ tienen holgura, de 2 y de 1. Sumando 5 a todo sale $x' = (0, 2, 5, 4, 1)$, que también es solución y tiene tiempos no negativos. Con las restricciones en el orden inverso, Bellman-Ford tarda una pasada más en estabilizarse y llega a lo mismo.

Ejemplo 7.29 (Un sistema sin solución)**.** Las restricciones $x_2 - x_1 \le 1$, $x_3 - x_2 \le -2$ y $x_1 - x_3 \le 0$ forman el ciclo $v_1 \to v_2 \to v_3 \to v_1$ de peso $1 - 2 + 0 = -1$, y sumarlas da $0 \le -1$. Bellman-Ford relaja primero las aristas de v_0 y luego las tres restricciones en ese orden.



	$d[v_1]$	$d[v_2]$	$d[v_3]$
pasada 1	-2	0	-2
pasada 2	-3	-1	-3
pasada 3	-4	-2	-4

Con $n = 4$ vértices, Bellman-Ford hace tres pasadas, y cada una baja todo en 1, el peso del ciclo. La revisión encuentra que (v_1, v_2) todavía relaja, $d[v_2] = -2 > d[v_1] + 1 = -3$, y devuelve falso. Los predecesores al final, $\pi[v_2] = v_1$, $\pi[v_3] = v_2$ y $\pi[v_1] = v_3$, recorren justo el ciclo: el propio π es el certificado de que no hay solución.

¿Qué solución sale, de las infinitas que hay? Por la arista (v_0, v_i) de peso 0, siempre $\delta(v_0, v_i) \leq 0$ (Cormen, ejercicio 24.4-3); si todos los b_k son no negativos, $x = 0$ ya es solución y es la que sale. Entre las soluciones con todas las $x_i \leq 0$, la de Bellman-Ford es la de suma máxima (ejercicio 24.4-8), y entre todas las soluciones minimiza $\max_i x_i - \min_i x_i$ (ejercicio 24.4-9): en calendarización, termina el proyecto en el menor intervalo posible.

Observación. El vértice v_0, con aristas de peso 0 hacia todos, es el mismo truco del vértice extra del algoritmo de Johnson (sección 9). Johnson lo agrega a un grafo cualquiera, corre Bellman-Ford desde él y usa $h(v) = \delta(v_0, v)$ para repesar cada arista como $w(u, v) + h(u) - h(v)$, que la desigualdad del triángulo hace no negativo. Visto desde aquí, h es una solución del sistema $h(v) - h(u) \leq w(u, v)$, con una restricción por arista: Johnson resuelve un sistema de restricciones de diferencia antes de correr Dijkstra.

Ojito...

La dirección de la arista. $x_j - x_i \leq b$ da la arista (v_i, v_j), no (v_j, v_i). Si se invierten todas, lo que sale es $-x$ para una solución x y hay que negar la respuesta; mezclar las dos convenciones da basura.

Otras formas de restricción. $x_j - x_i \geq b$ se reescribe $x_i - x_j \leq -b$. Una igualdad $x_i = x_j + b$ son dos desigualdades, $x_i - x_j \leq b$ y $x_j - x_i \leq -b$. Las desigualdades estrictas no entran directamente; con incógnitas enteras, $x_j - x_i < b$ equivale a $x_j - x_i \leq b - 1$.

Restricciones de una sola variable, como $x_i \leq b$ o $-x_i \leq b$ (Cormen, ejercicio 24.4-10). Es tentador usar v_0 como el cero, pero sus aristas de peso 0 agregarían a escondidas $x_i \leq 0$, y con $-x_1 \leq -5$ un sistema factible se volvería infactible. Lo correcto es una incógnita nueva z que haga de cero, con $x_i - z \leq b$ o $z - x_i \leq b$, resolver el sistema ampliado y devolver $x_i - z$, que es solución por el lema 7.25.

Resumen

Algoritmo	**Pesos**	**Qué relaja**	**Ciclos negativos**	**Costo**
BFS	todos iguales	cada arista una vez, por capas	no hay	$\Theta(n+m)$
Bellman-Ford	cualesquiera	todas, $n-1$ veces	los detecta	$\Theta(nm)$
Caminos-DAG	cualesquiera, grafo acíclico	cada arista una vez, en orden topológico	no hay	$\Theta(n+m)$
Dijkstra	≥ 0	cada arista una vez, por d creciente	no hay	$\mathcal{O}((n+m)\log n)$

El costo de Dijkstra es con montículo binario; con arreglo es $\mathcal{O}(n^2)$ y con montículo de Fibonacci, $\mathcal{O}(n\log n+m)$. Un sistema de restricciones de diferencia con p incógnitas y r restricciones se resuelve con Bellman-Ford desde el vértice extra v_0, en $\mathcal{O}(p^2+pr)$, o en $\mathcal{O}(p+pr)$ sin construir v_0. Las seis propiedades de la relajación, que sostienen todas las pruebas de esta sección, se demuestran en la sección 8.

8. Demostraciones y problemas de caminos más cortos

Complemento: Cormen, cap. 24.5 y problemas

Resumen rápido. Bellman-Ford, el algoritmo para DAG y Dijkstra hacen lo mismo: inicializan las estimaciones y relajan aristas. Solo cambian cuántas veces relajan cada arista y en qué orden. Por eso su corrección se parte en dos capas. Abajo hay seis propiedades de la relajación que valen para *cualquier* secuencia de relajaciones; arriba, un argumento corto por algoritmo de que su orden cumple la hipótesis de alguna de ellas. La sección 7 enunció las seis; aquí se demuestran completas y en orden de dependencia, junto con el lema auxiliar que pide la convergencia y la prueba de que el subgrafo predecesor es siempre un árbol. Un ejemplo con relajaciones intercaladas muestra la propiedad central, la relajación de caminos, y una tabla resume qué propiedad usa cada algoritmo. La segunda mitad resuelve con esas mismas herramientas los problemas del capítulo 24 de Cormen: la mejora de Yen a Bellman-Ford, las cajas anidadas, el arbitraje entre monedas, el ciclo de peso medio mínimo de Karp y los caminos bitónicos.

Las reglas del juego

Todo ocurre en un grafo dirigido $G = (V, E)$ con pesos $w : E \to \mathbb{R}$ y una fuente s. Los algoritmos de la sección 7 tocan las estimaciones solo de dos maneras. Inicializar-Fuente(G, s) (Algoritmo 7.1) pone $d[v] = \infty$ y $\pi[v] =$ nulo en todos los vértices y luego $d[s] = 0$. Relajar(u, v, w) (Algoritmo 7.2) pregunta si $d[v] > d[u] + w(u, v)$ y, si es así, asigna $d[v] \leftarrow d[u] + w(u, v)$ y $\pi[v] \leftarrow u$ en el mismo renglón. De ese pseudocódigo salen tres hechos que las pruebas usan sin parar:

(i) Relajar solo baja estimaciones: $d[v]$ nunca sube.

(ii) $d[v]$ y $\pi[v]$ cambian juntos. Por eso, si $\pi[v] = u$, la última relajación que cambió $d[v]$ fue Relajar(u, v, w).

(iii) Con $d[u] = \infty$ no pasa nada, porque $\infty + w(u, v) = \infty$ y la prueba $d[v] > \infty$ es falsa. Por eso $\pi[v]$ solo recibe vértices con d finita, y un vértice $v \neq s$ tiene $d[v]$ finita si y solo si $\pi[v]$ no es nulo: las dos cosas las asigna la misma relajación.

Salvo la desigualdad del triángulo, que habla solo de δ, todos los resultados de esta parte suponen lo mismo: *el grafo se inicializó con Inicializar-Fuente(G, s) y desde entonces d y π solo cambiaron por llamadas a Relajar sobre aristas de G*, en cualquier orden y cuantas veces se quiera. No se supone nada sobre ese orden, y ahí está la gracia: lo que se pruebe aquí vale para Bellman-Ford, para el DAG, para Dijkstra y para cualquier algoritmo que se invente mañana, mientras solo relaje.

De la sección 7 se usa lo siguiente. $\delta(s, v)$ es el menor peso de una caminata de s a v: vale ∞ si no hay ninguna y $-\infty$ si alguna toca un ciclo de peso negativo, con $a + \infty = \infty$ para $a \neq -\infty$ y $a + (-\infty) = -\infty$ para $a \neq \infty$. Cuando $\delta(s, v)$ es finita la alcanza un camino, sin vértices repetidos y con a lo más $n - 1$ aristas; un *camino más corto* es un camino de peso $\delta(s, v)$. El lema de subcaminos (Cormen, lema 24.1) dice que todo subcamino de un camino más corto es un camino más corto entre sus extremos. El subgrafo predecesor es $G_\pi = (V_\pi, E_\pi)$ con

$$V_\pi = \{v \in V : \pi[v] \neq \text{nulo}\} \cup \{s\}\,, \qquad E_\pi = \{(\pi[v], v) : v \in V_\pi - \{s\}\}\,,$$

y un árbol de caminos más cortos con raíz s es un subgrafo cuyos vértices son (1) los alcanzables desde s, que (2) es un árbol con raíz s y en el que (3) el único camino de s a cada v es un camino más corto en G.

La figura 1 es el mapa de lo que sigue. Cada flecha se lee “se usa para probar”. En azul, las seis propiedades; en blanco, los tres auxiliares.

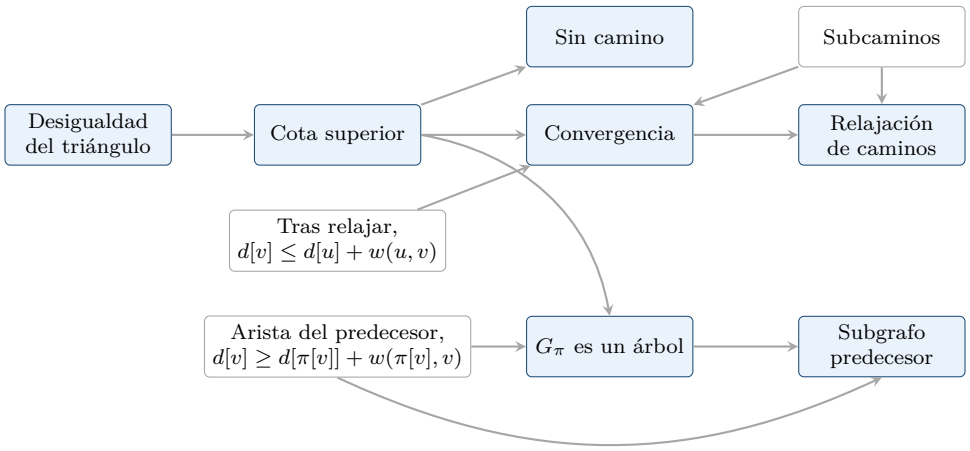


Figura 1: De qué depende cada propiedad de la relajación.

Triángulo, cota superior y sin camino

Lema 8.1 (Desigualdad del triángulo; Proposición 7.8)**.** *Para toda arista* $(u, v) \in E$*,*

$$\delta(s, v) \le \delta(s, u) + w(u, v).$$

Demostración. La idea cabe en una frase: ir a u por el mejor camino y luego tomar (u, v) es una manera de llegar a v, y la mejor manera no puede costar más que una cualquiera. Los infinitos obligan a separar tres casos según el valor de $\delta(s, u)$.

$\delta(s, u) = \infty$**.** El lado derecho vale ∞ y no hay nada que probar.

$\delta(s, u)$ **finito.** Sea p un camino más corto de s a u. Si se le pega la arista (u, v), queda una caminata de s a v que pesa $\delta(s, u) + w(u, v)$. Puede repetir v, y no importa: $\delta(s, v)$ es el mínimo sobre todas las caminatas de s a v y no puede pasar del peso de esta en particular.

$\delta(s, u) = -\infty$**.** Hay caminatas de s a u de peso tan negativo como se quiera. Pegándoles (u, v) salen caminatas a v igual de negativas, salvo la constante $w(u, v)$; entonces $\delta(s, v) = -\infty$ y la desigualdad vale con igualdad. □

Ojito…

La desigualdad es sobre las distancias, no sobre los pesos. Las aristas no tienen que cumplir ninguna desigualdad del triángulo: en el ejemplo 8.7, más abajo, $w(s, b) = 6$ es mayor que $w(s, a)+w(a, b) = 5$. La que sí la cumple es δ, en cualquier grafo y con pesos negativos o sin ellos.

Lema 8.2 (Cota superior; Proposición 7.9)**.** *Después de* Inicializar-Fuente(G, s) *se cumple* $d[v] \ge \delta(s, v)$ *para todo* $v \in V$*, y la desigualdad se mantiene tras cualquier secuencia de relajaciones. Además, en cuanto* $d[v]$ *alcanza* $\delta(s, v)$*, ya no cambia.*

Demostración. La primera parte es un invariante y se prueba por inducción sobre el número de relajaciones. Cuidado con la variable: no se induce sobre los vértices ni sobre las pasadas, sino sobre las llamadas a Relajar, una por una.

Base. Recién inicializado, $d[v] = \infty \geq \delta(s, v)$ para todo $v \neq s$. Para la fuente, $d[s] = 0$ y $\delta(s, s) \leq 0$, porque la caminata vacía de s a s pesa 0. ($\delta(s, s)$ vale 0 si ningún ciclo negativo pasa por s, y $-\infty$ si alguno pasa.)

Paso inductivo. Se llama Relajar(u, v, w) y, por hipótesis de inducción, $d[x] \geq \delta(s, x)$ para todo x antes de la llamada. La única estimación que puede cambiar es $d[v]$, y si cambia,

$$\begin{aligned} d[v] &= d[u] + w(u, v) \\ &\geq \delta(s, u) + w(u, v) \quad \textit{(hipótesis de inducción)} \\ &\geq \delta(s, v) \quad \textit{(Lema 8.1).} \end{aligned}$$

Las demás estimaciones no se mueven, así que el invariante sigue en pie.

Una vez alcanzada, no cambia. Si $d[v] = \delta(s, v)$, no puede bajar, por el invariante que se acaba de probar, y no puede subir, por el hecho (i). Se queda donde está. □

La segunda mitad del lema parece un detalle y es la pieza que más se usa. Es la que vuelve inofensivas las relajaciones de más: una vez que un vértice tiene su distancia, nada de lo que pase después se la quita.

Observación. Hay otra manera de ver la cota superior. Si $d[v]$ es finita, es el peso de una caminata de verdad de s a v: la que fueron armando las relajaciones. Se ve por la misma inducción. $d[s] = 0$ es la caminata vacía, y cuando Relajar asigna $d[v] \leftarrow d[u] + w(u, v)$, a la caminata que pesaba $d[u]$ se le pega (u, v). Como ninguna caminata pesa menos que $\delta(s, v)$, $d[v] \geq \delta(s, v)$. Las estimaciones no son adivinanzas; son pesos de caminatas que existen.

Corolario 8.3 (Sin camino; Proposición 7.10)**.** *Si no hay camino de s a v, entonces $d[v] = \delta(s, v) = \infty$ después de inicializar y después de cualquier secuencia de relajaciones.*

Demostración. Por la cota superior, $\infty = \delta(s, v) \leq d[v]$; entonces $d[v] = \infty$. □

Convergencia y relajación de caminos

La convergencia necesita saber qué deja una relajación, justo después de hacerse. Es el lema auxiliar de Cormen (lema 24.13), y es casi la definición de Relajar.

Lema 8.4 (Lo que deja una relajación)**.** *Para una arista (u, v) con $u \neq v$, inmediatamente después de Relajar(u, v, w) se cumple $d[v] \leq d[u] + w(u, v)$.*

Demostración. Si antes de la llamada $d[v] > d[u] + w(u, v)$, la llamada asigna justo $d[v] = d[u] + w(u, v)$. Si no, ya se cumplía $d[v] \leq d[u] + w(u, v)$, y la llamada no toca ni $d[u]$ ni $d[v]$. La condición $u \neq v$ solo excluye lazos: con un lazo de peso negativo, la asignación baja $d[u]$ junto con $d[v]$. Un camino nunca usa lazos, así que la convergencia no pierde nada. □

Lema 8.5 (Convergencia; Proposición 7.11)**.** *Sea $s \rightsquigarrow u \to v$ un camino más corto de s a v. Si $d[u] = \delta(s, u)$ en algún momento antes de llamar a Relajar(u, v, w), entonces $d[v] = \delta(s, v)$ en todo momento después de esa llamada.*

Demostración. Se juntan tres piezas: el lema 8.4 da una desigualdad, la cota superior da la otra y el lema de subcaminos pega las dos.

Por la cota superior, una vez que $d[u] = \delta(s,u)$ ya no cambia; en particular sigue valiendo $\delta(s,u)$ cuando se relaja (u,v). Justo después de esa llamada,

$$\begin{aligned} d[v] &\le d[u] + w(u,v) \quad \textit{(Lema 8.4)} \\ &= \delta(s,u) + w(u,v) \\ &= \delta(s,v) \quad \textit{(subcaminos)}. \end{aligned}$$

La última igualdad merece una línea. El tramo $s \rightsquigarrow u$ es un subcamino del camino más corto, así que pesa $\delta(s,u)$; el camino completo pesa entonces $\delta(s,u) + w(u,v)$, y por ser el más corto ese peso es $\delta(s,v)$. La cota superior da la desigualdad contraria, $d[v] \ge \delta(s,v)$. Entonces $d[v] = \delta(s,v)$, y por la segunda mitad de la cota superior se queda así. □

Lema 8.6 (Relajación de caminos; Proposición 7.12)**.** *Sea $p = \langle v_0, v_1, \ldots, v_k \rangle$ un camino más corto de $s = v_0$ a v_k. Si después de* Inicializar-Fuente(G,s) *ocurre una secuencia de relajaciones que incluye, en este orden, a $(v_0,v_1),(v_1,v_2),\ldots,(v_{k-1},v_k)$, entonces $d[v_k] = \delta(s,v_k)$ al terminar esas relajaciones y en todo momento posterior. No importa qué otras relajaciones haya ni cómo se intercalen con las de p.*

Demostración. Se prueba por inducción sobre i que, a partir de la relajación de (v_{i-1},v_i), se cumple $d[v_i] = \delta(s,v_i)$. Para $i = 0$ "a partir de" significa desde el principio.

Base, $i = 0$. Por la inicialización, $d[v_0] = d[s] = 0$. Falta ver que $\delta(s,s) = 0$, y aquí hay un detalle que Cormen da por hecho. Si un ciclo negativo pasara por s, darle vueltas antes de seguir p abarataría sin límite las caminatas a v_k, y no habría camino más corto a v_k. Como p existe, ningún ciclo negativo pasa por s, y $\delta(s,s) = 0 = d[s]$. Por la cota superior, $d[s]$ ya no cambia.

Paso inductivo. Supóngase que $d[v_{i-1}] = \delta(s,v_{i-1})$ desde que se relajó (v_{i-2},v_{i-1}), o desde el principio si $i = 1$. Las relajaciones intercaladas no rompen esa igualdad, por la cota superior. Cuando le toca a (v_{i-1},v_i), el tramo $\langle v_0,\ldots,v_{i-1},v_i\rangle$ es un subcamino de p y, por lo tanto, un camino más corto. La convergencia, con $u = v_{i-1}$ y $v = v_i$, da $d[v_i] = \delta(s,v_i)$ desde ese momento y para siempre.

Con $i = k$ queda el enunciado. □

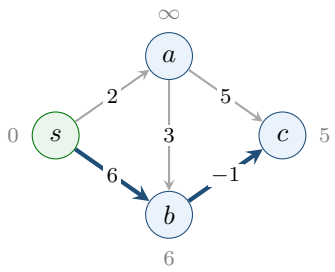
Idea clave

La relajación de caminos dice que lo único que importa es una *subsecuencia*: que las aristas de un camino más corto aparezcan en orden dentro de la secuencia de relajaciones. Lo que haya entre ellas no estorba, por la cota superior, y lo que se relaje antes de tiempo no cuenta. Un algoritmo no necesita conocer el camino más corto; le basta un orden que garantice esa subsecuencia para todos los caminos a la vez. Bellman-Ford la garantiza a la fuerza, con $n-1$ pasadas completas; el algoritmo para DAG la obtiene gratis del orden topológico.

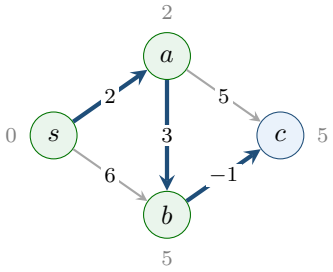
Ejemplo 8.7 (Relajación de caminos con relajaciones intercaladas)**.** Un grafo chico, con una arista negativa a propósito para que se vea que la propiedad no depende de Dijkstra. $V = \{s,a,b,c\}$ y $w(s,a) = 2$, $w(s,b) = 6$, $w(a,b) = 3$, $w(a,c) = 5$, $w(b,c) = -1$. Las distancias son $\delta(s,a) = 2$, $\delta(s,b) = 5$ y $\delta(s,c) = 4$, y el camino más corto a c es $p = \langle s,a,b,c\rangle$. Se relajan siete aristas en un orden cualquiera que contiene a las de p en orden (pasos 4, 6 y 7), mezcladas con otras:

Paso	Arista	$d[s]$	$d[a]$	$d[b]$	$d[c]$	¿Cambia?	Comentario
0		0	∞	∞	∞		inicialización
1	(b,c)	0	∞	∞	∞	no	$\infty + (-1) = \infty$
2	(s,b)	0	∞	6	∞	sí	$\pi[b] = s$
3	(b,c)	0	∞	6	5	sí	$\pi[c] = b$; $5 > \delta(s,c)$
4	(s,a)	0	**2**	6	5	sí	primera de p: a converge
5	(a,c)	0	2	6	5	no	$2 + 5 = 7 > 5$
6	(a,b)	0	2	**5**	5	sí	segunda de p: b converge, $\pi[b] = a$
7	(b,c)	0	2	5	**4**	sí	tercera de p: c converge

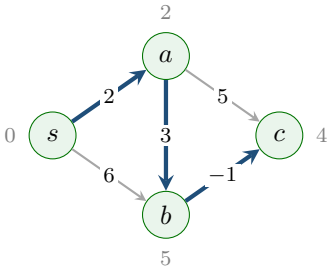
En verde, los vértices con $d[v] = \delta(s,v)$; las flechas azules son las aristas $(\pi[v], v)$ de G_π, y el número junto a cada vértice es $d[v]$.



tras el paso 3



tras el paso 6



tras el paso 7

Tras el paso 3, $d[c] = 5$ es el peso de un camino real, $s \to b \to c$, pero no del más corto: la cota superior se cumple y nada más. Tras el paso 6, b ya convergió, pero c no se entera hasta que se vuelve a relajar (b,c); la relajación del paso 3 se hizo cuando $d[b]$ todavía no era correcta y no cuenta. En el paso 7 se completa la subsecuencia $(s,a),(a,b),(b,c)$ y c queda en 4 para siempre. Los pasos 1, 3 y 5 sobran, pero tampoco estorban.

El mismo grafo explica por qué Bellman-Ford hace $n-1$ pasadas y por qué al DAG le basta una. Si cada pasada relaja las aristas en el orden (b,c), (a,b), (s,a), (a,c), (s,b), que recorre p al revés, cada pasada avanza una sola arista de p:

Tras la pasada	$d[s]$	$d[a]$	$d[b]$	$d[c]$
1	0	**2**	6	7
2	0	2	**5**	5
3	0	2	5	**4**

Son $3 = n-1$ pasadas, el peor caso. El grafo no tiene ciclos y s, a, b, c es un orden topológico. Caminos-DAG relaja las aristas agrupadas por origen en ese orden, (s,a), (s,b), (a,b), (a,c), (b,c), y termina en una pasada con $d = (0,2,5,4)$. No es suerte: en un orden topológico cada arista de cualquier camino sale de un vértice que va antes que el origen de la siguiente, así que las aristas de *todo* camino se relajan en orden. Aquí las de p caen en las posiciones 1, 3 y 5.

El subgrafo predecesor

Falta la otra mitad de la historia: los apuntadores π. Las dos pruebas que siguen se apoyan en una desigualdad que vale en todo momento para las aristas de G_π, la misma que en Cormen aparece dentro de la prueba del lema 24.16.

Lema 8.8 (Arista del predecesor)**.** *En todo momento, para cada vértice v con $\pi[v]$ no nulo, $d[\pi[v]]$ es*

finita y

$$d[v] \geq d[\pi[v]] + w(\pi[v], v).$$

Demostración. Sea $u = \pi[v]$. Por el hecho (ii), la última relajación que cambió $d[v]$ fue Relajar(u, v, w), y dejó $d[v] = d[u] + w(u, v)$ con el valor que $d[u]$ tenía entonces, que era finito por el hecho (iii). Desde ese momento $d[v]$ no ha cambiado y $d[u]$ solo pudo bajar, por el hecho (i). El lado derecho solo pudo bajar y la desigualdad se mantiene. □

Con estimaciones correctas la desigualdad se vuelve igualdad sobre las aristas del árbol; es lo que dice la última línea de la prueba del lema 8.10.

Lema 8.9 (G_π es un árbol)**.** *Supóngase que G no tiene ciclos de peso negativo alcanzables desde s. Después de* Inicializar-Fuente(G, s)*, el subgrafo predecesor G_π es un árbol con raíz s, y cualquier secuencia de relajaciones mantiene esa propiedad.*

Demostración. Al inicio $V_\pi = \{s\}$ y $E_\pi = \emptyset$, un árbol de un solo vértice. Hay que ver dos cosas después de cada relajación: que G_π no tiene ciclos y que de s hay un único camino a cada vértice de V_π.

G_π no tiene ciclos. Supóngase, buscando una contradicción, que alguna relajación cierra un ciclo $c = \langle v_0, v_1, \ldots, v_k \rangle$ en G_π, con $v_k = v_0$ y $\pi[v_i] = v_{i-1}$ para $i = 1, \ldots, k$. Numerando el ciclo desde donde convenga, la llamada que lo cierra es Relajar(v_{k-1}, v_k, w). Se miran las estimaciones justo antes de esa llamada.

Para $i = 1, \ldots, k-1$ el apuntador $\pi[v_i] = v_{i-1}$ ya estaba puesto, y el lema 8.8 da

$$d[v_i] \geq d[v_{i-1}] + w(v_{i-1}, v_i), \qquad i = 1, \ldots, k-1.$$

En la arista que cierra el ciclo la desigualdad es estricta, porque la llamada sí cambia $\pi[v_k]$, y eso solo ocurre si pasa la prueba de Relajar:

$$d[v_k] > d[v_{k-1}] + w(v_{k-1}, v_k).$$

Las k estimaciones del ciclo son finitas: $d[v_{k-1}]$ porque la prueba pasó, y las otras porque cada v_{i-1} con $1 \leq i \leq k-1$ es el predecesor de v_i. Sumando las k desigualdades,

$$\sum_{i=1}^{k} d[v_i] > \sum_{i=1}^{k} d[v_{i-1}] + \sum_{i=1}^{k} w(v_{i-1}, v_i).$$

Como $v_0 = v_k$, las dos sumas de estimaciones recorren los mismos vértices, una vez cada uno, y son el mismo número finito. Se cancelan y queda

$$0 > \sum_{i=1}^{k} w(v_{i-1}, v_i) = w(c).$$

El ciclo c es negativo. Además es alcanzable desde s, porque sus vértices tienen d finita y, por la cota superior, $\delta(s, v_i) \leq d[v_i] < \infty$. Eso contradice la hipótesis.

Hay un único camino de s a cada vértice de V_π. Primero, $\pi[s]$ sigue siendo nulo: cambiarlo exigiría bajar $d[s]$ de 0, y la cota superior lo impide, porque sin ciclos negativos alcanzables $\delta(s, s) = 0$. Ahora tómese $v \in V_\pi$ y sígase la cadena $v, \pi[v], \pi[\pi[v]], \ldots$. Por el lema 8.8 cada eslabón tiene d finita, así que es s o tiene π no nulo (hecho (iii)): la cadena no sale de V_π. Tampoco repite vértices, porque

G_π no tiene ciclos. Entonces se detiene, y solo puede detenerse en un vértice de V_π con π nulo, o sea en s. Leída al revés, es un camino de s a v en G_π.

Es el único. A cada vértice $z \neq s$ de V_π le entra en G_π exactamente una arista, $(\pi[z], z)$, y a s ninguna. Un camino que sale de s y llega a v tiene que llegar desde $\pi[v]$, que a su vez tiene que venir de $\pi[\pi[v]]$, y así hasta s: está forzado a ser la cadena. Un grafo dirigido con un único camino de s a cada vértice es un árbol con raíz s. □

Lema 8.10 (Subgrafo predecesor; Proposición 7.13)**.** *Supóngase que G no tiene ciclos negativos alcanzables desde s. Si después de* Inicializar-Fuente(G, s) *una secuencia de relajaciones deja $d[v] = \delta(s, v)$ para todo $v \in V$, entonces G_π es un árbol de caminos más cortos con raíz s.*

Demostración. Hay que comprobar las tres condiciones de la definición.

(1) V_π son los alcanzables. Sin ciclos negativos alcanzables, $\delta(s, v)$ es finita si y solo si v es alcanzable desde s. Por el hecho (iii), V_π es justo el conjunto de vértices con d finita (s incluido, con $d[s] = 0$). Como $d[v] = \delta(s, v)$, $V_\pi = \{v : \delta(s, v) < \infty\}$, que son los alcanzables.

(2) Es un árbol con raíz s. Es el lema 8.9.

(3) Sus caminos son caminos más cortos. Sea $v \in V_\pi$ y sea $p = \langle v_0, \ldots, v_k \rangle$ el camino de $s = v_0$ a $v = v_k$ en G_π. Para cada i, $\pi[v_i] = v_{i-1}$, y el lema 8.8 con $d = \delta$ dice

$$w(v_{i-1}, v_i) \leq \delta(s, v_i) - \delta(s, v_{i-1}).$$

Al sumar sobre el camino, el lado derecho se telescopa:

$$\begin{aligned} w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i) &\leq \sum_{i=1}^{k} \big(\delta(s, v_i) - \delta(s, v_{i-1})\big) \\ &= \delta(s, v_k) - \delta(s, v_0) \quad \textit{(suma telescópica)} \\ &= \delta(s, v) \quad \textit{($\delta(s, s) = 0$).} \end{aligned}$$

Ningún camino de s a v pesa menos que $\delta(s, v)$, así que $w(p) = \delta(s, v)$ y p es un camino más corto. De paso, todas las desigualdades de la suma eran igualdades: sobre las aristas del árbol el triángulo se cumple justo, $\delta(s, v_i) = \delta(s, v_{i-1}) + w(v_{i-1}, v_i)$. □

Ojito…

Las hipótesis de los lemas 8.9 y 8.10 son distintas, y las dos hacen falta.

- Sin ciclos negativos, G_π es árbol en todo momento, pero a media corrida no tiene por qué ser de caminos más cortos. Tras el paso 3 del ejemplo 8.7 el árbol contiene $s \to b \to c$, que no es el camino más corto ni a b ni a c. Solo cuando todas las d son correctas el árbol es el bueno.
- Con un ciclo negativo alcanzable, G_π puede dejar de ser árbol: en el ejemplo 8.13 los apuntadores forman un ciclo, y es justo el ciclo negativo. La prueba de aciclicidad solo usó la hipótesis al final, para descartar el ciclo. Leída sin ella dice algo más fuerte: *todo ciclo que aparezca en G_π es negativo.* Eso es lo que permite recuperar un ciclo negativo siguiendo apuntadores.

Quién usa qué

Con las seis propiedades probadas, la corrección de cada algoritmo se reduce a decir cuál de ellas aplica y por qué su orden de relajación cumple la hipótesis. La tabla 2 lo resume para los tres algoritmos de la sección 7 y para los dos de los problemas de esta sección.

Algoritmo	**Orden de relajación**	**Propiedades**	**Dónde entran**
Bellman-Ford (Alg. 7.3)	$n-1$ pasadas sobre todas las aristas	relajación de caminos, sin camino, triángulo, subgrafo predecesor	la pasada i relaja la i-ésima arista de todo camino más corto; la revisión final pasa por el triángulo y falla con un ciclo negativo por la suma alrededor del ciclo
Caminos-DAG (Alg. 7.4)	una pasada, orígenes en orden topológico	relajación de caminos, sin camino, subgrafo predecesor	el orden topológico pone en orden las aristas de cualquier camino
Dijkstra (Alg. 7.5)	cada arista una vez, orígenes en orden de d creciente	cota superior, sin camino, convergencia, subgrafo predecesor	el primer vértice no fijado de un camino más corto ya tiene su distancia, por convergencia
Yen (Alg. 8.1)	$\lceil n/2 \rceil$ pasadas de ida y regreso	relajación de caminos	cada pasada despacha dos corridas del camino
Bitónicos (Alg. 8.4)	cuatro pasadas en orden de peso	relajación de caminos	cada pasada despacha una corrida monótona

Tabla 2: Qué propiedad de la relajación sostiene a cada algoritmo.

Bellman-Ford. Sin ciclos negativos alcanzables, cada vértice alcanzable v tiene un camino más corto $p = \langle v_0, \ldots, v_k \rangle$ con $k \leq n-1$. La pasada i relaja todas las aristas, entre ellas (v_{i-1}, v_i); las de p quedan relajadas en orden y la relajación de caminos da $d[v] = \delta(s, v)$. Los no alcanzables se quedan en ∞ por la propiedad sin camino, y el árbol sale del subgrafo predecesor. La revisión final no encuentra nada que relajar porque $d[v] = \delta(s, v) \leq \delta(s, u) + w(u, v) = d[u] + w(u, v)$, por el triángulo. Con un ciclo negativo alcanzable $c = \langle v_0, \ldots, v_k \rangle$, si ninguna de sus aristas fallara la revisión, sumando $d[v_i] \leq d[v_{i-1}] + w(v_{i-1}, v_i)$ alrededor de c las estimaciones se cancelarían (son finitas, porque tras $n-1$ pasadas todo alcanzable tiene d finita) y quedaría $0 \leq w(c)$. Es la misma suma de la prueba del lema 8.9, con la desigualdad volteada.

Caminos-DAG. En un orden topológico cada v_{i-1} de un camino va antes que v_i, así que al procesar los orígenes en ese orden las aristas de cualquier camino se relajan en orden, cada una exactamente una vez. Es el orden "bueno" del ejemplo 8.7.

Dijkstra. Aquí la herramienta es la convergencia, porque el orden no se conoce de antemano. Si u fuera el primer vértice que se fija con $d[u] \neq \delta(s, u)$, no sería s, y sería alcanzable (si no, la propiedad sin camino lo dejaría correcto en ∞). Sea y el primer vértice no fijado de un camino más corto de s a u y x su predecesor en ese camino, ya fijado. Cuando x se fijó tenía $d[x] = \delta(s, x)$, porque u es el primero que falla, y en ese momento se relajó (x, y): por convergencia, $d[y] = \delta(s, y)$. Con pesos no negativos, $\delta(s, y) \leq \delta(s, u) \leq d[u] \leq d[y]$ (el tramo de y a u no resta, cota superior, y u se eligió con la d mínima), así que todo es igualdad y $d[u] = \delta(s, u)$, una contradicción. La traza de Dijkstra de la clase del 22 de octubre ilustra además la relajación de caminos: las aristas del camino A, D, E, F se relajan en los pasos 1, 4 y 5, en orden, y F termina en 12.

Problema: la mejora de Yen a Bellman-Ford

Planteamiento (Cormen, problema 24-1). Antes de la primera pasada se fija un orden lineal arbitrario $v_1, v_2, \ldots, v_n$ de los vértices y se parten las aristas en dos conjuntos:

$$E_f = \{(v_i, v_j) \in E : i < j\}, \qquad E_b = \{(v_i, v_j) \in E : i > j\},$$

las que van hacia adelante en el orden y las que van hacia atrás. Se supone que no hay lazos, para que toda arista caiga en uno de los dos. Cada pasada recorre los vértices en el orden $v_1, \ldots, v_n$ relajando sus aristas de E_f, y luego en el orden $v_n, \ldots, v_1$ relajando sus aristas de E_b. Yen mostró que así bastan $\lceil n/2 \rceil$ pasadas en lugar de $n-1$.

Dos DAG escondidos. $G_f = (V, E_f)$ no tiene ciclos y $v_1, \ldots, v_n$ es un orden topológico suyo: toda arista sube de índice, y un ciclo tendría que bajar alguna vez para regresar. Simétricamente, $G_b = (V, E_b)$ es acíclico con orden topológico $v_n, \ldots, v_1$. Cada mitad de pasada es exactamente Caminos-DAG sobre uno de los dos DAG; Yen alterna los dos.

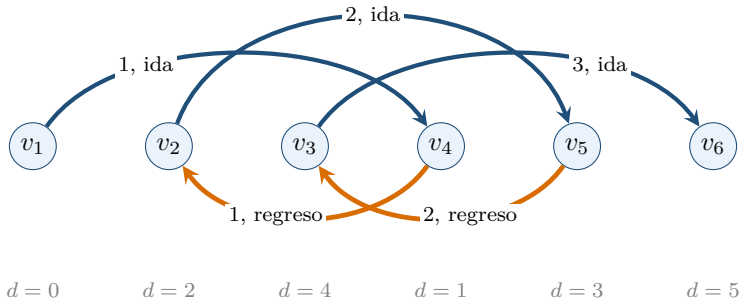
Por qué bastan $\lceil n/2 \rceil$ pasadas. Sea v alcanzable y $p = \langle u_0, u_1, \ldots, u_k \rangle$ un camino más corto de $s = u_0$ a $v = u_k$, con $k \le n-1$. Cada arista de p es de ida o de regreso. Se parte p en *corridas*: bloques máximos de aristas consecutivas del mismo tipo, $R_1, R_2, \ldots, R_r$, que alternan entre E_f y E_b, con $r \le k$. A lo largo de una corrida de E_f los orígenes suben de índice, y en la mitad de ida los vértices se procesan en índice creciente; entonces la corrida entera se relaja en el orden del camino dentro de una sola mitad de ida. Lo mismo pasa con las corridas de E_b en la mitad de regreso. Cada pasada despacha, entonces, una corrida de E_f y la de E_b que le sigue. Si p empieza con una corrida de E_b, la primera mitad de ida se desperdicia. En el peor caso hacen falta $\lceil (r+1)/2 \rceil$ pasadas, y como $r \le n-1$, eso es a lo más $\lceil n/2 \rceil$. Las aristas de p quedaron relajadas en orden, y la relajación de caminos da $d[v] = \delta(s, v)$.

Algoritmo 8.1 Bellman-Ford-Yen(G, w, s)

```
1:  procedimiento BELLMAN-FORD-YEN(G, w, s)
2:      fijar un orden lineal v_1, v_2, ..., v_n de V
3:      E_f ← {(v_i, v_j) ∈ E : i < j},    E_b ← {(v_i, v_j) ∈ E : i > j}
4:      INICIALIZAR-FUENTE(G, s)
5:      para p ← 1 hasta ⌈n/2⌉ hacer
6:          para i ← 1, 2, ..., n hacer                    // mitad de ida
7:              para todo (v_i, v_j) ∈ E_f hacer
8:                  RELAJAR(v_i, v_j, w)
9:              fin para
10:         fin para
11:         para i ← n, n − 1, ..., 1 hacer                // mitad de regreso
12:             para todo (v_i, v_j) ∈ E_b hacer
13:                 RELAJAR(v_i, v_j, w)
14:             fin para
15:         fin para
16:     fin para
17:     para todo (u, v) ∈ E hacer
18:         si d[v] > d[u] + w(u, v) entonces
19:             devolver falso                             // ciclo negativo alcanzable
20:         fin si
21:     fin para
22:     devolver verdadero
23: fin procedimiento
```

Costo. Las líneas 6 a 15 tocan cada arista exactamente una vez por pasada: cada una está en E_f o en E_b y se visita desde su origen. Con listas de adyacencia separadas para E_f y E_b, que la línea 3 arma una vez en $\Theta(n+m)$, una pasada cuesta $\Theta(n+m)$. En total: $\Theta(n+m)$ de preparación, $\lceil n/2 \rceil \cdot \Theta(n+m)$ de pasadas y $\Theta(m)$ de revisión, o sea $\Theta(nm)$ cuando todos los vértices son alcanzables ($m \geq n-1$). El orden de crecimiento no mejora; lo que se gana es la mitad de las pasadas. La revisión de las líneas 17 a 21 sigue siendo válida tras $\lceil n/2 \rceil$ pasadas: el argumento de Bellman-Ford solo necesita que los vértices de un ciclo negativo alcanzable tengan d finita, y la tienen, porque el argumento de las corridas vale para cualquier camino desde s, no solo para los más cortos.

Ejemplo 8.11 (Las pasadas de Yen)**.** Seis vértices en el orden lineal $v_1, \ldots, v_6$ y el camino $v_1 \to v_4 \to v_2 \to v_5 \to v_3 \to v_6$, con todas las aristas de peso 1 y fuente $s = v_1$. Arriba, en azul, las aristas de E_f; abajo, en naranja, las de E_b. Junto a cada arista va la pasada y la mitad en que se relaja con su origen ya correcto, y debajo de cada vértice su distancia final:



El camino tiene cinco corridas, tres de ida y dos de regreso. La pasada 1 resuelve v_4 en su mitad de ida y v_2 en la de regreso; la pasada 2, v_5 y v_3; la pasada 3, v_6. Son $3 = \lceil 6/2 \rceil$ pasadas. Bellman-Ford con las aristas en el orden inverso del camino necesita $5 = n - 1$.

Ojito...

Un lazo (v, v) no está ni en E_f ni en E_b. Si su peso es negativo es un ciclo negativo que el esquema no vería nunca, así que los lazos se revisan aparte; los de peso no negativo se pueden tirar, porque ningún camino más corto los usa.

Problema: cajas anidadas

Planteamiento (Cormen, problema 24-2). Una caja de k dimensiones $(x_1, \ldots, x_k)$ *anida* en otra $(y_1, \ldots, y_k)$ si existe una permutación σ de $\{1, \ldots, k\}$ con $x_{\sigma(i)} < y_i$ para todo i: se puede girar la primera para que quepa holgada dentro de la segunda. Cormen llama d a la dimensión; aquí es k, porque d ya es la estimación. Dadas n cajas, se busca la sucesión más larga $B_{i_1}, B_{i_2}, \ldots$ en la que cada caja anida en la siguiente.

Anidar es transitivo. Si $x_{\sigma(i)} < y_i$ para todo i y $y_{\tau(j)} < z_j$ para todo j, entonces $x_{\sigma(\tau(j))} < y_{\tau(j)} < z_j$: la permutación compuesta sirve de testigo.

Cómo probar si anida sin recorrer permutaciones. Se ordenan las dimensiones de las dos cajas. Escribiendo $x_{(i)}$ para la i-ésima más chica,

$$x \text{ anida en } y \iff x_{(i)} < y_{(i)} \text{ para todo } i.$$

De derecha a izquierda, la permutación que ordena es el testigo. De izquierda a derecha, las i dimensiones más chicas de y están emparejadas por σ con i dimensiones distintas de x, cada una menor que su

pareja y por lo tanto menor que $y_{(i)}$; hay al menos i valores de x por debajo de $y_{(i)}$, y entonces $x_{(i)} < y_{(i)}$. Cuesta $\mathcal{O}(k \log k)$ ordenar y $\mathcal{O}(k)$ comparar. El criterio se comparó contra la fuerza bruta sobre todas las permutaciones en 3000 pares aleatorios con $k \leq 4$.

La sucesión más larga es un camino más largo en un DAG. Un vértice por caja y una arista $B_i \to B_j$ si B_i anida en B_j. El grafo no tiene ciclos: un ciclo daría, por transitividad, una caja que anida en sí misma, y eso es imposible con desigualdades estrictas. La sucesión pedida es un camino con el mayor número de vértices. Se resuelve con Caminos-DAG poniendo peso -1 a cada arista, o directamente con la recurrencia

$$L(j) = 1 + \max\{L(i) : B_i \to B_j\},$$

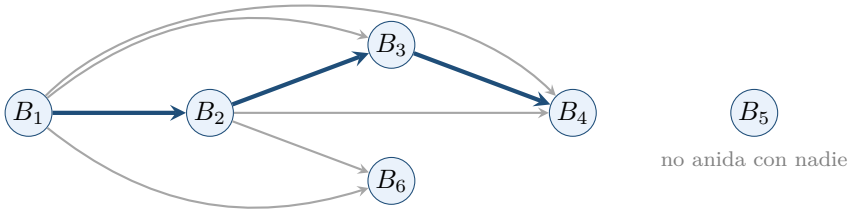
con $L(j) = 1$ si a B_j no le entra ninguna arista, evaluada en orden topológico. Hay un orden topológico gratis: ordenar las cajas por la suma de sus dimensiones, porque si B_i anida en B_j su suma es estrictamente menor. La sucesión se reconstruye guardando en cada caja de dónde vino su máximo, como un π.

Costo. Ordenar las dimensiones de cada caja cuesta $\mathcal{O}(nk \log k)$; comparar los $n(n-1)$ pares ordenados, $\mathcal{O}(n^2 k)$. El DAG tiene hasta $\Theta(n^2)$ aristas y el camino más largo cuesta $\mathcal{O}(n^2)$. Total: $\mathcal{O}(nk \log k + n^2 k)$.

Ejemplo 8.12 (Seis cajas de tres dimensiones)**.** Las cajas, sus dimensiones ordenadas, la suma y el resultado de la recurrencia en el orden de las sumas:

Caja	Dimensiones	Ordenadas	Suma	L	Viene de
B_1	$(5,1,3)$	$(1,3,5)$	9	1	
B_2	$(2,6,4)$	$(2,4,6)$	12	2	B_1
B_5	$(2,2,9)$	$(2,2,9)$	13	1	
B_3	$(7,3,5)$	$(3,5,7)$	15	3	B_2
B_6	$(3,7,8)$	$(3,7,8)$	18	3	B_2
B_4	$(6,9,4)$	$(4,6,9)$	19	**4**	B_3

Comparando las columnas ordenadas salen ocho aristas. En azul, la sucesión más larga:



La respuesta es B_1, B_2, B_3, B_4, de longitud 4. Ninguno de sus tres pasos se ve comparando coordenada a coordenada sin ordenar: $(5,1,3)$ contra $(2,6,4)$ falla en la primera, y sin embargo B_1 cabe en B_2 girándola. B_5 muestra que la suma solo sirve de orden: su suma queda entre las de B_2 y B_3, pero su dimensión 9 le impide entrar en cualquier otra, y ninguna cabe en ella.

Ojito...

La desigualdad tiene que ser estricta. Dos cajas iguales no anidan; si se usara $\leq$, cada par de cajas iguales formaría un ciclo y "camino más largo" dejaría de tener sentido.

Problema: arbitraje

Planteamiento (Cormen, problema 24-3). *Arbitraje* es aprovechar tipos de cambio inconsistentes para convertir una unidad de una moneda en más de una unidad de la misma moneda. Hay n monedas $c_1, \ldots, c_n$ y una tabla R en la que una unidad de c_i compra $R[i, j]$ unidades de c_j. Se pide decidir si existe un ciclo de monedas con

$$R[i_1, i_2] \cdot R[i_2, i_3] \cdots R[i_{k-1}, i_k] \cdot R[i_k, i_1] > 1$$

y, si existe, imprimirlo.

De productos a sumas. El logaritmo convierte productos en sumas y es creciente. Con $i_{k+1} = i_1$,

$$\prod_{j=1}^{k} R[i_j, i_{j+1}] > 1 \iff \sum_{j=1}^{k} \ln R[i_j, i_{j+1}] > 0 \iff \sum_{j=1}^{k} (-\ln R[i_j, i_{j+1}]) < 0.$$

Con pesos $w(c_i, c_j) = -\ln R[i, j]$, hay arbitraje si y solo si hay un ciclo de peso negativo, y Bellman-Ford lo detecta. Un tipo de cambio mayor que 1 da un peso negativo, y eso es lo normal aquí: Dijkstra no sirve. Si $R[i, j] = 0$ no hay arista, porque $\ln 0$ no existe. Para que todo ciclo sea alcanzable aunque la tabla tenga ceros, se agrega una fuente virtual s con aristas de peso 0 a todas las monedas; equivale a inicializar todas las estimaciones en 0. El grafo queda con $n + 1$ vértices: n es el número de monedas, no de vértices.

El algoritmo completo queda así; la recuperación del ciclo, en las líneas 12 a 21, se justifica después.

Algoritmo 8.2 Arbitraje(R, n)

```
1:  procedimiento ARBITRAJE(R, n)
2:      V ← {s, c_1, ..., c_n}; aristas (s, c_i) de peso 0
3:      aristas (c_i, c_j) de peso −ln R[i, j] para cada i ≠ j con R[i, j] > 0
4:      INICIALIZAR-FUENTE(G, s)
5:      para p ← 1 hasta n hacer                          // |V| − 1 = n pasadas
6:          para todo (u, v) ∈ E hacer
7:              RELAJAR(u, v, w)
8:          fin para
9:      fin para
10:     para todo (u, v) ∈ E hacer
11:         si d[v] > d[u] + w(u, v) + ε entonces
12:             RELAJAR(u, v, w)                          // ahora G_π tiene un ciclo
13:             x ← v
14:             para i ← 1 hasta n + 1 hacer
15:                 x ← π[x]                              // al final, x está en el ciclo
16:             fin para
17:             C ← ⟨x⟩,    y ← π[x]
18:             mientras y ≠ x hacer
19:                 anteponer y a C,    y ← π[y]
20:             fin mientras
21:             anteponer x a C
22:             devolver C                                // c_{i_1}, ..., c_{i_k}, c_{i_1}
23:         fin si
24:     fin para
25:     devolver no hay arbitraje
26: fin procedimiento
```

Cómo recuperar el ciclo. Si tras las n pasadas ($|V|-1=n$) la arista (u,v) todavía se puede relajar, se relaja y se siguen los apuntadores desde v durante $n+1$ pasos. El vértice al que se llega está en un ciclo de G_π, y ese ciclo es negativo. La razón tiene tres partes.

- Tras i pasadas, cada $d[x]$ es a lo más el peso de cualquier caminata de s a x con a lo más i aristas. Se ve por inducción sobre i: si la caminata termina en (y,x), tras $i-1$ pasadas $d[y]$ ya es a lo más el peso del resto, y en la pasada i se relaja (y,x) (lema 8.4). Entonces la relajación extra bajó $d[v]$ estrictamente por debajo del peso de toda caminata de s a v con a lo más n aristas.
- Si la cadena $v, \pi[v], \pi[\pi[v]], \ldots$ llegara a s sin repetir vértices, sería un camino de a lo más n aristas. Sumando el lema 8.8 a lo largo de ella saldría $d[v] \geq d[s] + w(\text{cadena}) = w(\text{cadena})$, porque a la fuente virtual no le entra ninguna arista y $d[s]$ se queda en 0. Eso contradice el punto anterior. Tampoco puede detenerse en otro vértice, porque todos los de la cadena tienen d finita y π no nulo. Entonces la cadena cae en un ciclo, y como solo hay $n+1$ vértices, tras $n+1$ pasos ya está dentro de él.
- Ese ciclo es negativo, por la versión fuerte del lema 8.9: todo ciclo que aparece en G_π lo es. Leído al derecho, es la sucesión de monedas.

Las líneas que no son obvias. En la línea 11, el ε (por ejemplo 10^{-12} relativo) evita que el redondeo de los logaritmos invente un arbitraje donde el producto es exactamente 1. La línea 12 relaja la arista antes de seguir apuntadores; es lo que garantiza el ciclo en G_π. Las líneas 14 a 16 dan $n+1=|V|$ saltos para caer seguro en el ciclo y no en un vértice que cuelga de él. En las líneas 17 a 21 se antepone porque los apuntadores van hacia atrás.

Costo. El grafo tiene $n+1$ vértices y, con la tabla llena, $n(n-1)+n=n^2$ aristas. Construirlo (líneas 2 y 3) cuesta $\Theta(n^2)$; las n pasadas, $n\cdot\Theta(n^2)=\Theta(n^3)$; la revisión, $\Theta(n^2)$, y la recuperación, $\Theta(n)$. Total: $\Theta(n^3)$. Floyd sobre los mismos pesos también cuesta $\Theta(n^3)$ y delata el ciclo como una entrada negativa en la diagonal; es más fácil de programar, aunque recuperar el ciclo pide guardar los intermedios.

Ejemplo 8.13 (Arbitraje entre dólar, rupia y yen)**.** Los tres tipos de cambio de Cormen (1 dólar compra 49 rupias, 1 rupia compra 2 yenes y 1 yen compra 0.0107 dólares), más los de regreso, con el margen que cobra una casa de cambio. $R[i,j]$ es cuántas unidades de la moneda j se compran con una de la i:

$R =$

	D	I	Y
D	1	49	92
I	0.0200	1	2
Y	0.0107	0.48	1

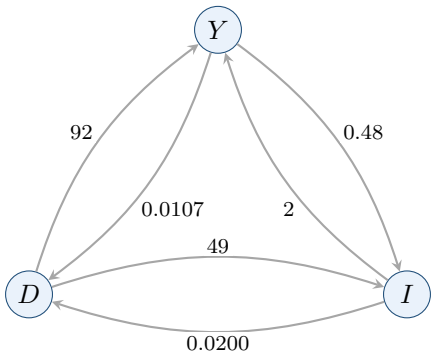
$w = -\ln R =$

	D	I	Y
D	0	-3.892	-4.522
I	3.912	0	-0.693
Y	4.538	0.734	0

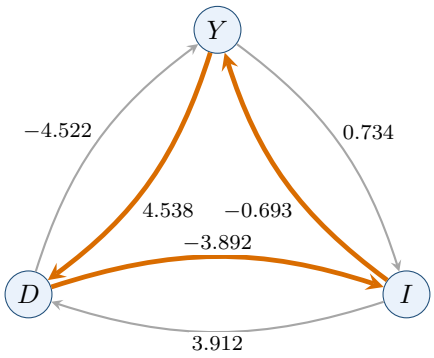
Un ciclo da ganancia si el producto de sus tipos pasa de 1, o sea, si la suma de sus pesos es negativa. Hay cinco ciclos:

Ciclo	**Producto de R**	**Suma de w**
$D \to I \to D$	0.9800	0.0202
$D \to Y \to D$	0.9844	0.0157
$I \to Y \to I$	0.9600	0.0408
$D \to I \to Y \to D$	1.0486	-0.0475
$D \to Y \to I \to D$	0.8832	0.1242

Solo $D \to I \to Y \to D$ gana: $49 \cdot 2 \cdot 0.0107 = 1.0486$, un 4.86 %, y su peso es $-3.892 - 0.693 + 4.538 = -0.0475 = -\ln 1.0486$.



tipos de cambio $R[i,j]$



pesos $-\ln R[i,j]$; ciclo de peso -0.0475

Para seguirlo a mano se corre Bellman-Ford desde D en lugar de la fuente virtual: la tabla está llena y D alcanza a las otras dos. Son $|V|-1=2$ pasadas, con las aristas en el orden (D,I), (D,Y), (I,D), (I,Y), (Y,D), (Y,I). La columna *candidato* es $d[u]+w(u,v)$:

Pasada	**Arista**	**Candidato**	**¿Cambia?**	$d[D]$	$d[I]$	$d[Y]$	
1	(D,I)	-3.8918	sí	0	-3.8918	∞	$\pi[I]=D$
	(D,Y)	-4.5218	sí	0	-3.8918	-4.5218	$\pi[Y]=D$
	(I,D)	0.0202	no	0	-3.8918	-4.5218	
	(I,Y)	-4.5850	sí	0	-3.8918	-4.5850	$\pi[Y]=I$
	(Y,D)	-0.0475	sí	-0.0475	-3.8918	-4.5850	$\pi[D]=Y$
	(Y,I)	-3.8510	no	-0.0475	-3.8918	-4.5850	
2	las seis		3 de 6	-0.0949	-3.9393	-4.6324	bajan D, I y Y
revisión	(D,I)	-3.9867	falla				$-3.9393 > -3.9867$

Ya en la pasada 1, al relajar (Y,D), la fuente recibe $\pi[D]=Y$ y G_π queda con el ciclo $D\to I\to Y\to D$: la fuente recibió un predecesor, y el lema 8.9 deja de valer porque su hipótesis falla. Cada vuelta baja las tres estimaciones en 0.0475 y nunca se estabilizan. La revisión encuentra que (D,I) todavía se puede relajar: hay un ciclo negativo. Se relaja ($\pi[I]$ ya era D) y se retrocede $|V|=3$ veces desde I: D, Y, I. Desde I se recorre π hasta volver: $\pi[I]=D$, $\pi[D]=Y$, $\pi[Y]=I$. Al derecho es $I\to Y\to D\to I$, el mismo ciclo $D\to I\to Y\to D$ empezando en otra moneda. Con la fuente virtual y tres pasadas falla la misma arista y sale el mismo ciclo.

Ojito...

Los logaritmos se redondean. Un ciclo con producto exactamente 1 puede sumar -10^{-16} y parecer arbitraje; por eso la línea 11 compara con tolerancia. Y para recuperar el ciclo hay que relajar primero la arista que falla y luego retroceder $|V|$ veces: retroceder menos puede dejar en un vértice que cuelga del ciclo sin estar en él.

Problema: ciclo de peso medio mínimo (Karp)

Planteamiento (Cormen, problema 24-5). El *peso medio* de un ciclo c con ℓ aristas es $\mu(c)=w(c)/\ell$, y $\mu^*=\min_c \mu(c)$ sobre todos los ciclos dirigidos de G. Un ciclo con $\mu(c)=\mu^*$ es un *ciclo de peso medio mínimo*. Se pide calcular μ^* en $\mathcal{O}(nm)$. Se supone que todo vértice es alcanzable desde una fuente s, y se define

$$\delta_k(s,v)=\text{peso mínimo de una caminata de } s \text{ a } v \text{ con exactamente } k \text{ aristas},$$

∞ si no hay ninguna. Aquí sí importa que sean caminatas: pueden repetir vértices, y con exactamente n aristas siempre repiten. La tabla de δ_k se llena por niveles, cada uno a partir del anterior: $\delta_k(s,v) = \min_{(u,v)\in E}\big(\delta_{k-1}(s,u) + w(u,v)\big)$, con $\delta_0(s,s) = 0$ y $\delta_0(s,v) = \infty$ para $v \neq s$.

Teorema 8.14 (Karp)**.** *Si G tiene al menos un ciclo y todo vértice es alcanzable desde s, entonces*

$$\mu^* = \min_{v\in V} \max_{0\le k\le n-1} \frac{\delta_n(s,v) - \delta_k(s,v)}{n-k},$$

donde el mínimo omite los v con $\delta_n(s,v) = \infty$ y el máximo omite los k con $\delta_k(s,v) = \infty$.

Demostración. Sea $F(v)$ el máximo de adentro. La prueba sigue los incisos de Cormen.

Paso 1: basta el caso $\mu^* = 0$. Si se suma una constante t al peso de cada arista, la media de todo ciclo sube t, así que μ^* sube t. Del otro lado, toda caminata de exactamente k aristas sube kt, así que $\delta_k(s,v)$ sube kt y cada cociente sube $\frac{nt-kt}{n-k} = t$. Los dos lados se mueven igual; con $t = -\mu^*$ el caso general se reduce a $\mu^* = 0$. De aquí en adelante, $\mu^* = 0$.

Paso 2: no hay ciclos negativos. Un ciclo negativo tendría media negativa, menor que μ^*. Entonces los caminos más cortos existen, no repiten vértices y tienen a lo más $n-1$ aristas:

$$\delta(s,v) = \min_{0\le k\le n-1} \delta_k(s,v),$$

finito para todo v porque todos son alcanzables.

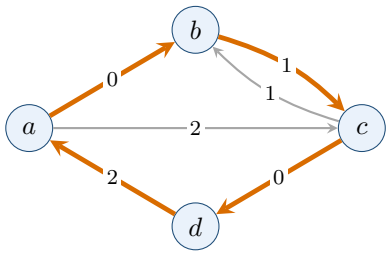
Paso 3: $F(v) \ge 0$. Sea v con $\delta_n(s,v)$ finita. Una caminata de n aristas sigue siendo una caminata, así que $\delta_n(s,v) \ge \delta(s,v) = \delta_{k^*}(s,v)$ para el k^* que alcanza el mínimo del paso 2. El cociente de ese k^* es ≥ 0, y el máximo también.

Paso 4: algún v tiene $F(v) = 0$. Sea c un ciclo de peso 0, que existe porque $\mu^* = 0$, y sea u un vértice de c. Primero una observación: si el tramo de c que va de u a otro de sus vértices v pesa x, entonces $\delta(s,v) = \delta(s,u) + x$. El resto del ciclo, de v a u, pesa $-x$; pegar un camino más corto con un tramo del ciclo da una caminata, de donde $\delta(s,v) \le \delta(s,u) + x$ y $\delta(s,u) \le \delta(s,v) - x$, y las dos juntas dan la igualdad.

Ahora se toma un camino más corto de s a u, de $j \le n-1$ aristas, y se alarga dando vueltas a c desde u hasta completar exactamente n aristas. Termina en algún vértice v de c, y lo recorrido sobre c pesa lo mismo que el tramo directo de u a v, porque cada vuelta completa pesa 0. Por la observación, esta caminata de n aristas pesa $\delta(s,u) + x = \delta(s,v)$. Entonces $\delta_n(s,v) \le \delta(s,v)$ y, con el paso 3, $\delta_n(s,v) = \delta(s,v) \le \delta_k(s,v)$ para todo k. Todos los cocientes de v son ≤ 0, así que $F(v) \le 0$, y con el paso 3, $F(v) = 0$.

Paso 5. Por el paso 3 el mínimo de los $F(v)$ es ≥ 0, y por el paso 4 vale 0, que es μ^*. Los v omitidos no cambian nada: con $\delta_n(s,v) = \infty$ su máximo sería ∞ y nunca ganaría el mínimo. □

Ejemplo 8.15 (Ciclo de peso medio mínimo con Karp)**.** El caso interesante es cuando el ciclo de media mínima no es el de peso total mínimo. $V = \{a,b,c,d\}$, $n = 4$, fuente $s = a$ y aristas $w(a,b) = 0$, $w(a,c) = 2$, $w(b,c) = 1$, $w(c,b) = 1$, $w(c,d) = 0$, $w(d,a) = 2$. Tiene tres ciclos:



b, c, b: peso 2, media 1

a, c, d, a: peso 4, media 4/3

a, b, c, d, a: peso 3, media 3/4

El ciclo más ligero es b, c, b, pero el de media mínima es el largo, en naranja: $\mu^* = 3/4$. La tabla de $\delta_k(a, v)$:

k	a	b	c	d
0	0	∞	∞	∞
1	∞	0	2	∞
2	∞	3	1	2
3	4	2	4	1
4	3	4	3	4

Hay ∞ en renglones donde el vértice sí es alcanzable, pero no con ese número exacto de aristas: a no se alcanza con 1 ni con 2. Por ejemplo, $\delta_2(a, b) = 3$ es $a \to c \to b$ y $\delta_4(a, a) = 3$ es la vuelta completa a, b, c, d, a. Para cada v se evalúa $(\delta_4(a, v) - \delta_k(a, v))/(4 - k)$ en los k con $\delta_k(a, v)$ finita:

v	**Cocientes**	**Máximo**
a	$k = 0$: $\frac{3-0}{4} = \frac{3}{4}$; $k = 3$: $\frac{3-4}{1} = -1$	**3/4**
b	$k = 1$: $\frac{4-0}{3} = \frac{4}{3}$; $k = 2$: $\frac{4-3}{2} = \frac{1}{2}$; $k = 3$: $\frac{4-2}{1} = 2$	2
c	$k = 1$: $\frac{3-2}{3} = \frac{1}{3}$; $k = 2$: $\frac{3-1}{2} = 1$; $k = 3$: $\frac{3-4}{1} = -1$	1
d	$k = 2$: $\frac{4-2}{2} = 1$; $k = 3$: $\frac{4-1}{1} = 3$	3

El mínimo de los máximos es $3/4$, en $v = a$, que está en el ciclo de media mínima, como predice el paso 4. Con los pesos corridos $w - 3/4$ del paso 1, la tabla da $\delta_4(a, a) = 0 = \delta_0(a, a)$: el vértice a alcanza su distancia con exactamente n aristas, como en la prueba.

El algoritmo llena esa tabla nivel por nivel y evalúa la fórmula del teorema 8.14.

Algoritmo 8.3 Karp(G, w, s)

```
1:  procedimiento Karp(G, w, s)
2:      D[k][v] ← ∞ para todo 0 ≤ k ≤ n y todo v ∈ V
3:      D[0][s] ← 0
4:      para k ← 1 hasta n hacer
5:          para todo (u, v) ∈ E hacer
6:              si D[k − 1][u] + w(u, v) < D[k][v] entonces
7:                  D[k][v] ← D[k − 1][u] + w(u, v)
8:              fin si
9:          fin para
10:     fin para
11:     μ ← ∞
12:     para todo v ∈ V con D[n][v] < ∞ hacer
13:         M ← −∞
14:         para k ← 0 hasta n − 1 hacer
15:             si D[k][v] < ∞ entonces
16:                 M ← máx(M, (D[n][v] − D[k][v])/(n − k))
17:             fin si
18:         fin para
19:         μ ← mín(μ, M)
20:     fin para
21:     devolver μ                                  // ∞ si no hay ciclos
22: fin procedimiento
```

Costo. Las líneas 2 y 3 llenan $(n+1)\cdot n$ casillas, $\Theta(n^2)$. Las líneas 4 a 10 son n niveles, y cada uno recorre las m aristas una vez: $\Theta(nm)$. Las líneas 11 a 20 evalúan n cocientes para cada uno de los n vértices: $\Theta(n^2)$. Como todos son alcanzables desde s, $m \geq n-1$ y $n^2 = \mathcal{O}(nm)$; el total es $\Theta(nm)$. La tabla ocupa $\Theta(n^2)$ de memoria. En la línea 6 se lee $D[k-1]$ y se escribe $D[k]$: si se actualizara en el lugar, sería Bellman-Ford y se perdería el "exactamente k aristas" que la fórmula necesita. Si ninguna fuente alcanza todo, se inicializa $D[0][v] = 0$ para todo v, que equivale a una fuente virtual con aristas de peso 0; con esa inicialización la fórmula coincidió con la fuerza bruta en 2000 grafos aleatorios.

Ojito...

Tres trampas al programarlo. La tabla no se actualiza en el lugar. Los ∞ se saltan: si $\delta_n(s,v) = \infty$ se omite el vértice entero, y si $\delta_k(s,v) = \infty$ se omite ese cociente. Y δ_k es sobre caminatas: restringida a caminos, que tienen a lo más $n-1$ aristas, δ_n sería ∞ en todos lados y la fórmula no diría nada.

Por el paso 1, μ^* es el mayor t tal que los pesos $w - t$ no tienen ciclos negativos. Una búsqueda binaria sobre t con Bellman-Ford también lo encuentra, aunque más lento; Karp lo saca de una sola tabla.

Problema: caminos bitónicos

Planteamiento (Cormen, problema 24-6). Una sucesión es *bitónica* si crece y luego decrece, o si se vuelve así con un corrimiento circular. $\langle 1,4,6,8,3,-2\rangle$, $\langle 9,2,-4,-10,-5\rangle$ y $\langle 1,2,3,4\rangle$ son bitónicas; $\langle 1,3,12,4,2,10\rangle$ no. Se da un grafo con pesos todos distintos y la promesa de que, para cada v, los pesos a lo largo de *cualquier* camino más corto de s a v forman una sucesión bitónica. Se piden los caminos más cortos lo más rápido posible.

Lema 8.16. *Una sucesión bitónica se parte en a lo más tres bloques consecutivos, cada uno monótono, con direcciones alternadas: sube, baja, sube, o baja, sube, baja (alguno puede faltar).*

Demostración. Algún corrimiento circular $r = \langle r_1, \ldots, r_\ell \rangle$ de la sucesión crece hasta un máximo r_q y luego decrece. Sean $U = \langle r_1, \ldots, r_q \rangle$, que sube, y $D = \langle r_{q+1}, \ldots, r_\ell \rangle$, que baja. La sucesión original es r cortada en algún punto j y con las dos partes intercambiadas, $\langle r_{j+1}, \ldots, r_\ell, r_1, \ldots, r_j \rangle$. Si el corte cae dentro de U ($j \leq q$), se lee como el final de U (sube), todo D (baja) y el principio de U (sube). Si cae dentro de D ($j > q$), se lee como el final de D (baja), todo U (sube) y el principio de D (baja). □

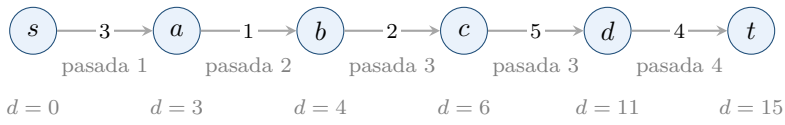
Idea. Se ordenan las aristas por peso y se hacen cuatro pasadas de relajación: en orden creciente de peso, decreciente, creciente y decreciente. Sea p un camino más corto de s a v. Un bloque creciente de p tiene pesos estrictamente crecientes, porque son distintos; una pasada en orden creciente lo relaja entonces en el orden del camino. Lo mismo con los bloques decrecientes y las pasadas decrecientes. Por el lema 8.16, el patrón de p es sube-baja-sube, y lo cubren las pasadas 1, 2 y 3, o baja-sube-baja, y lo cubren las pasadas 2, 3 y 4. En los dos casos las aristas de p quedan relajadas en orden, y la relajación de caminos da $d[v] = \delta(s, v)$.

Algoritmo 8.4 Caminos-Bitónicos(G, w, s)

```
1: procedimiento Caminos-Bitónicos(G, w, s)
2:     L ← las aristas de E ordenadas por peso creciente
3:     Inicializar-Fuente(G, s)
4:     para todo sentido en (creciente, decreciente, creciente, decreciente) hacer
5:         para todo (u, v) en L, recorrida en ese sentido hacer
6:             Relajar(u, v, w)
7:         fin para
8:     fin para
9: fin procedimiento
```

Costo. Ordenar cuesta $\mathcal{O}(m \log m) = \mathcal{O}(m \log n)$, porque $m \leq n^2$; inicializar, $\Theta(n)$, y las cuatro pasadas, $\Theta(m)$. Total: $\mathcal{O}(n + m \log n)$, dominado por el ordenamiento. Si los pesos son enteros en un rango chico y se ordenan en tiempo lineal, queda $\Theta(n + m)$. En 1500 grafos aleatorios que cumplen la promesa, las cuatro pasadas dieron siempre las distancias de Bellman-Ford.

Ejemplo 8.17 (Por qué hacen falta cuatro pasadas)**.** El grafo más simple que lo muestra es un solo camino. Sus pesos, $\langle 3, 1, 2, 5, 4 \rangle$, forman una sucesión bitónica: con el corrimiento $\langle 1, 2, 5, 4, 3 \rangle$ sube y luego baja. Como es el único camino, también es el más corto, y la promesa se cumple. Junto a cada arista va la pasada en que se relaja con su origen ya correcto, y debajo de cada vértice su distancia:



Tras las pasadas creciente y decreciente solo s, a y b tienen su distancia; la tercera pasada resuelve c y d, y t sigue en ∞ hasta la cuarta. El patrón es baja-sube-baja: el bloque $\langle 3 \rangle$, el bloque $\langle 1, 2, 5 \rangle$ y el bloque $\langle 4 \rangle$.

Ojito...

La solución que suele circular hace dos pasadas, creciente y luego decreciente, y solo cubre la forma sube-baja sin corrimiento. El enunciado de Cormen admite corrimientos circulares, y con ellos dos pasadas no bastan: en el ejemplo dejan mal a c, d y t, y en las pruebas aleatorias fallaron en 162 de 1500 grafos. Tres pasadas tampoco: ningún orden de tres cubre a la vez sube-baja-sube y baja-sube-baja, y creciente, decreciente, creciente falló en 2. Los pesos distintos también son necesarios: con empates, dos aristas consecutivas del mismo peso pueden relajarse en el orden equivocado dentro de una pasada.

9. Caminos más cortos entre todos los pares

Complemento: Cormen, cap. 25

Resumen rápido. El problema es calcular $\delta(u, v)$ para todo par de vértices de un grafo dirigido con pesos, que pueden ser negativos siempre que no haya ciclos negativos, y entregar la respuesta como dos matrices: D con los pesos y Π con los predecesores que permiten reconstruir cada camino. Correr n veces un algoritmo de fuente única es el punto de partida y la vara con que se mide lo demás. Siguen tres ideas. Una programación dinámica por número de aristas que resulta ser un producto de matrices con mín en lugar de suma y + en lugar de producto: $\Theta(n^4)$ tal cual y $\Theta(n^3 \log n)$ elevando al cuadrado. Floyd-Warshall, que cambia el estado a qué vértices se permiten como intermedios y baja a $\Theta(n^3)$; con él vienen Π, la versión en el lugar, la detección de ciclos negativos en la diagonal y la cerradura transitiva como caso booleano. Y Johnson, que repondera las aristas con un potencial calculado por Bellman-Ford para que ningún peso quede negativo y luego corre Dijkstra desde cada vértice; en grafos dispersos le gana a Floyd-Warshall. Una tabla final dice cuál conviene según la densidad.

El problema y su representación

Sea $G = (V, E)$ un grafo dirigido con pesos $w : E \to \mathbb{R}$. Se quiere el peso $\delta(u, v)$ de un camino más corto entre **todo** par de vértices, y algo que permita reconstruir esos caminos. Los pesos pueden ser negativos, pero se supone que no hay ciclos de peso negativo: con uno alcanzable, δ vale $-\infty$ para algunos pares y no existe camino más corto. Floyd-Warshall y Johnson, además, detectan esa situación en lugar de devolver basura.

La clase del 22 de octubre (sección 1) presentó Floyd con un ejemplo sobre el grafo dirigido de cuatro vértices; aquí va el tratamiento completo, y ese ejemplo se cita en lugar de repetirse. A diferencia de los algoritmos de fuente única, los de este capítulo trabajan sobre la matriz de pesos y numeran los vértices de 1 a n. Cuando un grafo tenga letras, $A = 1$, $B = 2$, y así.

Definición 9.1 (Matriz de pesos)**.** La matriz de pesos de G es $W = (w_{ij})$, de $n \times n$, con

$$w_{ij} = \begin{cases} 0 & \text{si } i = j, \\ w(i, j) & \text{si } i \neq j \text{ y } (i, j) \in E, \\ \infty & \text{si } i \neq j \text{ y } (i, j) \notin E. \end{cases}$$

Es la matriz pesada de la clase del 22 de octubre: ceros en la diagonal e ∞ donde no hay arista. Las matrices se escriben con mayúscula y sus casillas con minúscula, $D = (d_{ij})$, y un superíndice entre paréntesis marca la iteración, como en $D^{(k)}$.

Definición 9.2 (Distancias, predecesores y subgrafo de predecesores)**.** Al terminar, un algoritmo de todos los pares entrega $D = (d_{ij})$ con $d_{ij} = \delta(i, j)$ y una **matriz de predecesores** $\Pi = (\pi_{ij})$: π_{ij} es nulo si $i = j$ o si no hay camino de i a j, y en otro caso es el predecesor de j en algún camino más corto desde i. El **subgrafo de predecesores** de i es $G_{\pi,i} = (V_{\pi,i}, E_{\pi,i})$, con

$$V_{\pi,i} = \{j \in V : \pi_{ij} \neq \text{nulo}\} \cup \{i\}, \qquad E_{\pi,i} = \{(\pi_{ij}, j) : j \in V_{\pi,i} - \{i\}\}.$$

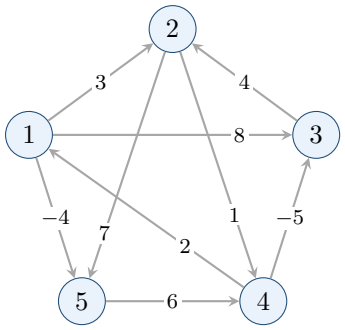
Se pide que cada $G_{\pi,i}$ sea un árbol de caminos más cortos con raíz i. Dicho de otro modo, el renglón i de Π es el arreglo $\pi[\cdot]$ que dejaría una corrida de fuente única desde i (sección 7), y con él se reconstruye cualquier camino que salga de i.

Algoritmo 9.1 Imprimir-Camino-Pares(Π, i, j)

```
1: procedimiento Imprimir-Camino-Pares(Π, i, j)
2:     si i = j entonces
3:         imprimir i
4:     si no si π_ij = nulo entonces
5:         imprimir que no hay camino de i a j
6:     si no
7:         Imprimir-Camino-Pares(Π, i, π_ij)        // de i al penúltimo
8:         imprimir j
9:     fin si
10: fin procedimiento
```

El procedimiento solo lee el renglón i: sube desde j por los predecesores hasta llegar a i y escribe el camino de regreso. La profundidad de la recursión es el número de aristas del camino, a lo más $n-1$, así que cuesta $\mathcal{O}(n)$. Supone que $G_{\pi,i}$ es un árbol; con una Π mal construida puede ciclarse para siempre, y más adelante se verá cómo pasa eso con los empates.

Ejemplo 9.3 (El grafo del capítulo)**.** Cormen usa un solo grafo en todo el capítulo (su figura 25.1), y esta sección también: cinco vértices, nueve aristas, dos de ellas negativas, y ningún ciclo negativo.



$$W = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

Dos de sus ciclos: $1 \to 5 \to 4 \to 1$ pesa $-4+6+2=4$, y $2 \to 4 \to 3 \to 2$ pesa $1-5+4=0$. Ese ciclo de peso cero importa cuando se hable de empates en Π.

Correr n veces un algoritmo de fuente única

La solución obvia es correr un algoritmo de la sección 7 desde cada vértice. Cuánto cuesta depende de cuál, y en Dijkstra, de la cola de prioridad:

- **Dijkstra** (algoritmo 7.5), solo con pesos no negativos. Con la cola en un arreglo, cada corrida hace n extracciones de $\Theta(n)$ y hasta m decrementos de $\mathcal{O}(1)$, o sea $\Theta(n^2)$, y las n corridas suman $\Theta(n^3)$. Con montículo binario cada corrida cuesta $\mathcal{O}((n+m)\log n)$ y el total es $\mathcal{O}(n(n+m)\log n)$, que Cormen escribe $\mathcal{O}(nm\log n)$ suponiendo $m=\Omega(n)$. Con montículo de Fibonacci, extraer cuesta $\mathcal{O}(\log n)$ y decrementar $\mathcal{O}(1)$, ambos amortizados: $\mathcal{O}(n\log n+m)$ por corrida y $\mathcal{O}(n^2\log n+nm)$ en total.
- **Bellman-Ford** (algoritmo 7.3), con pesos de cualquier signo: $\mathcal{O}(nm)$ por corrida y $\mathcal{O}(n^2m)$ en total, que en un grafo denso es $\mathcal{O}(n^4)$.

El resto de la sección persigue dos metas. Bajar ese $\mathcal{O}(n^4)$ a $\Theta(n^3)$ sin renunciar a los pesos negativos, que es lo que logra Floyd-Warshall, y alcanzar el costo de n veces Dijkstra con Fibonacci también con pesos negativos, que es lo que logra Johnson.

Ojito...

Al comparar con Floyd hay que comparar contra n corridas. El costo de Dijkstra que se cita siempre, $\mathcal{O}((n+m)\log n)$, es de una sola fuente; para todos los pares es $\mathcal{O}(n(n+m)\log n)$. En un grafo denso eso da $\mathcal{O}(n^3 \log n)$, peor que el $\Theta(n^3)$ de Floyd, y en uno disperso $\mathcal{O}(n^2 \log n)$, mucho mejor. La comparación completa está en la tabla del final de la sección.

Programación dinámica por número de aristas

Las dos programaciones dinámicas del capítulo se apoyan en la misma propiedad de estructura (Cormen, lema 24.1): *todo subcamino de un camino más corto es un camino más corto*. Si hubiera una forma más barata de recorrer un tramo, cambiarla dentro del camino completo lo abarataría, y entonces el camino no era el más corto. En particular, si p es un camino más corto de i a $j \neq i$ y su última arista es (k, j), entonces $\delta(i,j) = \delta(i,k) + w_{kj}$.

Definición 9.4 (Caminatas con a lo más r aristas)**.** $l_{ij}^{(r)}$ es el peso mínimo de una caminata de i a j con a lo más r aristas, o ∞ si no hay ninguna, y $L^{(r)} = (l_{ij}^{(r)})$.

Cormen usa la letra m para este contador; aquí es r, porque m ya es el número de aristas. La definición habla de caminatas porque eso es lo que construye la recurrencia de abajo, que puede repetir vértices. Sin ciclos negativos da lo mismo: quitarle un ciclo a una caminata le quita aristas y no la encarece, así que la mejor caminata con a lo más r aristas puede tomarse sin vértices repetidos.

Una caminata de i a j con a lo más r aristas, o tiene a lo más $r-1$, o termina con una arista (k, j) precedida por una caminata de i a k con a lo más $r-1$. Probar todos los penúltimos vértices k da la recurrencia

$$l_{ij}^{(0)} = \begin{cases} 0 & i = j \\ \infty & i \neq j \end{cases} \qquad l_{ij}^{(r)} = \min\Big(l_{ij}^{(r-1)},\ \min_{1\le k\le n}\left\{l_{ik}^{(r-1)} + w_{kj}\right\}\Big) = \min_{1\le k\le n}\left\{l_{ik}^{(r-1)} + w_{kj}\right\}.$$

La segunda igualdad usa $w_{jj} = 0$: el término $k = j$ vale $l_{ij}^{(r-1)}$, que es justo la opción de no agregar arista. De paso, $L^{(1)} = W$.

Proposición 9.5 (Cuándo parar)**.** *Si G no tiene ciclos negativos, $l_{ij}^{(r)} = \delta(i,j)$ para todo $r \geq n-1$.*

Demostración. Toda caminata de i a j pesa al menos $\delta(i,j)$: se le quitan ciclos, ninguno negativo, hasta dejar un camino, y el peso no sube. Por eso $l_{ij}^{(r)} \geq \delta(i,j)$ para todo r. Al revés, si $\delta(i,j) < \infty$ hay un camino más corto; no repite vértices, así que visita a lo más n y tiene a lo más $n-1$ aristas, y cuenta en $l_{ij}^{(r)}$ para todo $r \geq n-1$. Si $\delta(i,j) = \infty$ no hay caminata y los dos lados valen ∞. □

Extender caminos es multiplicar matrices

Algoritmo 9.2 Extender-Caminos(L, W)

```
1:  procedimiento Extender-Caminos(L, W)
2:      n ← número de renglones de L
3:      L' ← matriz nueva de n × n
4:      para i ← 1 hasta n hacer
5:          para j ← 1 hasta n hacer
6:              l'_ij ← ∞                                  // neutro del mínimo
7:              para k ← 1 hasta n hacer
8:                  l'_ij ← mín(l'_ij, l_ik + w_kj)        // k es el penúltimo
9:              fin para
10:         fin para
11:     fin para
12:     devolver L'
13: fin procedimiento
```

Extender-Caminos recibe $L^{(r-1)}$ y W y devuelve $L^{(r)}$: calcula la recurrencia en todas las casillas. La línea 6 arranca en ∞ porque es el neutro del mínimo, como 0 lo es de la suma; en la línea 8, k recorre los posibles penúltimos vértices. Son tres ciclos anidados de n vueltas con cuerpo $\mathcal{O}(1)$: $\Theta(n^3)$.

El código es el producto de matrices de siempre, $c_{ij} = \sum_k a_{ik} \cdot b_{kj}$, con dos sustituciones: la suma se vuelve mín y el producto se vuelve $+$. La tabla completa la analogía.

	Producto usual	**Producto** $(\text{mín}, +)$
Casilla	$c_{ij} = \sum_k a_{ik} \cdot b_{kj}$	$l'_{ij} = \text{mín}_k\,(l_{ik} + w_{kj})$
"Suma"	$+$	mín
"Producto"	$\cdot$	$+$
Neutro de la suma	0	∞
Neutro del producto	1	0
Identidad	I: unos en la diagonal, ceros fuera	$L^{(0)}$: ceros en la diagonal, ∞ fuera

Con esas operaciones, $(\mathbb{R} \cup \{\infty\}, \text{mín}, +)$ es un **semianillo**: mín y $+$ son asociativas y tienen neutro, y $+$ se distribuye sobre mín, $x + \text{mín}(y, z) = \text{mín}(x + y, x + z)$. Lo que no tiene es resta: no hay manera de deshacer un mínimo. Se escribe $A \cdot B$ para el producto $(\text{mín}, +)$ y W^r para sus potencias, de modo que $L^{(r)} = W^r$ y, por la proposición 9.5, la respuesta es W^{n-1}.

Proposición 9.6 (El producto $(\text{mín}, +)$ es asociativo)**.** *Para matrices A, B, C de $n \times n$, $(A \cdot B) \cdot C = A \cdot (B \cdot C)$.*

Demostración. Por la distributividad de $+$ sobre mín,

$$\begin{aligned}((A \cdot B) \cdot C)_{ij} &= \min_l \Big(\min_k (a_{ik} + b_{kl}) + c_{lj} \Big) = \min_{k,l} (a_{ik} + b_{kl} + c_{lj}) \\ &= \min_k \Big(a_{ik} + \min_l (b_{kl} + c_{lj}) \Big) = (A \cdot (B \cdot C))_{ij}.\end{aligned}$$

□

La consecuencia práctica es $W^{2r} = W^r \cdot W^r$, que también se lee directo: una caminata de a lo más $2r$ aristas se parte en dos de a lo más r cada una. No hace falta calcular las potencias una por una.

Algoritmo 9.3 Todos-Pares-Lento(W)

```
1: procedimiento Todos-Pares-Lento(W)
2:     n ← número de renglones de W
3:     L^(1) ← W
4:     para r ← 2 hasta n − 1 hacer
5:         L^(r) ← Extender-Caminos(L^(r−1), W)
6:     fin para
7:     devolver L^(n−1)
8: fin procedimiento
```

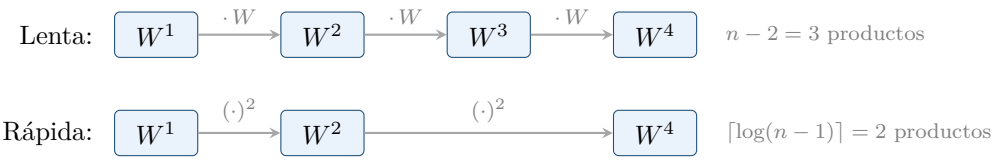
Algoritmo 9.4 Todos-Pares-Rápido(W)

```
1: procedimiento Todos-Pares-Rápido(W)
2:     n ← número de renglones de W
3:     L^(1) ← W,    r ← 1
4:     mientras r < n − 1 hacer
5:         L^(2r) ← Extender-Caminos(L^(r), L^(r))        // elevar al cuadrado
6:         r ← 2r
7:     fin mientras
8:     devolver L^(r)
9: fin procedimiento
```

Todos-Pares-Lento hace $n-2$ productos de $\Theta(n^3)$: $\Theta(n^4)$, lo mismo que n corridas de Bellman-Ford en un grafo denso. Como está escrito guarda todas las $L^{(r)}$, pero cada una solo necesita la anterior y dos matrices bastan.

Todos-Pares-Rápido calcula $W, W^2, W^4, \ldots$ elevando al cuadrado, la misma idea de la exponenciación rápida. Después de t vueltas $r = 2^t$, y el ciclo se detiene en la primera t con $2^t \geq n-1$, o sea $t = \lceil \log(n-1) \rceil$. La última matriz es $L^{(r)}$ con $n-1 \leq r < 2n-2$, y por la proposición 9.5 pasarse de $n-1$ no estropea nada. Costo: $\lceil \log(n-1) \rceil$ productos de $\Theta(n^3)$, $\Theta(n^3 \log n)$, con dos matrices de memoria. Para $n = 1000$ la versión lenta hace 998 productos y la rápida 10: del orden de 10^{12} operaciones contra 10^{10}. Floyd-Warshall, en la siguiente subsección, hace 10^9.

Ejemplo 9.7 (Lento contra cuadrados repetidos)**.** Con $n = 5$ la versión lenta calcula $L^{(2)}, L^{(3)}, L^{(4)}$ y la rápida solo $L^{(2)}$ y $L^{(4)} = L^{(2)} \cdot L^{(2)}$.



Sobre el grafo del ejemplo 9.3, en azul las casillas que cambian respecto a la matriz anterior:

$$L^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \qquad L^{(2)} = \begin{pmatrix} 0 & 3 & 8 & \mathbf{2} & -4 \\ \mathbf{3} & 0 & \mathbf{-4} & 1 & 7 \\ \infty & 4 & 0 & \mathbf{5} & \mathbf{11} \\ 2 & \mathbf{-1} & -5 & 0 & \mathbf{-2} \\ \mathbf{8} & \infty & \mathbf{1} & 6 & 0 \end{pmatrix}$$

$$L^{(3)} = \begin{pmatrix} 0 & 3 & \mathbf{-3} & 2 & -4 \\ 3 & 0 & -4 & 1 & \mathbf{-1} \\ \mathbf{7} & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & \mathbf{5} & 1 & 6 & 0 \end{pmatrix} \qquad L^{(4)} = \begin{pmatrix} 0 & \mathbf{1} & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & \mathbf{3} \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$

La rápida llega a la misma $L^{(4)}$, y $L^{(4)} \cdot W = L^{(4)}$, como pide la proposición 9.5. Una casilla baja en $L^{(r)}$ cuando el mejor camino con a lo más r aristas usa exactamente r. La $(1,2)$ vale 3 hasta $L^{(3)}$ y baja a 1 en $L^{(4)}$ por el camino $1 \to 5 \to 4 \to 3 \to 2$, de cuatro aristas y peso $-4+6-5+4=1$. Las dos versiones llegan a ese 1 por rutas distintas:

- Lenta, $L^{(4)} = L^{(3)} \cdot W$: los candidatos $l^{(3)}_{1k} + w_{k2}$ son $3, 3, 1, \infty, \infty$ y gana $k=3$, tres aristas hasta 3 y luego $3 \to 2$.
- Rápida, $L^{(4)} = L^{(2)} \cdot L^{(2)}$: los candidatos $l^{(2)}_{1k} + l^{(2)}_{k2}$ son $3, 3, 12, 1, \infty$ y gana $k=4$, dos aristas hasta 4 ($1 \to 5 \to 4$) y dos más hasta 2 ($4 \to 3 \to 2$).

En la rápida, k ya no es el penúltimo vértice sino el punto medio del camino.

Ojito...

- Pasarse de $n-1$ solo es inofensivo sin ciclos negativos. Con uno, $L^{(r)}$ sigue bajando para siempre. Un ciclo negativo de c aristas deja $l^{(r)}_{ii} < 0$ en sus vértices en cuanto $r \geq c$, y un ciclo tiene a lo más n aristas, así que basta revisar la diagonal de una $L^{(r)}$ con $r \geq n$. Con $r = n-1$ no alcanza: el ciclo $1 \to 2 \to 3 \to 1$ con pesos -1 deja la diagonal de $L^{(2)}$ en ceros y la de $L^{(3)}$ en -3.
- El algoritmo de Strassen no sirve para el producto $(\text{mín}, +)$: sus fórmulas restan, y el mínimo no tiene inverso.

Floyd-Warshall

La clase del 22 de octubre trae la recurrencia, el pseudocódigo en el lugar y un ejemplo sobre el grafo dirigido de cuatro vértices. Aquí se justifica cada pieza: por qué la recurrencia es correcta y dónde entra la hipótesis de no haber ciclos negativos, cómo se construye Π, por qué basta una matriz y cómo se detectan los ciclos negativos. El ejemplo grande usa pesos negativos, que es justo donde Floyd-Warshall tiene algo que Dijkstra no.

La idea es cambiar el estado de la programación dinámica: en lugar de cuántas aristas usa el camino, qué vértices puede usar como intermedios.

Definición 9.8 (Vértice intermedio)**.** Los **intermedios** de una caminata $p = \langle v_1, v_2, \ldots, v_l \rangle$ son los vértices que aparecen en las posiciones 2 a $l-1$, es decir, todos menos los extremos. Si G no tiene ciclos negativos, $d^{(k)}_{ij}$ es el peso mínimo de una caminata de i a j cuyos intermedios están todos en $\{1, \ldots, k\}$, o ∞ si no hay ninguna, y $D^{(k)} = (d^{(k)}_{ij})$.

Con $k=0$ no se permite ningún intermedio: la caminata es la arista directa o la vacía, de ahí $d^{(0)}_{ij} = w_{ij}$. Con $k=n$ se permite todo, así que $d^{(n)}_{ij} = \delta(i,j)$.

Lema 9.9 (Estructura de Floyd-Warshall)**.** *Si G no tiene ciclos negativos, para $k \geq 1$*

$$d^{(k)}_{ij} = \text{mín}(d^{(k-1)}_{ij},\ d^{(k-1)}_{ik} + d^{(k-1)}_{kj}).$$

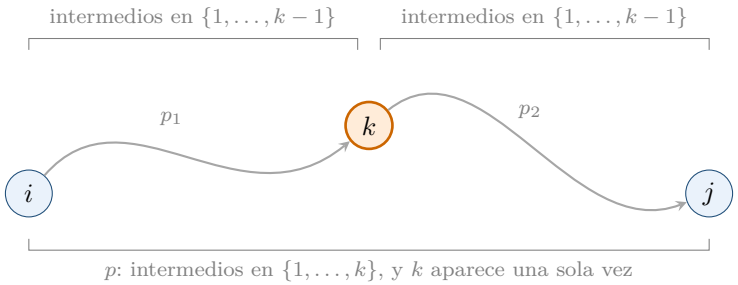
Demostración. Primero $\leq$. Una caminata con intermedios en $\{1, \ldots, k-1\}$ también los tiene en $\{1, \ldots, k\}$, así que $d^{(k)}_{ij} \leq d^{(k-1)}_{ij}$. Y pegar una caminata óptima de i a k con una de k a j, las dos con

intermedios en $\{1, \ldots, k-1\}$, da una caminata de i a j cuyos intermedios son los de ambas más el propio k: $d_{ij}^{(k)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$.

Ahora $\geq$, y aquí entra la hipótesis. Sea p una caminata óptima de i a j con intermedios en $\{1, \ldots, k\}$. Si p repite un vértice, contiene un ciclo; ese ciclo no es negativo, y quitarlo deja una caminata con los mismos extremos, intermedios en el mismo conjunto y peso no mayor. Repitiendo, p se puede tomar sin vértices repetidos. Hay dos casos.

- k no es intermedio de p. Entonces todos sus intermedios están en $\{1, \ldots, k-1\}$, así que $d_{ij}^{(k)} = w(p) \geq d_{ij}^{(k-1)}$.
- k es intermedio de p. Aparece exactamente una vez, porque p no repite vértices. Partiendo p en k quedan p_1 de i a k y p_2 de k a j, y en ninguno de los dos k es intermedio: es un extremo y no aparece en otra parte. Los dos usan solo $\{1, \ldots, k-1\}$, así que $d_{ij}^{(k)} = w(p_1) + w(p_2) \geq d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$. □

La figura es la 25.3 de Cormen: el camino óptimo que pasa por k se parte en dos tramos que solo usan intermedios menores que k.



Idea clave

La hipótesis de no haber ciclos negativos sirve para una sola cosa: poder tomar el óptimo sin vértices repetidos, y con eso que k aparezca a lo más una vez. Si un ciclo negativo pasara por k, la mejor caminata daría vueltas a ese ciclo sin límite, k aparecería tantas veces como se quisiera y no habría manera de partirla en solo dos tramos. Ganada la hipótesis, cada casilla elige entre dos opciones, usar k o no, en lugar de entre n penúltimos vértices como en la subsección anterior. Las dos programaciones dinámicas tienen n^3 estados (n^2 pares por n etapas); la diferencia entre $\Theta(n^4)$ y $\Theta(n^3)$ es lo que cuesta resolver cada uno.

Los predecesores

Π se construye con una regla igual de simple. Al principio el predecesor de j desde i es i si hay arista, y nulo si no:

$$\pi_{ij}^{(0)} = \begin{cases} \text{nulo} & \text{si } i = j \text{ o } w_{ij} = \infty, \\ i & \text{si } i \neq j \text{ y } w_{ij} < \infty. \end{cases}$$

En la vuelta k, el predecesor solo cambia si pasar por k mejora estrictamente:

$$\pi_{ij}^{(k)} = \begin{cases} \pi_{ij}^{(k-1)} & \text{si } d_{ij}^{(k-1)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)}, \\ \pi_{kj}^{(k-1)} & \text{si } d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)}. \end{cases}$$

Si el mejor camino pasa por k, su último tramo es un camino óptimo de k a j, así que el predecesor de j es el que ya se tenía para el par (k, j). En un empate se conserva el predecesor viejo, y eso

no es un detalle de estilo: la otra elección rompe Π (Cormen, ejercicio 25.2-5). Que cada $G_{\pi,i}$ salga un árbol de caminos más cortos es el ejercicio 25.2-3 de Cormen; la clave es mostrar que $\pi_{ij}^{(k)} = l$ implica $d_{ij}^{(k)} \geq d_{il}^{(k)} + w_{lj}$, lo que impide ciclos en $G_{\pi,i}$, y luego adaptar el argumento del subgrafo de predecesores de fuente única (proposición 7.13).

El algoritmo completo

La versión de abajo junta la recurrencia sin superíndices, que es la de la clase, con el cálculo de Π y la prueba de la diagonal.

Algoritmo 9.5 Floyd-Warshall(W), con Π y en el lugar

```
 1: procedimiento FLOYD-WARSHALL(W)
 2:     n ← número de renglones de W;    D ← W
 3:     para i ← 1 hasta n hacer
 4:         para j ← 1 hasta n hacer
 5:             π_ij ← i si i ≠ j y w_ij < ∞; si no, π_ij ← NULO
 6:         fin para
 7:     fin para
 8:     para k ← 1 hasta n hacer                    // k va afuera, siempre
 9:         para i ← 1 hasta n hacer
10:             para j ← 1 hasta n hacer
11:                 si d_ik + d_kj < d_ij entonces
12:                     d_ij ← d_ik + d_kj
13:                     π_ij ← π_kj
14:                 fin si
15:             fin para
16:         fin para
17:     fin para
18:     para i ← 1 hasta n hacer
19:         si d_ii < 0 entonces
20:             devolver "hay un ciclo de peso negativo"
21:         fin si
22:     fin para
23:     devolver D, Π
24: fin procedimiento
```

- Líneas 3 a 7: $\Pi^{(0)}$, en $\Theta(n^2)$.
- Línea 8: k tiene que ser el ciclo de afuera. La recurrencia necesita $D^{(k-1)}$ completa antes de empezar con k.
- Línea 11: la desigualdad es estricta. En un empate no se toca nada, que es el $\leq$ de la regla de Π.
- Línea 13: π_{kj} se lee de la misma matriz que se está modificando; la proposición 9.10 dice por qué está bien.
- Líneas 18 a 22: el teorema 9.11. Si hay un ciclo negativo, D y Π no significan nada y no deben usarse.

Costo. La inicialización es $\Theta(n^2)$, el triple ciclo ejecuta su cuerpo de $\mathcal{O}(1)$ exactamente n^3 veces y la

diagonal cuesta $\Theta(n)$: $\Theta(n^3)$ en tiempo y $\Theta(n^2)$ en memoria. No hay mejor caso. El costo depende solo de n, no de m ni de los pesos, y eso es una virtud en grafos densos y un defecto en los dispersos.

Proposición 9.10 (Basta una matriz)**.** *Si G no tiene ciclos negativos, durante la vuelta k el renglón k y la columna k de D y de Π no cambian. Por eso la versión sin superíndices calcula exactamente $D^{(n)}$ y $\Pi^{(n)}$, con $\Theta(n^2)$ de memoria.*

Demostración. Primero, $d_{kk}^{(k-1)} = 0$: la caminata vacía de k a k pesa 0, y ninguna caminata cerrada pesa menos porque no hay ciclos negativos. Entonces

$$d_{ik}^{(k)} = \min(d_{ik}^{(k-1)},\ d_{ik}^{(k-1)} + d_{kk}^{(k-1)}) = d_{ik}^{(k-1)},$$

y lo mismo con $d_{kj}^{(k)}$. En Π, π_{kj} solo cambiaría si $d_{kk} + d_{kj} < d_{kj}$, es decir, si $d_{kk} < 0$. En la vuelta k las únicas casillas que se leen además de d_{ij} son d_{ik}, d_{kj} y π_{kj}, y valen lo mismo antes y después de que el ciclo pase por ellas. Leerlas ya actualizadas o todavía sin actualizar da igual, y por inducción sobre k la matriz única coincide con $D^{(k)}$ y $\Pi^{(k)}$ al final de cada vuelta. □

Es la misma pregunta que surge en la mochila 0-1 con una sola fila, donde el orden del recorrido sí importa porque la casilla que se lee puede haber cambiado en la misma pasada. Aquí no importa porque las casillas que se leen, el renglón y la columna k, son justo las que no cambian.

Teorema 9.11 (La diagonal delata los ciclos negativos)**.** *Sin suponer nada sobre los ciclos, sea D la matriz que deja Floyd-Warshall, por capas o en el lugar. G tiene un ciclo de peso negativo si y solo si $d_{ii} < 0$ para algún i.*

Demostración. Sin la hipótesis, $d_{ij}^{(k)}$ ya no se puede definir como un mínimo, así que la prueba habla de los números que calcula la recurrencia. Dos hechos, los dos por inducción sobre k:

(a) Cada valor finito es el peso de alguna caminata de i a j. Para $k = 0$ es una arista o la caminata vacía, y cada actualización pega una caminata de i a k con una de k a j.

(b) $d_{ij}^{(k)} \le w(p)$ para todo camino p de i a j con intermedios en $\{1, \ldots, k\}$. Si k no está en p, vale porque $d_{ij}^{(k)} \le d_{ij}^{(k-1)}$; si está, aparece una vez y p se parte en dos caminos que solo usan $\{1, \ldots, k-1\}$, como en el lema 9.9.

La versión en el lugar cumple (a) por la misma razón, y sus valores nunca quedan arriba de los de la versión por capas, porque cada actualización toma el mínimo de números que ya son menores o iguales. Así que también cumple (b).

($\Leftarrow$) Si $d_{ii} < 0$, por (a) hay una caminata cerrada de peso negativo. Si no es un ciclo, repite un vértice a la mitad y se parte en dos caminatas cerradas más cortas cuyos pesos suman el total; una de ellas es negativa. Repitiendo se llega a un ciclo negativo.

($\Rightarrow$) Sea C un ciclo negativo y q su vértice de número más alto. Un lazo (q, q) no cuenta, porque $w_{qq} = 0$ por definición (ver el ojo de abajo), así que C tiene otro vértice i. Los dos tramos de C, de i a q y de q a i, son caminos cuyos intermedios son otros vértices de C, todos menores que q. Por (b) con $k = q - 1$,

$$d_{ii}^{(q)} \le d_{iq}^{(q-1)} + d_{qi}^{(q-1)} \le w(C) < 0,$$

y como los valores nunca suben, $d_{ii}^{(n)} < 0$. □

Ejemplo 9.12 (Floyd-Warshall sobre el grafo del capítulo)**.** Las seis parejas $D^{(k)}$, $\Pi^{(k)}$ sobre el grafo del ejemplo 9.3; en azul, las casillas que cambian en la vuelta k. Coinciden con la figura 25.4 de Cormen.

$$D^{(0)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \qquad \Pi^{(0)} = \begin{pmatrix} \text{NULO} & 1 & 1 & \text{NULO} & 1 \\ \text{NULO} & \text{NULO} & \text{NULO} & 2 & 2 \\ \text{NULO} & 3 & \text{NULO} & \text{NULO} & \text{NULO} \\ 4 & \text{NULO} & 4 & \text{NULO} & \text{NULO} \\ \text{NULO} & \text{NULO} & \text{NULO} & 5 & \text{NULO} \end{pmatrix}$$

$$D^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \mathbf{5} & -5 & 0 & \mathbf{-2} \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \qquad \Pi^{(1)} = \begin{pmatrix} \text{NULO} & 1 & 1 & \text{NULO} & 1 \\ \text{NULO} & \text{NULO} & \text{NULO} & 2 & 2 \\ \text{NULO} & 3 & \text{NULO} & \text{NULO} & \text{NULO} \\ 4 & \mathbf{1} & 4 & \text{NULO} & \mathbf{1} \\ \text{NULO} & \text{NULO} & \text{NULO} & 5 & \text{NULO} \end{pmatrix}$$

$$D^{(2)} = \begin{pmatrix} 0 & 3 & 8 & \mathbf{4} & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \mathbf{5} & \mathbf{11} \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \qquad \Pi^{(2)} = \begin{pmatrix} \text{NULO} & 1 & 1 & \mathbf{2} & 1 \\ \text{NULO} & \text{NULO} & \text{NULO} & 2 & 2 \\ \text{NULO} & 3 & \text{NULO} & \mathbf{2} & \mathbf{2} \\ 4 & 1 & 4 & \text{NULO} & 1 \\ \text{NULO} & \text{NULO} & \text{NULO} & 5 & \text{NULO} \end{pmatrix}$$

$$D^{(3)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & \mathbf{-1} & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \qquad \Pi^{(3)} = \begin{pmatrix} \text{NULO} & 1 & 1 & 2 & 1 \\ \text{NULO} & \text{NULO} & \text{NULO} & 2 & 2 \\ \text{NULO} & 3 & \text{NULO} & 2 & 2 \\ 4 & \mathbf{3} & 4 & \text{NULO} & 1 \\ \text{NULO} & \text{NULO} & \text{NULO} & 5 & \text{NULO} \end{pmatrix}$$

$$D^{(4)} = \begin{pmatrix} 0 & 3 & \mathbf{-1} & 4 & -4 \\ \mathbf{3} & 0 & \mathbf{-4} & 1 & \mathbf{-1} \\ \mathbf{7} & 4 & 0 & 5 & \mathbf{3} \\ 2 & -1 & -5 & 0 & -2 \\ \mathbf{8} & \mathbf{5} & \mathbf{1} & 6 & 0 \end{pmatrix} \qquad \Pi^{(4)} = \begin{pmatrix} \text{NULO} & 1 & \mathbf{4} & 2 & 1 \\ \mathbf{4} & \text{NULO} & \mathbf{4} & 2 & \mathbf{1} \\ \mathbf{4} & 3 & \text{NULO} & 2 & \mathbf{1} \\ 4 & 3 & 4 & \text{NULO} & 1 \\ \mathbf{4} & \mathbf{3} & \mathbf{4} & 5 & \text{NULO} \end{pmatrix}$$

$$D^{(5)} = \begin{pmatrix} 0 & \mathbf{1} & \mathbf{-3} & \mathbf{2} & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \qquad \Pi^{(5)} = \begin{pmatrix} \text{NULO} & \mathbf{3} & 4 & \mathbf{5} & 1 \\ 4 & \text{NULO} & 4 & 2 & 1 \\ 4 & 3 & \text{NULO} & 2 & 1 \\ 4 & 3 & 4 & \text{NULO} & 1 \\ 4 & 3 & 4 & 5 & \text{NULO} \end{pmatrix}$$

Qué pasa en cada vuelta. Ninguna casilla del renglón k ni de la columna k cambia en la vuelta k, como dice la proposición 9.10.

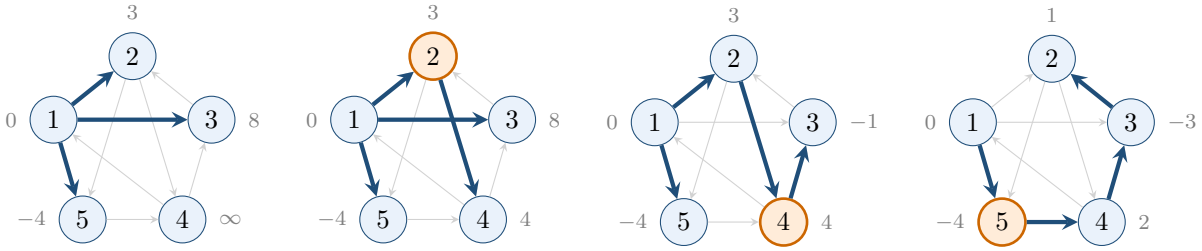
- $k = 1$: solo 4 entra a 1, así que solo el renglón 4 puede mejorar pasando por 1: $d_{42} = d_{41} + d_{12} = 2 + 3 = 5$ y $d_{45} = 2 + (-4) = -2$, las dos con predecesor $\pi_{12} = \pi_{15} = 1$.
- $k = 2$: a 2 entran aristas desde 1 y 3. El renglón 1 gana $1 \to 2 \to 4$, con $d_{14} = 3 + 1 = 4$, y el 3 gana $3 \to 2 \to 4$ y $3 \to 2 \to 5$, con 5 y 11.
- $k = 3$: un solo cambio, y es la mejora de un valor finito: $d_{42} = d_{43} + d_{32} = -5 + 4 = -1 < 5$ por la arista negativa $4 \to 3$, y $\pi_{42} \leftarrow \pi_{32} = 3$.
- $k = 4$: la vuelta grande, nueve casillas. Casi todos los caminos buenos pasan por 4: tiene las salidas $4 \to 1$ y $4 \to 3$, la negativa, y es la única salida de 5, cuyo renglón se llena completo. Por ejemplo, $d_{52} = d_{54} + d_{42} = 6 + (-1) = 5$ con $\pi_{52} \leftarrow \pi_{42} = 3$: el predecesor viene del renglón 4, que ya sabía llegar a 2 por 3.

- $k = 5$: solo mejora el renglón 1, gracias a la arista $1 \to 5$ de peso -4; por ejemplo, $d_{12} = d_{15} + d_{52} = -4 + 5 = 1$ y $\pi_{12} \leftarrow \pi_{52} = 3$. Todo camino que sale de 5 empieza con $5 \to 4$, y los demás vértices ya llegaban a 4 más barato sin pasar por 5.

Un detalle de la última vuelta: d_{13} baja de -1 a -3 y π_{13} *no* cambia. El camino nuevo, $1 \to 5 \to 4 \to 3$, termina con la misma arista $4 \to 3$ que el anterior, $1 \to 2 \to 4 \to 3$, así que $\pi_{53} = 4 = \pi_{13}$. Cambiar D no implica cambiar Π.

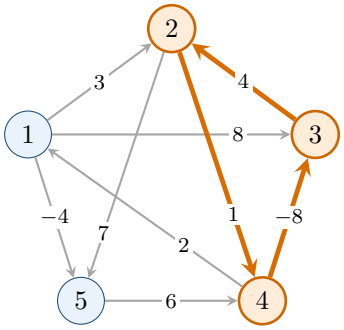
Reconstrucción. Imprimir-Camino-Pares$(\Pi^{(5)}, 1, 2)$ sube por el renglón 1: $\pi_{12} = 3$, $\pi_{13} = 4$, $\pi_{14} = 5$, $\pi_{15} = 1$, e imprime $1, 5, 4, 3, 2$, con peso $-4 + 6 - 5 + 4 = 1 = d_{12}$. De 3 a 5 sale $3, 2, 4, 1, 5$, con peso $4 + 1 + 2 - 4 = 3$, y de 2 a 1 sale $2, 4, 1$, con peso 3. Los 20 caminos que produce $\Pi^{(5)}$ pesan lo que dice $D^{(5)}$, y $D^{(5)}$ es la misma $L^{(4)}$ del ejemplo 9.7.

Cómo crece el árbol desde 1. El renglón 1 de $\Pi^{(k)}$ define $G_{\pi,1}$ en cada etapa. En azul las aristas del árbol, en naranja el vértice k de la vuelta, y junto a cada vértice $d_{1j}^{(k)}$:



Al principio el árbol son las aristas directas desde 1. En $k = 2$ aparece $2 \to 4$, con $d_{14} = 3 + 1 = 4$. En $k = 4$ la arista $4 \to 3$ reemplaza a $1 \to 3$, con $d_{13} = 4 - 5 = -1$. En $k = 5$ el árbol se vuelve un solo camino, $1 \to 5 \to 4 \to 3 \to 2$. A diferencia de Dijkstra, ningún valor es definitivo antes de la última vuelta: d_{12} valió 3 desde el principio y cambió hasta $k = 5$.

Ejemplo 9.13 (Caso borde: un ciclo negativo)**.** El grafo del ejemplo 9.3 con un solo cambio: la arista $4 \to 3$ pesa -8 en lugar de -5, y el ciclo $2 \to 4 \to 3 \to 2$ pasa a pesar $1 - 8 + 4 = -3$. La tabla da la diagonal de $D^{(k)}$ en cada vuelta de la versión por capas.



ciclo $2 \to 4 \to 3 \to 2$: $1 - 8 + 4 = -3$

k	d_{11}	d_{22}	d_{33}	d_{44}	d_{55}
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	-3	0
4	0	-3	-3	-6	0
5	0	-3	-3	-6	0

- La primera casilla negativa aparece en $k = 3$, no en $k = 4$. Para $i = 4$ el ciclo, leído como $4 \to 3 \to 2 \to 4$, solo tiene intermedios 3 y 2, y ya cuenta en $D^{(3)}$. Es la prueba del teorema 9.11 vista desde el vértice más alto del ciclo, que aquí es su propio extremo.
- En $k = 4$ los otros dos vértices del ciclo bajan a -3, y d_{44} baja a $-6 = d_{44} + d_{44}$: la caminata da dos vueltas al ciclo. Con ciclos negativos los números ya no son pesos de caminos más cortos.

- d_{11} y d_{55} se quedan en 0, aunque 1 y 5 alcanzan el ciclo y se alcanzan desde él. La diagonal avisa que hay un ciclo, no qué pares están afectados: aquí los 25 pares tienen en realidad $\delta = -\infty$, mientras $D^{(5)}$ muestra números finitos como $d_{12} = -2$. Para marcarlos se pone $d_{ij} = -\infty$ cuando existe k con $d_{kk} < 0$, $d_{ik} < \infty$ y $d_{kj} < \infty$: i llega a un ciclo negativo y de ahí a j.
- La versión en el lugar también lo detecta, con otra diagonal: $0, -5, -5, -8, -16$. La proposición 9.10 deja de valer cuando $d_{kk} < 0$, pero el teorema 9.11 no.

Ojito...

- k **va afuera.** Con el ciclo de k adentro, en el orden i, j, k, el algoritmo es otro y falla: en el grafo del capítulo da $d_{12} = 3$ y $d_{13} = 8$ en lugar de 1 y -3, y en 2000 grafos aleatorios chicos falló en 721. Curiosamente, en esas mismas pruebas repetir tres veces la versión mal ordenada dio el resultado correcto todas las veces. Es una observación experimental, no una razón para programarlo así.
- **Aritmética con ∞.** En código, ∞ suele ser un entero grande. Entonces $\infty + (-5) < \infty$ inventa caminos que no existen, y $\infty + \infty$ puede desbordar. Hay que saltarse la actualización si $d_{ik} = \infty$ o $d_{kj} = \infty$.
- **Lazos negativos.** La matriz de pesos pone $w_{ii} = 0$ siempre, así que un lazo (i, i) negativo, que es un ciclo negativo, desaparece y nadie lo detecta. Si el grafo puede tener lazos, hay que poner $w_{ii} = \min(0, w(i, i))$; con eso d_{ii} sale negativo.
- **Grafos no dirigidos.** Una arista no dirigida es un par de arcos (u, v) y (v, u). Si pesa -1, el ciclo $u \to v \to u$ pesa -2: con una sola arista negativa ya hay ciclo negativo, y Floyd-Warshall lo reporta.
- **Los números explotan.** Con un ciclo negativo los valores bajan muy rápido: en el grafo dirigido completo de n vértices con todos los pesos en -1, el mínimo de $D^{(n)}$ es -2^n (verificado de $n = 2$ a 8). Con pesos grandes eso desborda. Revisar $d_{kk} < 0$ al terminar cada vuelta y detenerse en cuanto aparezca cuesta $\Theta(n)$ extra por vuelta.

Cerradura transitiva

A veces no interesa cuánto cuesta ir de i a j, sino si se puede.

Definición 9.14 (Cerradura transitiva)**.** La cerradura transitiva de G es $G^* = (V, E^*)$ con $E^* = \{(i, j) : \text{hay un camino de } i \text{ a } j \text{ en } G\}$. Como el camino vacío cuenta, E^* incluye todos los pares (i, i).

Es el esquema de Floyd-Warshall con un camino que existe o no existe en lugar de un peso. Sea $t_{ij}^{(k)} \in \{0, 1\}$ igual a 1 si hay un camino de i a j con intermedios en $\{1, \ldots, k\}$. Entonces $t_{ij}^{(0)} = 1$ si $i = j$ o $(i, j) \in E$, y 0 si no, y

$$t_{ij}^{(k)} = t_{ij}^{(k-1)} \vee (t_{ik}^{(k-1)} \wedge t_{kj}^{(k-1)}),$$

que es el lema 9.9 con $\vee$ en lugar de mín y $\wedge$ en lugar de $+$. Aquí no hace falta ninguna hipótesis sobre ciclos: si hay alguna caminata de i a j, hay un camino. Al final, $(i, j) \in E^*$ si y solo si $t_{ij}^{(n)} = 1$.

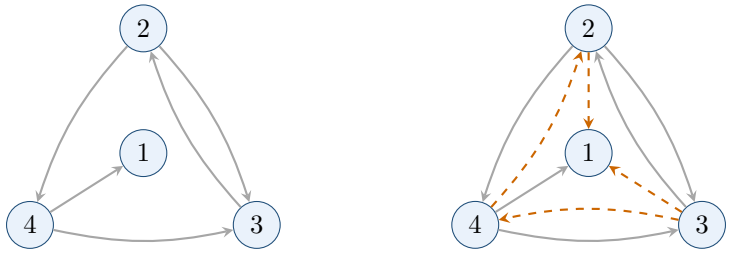
Algoritmo 9.6 Cerradura-Transitiva(G), en el lugar

```
1:  procedimiento Cerradura-Transitiva(G)
2:      para i ← 1 hasta n hacer
3:          para j ← 1 hasta n hacer
4:              t_ij ← 1 si i = j o (i, j) ∈ E; si no, t_ij ← 0
5:          fin para
6:      fin para
7:      para k ← 1 hasta n hacer
8:          para i ← 1 hasta n hacer
9:              para j ← 1 hasta n hacer
10:                 t_ij ← t_ij ∨ (t_ik ∧ t_kj)
11:             fin para
12:         fin para
13:     fin para
14:     devolver T
15: fin procedimiento
```

Costo. $\Theta(n^3)$, igual que Floyd-Warshall. La versión en el lugar es correcta por el argumento de la proposición 9.10, con $t_{kk} = 1$ en el papel de $d_{kk} = 0$. La ganancia es práctica: operaciones lógicas sobre bits en lugar de aritmética sobre palabras, y la memoria baja por un factor del tamaño de la palabra. Si cada renglón de T se guarda como un vector de bits, la línea 10 para i fija y todo j se vuelve "si $t_{ik} = 1$, renglón $i \leftarrow$ renglón i OR renglón k", y con palabras de b bits el total baja a $\Theta(n^3/b)$ operaciones de palabra. Hay dos alternativas. Poner peso 1 a cada arista y correr Floyd-Warshall: hay camino si y solo si $d_{ij} < n$. O, en grafos dispersos, un BFS o DFS desde cada vértice (sección 4), en $\Theta(n(n+m))$.

El nombre doble del algoritmo viene de aquí. Según las notas del capítulo 25 de Cormen, Floyd basó su algoritmo de caminos más cortos en un teorema de Warshall que describe cómo calcular la cerradura transitiva de matrices booleanas.

Ejemplo 9.15 (Cerradura transitiva, figura 25.5 de Cormen)**.** Un grafo de cuatro vértices con aristas $2 \to 3$, $2 \to 4$, $3 \to 2$, $4 \to 1$ y $4 \to 3$. Se dibuja con 1 al centro para que G^* quede sin cruces; en naranja punteado, las aristas que agrega la cerradura, sin dibujar los pares (i, i).



G $\qquad$ G^*

$$T^{(0)} = T^{(1)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{pmatrix} \quad T^{(2)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & \mathbf{1} \\ 1 & 0 & 1 & 1 \end{pmatrix} \quad T^{(3)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & \mathbf{1} & 1 & 1 \end{pmatrix} \quad T^{(4)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ \mathbf{1} & 1 & 1 & 1 \\ \mathbf{1} & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$$

- $k = 1$: nada cambia. 1 no tiene aristas de salida, así que pasar por 1 no lleva a ningún lado.
- $k = 2$: $t_{34} = t_{32} \wedge t_{24} = 1$, por $3 \to 2 \to 4$.

- $k = 3$: $t_{42} = t_{43} \wedge t_{32} = 1$, por $4 \to 3 \to 2$.
- $k = 4$: $t_{21} = t_{24} \wedge t_{41} = 1$ y $t_{31} = t_{34} \wedge t_{41} = 1$; el segundo usa el t_{34} que apareció en $k = 2$, por $3 \to 2 \to 4 \to 1$.

Al final 1 solo se alcanza a sí mismo, y 2, 3 y 4 alcanzan a todos.

Johnson: reponderar para poder usar Dijkstra

Floyd-Warshall cuesta $\Theta(n^3)$ aunque el grafo tenga $m = 2n$ aristas. En un grafo disperso, n corridas de Dijkstra harían del orden de $n^2 \log n$, pero Dijkstra no admite pesos negativos, y n corridas de Bellman-Ford cuestan $\mathcal{O}(n^2 m)$. El algoritmo de Johnson (1977) junta lo mejor de los dos con una idea que vale más que el algoritmo mismo: cambiar los pesos sin cambiar los caminos más cortos.

Definición 9.16 (Reponderación)**.** Dada una función $h : V \to \mathbb{R}$ cualquiera, la **reponderación** de w por h es

$$\hat{w}(u, v) = w(u, v) + h(u) - h(v),$$

y $\hat{\delta}$ denota los pesos de caminos más cortos con $\hat{w}$.

A h se le suele llamar **potencial**, y a $\hat{w}$, **costos reducidos**; Cormen no le da nombre. El gorro sí es de Cormen. Algunas notas escriben w', pero aquí $G' = (V', E')$ va a ser el grafo aumentado con un vértice nuevo, y w' se leería como "los pesos de G'", que son otra cosa: en G' los pesos siguen siendo w.

Lema 9.17 (La reponderación conserva los caminos más cortos)**.** *Sea $h : V \to \mathbb{R}$ cualquiera y $\hat{w}$ la reponderación por h.*

(a) Para toda caminata $p = \langle v_0, v_1, \ldots, v_k \rangle$, $\hat{w}(p) = w(p) + h(v_0) - h(v_k)$.

(b) p es un camino más corto de v_0 a v_k con w si y solo si lo es con $\hat{w}$.

(c) G tiene un ciclo negativo con w si y solo si lo tiene con $\hat{w}$.

Demostración. La suma es telescópica:

$$\hat{w}(p) = \sum_{i=1}^{k} \big(w(v_{i-1}, v_i) + h(v_{i-1}) - h(v_i)\big) = w(p) + \sum_{i=1}^{k} \big(h(v_{i-1}) - h(v_i)\big) = w(p) + h(v_0) - h(v_k),$$

porque cada $h(v_i)$ intermedio aparece una vez sumando y una vez restando. El término $h(v_0) - h(v_k)$ depende solo de los extremos: a todas las caminatas de v_0 a v_k se les suma la misma cantidad, así que una pesa menos que otra con w si y solo si pesa menos con $\hat{w}$, y eso es (b). Para un ciclo, $v_0 = v_k$ y (a) da $\hat{w}(c) = w(c)$, que es (c). □

Corolario 9.18. *Si G no tiene ciclos negativos, $\delta(u, v) = \hat{\delta}(u, v) + h(v) - h(u)$ para todo par, con ∞ en los dos lados cuando no hay camino.*

Basta aplicar (a) a un camino más corto p de u a v, que por (b) también lo es con $\hat{w}$. Para no equivocarse con el signo: con una sola arista, $\hat{w} = w + h(u) - h(v)$, y despejando, $w = \hat{w} + h(v) - h(u)$.

Lo que el lema no dice es que se conserve el orden entre destinos distintos, porque cada destino tiene su propia $h(v)$. Dijkstra con $\hat{w}$ puede extraer los vértices en un orden que no tiene nada que ver con las distancias reales, y aun así las distancias salen bien; el ejemplo 9.22 lo muestra.

Cómo escoger h

El lema vale para cualquier h. La gracia es encontrar una que deje todos los pesos no negativos, para poder correr Dijkstra. Se agrega un vértice nuevo s con una arista de peso 0 hacia cada vértice: el **grafo aumentado** es $G' = (V', E')$, con $V' = V \cup \{s\}$, $E' = E \cup \{(s, v) : v \in V\}$ y $w(s, v) = 0$. A s no entra ninguna arista, así que ningún ciclo pasa por s: G' tiene un ciclo negativo si y solo si G lo tiene, y ningún camino más corto entre vértices de G pasa por s.

Proposición 9.19 (El potencial de Johnson)**.** *Si G' no tiene ciclos negativos y $h(v) = \delta(s, v)$ para todo $v \in V'$, entonces $\hat{w}(u, v) \geq 0$ para toda $(u, v) \in E'$.*

Demostración. Todo v es alcanzable desde s por la arista (s, v), así que $h(v)$ es finito, y de hecho $h(v) \leq 0$. Por la desigualdad del triángulo (proposición 7.8), $\delta(s, v) \leq \delta(s, u) + w(u, v)$ para toda arista (u, v), es decir, $h(v) \leq h(u) + w(u, v)$, que reescrita es $\hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0$. Las aristas de s quedan con $\hat{w}(s, v) = 0 + 0 - h(v) \geq 0$. □

La prueba dice algo más: $\hat{w}(u, v) = 0$ exactamente cuando la desigualdad del triángulo es una igualdad, es decir, cuando la arista está sobre algún camino más corto desde s. Y no hace falta que h sea $\delta(s, \cdot)$: sirve cualquier h con $h(v) - h(u) \leq w(u, v)$ en cada arista, que es una solución de un sistema de restricciones de diferencia (Cormen, sección 24.4), y ese sistema se resuelve con la misma construcción del vértice nuevo.

Algoritmo 9.7 Johnson(G, w)

```
1:  procedimiento JOHNSON(G, w)
2:      V' ← V ∪ {s},    E' ← E ∪ {(s, v) : v ∈ V}          // s es un vértice nuevo
3:      w(s, v) ← 0 para todo v ∈ V
4:      si BELLMAN-FORD(G', w, s) devuelve falso entonces
5:          devolver "G tiene un ciclo de peso negativo"
6:      fin si
7:      h(v) ← d[v] para todo v ∈ V                          // d[v] = δ(s, v)
8:      ŵ(u, v) ← w(u, v) + h(u) − h(v) para toda (u, v) ∈ E   // ≥ 0
9:      para todo u ∈ V hacer
10:         DIJKSTRA(G, ŵ, u)                               // deja d[v] = δ̂(u, v)
11:         para todo v ∈ V hacer
12:             d_uv ← d[v] + h(v) − h(u)                   // si d[v] = ∞, queda ∞
13:         fin para
14:     fin para
15:     devolver D
16: fin procedimiento
```

Bellman-Ford y Dijkstra son los algoritmos 7.3 y 7.5; $d[v]$ es el arreglo de distancias que dejan, y d_{uv} la casilla de la matriz de salida.

- Línea 4: la única corrida de Bellman-Ford hace dos trabajos. Calcula h y detecta ciclos negativos en todo G, porque desde s se alcanza todo. Si se corriera desde un vértice de G, solo vería los ciclos alcanzables desde él y dejaría $h = \infty$ en los vértices que no alcanza (Cormen, ejercicio 25.3-6).
- Línea 7: se copia $d[v]$ en $h(v)$ porque cada Dijkstra de la línea 10 sobrescribe d. Olvidar esa copia es el error de implementación más común.

- Línea 10: Dijkstra corre sobre G, no sobre G'. Daría lo mismo, porque desde $u \in V$ no se llega a s, pero no hace falta.
- Línea 12: si $d[v] = \infty$, la casilla queda en ∞. Con enteros y un centinela grande, sumarle $h(v) - h(u)$ desborda o da un número finito sin sentido; hay que revisar antes.
- Los predecesores salen gratis, aunque Cormen no los calcula. Por el lema 9.17, el árbol que Dijkstra construye con $\hat{w}$ desde u es también árbol de caminos más cortos con w, y el renglón u de Π es el arreglo π de esa corrida, sin corrección.

Teorema 9.20 (Correctitud de Johnson)**.** *Si G tiene un ciclo negativo, Johnson lo reporta. Si no, devuelve D con $d_{uv} = \delta(u, v)$ para todo par.*

Demostración. Bellman-Ford devuelve falso si y solo si hay un ciclo negativo alcanzable desde la fuente (sección 7; Cormen, teorema 24.4). En G' todo es alcanzable desde s, así que devuelve falso si y solo si G' tiene un ciclo negativo, y eso pasa si y solo si G lo tiene. Si devuelve verdadero, deja $d[v] = \delta(s, v) = h(v)$; por la proposición 9.19, $\hat{w} \geq 0$, así que cada Dijkstra calcula $\hat{\delta}(u, \cdot)$ correctamente, y el corolario 9.18 lo convierte en $\delta(u, \cdot)$. □

Costo. Construir G' (líneas 2 y 3) cuesta $\Theta(n + m)$: copiar las listas y agregar n aristas. Bellman-Ford en G' (línea 4) hace n pasadas sobre $m + n$ aristas, $\mathcal{O}((n + 1)(m + n)) = \mathcal{O}(nm + n^2)$. Copiar h (línea 7) cuesta $\Theta(n)$, reponderar (línea 8) $\Theta(m)$, y deshacer la reponderación (líneas 11 a 13) $\Theta(n^2)$, una suma por casilla de D. Falta lo que domina, las n corridas de Dijkstra de las líneas 9 y 10, y eso depende de la cola de prioridad. Una corrida de Dijkstra hace n inserciones, n extracciones del mínimo y a lo más m decrementos de llave, uno por relajación exitosa. Con montículo de Fibonacci eso es $n \cdot \mathcal{O}(1) + n \cdot \mathcal{O}(\log n) + m \cdot \mathcal{O}(1) = \mathcal{O}(n \log n + m)$ por corrida y $\mathcal{O}(n^2 \log n + nm)$ las n corridas. Bellman-Ford queda absorbido: su nm por el de las corridas y su n^2 por el $n^2 \log n$. El total es

$$\mathcal{O}\Big(nm + n^2\Big) + \mathcal{O}\Big(n^2 \log n + nm\Big) + \Theta\Big(n^2\Big) = \mathcal{O}\Big(n^2 \log n + nm\Big).$$

Con el montículo binario de la libreta son $\mathcal{O}((n + m) \log n)$ por corrida y $\mathcal{O}(n(n + m) \log n)$ en total, que Cormen escribe $\mathcal{O}(nm \log n)$. Con un arreglo, $\mathcal{O}(n^3)$, y entonces no tiene caso: se iguala a Floyd-Warshall con más trabajo. En los tres casos el costo es el de n veces Dijkstra con pesos no negativos; admitir pesos negativos cuesta una corrida de Bellman-Ford, que no domina.

La memoria es $\Theta(n + m)$ de trabajo más los $\Theta(n^2)$ de la salida. A diferencia de Floyd-Warshall, Johnson produce D renglón por renglón: si solo se necesitan algunos renglones, nunca hace falta la matriz completa.

Ejemplo 9.21 (El contraejemplo de Dijkstra, resuelto)**.** La clase del 22 de octubre muestra que Dijkstra falla con las aristas $A \to B$ de peso 2, $A \to C$ de peso 3 y $C \to B$ de peso -2: desde A fija B con $d = 2$, pero A, C, B cuesta 1. Es el ejemplo natural para Johnson.

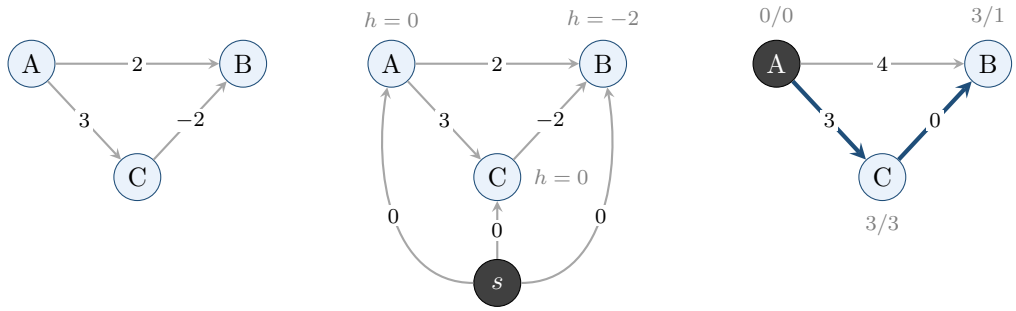
Bellman-Ford en G'. Con las aristas en el orden (s, A), (s, B), (s, C), (A, B), (A, C), (C, B), la primera pasada pone $d = 0$ en A, B y C por las aristas de s, y luego (C, B) baja B a $0 - 2 = -2$. Las pasadas 2 y 3 no cambian nada y la verificación devuelve verdadero: $h(A) = 0$, $h(B) = -2$ y $h(C) = 0$.

Reponderación. $\hat{w}(A, B) = 2 + 0 - (-2) = 4$, $\hat{w}(A, C) = 3 + 0 - 0 = 3$ y $\hat{w}(C, B) = -2 + 0 - (-2) = 0$.

Dijkstra desde A con $\hat{w}$, en el formato de la clase; en negritas, las d ya fijas.

Paso	**Sale**	A	B	C	**Relajaciones**
0		0	∞	∞	
1	A	**0**	4	3	B y C desde A
2	C	**0**	3	**3**	B: $3+0=3<4$
3	B	**0**	**3**	**3**	
$\delta(A,v)$		0	$3+(-2)-0=1$	$3+0-0=3$	

B empieza en 4 por la arista directa y baja a 3 al pasar por C, como debe. Desde C, $\hat{\delta}(C,B)=0$ da $\delta(C,B)=0+(-2)-0=-2$; desde B no se alcanza nada y los ∞ se quedan. Las tres instantáneas son el grafo original, el aumentado con h y el reponderado con el árbol de Dijkstra desde A, con $\hat{\delta}(A,v)/\delta(A,v)$ junto a cada vértice:

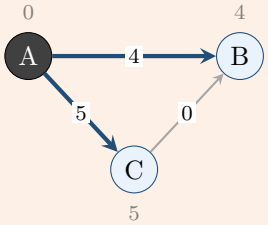


(a) G con w (b) G' y $h=\delta(s,\cdot)$ (c) $\hat{w}$, Dijkstra desde A

Floyd-Warshall, con $A=1$, $B=2$ y $C=3$, llega a lo mismo: d_{AB} baja de 2 a 1 en la vuelta $k=3$, la de C.

Ojito...

Sumar una constante no sirve (Cormen, ejercicio 25.3-4). La tentación es restarle a cada arista el peso mínimo, $w^*=-2$, para que todas queden no negativas: $A\to B$ pasa a 4, $A\to C$ a 5 y $C\to B$ a 0. Dijkstra desde A elige ahora la arista directa, 4 contra $5+0=5$, y responde que el mejor camino a B es $A\to B$, de peso original 2. Está mal. Sumarle $|w^*|$ a cada arista le suma $k\,|w^*|$ a un camino de k aristas, y castiga a los largos: A, C, B recibe $+4$ y la arista directa solo $+2$. La corrección de Johnson depende solo de los extremos, no de cuántas aristas tiene el camino.



$w(u,v)-w^*$, con $w^*=-2$: árbol equivocado

Ejemplo 9.22 (La figura 25.6 de Cormen, completa)**.** Es la corrida de Johnson sobre el grafo del ejemplo 9.3.

Paso 1: Bellman-Ford en G', con las cinco aristas (s,v) primero y luego las de E en orden lexicográfico. Converge en dos pasadas, aunque hace las $|V'|-1=5$:

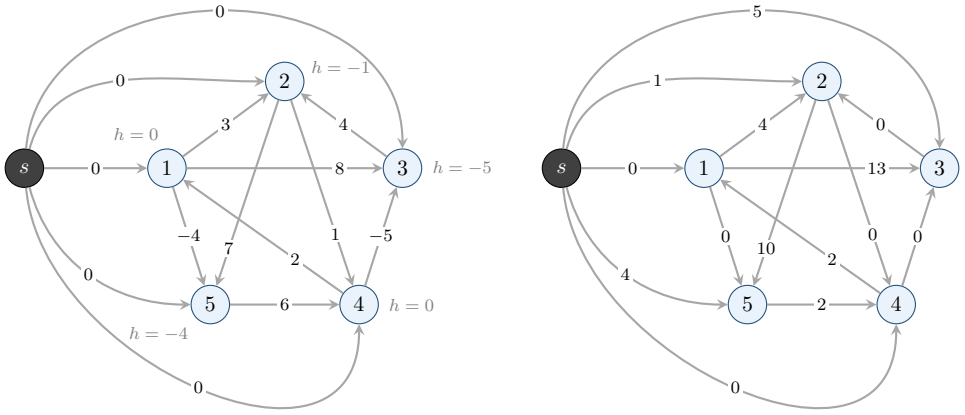
Pasada	s	1	2	3	4	5	**Cambios**
0	0	∞	∞	∞	∞	∞	
1	0	0	0	-5	0	-4	las cinco (s,v); $(1,5)$: -4; $(4,3)$: -5
2	0	0	-1	-5	0	-4	$(3,2)$: $-5+4=-1$
3 a 5	0	0	-1	-5	0	-4	ninguno
h	0	0	-1	-5	0	-4	verificación: verdadero

En la pasada 1, $(1,5)$ con $d[1]=0$ da $d[5]=-4$, y $(4,3)$ da $d[3]=-5$; en la pasada 2, $(3,2)$ aprovecha ese -5: $d[2]=-5+4=-1$. Todos los h son ≤ 0, como debe ser.

Paso 2: reponderar, con $\hat{w}(u,v)=w(u,v)+h(u)-h(v)$:

(u,v)	$(1,2)$	$(1,3)$	$(1,5)$	$(2,4)$	$(2,5)$	$(3,2)$	$(4,1)$	$(4,3)$	$(5,4)$
$w(u,v)$	3	8	-4	1	7	4	2	-5	6
$h(u)$	0	0	0	-1	-1	-5	0	0	-4
$h(v)$	-1	-5	-4	0	-4	-1	0	-5	0
$\hat{w}(u,v)$	4	13	0	0	10	0	2	0	2

Las aristas de s quedan con $\hat{w}(s,v)=-h(v)$, o sea $0,1,5,0,4$. Las de peso 0 son las que están sobre algún camino más corto desde s. Los dos grafos, (a) con w y h, y (b) con $\hat{w}$:



(a) G' con los pesos w (b) G' con $\hat{w}$

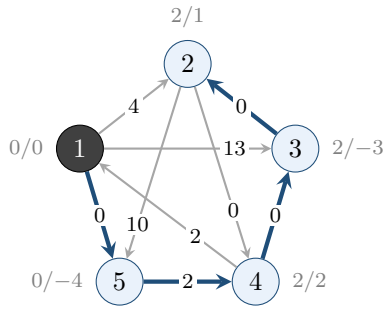
Paso 3: Dijkstra desde cada vértice con $\hat{w}$. Desde el vértice 1:

Paso	**Sale**	1	2	3	4	5	**Relajaciones**
0		0	∞	∞	∞	∞	
1	1	**0**	4	13	∞	0	2, 3, 5 desde 1
2	5	**0**	4	13	2	**0**	4: $0+2$
3	4	**0**	4	2	**2**	**0**	3: $2+0=2<13$
4	3	**0**	2	**2**	**2**	**0**	2: $2+0=2<4$
5	2	**0**	**2**	**2**	**2**	**0**	ninguna
$h(v)-h(1)$		0	-1	-5	0	-4	
$\delta(1,v)$		0	1	-3	2	-4	

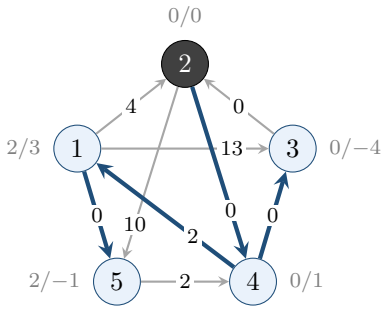
El vértice 2 empieza en 4 por la arista directa y baja a 2 por el camino $1,5,4,3,2$, que pesa $0+2+0+0=2$ con $\hat{w}$ y $-4+6-5+4=1$ con w. El último renglón aplica el corolario 9.18; por ejemplo, $\delta(1,3)=$

$2 + (-5) - 0 = -3$.

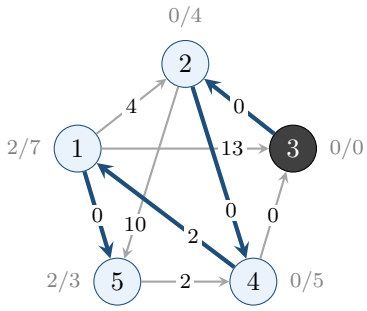
Las cinco corridas, con el árbol de Dijkstra en azul, la fuente en negro y $\hat{\delta}(u,v)/\delta(u,v)$ junto a cada vértice:



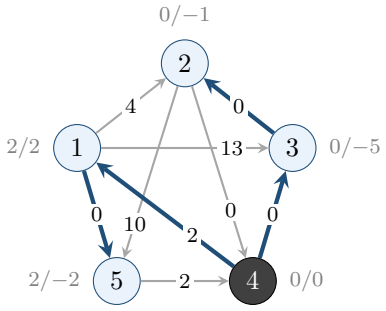
(c) fuente 1



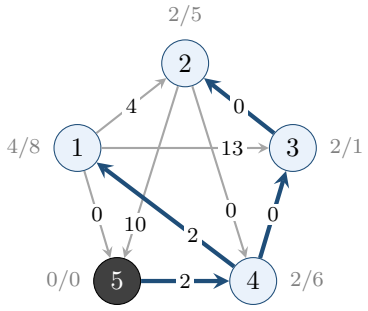
(d) fuente 2



(e) fuente 3



(f) fuente 4



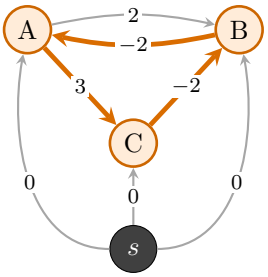
(g) fuente 5

Desde el vértice 5, Dijkstra extrae 4, 3 y 2 empatados en $\hat{\delta} = 2$, pero sus distancias reales son 6, 1 y 5: el orden de extracción no dice nada sobre w. Juntando los renglones,

$$D = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix},$$

que es la $L^{(4)}$ del ejemplo 9.7 y la $D^{(5)}$ del ejemplo 9.12.

Ejemplo 9.23 (Caso borde: Johnson ante un ciclo negativo)**.** Al grafo del ejemplo 9.21 se le agrega la arista $B \to A$ con peso -2. El ciclo A, C, B, A pesa $3 - 2 - 2 = -1$; el ciclo A, B, A pesa 0 y no estorba.



Bellman-Ford en G', con las aristas en el orden (s,A), (s,B), (s,C), (A,B), (A,C), (C,B), (B,A):

Pasada	s	A	B	C	**Cambios**
0	0	∞	∞	∞	
1	0	-4	-2	0	las tres (s, v); (C, B): -2; (B, A): -4
2	0	-5	-3	-1	(A, C), (C, B), (B, A)
3	0	-6	-4	-2	(A, C), (C, B), (B, A)
Verificación: (A, C) da $-6 + 3 = -3 < -2$, devuelve falso					

Cada pasada le da una vuelta más al ciclo, y los tres valores bajan 1, el peso del ciclo. No hay punto fijo: después de las $|V'| - 1 = 3$ pasadas todavía hay una arista que relaja, la verificación devuelve falso y Johnson se detiene en la línea 5 sin calcular ningún h. Si siguiera con estos d, la desigualdad del triángulo no valdría y saldrían pesos $\hat{w}$ negativos.

Ojito...

- **Deshacer la reponderación, y con el signo correcto.** Sin la corrección se devuelve $\hat{\delta}$: en el ejemplo 9.22, $\hat{\delta}(1, 3) = 2$ pero $\delta(1, 3) = -3$. Se suma $h(v) - h(u)$, no al revés.
- **Pesos reales.** Con punto flotante, $\hat{w}$ puede salir -10^{-16} por redondeo, y Dijkstra supone pesos no negativos. Conviene redondear a 0 los negativos diminutos o trabajar con enteros.
- **Un ciclo negativo en cualquier parte tumba todo.** Johnson rechaza la entrada completa aunque muchos pares no estén afectados. Marcar con $-\infty$ solo los afectados ya es otro algoritmo.
- **Grafos no dirigidos.** Una arista negativa son dos arcos que forman un ciclo negativo de dos aristas, y Johnson rechaza el grafo. Con pesos no negativos, $h \equiv 0$ y Johnson se reduce a n veces Dijkstra.
- **Empates.** $\hat{w}$ produce muchas aristas de peso 0, y Dijkstra empata seguido. Las distancias salen bien con cualquier desempate, pero los árboles pueden salir distintos a los de Cormen.

Qué algoritmo usar

La tabla junta todo lo de la sección. "Sí" en la columna de pesos negativos quiere decir "sí, siempre que no haya ciclos negativos", y todos los que admiten negativos también detectan esos ciclos: Floyd-Warshall y los cuadrados repetidos con la diagonal, Bellman-Ford y Johnson con la verificación final.

Algoritmo	**Pesos negativos**	**General**	**Disperso** $m = \Theta(n)$	**Denso** $m = \Theta(n^2)$
Floyd-Warshall	sí	$\Theta(n^3)$	$\Theta(n^3)$	$\Theta(n^3)$
Cuadrados repetidos	sí	$\Theta(n^3 \log n)$	$\Theta(n^3 \log n)$	$\Theta(n^3 \log n)$
n veces Dijkstra, arreglo	no	$\Theta(n^3)$	$\Theta(n^3)$	$\Theta(n^3)$
n veces Dijkstra, binario	no	$\mathcal{O}(n(n+m) \log n)$	$\mathcal{O}(n^2 \log n)$	$\mathcal{O}(n^3 \log n)$
n veces Dijkstra, Fibonacci	no	$\mathcal{O}(n^2 \log n + nm)$	$\mathcal{O}(n^2 \log n)$	$\mathcal{O}(n^3)$
n veces Bellman-Ford	sí	$\mathcal{O}(n^2 m)$	$\mathcal{O}(n^3)$	$\mathcal{O}(n^4)$
Johnson, binario	sí	$\mathcal{O}(n(n+m) \log n)$	$\mathcal{O}(n^2 \log n)$	$\mathcal{O}(n^3 \log n)$
Johnson, Fibonacci	sí	$\mathcal{O}(n^2 \log n + nm)$	$\mathcal{O}(n^2 \log n)$	$\mathcal{O}(n^3)$

Para dimensionar, con $n = 10^4$ y solo el término dominante de cada cota, sin constantes:

Cota	**Disperso**, $m = 5 \cdot 10^4$	**Denso**, $m \approx 10^8$
Floyd-Warshall, n^3	10^{12}	10^{12}
Dijkstra o Johnson binario, $n(n+m)\log n$	$8.0 \cdot 10^9$	$1.3 \cdot 10^{13}$
Dijkstra o Johnson Fibonacci, $n^2 \log n + nm$	$1.8 \cdot 10^9$	$1.0 \cdot 10^{12}$
n veces Bellman-Ford, $n^2 m$	$5.0 \cdot 10^{12}$	$1.0 \cdot 10^{16}$

Dónde está el cruce. Con montículo binario, Johnson le gana a Floyd-Warshall cuando $m = o(n^2/\log n)$, porque entonces $n(n+m)\log n = o(n^3)$. Con Fibonacci, cuando $m = o(n^2)$; en el caso denso empatan asintóticamente. En la práctica, en grafos densos gana Floyd-Warshall: tres ciclos anidados sobre una matriz contigua tienen constantes mucho menores que un montículo de Fibonacci, cuyas constantes son famosamente grandes.

Idea clave

Grafo denso o chico: Floyd-Warshall, por sencillez y constantes, con pesos de cualquier signo. Grafo grande y disperso con pesos no negativos: n veces Dijkstra. Grafo grande y disperso con pesos negativos: Johnson, que cuesta lo mismo que n veces Dijkstra más una corrida de Bellman-Ford. Si solo importa si hay camino: cerradura transitiva con bits, o un BFS por vértice si el grafo es disperso.

Observación. En grafos con $m = \Theta(n^{1+\varepsilon})$ y $0 < \varepsilon \leq 1$ fijo, se puede igualar a Fibonacci con una estructura sencilla (Cormen, problema 25-2). Un montículo d-ario con $d = n^\varepsilon$ tiene altura $\log_d n = 1/\varepsilon = \mathcal{O}(1)$: insertar y decrementar cuestan $\mathcal{O}(1)$ sin amortizar, y extraer, que compara con d hijos por nivel, cuesta $\mathcal{O}(d \log_d n) = \mathcal{O}(n^\varepsilon)$. Las n extracciones de Dijkstra suman $\mathcal{O}(n^{1+\varepsilon}) = \mathcal{O}(m)$ y los m decrementos, $\mathcal{O}(m)$: Dijkstra queda en $\mathcal{O}(m)$, y Johnson en $\mathcal{O}(nm)$. Es el mismo truco que motiva a Fibonacci: abaratar Decrementar-Llave, que es lo que Dijkstra más repite.

10. Flujo máximo: Ford-Fulkerson y Edmonds-Karp *Complemento: Cormen, cap. 26.1 y 26.2*

Resumen rápido. Una red de flujo es un grafo dirigido con una capacidad en cada arista, una fuente s y un sumidero t; el problema es mandar de s a t tanto material como se pueda sin rebasar ninguna capacidad. La sección construye el método de Ford-Fulkerson: empezar con flujo cero y aumentar mientras exista un camino de s a t en la red residual, el grafo que registra cuánto cabe todavía en cada arista y cuánto se puede regresar. Regresar flujo es lo que separa al método de un voraz, que en la red de ejemplo se atora en 19 cuando el máximo es 23. El teorema de flujo máximo y corte mínimo prueba que, cuando ya no hay camino, el flujo es óptimo, y entrega un corte que lo certifica. Con capacidades enteras Ford-Fulkerson cuesta $\mathcal{O}(m\,|f^*|)$, y una mala elección de caminos alcanza esa cota: dos millones de aumentos donde bastan dos. Edmonds-Karp escoge siempre el camino con menos aristas, con el BFS de la clase del 22 de octubre, y baja el costo a $\mathcal{O}(nm^2)$ sin importar las capacidades.

Redes de flujo

Una compañía fabrica discos de hockey en Vancouver y los guarda en una bodega en Winnipeg. Renta espacio en camiones que cubren rutas entre ciudades, y cada ruta admite a lo más cierto número de cajas por día. La pregunta es cuántas cajas diarias pueden llegar a Winnipeg. No importa por dónde viajen ni cuánto cueste: importa la tasa. Ese es el problema de flujo máximo, y el mismo modelo sirve para líquido en tuberías, corriente en una red eléctrica o paquetes en una red de datos. El ejemplo es de Cormen, y su red se usa en toda la sección.

Definición 10.1 (Red de flujo)**.** Una **red de flujo** (*flow network*) es un grafo dirigido $G = (V, E)$ en el que cada arista (u, v) tiene una **capacidad** $c(u, v) \geq 0$, junto con dos vértices distinguidos: la **fuente** s y el **sumidero** t. Por comodidad, $c(u, v) = 0$ si $(u, v) \notin E$. Se pide además que no haya lazos, que no haya **aristas antiparalelas** (si $(u, v) \in E$, entonces $(v, u) \notin E$) y que todo vértice esté en algún camino de s a t.

La última condición tiene una consecuencia que se usa en todos los análisis: todo vértice salvo s tiene al menos una arista de entrada, así que $m \geq n - 1$ y $\mathcal{O}(n + m) = \mathcal{O}(m)$.

Definición 10.2 (Flujo)**.** Un **flujo** en G es una función $f : V \times V \to \mathbb{R}$ que cumple dos propiedades:

- **Restricción de capacidad**: $0 \leq f(u, v) \leq c(u, v)$ para todos $u, v \in V$.
- **Conservación**: para todo $u \in V - \{s, t\}$,

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v).$$

El **valor** del flujo es lo que sale de la fuente menos lo que entra a ella:

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s).$$

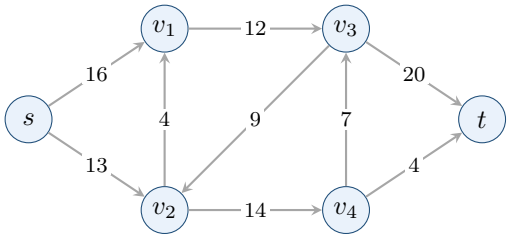
El **problema de flujo máximo** pide, dados G, s y t, un flujo de valor máximo. A un flujo así se le llama f^*.

La conservación dice que lo que entra a un vértice intermedio sale de él; cuando el material es corriente eléctrica, es la ley de corrientes de Kirchhoff. La restricción de capacidad obliga $f(u, v) = 0$ cuando $(u, v) \notin E$, porque ahí $c(u, v) = 0$.

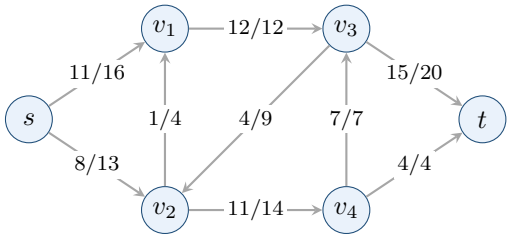
Ojito...

Las barras de $|f|$ no son valor absoluto ni cardinalidad: denotan el valor del flujo. Y el término que resta lo que entra a s no sobra. En una red de flujo normalmente no entra nada a la fuente, pero la definición se aplica también a flujos en redes residuales, que sí tienen aristas hacia s.

Ejemplo 10.3 (La red de Cormen)**.** La red tiene $V = \{s, v_1, v_2, v_3, v_4, t\}$ y nueve aristas: $n = 6$ y $m = 9$. Las ciudades son Vancouver (s), Edmonton (v_1), Calgary (v_2), Saskatoon (v_3), Regina (v_4) y Winnipeg (t). A la izquierda van las capacidades; a la derecha, un flujo, con cada arista etiquetada $f(u, v)/c(u, v)$. La diagonal separa los dos números, no los divide.



capacidades $c(u, v)$



flujo $f(u, v)/c(u, v)$, $|f| = 19$

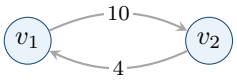
El flujo respeta todas las capacidades. A v_2 le entran $f(s, v_2) + f(v_3, v_2) = 8 + 4 = 12$ unidades y le salen $f(v_2, v_1) + f(v_2, v_4) = 1 + 11 = 12$; en v_1, v_3 y v_4 entran y salen 12, 19 y 11, respectivamente. El valor es $|f| = f(s, v_1) + f(s, v_2) = 11 + 8 = 19$. El máximo de esta red, se verá, es 23.

Observación. Cormen escribe el flujo de una arista como atributo, $(u, v).f$, y dentro de la $\mathcal{O}$ abrevia $|V|$ y $|E|$ como V y E: su $\mathcal{O}(VE^2)$ es el $\mathcal{O}(nm^2)$ de esta sección.

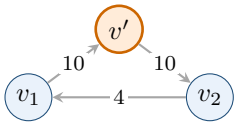
Aristas antiparalelas

La definición prohíbe los pares (u, v), (v, u), y no por capricho. La red residual, que viene enseguida, le asigna a cada par ordenado una sola capacidad residual, y con un par antiparalelo no sabría si (u, v) es una arista original o el regreso de (v, u). Pero los pares aparecen solos en cualquier modelo: si la transportista ofrece además 10 cajas diarias de Edmonton a Calgary, la red tendría (v_1, v_2) junto a (v_2, v_1).

La salida es partir una de las dos con un vértice nuevo v': se reemplaza (v_1, v_2) por (v_1, v') y (v', v_2), las dos con la capacidad original. Todo lo que pasaba por la arista vieja pasa ahora por el camino de dos aristas, y al revés, así que las dos redes admiten los mismos valores de flujo (Cormen, ejercicio 26.1-1). Cuesta un vértice y una arista por cada par.



par antiparalelo: prohibido



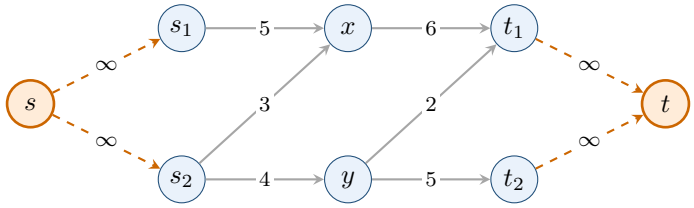
se parte (v_1, v_2) con v'

Con la red de Cormen más la arista (v_1, v_2) de capacidad 10, la versión con el par y la versión partida tienen el mismo flujo máximo, 23, que además es el de la red sin la arista extra: más abajo se verá por qué esa arista no ayuda.

Varias fuentes y varios sumideros

Si hay fuentes $s_1, \ldots, s_k$ y sumideros $t_1, \ldots, t_\ell$, se agrega una **superfuente** s con aristas (s, s_i) de capacidad ∞ y un **supersumidero** t con aristas (t_j, t) de capacidad ∞. La superfuente le da a cada s_i todo lo que pida y el supersumidero recibe todo lo que llegue, así que los flujos de la red original corresponden uno a uno con los de la red nueva, con el mismo valor (Cormen, ejercicio 26.1-2). Cormen cuenta las fuentes y los sumideros con m y n; aquí esas letras ya son $|E|$ y $|V|$, y por eso se usan k y ℓ.

En la figura, lo que se agrega va en naranja. La red original tiene dos fuentes, dos sumideros y dos vértices intermedios, x y y.



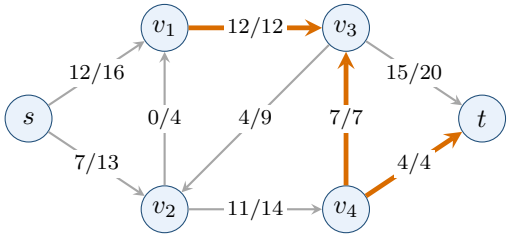
En una implementación, ∞ se reemplaza por un número grande, por ejemplo la suma de todas las capacidades originales. Como esas son finitas, el flujo máximo también lo es (Cormen, ejercicio 26.2-5).

La red residual

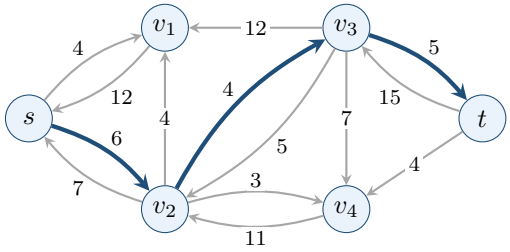
Por qué no basta un voraz

La idea más directa es la de la clase del 8 de octubre: mientras haya un camino de s a t cuyas aristas tengan holgura, $f(u, v) < c(u, v)$, mandar por él todo lo que permita su arista más angosta. Toma lo mejor que ve en el momento y nunca revisa una decisión ya tomada. En esta red eso no alcanza.

Ejemplo 10.4 (El voraz se atora en 19)**.** El voraz empieza con el camino s, v_1, v_3, v_2, v_4, t, que admite $\min(16, 12, 9, 14, 4) = 4$ unidades. Después busca con BFS entre las aristas con holgura y encuentra s, v_1, v_3, t, con 8, y s, v_2, v_4, v_3, t, con 7. Lleva 19. A la izquierda está ese flujo: (v_1, v_3), (v_4, v_3) y (v_4, t), en naranja, quedaron saturadas, y desde s solo se llega a v_1, v_2 y v_4 por aristas con holgura. No hay más camino y el voraz se detiene.



flujo del voraz, $|f| = 19$



su red residual G_f

Pero 19 no es el máximo. Estorban las 4 unidades que el primer camino mandó de v_3 a v_2: ocupan capacidad de (v_1, v_3) que hace falta para llegar a t por arriba. A la derecha está la red residual de ese flujo, que se define enseguida. En ella sí hay camino, s, v_2, v_3, t, y su arista de en medio, (v_2, v_3), no existe en G: es el regreso de las 4 unidades de (v_3, v_2). Mandar 4 por ese camino deja $f(s, v_2) = 11$, $f(v_3, v_2) = 0$ y $f(v_3, t) = 19$, de valor 23. Dicho en cajas: v_2 recibe ahora de s lo que antes le llegaba de v_3, y v_3 manda esas 4 unidades directo a t.

El voraz no siempre falla: si empieza por s, v_1, v_3, t, llega a 23. Falla cuando una decisión temprana le estorba a una posterior y no tiene forma de deshacerla. La red residual es justo esa forma.

Capacidad residual

Definición 10.5 (Red residual)**.** Sea f un flujo en G. La **capacidad residual** (*residual capacity*) de un par $u, v \in V$ es

$$c_f(u,v) = \begin{cases} c(u,v) - f(u,v) & \text{si } (u,v) \in E, \\ f(v,u) & \text{si } (v,u) \in E, \\ 0 & \text{en otro caso.} \end{cases}$$

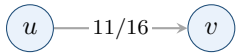
La **red residual** de G inducida por f es $G_f = (V, E_f)$, con

$$E_f = \{(u,v) \in V \times V : c_f(u,v) > 0\}.$$

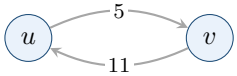
Sus aristas se llaman **aristas residuales**.

El primer renglón es el espacio que queda en la arista. El segundo es el **regreso de flujo**: si de v a u van $f(v,u)$ unidades, se puede cancelar cualquier parte de ellas, y visto desde el resultado eso equivale a mandar flujo de u a v. Como no hay aristas antiparalelas, a cada par ordenado le toca exactamente un renglón. Cada arista residual es una arista de E o la inversa de una, así que $|E_f| \leq 2m$. G_f no es una red de flujo, porque puede tener pares antiparalelos, pero un flujo en G_f se define igual que en una red, con las capacidades c_f.

Ejemplo 10.6 (Una arista y sus dos residuales)**.** Si $c(u,v) = 16$ y $f(u,v) = 11$, caben 5 unidades más, $c_f(u,v) = 5$, y se pueden regresar hasta 11, $c_f(v,u) = 11$.



en G: $f(u,v)/c(u,v)$



en G_f: 5 de ida, 11 de regreso

Una arista saturada solo deja la residual de regreso, y una vacía solo la de ida. En las figuras de esta sección las aristas residuales de capacidad 0 no se dibujan.

Aumento

Definición 10.7 (Aumento)**.** Si f es un flujo en G y f' uno en G_f, el **aumento** de f por f' es la función $f \uparrow f'$ dada por

$$(f \uparrow f')(u,v) = \begin{cases} f(u,v) + f'(u,v) - f'(v,u) & \text{si } (u,v) \in E, \\ 0 & \text{en otro caso.} \end{cases}$$

Se suma lo que f' manda por (u,v) y se resta lo que manda por la arista de regreso, (v,u). Empujar por la de regreso se llama **cancelación**: mandar 5 cajas de u a v y 2 de v a u es, visto desde el resultado, mandar 3 de u a v.

Lema 10.8 (Cormen, lema 26.1)**.** *Si f es un flujo en G y f' es un flujo en G_f, entonces $f \uparrow f'$ es un flujo en G y $|f \uparrow f'| = |f| + |f'|$.*

Demostración. Sea $h = f \uparrow f'$. Primero la capacidad. Fuera de E, h vale 0. Si $(u,v) \in E$, como $f'(v,u) \leq c_f(v,u) = f(u,v)$,

$$h(u,v) \geq f(u,v) + f'(u,v) - f(u,v) = f'(u,v) \geq 0,$$

y como $f'(v,u) \geq 0$ y $f'(u,v) \leq c_f(u,v) = c(u,v) - f(u,v)$,

$$h(u,v) \leq f(u,v) + f'(u,v) \leq c(u,v).$$

Para la conservación y el valor conviene mirar el flujo en neto. Para cualquier función g en $V \times V$ sea $\bar{g}(u,v) = g(u,v) - g(v,u)$, lo que g manda de u a v descontando lo que manda de v a u. La conservación en u dice $\sum_v \bar{g}(u,v) = 0$, y el valor es $|g| = \sum_v \bar{g}(s,v)$. Basta probar que para todo par

$$\bar{h}(u,v) = \bar{f}(u,v) + \bar{f}'(u,v), \qquad (*)$$

porque al sumar sobre v la conservación de f y la de f' dan la de h, y en $u = s$ queda $|h| = |f| + |f'|$. Hay tres casos. Si $(u,v) \in E$, entonces $(v,u) \notin E$, así que $f(v,u) = 0$ y $h(v,u) = 0$, y $\bar{h}(u,v) = f(u,v) + f'(u,v) - f'(v,u) = \bar{f}(u,v) + \bar{f}'(u,v)$. Si $(v,u) \in E$, es el caso anterior con los papeles cambiados, y los dos lados de $(*)$ solo cambian de signo. Si ninguna de las dos está en E, entonces f, h y f' valen 0 en los dos sentidos (f' porque ahí $c_f = 0$), y los dos lados son 0. □

Cormen prueba la parte del valor separando los vecinos de s en los de salida y los de entrada, y deja un paso de las sumas como ejercicio (26.2-1). El flujo neto $\bar{g}$ resuelve la conservación y el valor de un golpe.

Caminos aumentantes

Definición 10.9 (Camino aumentante)**.** Un **camino aumentante** (*augmenting path*) es un camino p de s a t en la red residual G_f. Su **capacidad residual** es

$$c_f(p) = \min\{c_f(u,v) : (u,v) \text{ está en } p\},$$

lo más que se puede mandar por todas sus aristas a la vez. El flujo f_p en G_f vale $c_f(p)$ en las aristas de p y 0 en las demás.

Lema 10.10 (Cormen, lema 26.2 y corolario 26.3)**.** *Si p es un camino aumentante, f_p es un flujo en G_f con $|f_p| = c_f(p) > 0$, y $f \uparrow f_p$ es un flujo en G con $|f \uparrow f_p| = |f| + c_f(p) > |f|$.*

Demostración. En cada arista de p, f_p vale $c_f(p)$, que no pasa de $c_f(u,v)$ porque es el mínimo; fuera de p vale 0. Como p no repite vértices, a cada vértice intermedio de p le entra exactamente una arista de p y le sale exactamente una, las dos con $c_f(p)$; los vértices fuera de p no tocan ninguna. De s sale una arista de p y no entra ninguna, así que $|f_p| = c_f(p)$, que es positivo porque toda arista residual tiene capacidad positiva. La segunda parte es el lema 10.8 con $f' = f_p$. □

Cormen deja la primera parte como ejercicio (26.2-7). Con este lema el método ya está completo: mientras haya un camino aumentante, aumentar por él, y cada vuelta sube el valor. Falta saber que cuando no queda ningún camino el flujo es máximo, y para eso hacen falta los cortes.

Cortes y el teorema de flujo máximo y corte mínimo

Definición 10.11 (Corte)**.** Un **corte** (*cut*) (S,T) de la red es una partición de V con $s \in S$ y $t \in T = V - S$. El **flujo neto** que lo cruza es

$$f(S,T) = \sum_{u \in S} \sum_{v \in T} f(u,v) - \sum_{u \in S} \sum_{v \in T} f(v,u),$$

y su **capacidad** es

$$c(S,T) = \sum_{u\in S}\sum_{v\in T} c(u,v).$$

Un **corte mínimo** es uno de capacidad mínima entre todos los cortes de la red.

La asimetría es deliberada. El flujo neto cuenta lo que va de S a T y resta lo que regresa de T a S; la capacidad solo cuenta las aristas de S a T, porque una arista de regreso no le agrega espacio al cruce: lo mejor que puede hacer es no llevar nada. Es la misma idea de partir los vértices en dos lados que sostiene a Kruskal (sección 6), con tres diferencias: aquí el grafo es dirigido, s y t quedan en lados opuestos, y la capacidad mide un solo sentido.

Ejemplo 10.12 (Un corte de la red de Cormen)**.** Con el flujo de valor 19 del ejemplo 10.3, tómese $S = \{s, v_1, v_2\}$ y $T = \{v_3, v_4, t\}$ (es el de la izquierda en la figura del ejemplo 10.18). De S a T cruzan (v_1, v_3) y (v_2, v_4); de T a S, (v_3, v_2). Entonces

$$f(S,T) = 12 + 11 - 4 = 19, \qquad c(S,T) = 12 + 14 = 26.$$

La capacidad de (v_3, v_2), 9, no se suma a $c(S,T)$, pero su flujo, 4, sí se resta en $f(S,T)$. Confundir las dos cuentas rompe la prueba del corolario 10.14.

Que el flujo neto haya salido igual a $|f|$ no es casualidad. Todo lo que sale de s tiene que cruzar el corte para llegar a t, y lo que cruza de regreso se descuenta.

Lema 10.13 (Cormen, lema 26.4)**.** *Para todo flujo f y todo corte (S,T), $f(S,T) = |f|$.*

Demostración. Para cada $u \in S - \{s\}$, la conservación dice $\sum_v f(u,v) - \sum_v f(v,u) = 0$ ($u \neq t$ porque $t \in T$). Sumar todos esos ceros a la definición de $|f|$ no la cambia:

$$|f| = \sum_{u\in S}\Big(\sum_{v\in V} f(u,v) - \sum_{v\in V} f(v,u)\Big).$$

Se parte cada suma interior en $v \in S$ y $v \in T$:

$$|f| = \sum_{u\in S}\sum_{v\in S} f(u,v) - \sum_{u\in S}\sum_{v\in S} f(v,u) + \sum_{u\in S}\sum_{v\in T} f(u,v) - \sum_{u\in S}\sum_{v\in T} f(v,u).$$

Las dos primeras sumas tienen exactamente los mismos términos, $f(x,y)$ con $x, y \in S$, y se cancelan. Las dos últimas son $f(S,T)$. □

En la red de Cormen hay $2^4 = 16$ cortes, uno por cada subconjunto de $\{v_1, v_2, v_3, v_4\}$ que acompaña a s, y con el flujo de valor 19 los 16 dan $f(S,T) = 19$.

Corolario 10.14 (Cormen, corolario 26.5)**.** *Para todo flujo f y todo corte (S,T), $|f| \leq c(S,T)$.*

Demostración.

$$|f| = f(S,T) = \sum_{u\in S}\sum_{v\in T} f(u,v) - \sum_{u\in S}\sum_{v\in T} f(v,u) \leq \sum_{u\in S}\sum_{v\in T} f(u,v) \leq \sum_{u\in S}\sum_{v\in T} c(u,v) = c(S,T).$$

La primera desigualdad tira términos no negativos que se restaban; la segunda es la restricción de capacidad. □

Cualquier corte es una cota superior para cualquier flujo. En la red de Cormen, el corte del ejemplo 10.12 dice que ningún flujo pasa de 26, y el corte $(\{s\}, V - \{s\})$ dice que ninguno pasa de $16 + 13 = 29$. La mejor de esas cotas la da un corte mínimo, y el teorema central dice que esa cota se alcanza.

Teorema 10.15 (Flujo máximo y corte mínimo; Cormen, teorema 26.6)**.** *Si f es un flujo en G, son equivalentes:*

1. *f es un flujo máximo en G.*
2. *La red residual G_f no tiene caminos aumentantes.*
3. *$|f| = c(S,T)$ para algún corte (S,T) de G.*

Demostración. (1) $\Rightarrow$ (2). Si hubiera un camino aumentante p, por el lema 10.10 el flujo $f \uparrow f_p$ tendría valor mayor que $|f|$, y f no sería máximo.

(2) $\Rightarrow$ (3). Sea

$$S = \{v \in V : \text{hay un camino de } s \text{ a } v \text{ en } G_f\}$$

y $T = V - S$. Es un corte: $s \in S$, y $t \notin S$ porque en G_f no hay camino de s a t. Sean $u \in S$ y $v \in T$. Si $(u,v) \in E$, entonces $f(u,v) = c(u,v)$: de lo contrario $c_f(u,v) > 0$, y v sería alcanzable desde s pasando por u. Si $(v,u) \in E$, entonces $f(v,u) = 0$: de lo contrario $c_f(u,v) = f(v,u) > 0$, y otra vez v estaría en S. Si ninguna de las dos está en E, las dos valen 0 en f y en c. Así

$$f(S,T) = \sum_{u \in S} \sum_{v \in T} c(u,v) - \sum_{u \in S} \sum_{v \in T} 0 = c(S,T),$$

y por el lema 10.13, $|f| = f(S,T) = c(S,T)$.

(3) $\Rightarrow$ (1). Por el corolario 10.14, $|f'| \leq c(S,T)$ para todo flujo f'. Si f alcanza esa cota, ningún flujo lo supera. □

Del teorema salen dos cosas. La primera, que el valor de un flujo máximo es igual a la capacidad de un corte mínimo. La segunda, que ese corte se obtiene con una búsqueda desde s sobre la red residual final, en $\mathcal{O}(n+m) = \mathcal{O}(m)$. El corte es un **certificado**: cualquiera puede comprobar que un flujo es máximo sumando unas cuantas capacidades, sin volver a correr nada.

Idea clave

Las dos mitades del teorema hacen trabajos distintos. El corolario dice que ningún flujo pasa de ningún corte: los cortes son cotas superiores. La construcción de (2) $\Rightarrow$ (3) dice que, cuando ya no hay camino aumentante, lo alcanzable en G_f forma un corte que el flujo llena exactamente: aristas de ida saturadas, aristas de vuelta vacías. Flujo y corte se encuentran, y ninguno de los dos se puede mejorar.

El método de Ford-Fulkerson

El método junta todo lo anterior: empezar con flujo cero y, mientras la red residual tenga un camino aumentante, aumentar por él. Cormen lo llama *método* y no algoritmo porque no dice cómo se encuentra el camino. Cada manera de buscarlo es una implementación distinta, con su propio costo.

Algoritmo 10.1 Ford-Fulkerson(G, s, t)

```
1:  procedimiento Ford-Fulkerson(G, s, t)
2:      para todo (u, v) ∈ E hacer
3:          f(u, v) ← 0
4:      fin para
5:      mientras existe un camino p de s a t en la red residual G_f hacer
6:          c_f(p) ← mín {c_f(u, v) : (u, v) está en p}
7:          para todo (u, v) en p hacer
8:              si (u, v) ∈ E entonces
9:                  f(u, v) ← f(u, v) + c_f(p)          // empuja por una arista de G
10:             si no
11:                 f(v, u) ← f(v, u) − c_f(p)          // cancela flujo de (v, u)
12:             fin si
13:         fin para
14:     fin mientras
15:     devolver f
16: fin procedimiento
```

Las líneas que no son obvias:

- **Línea 5.** "Existe un camino" es cualquier búsqueda desde s sobre G_f, BFS o DFS. Si llega a t devuelve el camino; si no, el ciclo termina, y por el teorema 10.15 el flujo es máximo.
- **Línea 6.** El cuello de botella del camino.
- **Líneas 8 a 12.** Cada arista residual de p es una arista de G o la inversa de una, nunca las dos cosas, porque G no tiene antiparalelas. En el primer caso se empuja flujo; en el segundo se cancela flujo que iba en sentido contrario. Es el aumento $f \uparrow f_p$ escrito arista por arista.

Estructura de datos. G_f nunca se construye. Se guarda $G' = (V, E')$, con

$$E' = \{(u, v) : (u, v) \in E \text{ o } (v, u) \in E\},$$

así que la lista de cada vértice trae a sus vecinos de salida y también a los de entrada. Cada entrada sabe si va hacia adelante o de regreso y apunta a la arista original, que guarda c y f. Así $c_f(u, v)$ cuesta $\mathcal{O}(1)$: $c - f$ hacia adelante, f de regreso. Las aristas de G_f son las de G' con $c_f > 0$, y se filtran durante la búsqueda. Sin antiparalelas, $|E'| = 2m$.

Costo, término por término.

- Líneas 2 a 4: $\Theta(m)$.
- Línea 5: una búsqueda sobre G' cuesta $\mathcal{O}(n + 2m)$, y como $m \geq n - 1$, eso es $\mathcal{O}(m)$.
- Líneas 6 a 13: p no repite vértices, así que tiene a lo más $n - 1$ aristas: $\mathcal{O}(n) \subseteq \mathcal{O}(m)$.
- Número de iteraciones. Si las capacidades son enteras, por inducción f y todas las c_f son enteras: empiezan en 0 y cada actualización suma o resta el entero $c_f(p)$. Entonces $c_f(p) \geq 1$ y, por el lema 10.10, $|f|$ sube al menos 1 en cada iteración. Como nunca pasa de $|f^*|$, hay a lo más $|f^*|$ iteraciones.

Resultado

Con capacidades enteras, Ford-Fulkerson hace a lo más $|f^*|$ aumentos y cuesta

$$\mathcal{O}(m) + |f^*| \cdot \mathcal{O}(m) = \mathcal{O}(m\,|f^*|)\,.$$

El costo depende del valor del flujo y no solo del tamaño de la red: es pseudopolinomial, porque $|f^*|$ puede ser exponencial en el número de bits de la entrada.

Corolario 10.16 (Integralidad)**.** *Si todas las capacidades son enteras, Ford-Fulkerson devuelve un flujo máximo con $f(u, v)$ entero en cada arista.*

Demostración. Es la misma inducción del costo: f empieza en 0 y cada actualización suma o resta un entero. Al terminar, el teorema 10.15 dice que el flujo es máximo. □

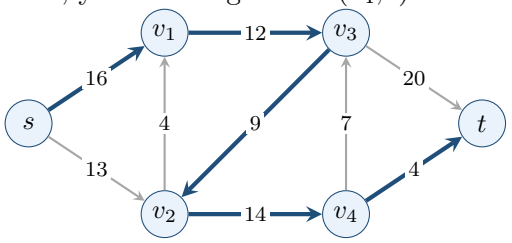
Cormen lo enuncia hasta la sección 26.3 (teorema 26.10), y es lo que hace funcionar el emparejamiento bipartito de la sección 11. Con capacidades racionales todo se multiplica por el común denominador y el argumento sigue igual. Con irracionales el método puede no terminar, y Cormen advierte que el valor puede ni siquiera converger al máximo.

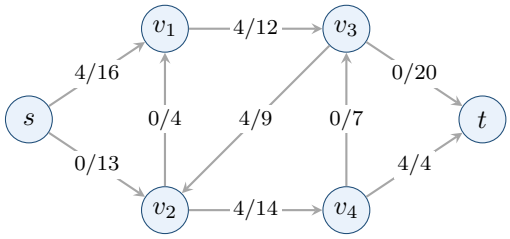
Ejemplo 10.17 (La ejecución de la figura 26.6 de Cormen)**.** Ford-Fulkerson sobre la red de Cormen, desde flujo cero, con los caminos que escoge Cormen. Primero el resumen; en la última columna, el valor después de cada aumento.

	Camino aumentante p	$c_f(p)$	**Arista crítica**	$\lvert f\rvert$
(a)	s, v_1, v_3, v_2, v_4, t	$\min(16, 12, 9, 14, 4) = 4$	(v_4, t)	4
(b)	s, v_2, v_1, v_3, t	$\min(13, 4, 8, 20) = 4$	(v_2, v_1)	8
(c)	s, v_1, v_2, v_3, t	$\min(12, 4, 4, 16) = 4$	(v_1, v_2) y (v_2, v_3)	12
(d)	s, v_2, v_4, v_3, t	$\min(9, 10, 7, 12) = 7$	(v_4, v_3)	19
(e)	s, v_1, v_3, t	$\min(8, 4, 5) = 4$	(v_1, v_3)	23
(f)	no hay camino			23

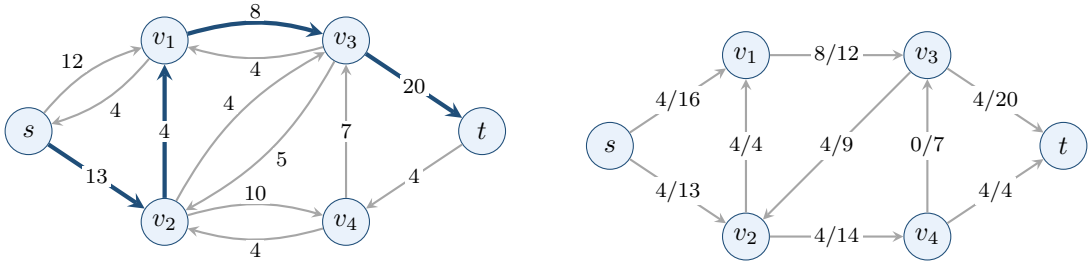
En cada renglón de figuras, a la izquierda está G_f con el camino aumentante en azul y a la derecha el flujo que resulta del aumento.

(a) El flujo es cero, así que $G_f = G$ y cada arista residual lleva la capacidad completa. El camino tiene cinco aristas, y la más angosta es (v_4, t).

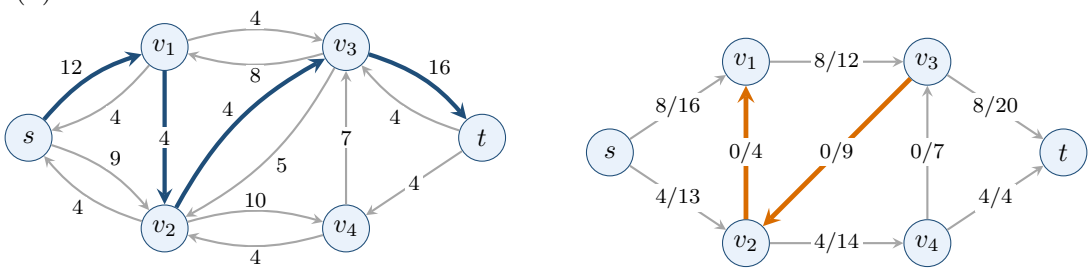




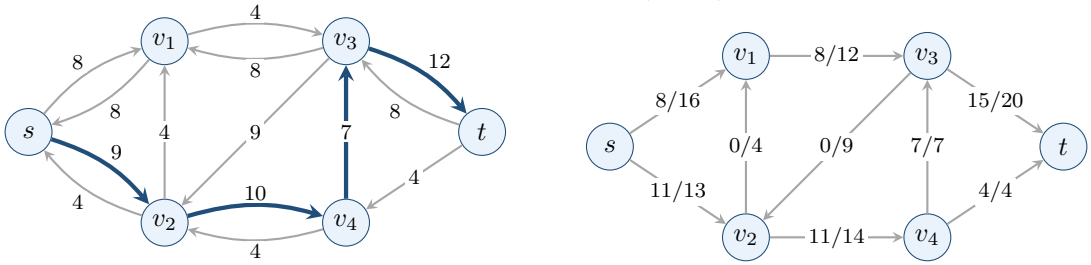
(b) Ya aparecen aristas de regreso, una por cada arista con flujo: (v_1, s), (v_3, v_1), (v_2, v_3), (v_4, v_2) y (t, v_4). El camino satura (v_2, v_1).



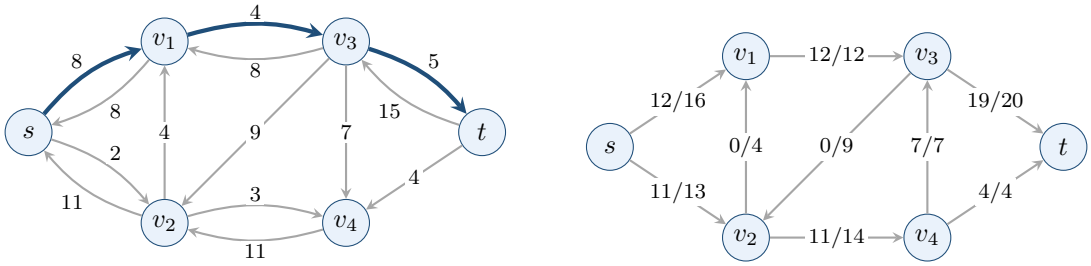
(c) El camino usa dos aristas de regreso: (v_1, v_2), inversa de (v_2, v_1), y (v_2, v_3), inversa de (v_3, v_2). El aumento cancela las 4 unidades de cada una, y $f(v_2, v_1)$ y $f(v_3, v_2)$ vuelven a 0 (en naranja). El flujo de valor 12 se lee ahora como 8 unidades por s, v_1, v_3, t y 4 por s, v_2, v_4, t: el método deshizo el rodeo de (a) y (b).



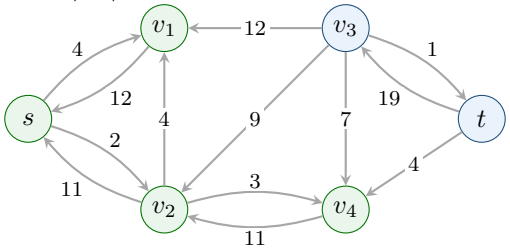
(d) Un camino de cuatro aristas, sin regresos, que satura (v_4, v_3).



(e) El último aumento satura (v_1, v_3). El flujo de la derecha vale 23.



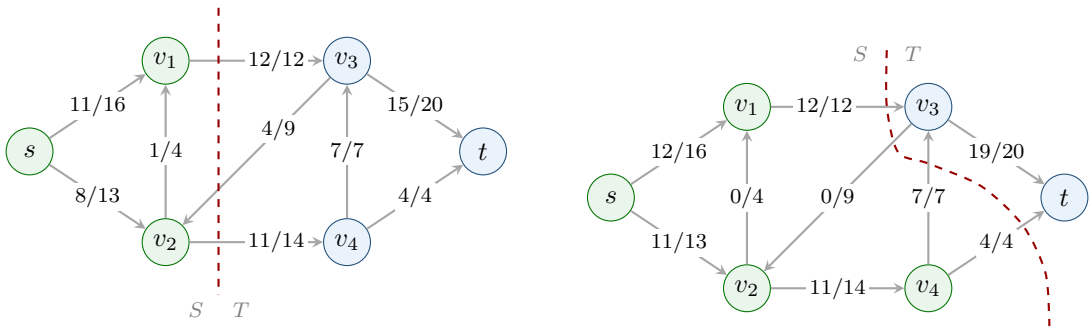
(f) La red residual final. Desde s se llega a v_1 y a v_2; desde v_2, a v_4; desde v_1 y v_4 solo se regresa. t no es alcanzable y el ciclo termina con $|f^*| = 23$. En verde, los vértices alcanzables desde s.



Ojito...

Los caminos de la figura 26.6 no salen de un BFS. Cormen no dice cómo los eligió, y el primero tiene cinco aristas cuando hay caminos de tres, como s, v_1, v_3, t. Quien intente reproducir la figura con un BFS obtiene otra ejecución, la de Edmonds-Karp, que aparece más adelante. Estos caminos sirven para que se vea una cancelación, y cualquier elección de caminos es una ejecución válida del método.

Ejemplo 10.18 (El corte mínimo)**.** Los vértices verdes de la red residual final forman $S = \{s, v_1, v_2, v_4\}$, con $T = \{v_3, t\}$. A la derecha está ese corte sobre el flujo máximo; a la izquierda, para contrastar, el corte del ejemplo 10.12 sobre el flujo de valor 19. En verde, los vértices de S.



De S a T cruzan (v_1, v_3) con 12/12, (v_4, v_3) con 7/7 y (v_4, t) con 4/4: todas saturadas, como exige la prueba de (2) ⇒ (3). La única de T a S es (v_3, v_2), con 0/9: vacía. Entonces

$$f(S,T) = 12 + 7 + 4 - 0 = 23 = c(S,T).$$

En el corte de la izquierda, en cambio, sobra holgura en (v_2, v_4), con 11/14, y regresan 4 unidades por (v_3, v_2). La red tiene 16 cortes; los seis más baratos son

S	$c(S,T)$	S	$c(S,T)$
$\{s, v_1, v_2, v_4\}$	23	$\{s, v_1, v_2\}$	26
$\{s, v_1, v_2, v_3, v_4\}$	24	$\{s\}$	29
$\{s, v_1\}$	25	$\{s, v_2, v_4\}$	31

y el mínimo es único. La lectura física: a Saskatoon solo pueden entrar $12 + 7 = 19$ cajas diarias, aunque la ruta a Winnipeg aguante 20, y por Regina pasan directo 4 más. No hay manera de enrutar que pase de 23. También se ve por qué la arista extra (v_1, v_2) del apartado de antiparalelas no ayuda: sus dos extremos están en S, así que no cruza el corte mínimo.

Idea clave

Ford-Fulkerson parece un voraz: toma un camino cualquiera y lo llena hasta su cuello de botella. La diferencia está en las aristas de regreso. La decisión de la iteración (a), mandar 4 unidades por (v_3, v_2), se deshizo en (c), y el voraz del ejemplo 10.4, que no podía deshacerla, se atoró en 19. Dijkstra y Kruskal nunca revisan una decisión, y no les hace falta: Dijkstra porque con pesos no negativos el vértice que fija ya tiene su distancia definitiva, Kruskal porque la arista más ligera que cruza un corte siempre es segura. En flujo no hay una propiedad así para los caminos: ninguna elección local es segura. Las aristas de regreso vuelven provisional cada decisión, y el teorema 10.15 garantiza que, cuando ya no hay camino, no queda nada que corregir.

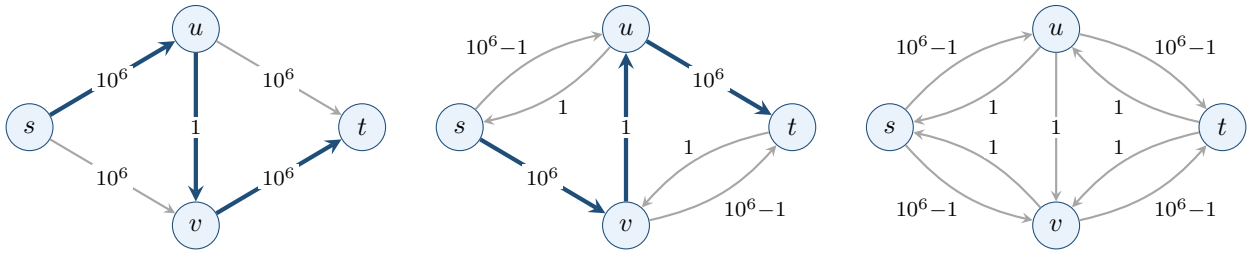
Cuando la elección del camino importa

Ford-Fulkerson no dice cómo escoger el camino, y eso decide el costo. La cota $\mathcal{O}(m\,|f^*|)$ no es pesimismo del análisis: hay redes donde se alcanza.

Ejemplo 10.19 (El caso patológico; Cormen, figura 26.7)**.** La red tiene $V = \{s, u, v, t\}$, con $c(s,u) = c(s,v) = c(u,t) = c(v,t) = 10^6$ y $c(u,v) = 1$. El flujo máximo vale $2 \cdot 10^6$: 10^6 por s, u, t y 10^6 por s, v, t, y el corte $(\{s\}, \{u, v, t\})$ tiene capacidad $2 \cdot 10^6$. Supóngase que Ford-Fulkerson elige s, u, v, t en las iteraciones impares y s, v, u, t en las pares. Cada camino pasa por (u, v) o por su inversa, de capacidad residual 1, así que el flujo sube de uno en uno.

i	**Camino**	$c_f(p)$	$f(u,v)$ después	$\lvert f\rvert$
1	s, u, v, t	1	1	1
2	s, v, u, t	1	0	2
3	s, u, v, t	1	1	3
⋮				⋮
$2 \cdot 10^6$	s, v, u, t	1	0	$2 \cdot 10^6$

Las tres instantáneas son la red original con el primer camino, la residual después del primer aumento con el segundo camino, y la residual después del segundo. En la tercera, (u, v) está otra vez disponible con capacidad 1 y las cuatro aristas de los lados perdieron una unidad: el ciclo se repite.



Nada en el método impide esa elección; la línea 5 solo pide que exista un camino. Son $2 \cdot 10^6$ iteraciones de $\mathcal{O}(m)$ cada una, y la cota $\Theta(m\,|f^*|)$ se alcanza. Con C en lugar de 10^6 la estrategia alternada hace $2C$ aumentos; se comprobó simulando con $C = 1, 2, 3, 10, 1000$ y 10^6. Mientras tanto, los caminos s, u, t y s, v, t, de dos aristas cada uno, resuelven la red en dos aumentos.

Edmonds-Karp

Edmonds-Karp fija la elección: el camino aumentante es siempre un camino de s a t con el menor número de aristas en G_f, y lo encuentra un BFS. En el ejemplo patológico eso ya basta. Los caminos más cortos son s, u, t y s, v, t, de dos aristas, y el de tres que pasa por (u, v) nunca se toca: dos aumentos contra dos millones.

El BFS es el de la clase del 22 de octubre (sección 1; a fondo en la sección 4) con dos cambios: recorre las listas de G', que incluyen a los vecinos de entrada, y solo cruza (u, v) si $c_f(u,v) > 0$. Recibe f porque con él calcula cada $c_f(u,v)$ en $\mathcal{O}(1)$, con la estructura G' de Ford-Fulkerson; la red residual sigue sin construirse. Su $d[v]$ es la **distancia residual** $\delta_f(s,v)$: el número de aristas de un camino más corto de s a v en G_f, o ∞ si no hay camino.

Algoritmo 10.2 BFS-Residual(G, f, s)

```
1: procedimiento BFS-Residual(G, f, s)
2:     para todo v ∈ V hacer
3:         visto[v] ← falso,    d[v] ← ∞,    π[v] ← NULO
4:     fin para
5:     visto[s] ← verdadero,    d[s] ← 0
6:     Q ← cola vacía;  Encolar(Q, s)
7:     mientras Q no está vacía hacer
8:         u ← Desencolar(Q)
9:         para todo v con (u, v) ∈ E o (v, u) ∈ E hacer          // lista de u en G'
10:            si no visto[v] y c_f(u, v) > 0 entonces
11:                visto[v] ← verdadero,    d[v] ← d[u] + 1,    π[v] ← u
12:                Encolar(Q, v)
13:            fin si
14:        fin para
15:    fin mientras
16: fin procedimiento
```

Algoritmo 10.3 Edmonds-Karp(G, s, t)

```
1: procedimiento Edmonds-Karp(G, s, t)
2:     para todo (u, v) ∈ E hacer
3:         f(u, v) ← 0
4:     fin para
5:     ciclo
6:         BFS-Residual(G, f, s)                                  // llena visto, d y π
7:         si no visto[t] entonces
8:             devolver f                                         // no hay camino: f es máximo
9:         fin si
10:        Δ ← ∞,    v ← t
11:        mientras v ≠ s hacer                                   // primera pasada: cuello de botella
12:            Δ ← mín(Δ, c_f(π[v], v)),    v ← π[v]
13:        fin mientras
14:        v ← t
15:        mientras v ≠ s hacer                                   // segunda pasada: aumento
16:            u ← π[v]
17:            si (u, v) ∈ E entonces
18:                f(u, v) ← f(u, v) + Δ
19:            si no
20:                f(v, u) ← f(v, u) − Δ
21:            fin si
22:            v ← u
23:        fin mientras
24:    fin ciclo
25: fin procedimiento
```

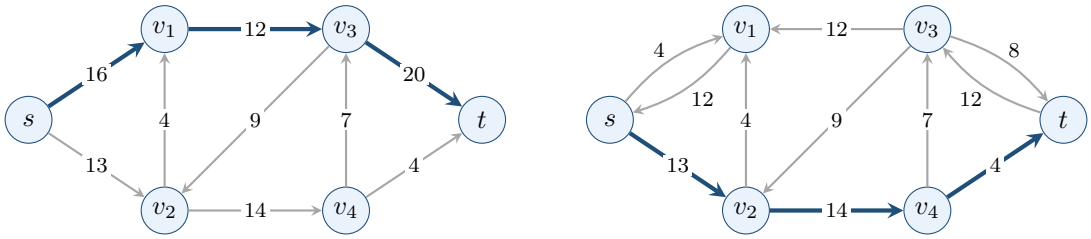
Las líneas que no son obvias en Edmonds-Karp:

- **Línea 6.** Después del BFS, $d[v] = \delta_f(s, v)$ y siguiendo π desde t se recorre un camino más corto, al revés.
- **Líneas 10 a 13.** Primera pasada de t a s para saber el cuello de botella; al salir, $\Delta = c_f(p)$. Hacen falta dos pasadas porque $c_f(p)$ solo se conoce después de recorrer todo el camino.
- **Líneas 14 a 23.** Segunda pasada: la misma actualización que las líneas 7 a 13 de Ford-Fulkerson.
- El BFS puede detenerse en cuanto descubre t. Ahorra trabajo y no cambia la cota.

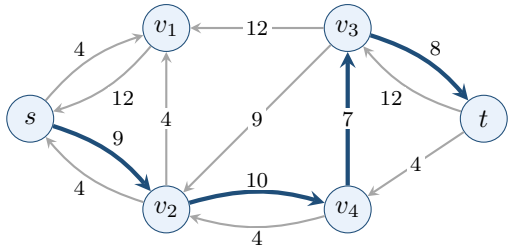
Ejemplo 10.20 (Edmonds-Karp sobre la red de Cormen; ejercicio 26.2-3)**.** Misma red, pero ahora cada camino lo elige un BFS que revisa los vecinos en el orden s, v_1, v_2, v_3, v_4, t. La tabla da las distancias residuales antes de cada aumento.

	$\delta_f(s,\cdot)$ antes del aumento								
Iter.	v_1	v_2	v_3	v_4	t	**Camino**	$c_f(p)$	**Crítica**	$\lvert f\rvert$
1	1	1	2	2	3	s, v_1, v_3, t	12	(v_1, v_3)	12
2	1	1	3	2	3	s, v_2, v_4, t	4	(v_4, t)	16
3	1	1	3	2	4	s, v_2, v_4, v_3, t	7	(v_4, v_3)	23
fin	1	1	∞	2	∞	no hay camino			

Las redes residuales de las iteraciones 1 y 2, con su camino en azul:



y la de la iteración 3:



La red residual final es la misma del renglón (f) del ejemplo 10.17, y el flujo final es idéntico al de (e). Son tres aumentos en lugar de cinco, y ninguno usa una arista de regreso. Las distancias ilustran el lema que sigue: $\delta_f(s, v_3)$ pasa de 2 a 3 y luego a ∞, $\delta_f(s, t)$ va de 3 a 3 a 4, y ninguna baja; las longitudes de los caminos, $3, 3, 4$, tampoco. Cada aumento tiene exactamente una arista crítica, y ninguna se repite. Hay un empate: s, v_1, v_3, t y s, v_2, v_4, t miden 3, y si el BFS revisara v_2 antes que v_1 saldrían en orden inverso.

Por qué Edmonds-Karp es polinomial

Todo el análisis descansa en un hecho sobre las distancias residuales: con caminos más cortos, ningún vértice se acerca a s.

Lema 10.21 (Distancias residuales; Cormen, lema 26.7)**.** *Durante Edmonds-Karp, para todo $v \in V$, la distancia residual $\delta_f(s,v)$ nunca baja de un aumento al siguiente.*

Demostración. Supóngase que sí baja alguna, y tómese el primer aumento que baja una distancia: f es el flujo antes de él y f' el de después. Entre los vértices cuya distancia bajó, sea v el de $\delta_{f'}(s,v)$ más chica. Así $\delta_{f'}(s,v) < \delta_f(s,v)$, y $v \neq s$ porque $\delta(s,s) = 0$ siempre. Sea u el vértice anterior a v en un camino más corto de s a v en $G_{f'}$: $(u,v) \in E_{f'}$ y $\delta_{f'}(s,u) = \delta_{f'}(s,v) - 1$. Como u quedó más cerca que v, por la elección de v su distancia no bajó:

$$\delta_{f'}(s,u) \geq \delta_f(s,u).$$

Si (u,v) ya estaba en E_f, la desigualdad del triángulo para distancias en aristas (propiedad 7.8, con todos los pesos iguales a 1) daría

$$\delta_f(s,v) \leq \delta_f(s,u) + 1 \leq \delta_{f'}(s,u) + 1 = \delta_{f'}(s,v),$$

contra la hipótesis de que la distancia de v bajó. Entonces $(u,v) \notin E_f$ y $(u,v) \in E_{f'}$: la arista apareció con el aumento. La capacidad residual de (u,v) solo sube si el flujo entre u y v cambia a favor de u, es decir, si el camino aumentante usó la arista residual (v,u). Ese camino es un camino más corto en G_f, así que $\delta_f(s,u) = \delta_f(s,v) + 1$, y

$$\delta_f(s,v) = \delta_f(s,u) - 1 \leq \delta_{f'}(s,u) - 1 = \delta_{f'}(s,v) - 2,$$

otra contradicción con $\delta_{f'}(s,v) < \delta_f(s,v)$. □

Cormen enuncia el lema para $v \in V - \{s,t\}$, pero la prueba no usa esa restricción; en t la distancia tampoco baja, como se vio en el ejemplo.

Ojito...

El lema dice "nunca baja", no "siempre sube". En el ejemplo 10.20, $\delta_f(s,v_1)$ y $\delta_f(s,v_2)$ se quedan en 1 todo el tiempo. Y la garantía es del BFS: con DFS los caminos no son los más cortos y el lema deja de valer. Un Dijkstra que use las capacidades como pesos tampoco es Edmonds-Karp: a lo largo de un camino las capacidades no se suman, se toma su mínimo.

Definición 10.22 (Arista crítica)**.** Una arista residual (u,v) de un camino aumentante p es **crítica** si $c_f(u,v) = c_f(p)$: es un cuello de botella.

Todo camino aumentante tiene al menos una arista crítica, y después del aumento esa arista sale de G_f: si era una arista de E, quedó saturada; si era la inversa de una, el flujo de esa arista volvió a 0.

Teorema 10.23 (Cormen, teorema 26.8)**.** *Edmonds-Karp hace a lo más nm aumentos.*

Demostración. Sea (u,v) crítica cuando el flujo es f. Como el camino es uno de los más cortos, su tramo hasta v también lo es, y $\delta_f(s,v) = \delta_f(s,u) + 1$. Después del aumento (u,v) sale de G_f, y para volver a ser crítica primero tiene que reaparecer. Eso solo pasa si un aumento posterior, con flujo f', empuja por (v,u); ese camino también es de los más cortos, así que $\delta_{f'}(s,u) = \delta_{f'}(s,v) + 1$. Por el lema 10.21, $\delta_{f'}(s,v) \geq \delta_f(s,v)$, y entonces

$$\delta_{f'}(s,u) = \delta_{f'}(s,v) + 1 \geq \delta_f(s,v) + 1 = \delta_f(s,u) + 2.$$

Como las distancias nunca bajan, entre dos veces seguidas que (u,v) es crítica la distancia de s a u sube al menos 2. Además, cada vez que es crítica, u está en un camino aumentante antes que t,

y el tramo de s a u de ese camino no repite vértices ni pasa por t: tiene a lo más $n-1$ vértices, y $\delta_f(s,u) \le n-2$. Empezando en al menos 0 y subiendo de 2 en 2 sin pasar de $n-2$, (u,v) es crítica a lo más $(n-2)/2+1 = n/2$ veces. Las aristas residuales posibles son las de E y sus inversas, a lo más $2m$, así que hay a lo más $2m \cdot n/2 = nm$ eventos críticos. Cada aumento tiene al menos uno. □

Cormen cuenta "$\mathcal{O}(E)$ pares de vértices" sin precisar; la cuenta exacta, $2m$ aristas residuales posibles con $n/2$ eventos cada una, da la cota nm.

Costo. Cada iteración hace un BFS sobre G', de $\mathcal{O}(n+2m) = \mathcal{O}(m)$ con listas, por la misma cuenta del BFS de la libreta, y dos pasadas por el camino, de $\mathcal{O}(n)$. Son a lo más nm iteraciones, más un último BFS que no encuentra camino.

Resultado

Edmonds-Karp hace $\mathcal{O}(nm)$ aumentos y cuesta

$$\mathcal{O}(nm) \cdot \mathcal{O}(m) + \mathcal{O}(m) = \mathcal{O}\left(nm^2\right),$$

sin importar las capacidades.

La cota no usa que las capacidades sean enteras: ni el lema ni el teorema lo suponen, así que Edmonds-Karp termina con capacidades reales cualesquiera. Y como sigue siendo una implementación de Ford-Fulkerson, con capacidades enteras también cumple $\mathcal{O}(m\,|f^*|)$; vale la menor de las dos cotas. En la red de Cormen las cotas de aumentos son $nm = 54$ y $|f^*| = 23$, y bastan 3. En el caso patológico son $nm = 20$ contra $2 \cdot 10^6$, y bastan 2.

Con matriz de adyacencia cada BFS cuesta $\Theta(n^2)$ y el total sube a $\mathcal{O}(n^3 m)$; la tabla de matriz contra listas de la clase del 22 de octubre aplica tal cual. En un grafo denso, $m = \Theta(n^2)$ y $\mathcal{O}(nm^2) = \mathcal{O}(n^5)$. Eso motiva los algoritmos de empujar y reetiquetar de la sección 12, que no buscan caminos aumentantes y llegan a $\mathcal{O}(n^2 m)$ y a $\mathcal{O}(n^3)$.

Errores comunes

Ojito...

- **Antiparalelas en la entrada.** La definición de c_f supone que no hay. Un grafo no dirigido, donde cada arista se modela con dos dirigidas, las tiene siempre. Hay dos salidas: partir cada par con un vértice v', o usar $c_f(u,v) = c(u,v) - f(u,v) + f(v,u)$. Una implementación con matriz de capacidades residuales $r[u][v]$, que al aumentar resta en $r[u][v]$ y suma en $r[v][u]$, hace lo segundo sin pensarlo.
- **El flujo máximo no es único; su valor sí.** Aumentar el flujo de valor 19 del ejemplo 10.3 por s, v_2, v_3, t da un flujo de valor 23 con $f(v_2,v_1) = 1$; el de la figura 26.6 vale 23 con $f(v_2,v_1) = 0$. También puede haber varios cortes mínimos, y la búsqueda final entrega uno. En la red de Cormen el corte mínimo es único.
- **La red es dirigida.** El flujo máximo de s a t no es el de t a s. En la red de Cormen no sale ninguna arista de t, y el flujo de t a s vale 0.
- **Vértices fuera de todo camino de s a t.** La definición los excluye. Si aparecen, hay un flujo máximo que no pasa por ellos (Cormen, ejercicio 26.1-3), y se pueden borrar antes de empezar.

Resumen

	Ford-Fulkerson	**Edmonds-Karp**
Camino aumentante	cualquiera	el de menos aristas, con BFS
Número de aumentos	$\leq \lvert f^* \rvert$ con capacidades enteras	$\leq nm$
Costo	$\mathcal{O}(m \lvert f^* \rvert)$ con capacidades enteras	$\mathcal{O}(nm^2)$
Capacidades irracionales	puede no terminar	termina
Red de Cormen	5 aumentos con los caminos de Cormen	3 aumentos
Caso patológico	hasta $2 \cdot 10^6$ aumentos	2 aumentos

Lo que se lleva de la sección: el flujo se mejora por caminos aumentantes en la red residual, cuyas aristas de regreso permiten deshacer decisiones; cuando ya no hay camino, el teorema 10.15 entrega a la vez la optimalidad y un corte que la certifica; y la regla para escoger el camino es lo que separa un algoritmo pseudopolinomial de uno polinomial.

11. Emparejamiento bipartito máximo

Complemento: Cormen, cap. 26.3

Resumen rápido. Un emparejamiento es un conjunto de aristas que no comparten extremos: cada máquina hace a lo más una tarea, cada alumno toma a lo más un proyecto. En un grafo bipartito, el emparejamiento más grande se encuentra con flujo máximo: se agregan una fuente y un sumidero, todas las capacidades valen 1, y los flujos enteros de esa red resultan ser exactamente los emparejamientos. Como Ford-Fulkerson solo produce flujos enteros, el problema queda resuelto en $\mathcal{O}(nm)$. Leído sobre el grafo original, cada camino aumentante es una cadena de reasignaciones que termina en un vértice libre, y el teorema de Berge dice que un emparejamiento es máximo justo cuando ya no queda ninguna. Después vienen el teorema de Hall, que dice cuándo no hay emparejamiento perfecto y entrega un testigo que se revisa a mano, el de König, la idea de Hopcroft-Karp, que baja el costo a $\mathcal{O}(\sqrt{n}\,m)$, y la razón por la que en grafos no bipartitos todo esto se queda corto.

El problema

Cormen lo plantea con máquinas y tareas. Hay un conjunto L de máquinas y un conjunto R de tareas que se ejecutan al mismo tiempo; la arista $\{u, v\}$ dice que la máquina u sabe hacer la tarea v. Cada máquina hace a lo más una tarea y cada tarea la hace a lo más una máquina. La pregunta es a cuántas máquinas se les puede dar trabajo a la vez. Con alumnos y proyectos es lo mismo: cada alumno acepta algunos proyectos, cada proyecto admite un alumno, y se quiere acomodar a tantos alumnos como se pueda.

El grafo es bipartito: $V = L \cup R$ con $L \cap R = \emptyset$, y toda arista une un vértice de L con uno de R. Aquí los lados se llaman L (izquierda) y R (derecha), como en Cormen; son los V_1 y V_2 de la sección 3.

Definición 11.1 (Emparejamiento)**.** En un grafo no dirigido $G = (V, E)$, un **emparejamiento** (*matching*) es un conjunto $M \subseteq E$ tal que cada vértice es incidente en a lo más una arista de M. Un vértice incidente en una arista de M está **emparejado**; si no, está **libre**. Si $\{u, v\} \in M$, v es la **pareja** de u y u la de v.

Definición 11.2 (Máximo, maximal y perfecto)**.** Un emparejamiento M es

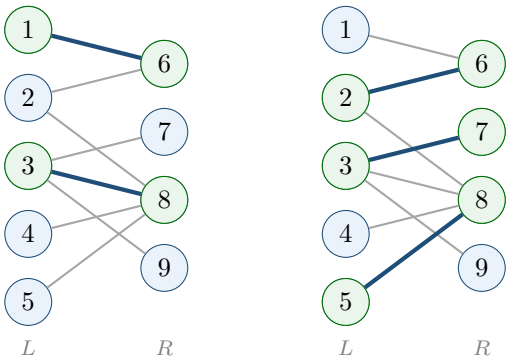
- **máximo** (*maximum*) si $|M| \geq |M'|$ para todo emparejamiento M' de G;
- **maximal** (*maximal*) si no está contenido propiamente en otro emparejamiento, es decir, si ninguna arista de $E - M$ se le puede agregar;
- **perfecto** (*perfect*) si todo vértice de G está emparejado.

Todo máximo es maximal, pero no al revés, y esa diferencia es todo el problema. Un perfecto es máximo, y en un bipartito exige $|L| = |R|$. El **problema del emparejamiento bipartito máximo** pide, dado G bipartito, un emparejamiento máximo. Cormen supone además que todo vértice tiene al menos una arista; de ahí sale $m \geq n/2$, que se usa en las cotas de tiempo. Un vértice aislado no cambia la respuesta y se quita en $\Theta(n)$ antes de empezar.

Ejemplo 11.3 (Figura 26.8 de Cormen: maximal contra máximo)**.** Los vértices se numeran como en el ejercicio 26.3-1 de Cormen: $L = \{1, 2, 3, 4, 5\}$ y $R = \{6, 7, 8, 9\}$, de arriba abajo, con

$$E = \{\{1,6\}, \{2,6\}, \{2,8\}, \{3,7\}, \{3,8\}, \{3,9\}, \{4,8\}, \{5,8\}\},$$

$n = 9$ y $m = 8$. En azul van las aristas del emparejamiento y en verde los vértices emparejados.



(a) maximal, $|M_a| = 2$ (b) máximo, $|M_b| = 3$

En (a), $M_a = \{\{1,6\},\{3,8\}\}$ es maximal: cada una de las otras seis aristas toca a 1, 3, 6 u 8. En (b), $M_b = \{\{2,6\},\{3,7\},\{5,8\}\}$ es máximo, y el argumento es corto: 7 y 9 solo tienen de vecino a 3, así que a lo más uno de los dos queda emparejado, y de R se emparejan a lo más tres vértices. Cormen no usa la palabra *maximal*; el pie de su figura dice solo que (a) tiene cardinalidad 2.

La forma obvia de construir un emparejamiento es voraz: recorrer las aristas y tomar cada una que no choque con las ya tomadas. Lo que sale es maximal por construcción, pero su tamaño depende del orden. En la figura 26.8, con las aristas en el orden en que se listaron arriba, el voraz toma $\{1,6\}$, $\{2,8\}$ y $\{3,7\}$, que es máximo; si empieza por $\{1,6\}$ y $\{3,8\}$, se atora en M_a. Lo único que garantiza es la mitad.

Proposición 11.4. *Si M es un emparejamiento maximal y M^* uno máximo, entonces $|M^*| \leq 2\,|M|$.*

Demostración. Cada arista de M^* comparte al menos un extremo con alguna arista de M; si no, se le podría agregar a M, que no sería maximal. A cada arista de M^* se le asigna uno de esos extremos compartidos. Los vértices asignados son distintos, porque dos aristas de M^* no comparten extremos, y todos son extremos de aristas de M, que en total tienen $2\,|M|$. Entonces $|M^*| \leq 2\,|M|$. □

La cota se alcanza: en el camino a–b–c–d, el voraz que empieza por la arista $\{b,c\}$ ya no puede agregar nada y se queda con 1, mientras que $\{\{a,b\},\{c,d\}\}$ mide 2. Cormen no incluye esta proposición en 26.3; está aquí porque dice exactamente cuánto puede perder el voraz.

Ojito…

Tres confusiones que conviene separar desde el principio. Maximal no es máximo: M_a es maximal y mide 2, y el máximo mide 3. El máximo no es único: la figura 26.8 tiene 10 emparejamientos de tamaño 3, y distintas reglas de desempate llegan a distintos máximos, todos del mismo tamaño. Y máximo no es perfecto: ahí $|L| = 5$ y $|R| = 4$, así que algún vértice de L se queda libre siempre.

Para pasar de M_a a un máximo hay que deshacer algo: sacar la arista $\{3,8\}$ y meter dos nuevas. La pregunta es cómo encontrar esos reacomodos sin probar todas las combinaciones. Cormen responde con flujo máximo; leída sobre el grafo original, la misma respuesta son los caminos aumentantes.

Reducción a flujo máximo

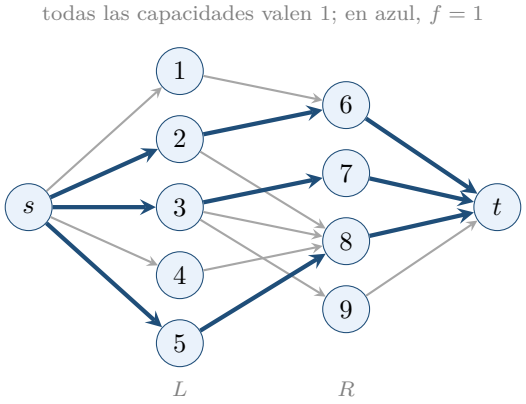
Definición 11.5 (Red de flujo correspondiente)**.** Sea $G = (V, E)$ bipartito con $V = L \cup R$. Su **red de flujo correspondiente** es $G' = (V', E')$ con $V' = V \cup \{s,t\}$ y

$$E' = \{(s,u) : u \in L\} \cup \{(u,v) : u \in L,\ v \in R,\ \{u,v\} \in E\} \cup \{(v,t) : v \in R\},$$

con capacidad $c(u, v) = 1$ en todas las aristas de E'.

Cada arista de G se orienta de L a R, la fuente s apunta a todo L y todo R apunta al sumidero t. La red tiene $|E'| = m + n$ aristas, y con $m \geq n/2$ queda $m \leq |E'| \leq 3m$, o sea $|E'| = \Theta(m)$. Todo el modelado está en las capacidades de (s, u) y (v, t), que son capacidades de vértice disfrazadas. A $u \in L$ solo le entra la arista (s, u), así que le entra a lo más una unidad, y por conservación esa unidad sale por una sola arista hacia R. Del lado de R pasa lo mismo con (v, t).

Para la figura 26.8 la red tiene 11 vértices y $|E'| = 5 + 8 + 4 = 17$ aristas; es el panel (c) de esa figura. En azul va el flujo que corresponde a M_b: una unidad por cada uno de los caminos $s, 2, 6, t$, $s, 3, 7, t$ y $s, 5, 8, t$, de valor $|f| = 3$.



Algoritmo 11.1 Emparejamiento-Bipartito(G, L, R)

```
1:  procedimiento Emparejamiento-Bipartito(G, L, R)
2:      V' ← V ∪ {s, t},    E' ← ∅
3:      para todo u ∈ L hacer
4:          E' ← E' ∪ {(s, u)}
5:      fin para
6:      para todo {u, v} ∈ E con u ∈ L, v ∈ R hacer
7:          E' ← E' ∪ {(u, v)}                          // solo de L a R
8:      fin para
9:      para todo v ∈ R hacer
10:         E' ← E' ∪ {(v, t)}
11:     fin para
12:     c(e) ← 1 para toda e ∈ E'
13:     f ← Ford-Fulkerson(G', s, t)
14:     devolver M = {{u, v} : u ∈ L, v ∈ R, f(u, v) = 1}
15: fin procedimiento
```

La línea 13 es el algoritmo 10.1 de la sección 10, con cualquier forma de buscar el camino; con BFS es Edmonds-Karp (algoritmo 10.3). La línea 14 lee el emparejamiento en el flujo, y que eso funcione es justo lo que hay que demostrar: que los flujos enteros son emparejamientos y que el flujo que devuelve Ford-Fulkerson es entero.

Lema 11.6 (Emparejamientos y flujos enteros)**.** *Si M es un emparejamiento de G, existe un flujo entero f en G' con $|f| = |M|$. Recíprocamente, si f es un flujo entero en G', entonces $M = \{\{u, v\} : u \in L,\ v \in R,\ f(u, v) = 1\}$ es un emparejamiento de G con $|M| = |f|$.*

Demostración. Ida. Por cada $\{u, v\} \in M$, con $u \in L$, se pone $f(s, u) = f(u, v) = f(v, t) = 1$, y 0 en las demás aristas. Las capacidades se respetan: cada $u \in L$ está en a lo más una arista de M, así que (s, u) recibe a lo más una unidad, y lo mismo (v, t) del lado de R. Se conserva el flujo: a un $u \in L$ emparejado le entra 1 por (s, u) y le sale 1 por (u, v), y a uno libre no le entra ni le sale nada; en R igual. A s no le entra ninguna arista, así que $|f| = \sum_{u \in L} f(s, u)$, que cuenta los vértices emparejados de L: $|f| = |M|$.

Vuelta. Como f es entero y $0 \leq f \leq 1$, cada arista lleva 0 o 1. A $u \in L$ solo le entra (s, u), así que le entra 0 o 1; por conservación le sale lo mismo, y como cada arista de salida lleva 0 o 1, a lo más una arista (u, v) lleva 1. Del lado de R, a v solo le sale (v, t), y el mismo argumento dice que a lo más una arista (u, v) que le entra lleva 1. Entonces ningún vértice está en dos aristas de M, y M es un emparejamiento. Por último, sumando la conservación en cada $u \in L$, $|f| = \sum_{u \in L} f(s, u) = \sum_{u \in L} \sum_{v \in R} f(u, v) = |M|$. □

Ojito...

El flujo tiene que ser entero. En $K_{2,2}$, con $L = \{a, b\}$ y $R = \{x, y\}$, poner $1/2$ en las cuatro aristas de en medio y 1 en las de s y de t es un flujo válido de valor 2, máximo, y de él no se lee ningún emparejamiento: no hay arista con $f = 1$. Un algoritmo de flujo máximo cualquiera podría devolver ese flujo. Lo que salva a la reducción es el siguiente teorema, que vale para los métodos de caminos aumentantes.

Teorema 11.7 (Integralidad)**.** *Si todas las capacidades son enteras, el flujo máximo f que devuelve Ford-Fulkerson tiene $|f|$ entero y $f(u, v)$ entero en cada arista.*

Demostración. Por inducción sobre las iteraciones. Al principio $f = 0$. Si f es entero, todas las capacidades residuales son enteras, porque son de la forma $c(u, v) - f(u, v)$ o $f(v, u)$ (definición 10.5). Entonces $c_f(p) = \min\{c_f(u, v) : (u, v) \text{ está en } p\}$ es un entero, y el aumento suma o resta ese entero en cada arista del camino: f sigue siendo entero. Es el mismo argumento que acota el número de iteraciones de Ford-Fulkerson en la sección 10. □

Corolario 11.8. *El tamaño de un emparejamiento máximo de G es igual al valor de un flujo máximo en G', y el emparejamiento que devuelve* Emparejamiento-Bipartito *es máximo.*

Demostración. Sea f^* el flujo máximo que devuelve Ford-Fulkerson. Es entero por el teorema 11.7, así que el lema 11.6 lo convierte en un emparejamiento M con $|M| = |f^*|$. Si M' es cualquier otro emparejamiento, el mismo lema da un flujo f' con $|f'| = |M'|$, y como f^* es máximo, $|M'| = |f'| \leq |f^*| = |M|$. □

Cuánto cuesta

- Construir G' (líneas 2 a 12) cuesta $\Theta(n + m)$: una pasada por L, una por E y una por R.
- Cada iteración de Ford-Fulkerson busca un camino de s a t en G'_f con BFS o DFS. La residual tiene $n + 2$ vértices y a lo más $2|E'| = 2(m + n)$ aristas, así que la búsqueda cuesta $\mathcal{O}(n + m) = \mathcal{O}(m)$, usando $n \leq 2m$. Actualizar el flujo a lo largo del camino es $\mathcal{O}(n)$ y queda absorbido.
- Cada aumento sube $|f|$ en $c_f(p)$, que aquí vale exactamente 1 porque todas las capacidades residuales son 0 o 1. El flujo máximo es $|M^*| \leq \min(|L|, |R|) \leq n/2$, así que hay a lo más $n/2$ aumentos y una búsqueda final que falla.

En total, $\mathcal{O}(n) \cdot \mathcal{O}(m) = \mathcal{O}(nm)$. La cota no depende de cómo se elija el camino: vale con BFS y con DFS. Es la cota $\mathcal{O}(m\,|f^*|)$ de Ford-Fulkerson con $|f^*| \leq n/2$; la de Edmonds-Karp en general, $\mathcal{O}(nm^2)$, aquí es peor y no hace falta. Cormen escribe $\min(L, R)$ para la cota del flujo; se lee $\min(|L|, |R|)$.

Ojito...

Las aristas de G' van solo de L a R. Si se dejaran en los dos sentidos, un camino de flujo podría cruzar de L a R, regresar a L por una arista al revés y volver a cruzar, y una unidad de flujo ya no correspondería a una arista del emparejamiento: la prueba del lema se cae en el paso que dice que a $u \in L$ solo le entra (s, u). Además, las redes de flujo de la sección 10 no admiten aristas antiparalelas, y con la orientación de L a R no aparece ninguna.

Caminos aumentantes en el grafo original

Ford-Fulkerson trabaja sobre G', pero lo que hace se entiende mejor en G. La correspondencia que sigue no la escribe Cormen en 26.3 (la deja implícita en el problema 26-6); es la clave para leer cualquier traza del algoritmo y para escribir una versión que no construye G'.

Definición 11.9 (Camino aumentante respecto a M)**.** Sea M un emparejamiento de G bipartito. Un camino es **alternante** si sus aristas están, una sí y una no, en $E - M$ y en M. Un **camino aumentante** respecto a M (*augmenting path*) es un camino alternante que empieza en un vértice libre de L y termina en un vértice libre de R.

La primera y la última arista de un camino aumentante están fuera de M, así que tiene k aristas de M y $k+1$ fuera de M, y su longitud $2k+1$ es impar. La **diferencia simétrica** de dos conjuntos es $A \oplus B = (A - B) \cup (B - A)$: lo que está en uno solo de los dos.

Proposición 11.10 (La residual de un emparejamiento)**.** *Sea M un emparejamiento de G y f el flujo entero que le corresponde en G' por el lema 11.6. La red residual G'_f tiene exactamente estas aristas, con $u \in L$ y $v \in R$:*

- *(s, u) si u está libre, y (u, s) si está emparejado;*
- *(u, v) si $\{u, v\} \in E - M$, y (v, u) si $\{u, v\} \in M$;*
- *(v, t) si v está libre, y (t, v) si está emparejado.*

En consecuencia, los caminos simples de s a t en G'_f son exactamente los $s, u_0, v_0, u_1, v_1, \ldots, u_k, v_k, t$ tales que $P = u_0, v_0, \ldots, u_k, v_k$ es un camino aumentante respecto a M, y aumentar el flujo por uno de ellos produce el flujo de $M \oplus P$.

Demostración. En G' todas las capacidades valen 1 y f solo toma los valores 0 y 1. Por la definición de red residual (definición 10.5), una arista $(x, y) \in E'$ aparece en G'_f hacia adelante si $f(x, y) = 0$ y aparece invertida, como (y, x), si $f(x, y) = 1$. Como f vale 1 exactamente en las (s, u) y (v, t) de vértices emparejados y en las (u, v) con $\{u, v\} \in M$, salen las tres líneas de la lista.

Sea ahora un camino simple de s a t en G'_f. No usa aristas que entran a s ni que salen de t, porque no regresa a s y termina en t. Su primera arista es entonces (s, u_0) con u_0 libre. Desde un vértice de L las únicas salidas que quedan son aristas (u, v) con $\{u, v\} \notin M$. Desde un vértice $v \in R$ hay una sola salida: hacia t si v está libre, y hacia su pareja si está emparejado. Así, el camino alterna una arista fuera de M, de L a R, con una de M, de R a L, hasta llegar a un vértice libre de R, desde donde sale a t. Como el camino es simple, los vértices no se repiten, y lo que queda entre s y t es un camino aumentante. Al revés, todo camino aumentante con s al principio y t al final es un camino en G'_f.

Su capacidad residual es 1. Aumentar pone en 1 el flujo de (s, u_0), de (v_k, t) y de cada (u_i, v_i), que estaban en 0, y regresa a 0 el de cada arista (u_{i+1}, v_i) de M, que el camino recorre al revés. Las aristas de M que están en P salen y las de P fuera de M entran: es el flujo de $M \oplus P$. □

La frase clave de la prueba es que un vértice emparejado de R tiene una sola salida en la residual, hacia su pareja. Por eso los caminos alternan solos, sin que nadie se lo pida al algoritmo.

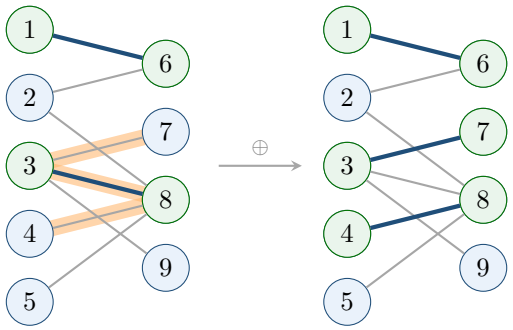
Lema 11.11. *Si P es un camino aumentante respecto a M, entonces $M \oplus P$ es un emparejamiento y $|M \oplus P| = |M| + 1$.*

Demostración. P tiene k aristas en M y $k+1$ fuera; $M \oplus P$ quita las primeras y agrega las segundas, así que mide $|M| + 1$. Falta ver que sigue siendo emparejamiento. Un vértice interior de P toca dos aristas de P, una de M y otra no, y ninguna otra arista de M: cambia una por la otra y se queda con exactamente una. Los extremos de P estaban libres y ganan una sola arista, la primera o la última de P. Los vértices fuera de P no cambian. □

Ejemplo 11.12 (Del maximal al máximo con un camino aumentante)**.** Se vuelve a la figura 26.8. Respecto a M_a los vértices libres son 2, 4 y 5 en L, y 7 y 9 en R. Hay seis caminos aumentantes, todos de longitud 3: de 2, 4 o 5 a 8, de 8 a su pareja 3, y de 3 a 7 o a 9. Todos pasan por la arista $\{3, 8\}$ de M_a, la que estorba: ocupa a la vez al único vecino de 7 y 9 y al único vecino de 4 y 5. Con $P = 4$–8–3–7:

Arista de P	**¿Está en M_a?**	**En $M_a \oplus P$**
$\{4, 8\}$	no	entra
$\{3, 8\}$	sí	sale
$\{3, 7\}$	no	entra

y $M_a \oplus P = \{\{1, 6\}, \{3, 7\}, \{4, 8\}\}$, de tamaño 3. En la figura, el trazo naranja es P dibujado sobre M_a.



M_a y el camino P $M_a \oplus P$

Respecto a M_b no hay ningún camino aumentante, como debe ser para un máximo.

Teorema 11.13 (Berge)**.** *Un emparejamiento M es máximo si y solo si no existe ningún camino aumentante respecto a M.*

Demostración. Si hay un camino aumentante P, el lema 11.11 da un emparejamiento $M \oplus P$ más grande, y M no es máximo.

Al revés, supóngase que M no es máximo y sea M^* un emparejamiento con $|M^*| > |M|$. En el grafo $H = (V, M \oplus M^*)$ cada vértice tiene grado a lo más 2: una arista de M y una de M^*, como mucho. Un grafo así se parte en caminos y ciclos, y en cada uno las aristas alternan entre $M - M^*$ y $M^* - M$, porque dos aristas del mismo emparejamiento no comparten vértice. Un ciclo alternante tiene tantas aristas de un emparejamiento como del otro. Como $M \oplus M^*$ tiene más aristas de M^* que de M, algún camino de H tiene más aristas de M^* que de M; alternando, empieza y termina con aristas de $M^* - M$. Sus dos extremos están libres en M. Si un extremo x tuviera una arista $e \in M$, esa arista no estaría en

M^*, porque la única arista de M^* que toca a x es la del camino, que no es de M; entonces e estaría en H y x tendría grado 2, y no sería extremo. Ese camino es aumentante respecto a M: tiene longitud impar, así que un extremo está en L y el otro en R, y se lee desde el de L. □

El nombre viene de Berge (1957); Cormen lo deja como parte del problema 26-6 sin nombrarlo. Con la proposición 11.10, el teorema de Berge para bipartitos es la equivalencia entre flujo máximo y ausencia de caminos aumentantes del teorema 10.15. La prueba de arriba, en cambio, no usa flujo, y usa la bipartición solo para nombrar los extremos del camino. Por eso sigue valiendo en cualquier grafo, cosa que se retoma al final de la sección.

El algoritmo escrito sobre G

Con la proposición 11.10, Ford-Fulkerson con BFS sobre G' se escribe sin construir G'. Es el BFS de la clase del 22 de octubre con dos cambios: arranca con todos los vértices libres de L en la cola, y al llegar a un vértice emparejado de R salta directo a su pareja. El arreglo pareja[v] guarda la pareja de cada vértice, o nulo si está libre.

Algoritmo 11.2 Camino-Aumentante(G, M)

```
1:  procedimiento Camino-Aumentante(G, M)
2:      para todo v ∈ V hacer
3:          visto[v] ← falso,    π[v] ← nulo
4:      fin para
5:      Q ← cola vacía
6:      para todo u ∈ L libre en M hacer
7:          visto[u] ← verdadero;  Encolar(Q, u)
8:      fin para
9:      mientras Q no está vacía hacer
10:         u ← Desencolar(Q)                          // siempre u ∈ L
11:         para todo v adyacente a u con {u, v} ∉ M hacer
12:             si no visto[v] entonces
13:                 visto[v] ← verdadero,    π[v] ← u
14:                 si v está libre en M entonces
15:                     devolver el camino de v hacia atrás por π
16:                 fin si
17:                 w ← pareja[v]                      // {w, v} ∈ M
18:                 visto[w] ← verdadero,    π[w] ← v
19:                 Encolar(Q, w)
20:             fin si
21:         fin para
22:     fin mientras
23:     devolver nulo
24: fin procedimiento
```

Las líneas que no son obvias en Camino-Aumentante:

- Líneas 6 a 8: todos los libres de L son raíces a la vez. En G' son el primer nivel del BFS desde s, que aquí ya no aparece.
- Línea 11: desde L solo se avanza por aristas fuera de M, que son las aristas (u, v) de la residual.

- Líneas 14 a 16: el primer vértice libre de R que se descubre cierra el camino; en G'_f es la arista (v, t).
- Líneas 17 a 19: desde un v emparejado solo se puede seguir a su pareja. No hace falta preguntar si w ya fue visto: w está emparejado, así que no fue raíz, y solo se llega a él desde v, que se acaba de visitar por primera vez. Por eso en la cola solo hay vértices de L.

Algoritmo 11.3 Emparejamiento-Máximo(G, L, R)

```
1:  procedimiento Emparejamiento-Máximo(G, L, R)
2:      M ← ∅
3:      repetir
4:          P ← Camino-Aumentante(G, M)
5:          si P ≠ nulo entonces
6:              M ← M ⊕ P                          // crece en uno
7:          fin si
8:      hasta que P = nulo
9:      devolver M
10: fin procedimiento
```

Emparejamiento-Máximo termina con un máximo por el teorema de Berge: Camino-Aumentante devuelve nulo solo cuando t no es alcanzable en G'_f, es decir, cuando no hay camino aumentante. Una llamada visita cada vértice una vez y revisa cada arista una vez desde su extremo en L: $\Theta(n + m)$. Hay a lo más $|M^*| + 1 \leq n/2 + 1$ llamadas, y $M \oplus P$ cuesta $\mathcal{O}(n)$ actualizando pareja[ ]. En total $\mathcal{O}(n(n + m)) = \mathcal{O}(nm)$ sin vértices aislados, igual que la reducción, y no es casualidad: es el mismo algoritmo. Con los vecinos en el mismo orden, elige exactamente los mismos caminos que Edmonds-Karp sobre G'.

Ojito...

Cormen advierte que un camino aumentante respecto a M está "relacionado con, pero es distinto de" un camino aumentante en una red de flujo. El de G va de un libre de L a un libre de R y alterna; el de G'_f va de s a t. Se corresponden uno a uno por la proposición 11.10, pero no son el mismo objeto. Y no cualquier camino alternante sirve: uno que termina en un vértice emparejado no es aumentante, y aplicarle $\oplus$ no hace crecer a M.

Ejemplo: alumnos y proyectos

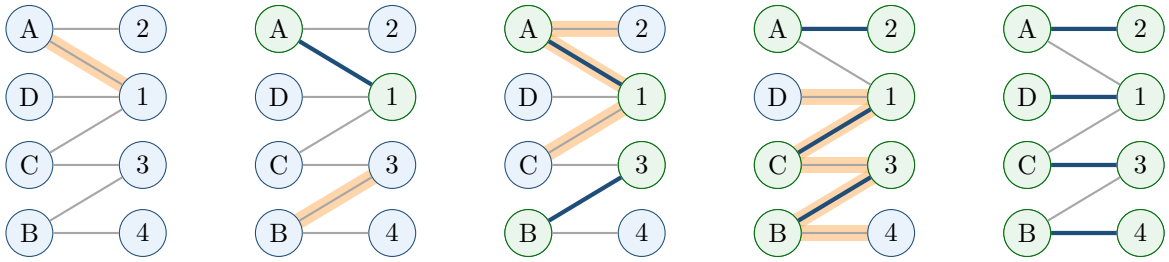
Ejemplo 11.14 (El emparejamiento crece de 1 a 4)**.** Cuatro alumnos, Ana, Beto, Caro y Dani, forman $L = \{A, B, C, D\}$, y cuatro proyectos forman $R = \{1, 2, 3, 4\}$. Cada alumno acepta:

Alumno	Proyectos que acepta
A (Ana)	1, 2
B (Beto)	3, 4
C (Caro)	1, 3
D (Dani)	1

Se abrevia la arista $\{A, 1\}$ como $A1$. Entonces $E = \{A1, A2, B3, B4, C1, C3, D1\}$, $n = 8$, $m = 7$ y $|E'| = 15$. El proyecto 1 es el más pedido y Dani no tiene alternativa. Se corre Emparejamiento-Máximo, que es Edmonds-Karp sobre G', con los vecinos en orden alfabético y numérico.

It.	Camino en G'_f	En G	M después	$\lvert M\rvert$
1	$s, A, 1, t$	A–1	$\{A1\}$	1
2	$s, B, 3, t$	B–3	$\{A1, B3\}$	2
3	$s, C, 1, A, 2, t$	C–1–A–2	$\{A2, B3, C1\}$	3
4	$s, D, 1, C, 3, B, 4, t$	D–1–C–3–B–4	$\{A2, B4, C3, D1\}$	4

En cada instantánea, el trazo naranja es el camino que se aplica en esa iteración, dibujado sobre el M de antes. Los vértices se dibujan en el orden A, D, C, B y 2, 1, 3, 4 para que ninguna arista se cruce; el BFS los recorre en orden alfabético.



it. 1: A–1 | it. 2: B–3 | it. 3: C–1–A–2 | it. 4: D–1–C–3–B–4 | final

Las dos primeras iteraciones son voraces. En la tercera, Caro quiere el 1, que es de Ana, o el 3, que es de Beto; el BFS encuentra primero que Ana puede mudarse al 2, que está libre. En la cuarta, Dani solo acepta el 1, que ahora es de Caro: Caro se muda al 3, que es de Beto, y Beto al 4. Cada camino aumentante es una cadena de reasignaciones que termina en un proyecto libre, y el final es perfecto.

Las búsquedas de las iteraciones 3 y 4

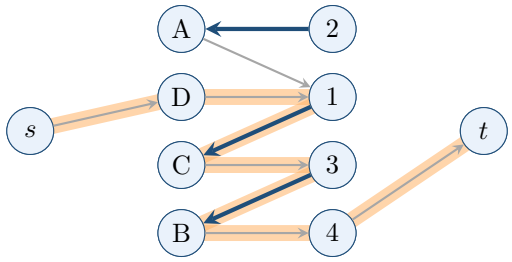
Con $M = \{A1, B3\}$, la parte de G'_f que importa tiene (s, C) y (s, D), porque C y D están libres; $(C, 1)$, $(C, 3)$, $(D, 1)$, $(A, 2)$ y $(B, 4)$, aristas fuera de M; $(1, A)$ y $(3, B)$, las aristas de M invertidas; y $(2, t)$ y $(4, t)$. El BFS desde s:

Sale	Descubre	Cola después
s	C, D	[C, D]
C	1, 3	[D, 1, 3]
D	nada: 1 ya visto	[1, 3]
1	A, por la arista invertida de $A1$	[3, A]
3	B, por la arista invertida de $B3$	[A, B]
A	2	[B, 2]
B	4	[2, 4]
2	t: se detiene	

Las distancias quedan $d[C] = d[D] = 1$, $d[1] = d[3] = 2$, $d[A] = d[B] = 3$, $d[2] = d[4] = 4$ y $d[t] = 5$. Siguiendo π desde t se recupera $t, 2, A, 1, C, s$, o sea el camino C–1–A–2. En Camino-Aumentante es la misma búsqueda sin s ni t y con el paso de cada vértice de R a su pareja hecho en el acto: la cola pasa por [C, D], [D, A, B] y [A, B], y al sacar A se descubre 2, que está libre.

En la iteración 4, con $M = \{A2, B3, C1\}$, la residual deja una sola ruta, $s, D, 1, C, 3, B, 4, t$, con $d[t] = 7$. A y 2 ni siquiera se alcanzan: a A solo se entra desde 2, por la arista invertida de $A2$, y a 2 solo desde t. Las aristas de M aparecen invertidas, de R a L; son las que permiten deshacer una asignación, la cancelación de flujo de la sección 10.

gris: arista de $E - M$, de L a R; azul: arista de M, invertida
(se omiten las que entran a s y las que salen de t)



Cuando no hay emparejamiento perfecto: Hall y König

La **vecindad** de $X \subseteq V$ es $N(X) = \{y \in V : \{x, y\} \in E \text{ para algún } x \in X\}$. Si un conjunto X de alumnos acepta entre todos menos proyectos que alumnos tiene, $|N(X)| < |X|$, es imposible acomodarlos a todos. Lo notable es que esa es la única obstrucción.

Teorema 11.15 (Hall)**.** *Sea G bipartito con $V = L \cup R$. Existe un emparejamiento que cubre todo L si y solo si $|A| \leq |N(A)|$ para todo $A \subseteq L$. En particular, si $|L| = |R|$, G tiene un emparejamiento perfecto si y solo si se cumple esa condición.*

Cormen lo pide en el ejercicio 26.3-4, en la versión con $|L| = |R|$.

Demostración. Si un emparejamiento cubre L, cada $x \in A$ tiene una pareja distinta, y todas están en $N(A)$: $|A| \leq |N(A)|$.

La vuelta se prueba por contrapositiva, y el flujo la hace constructiva. Supóngase que el emparejamiento máximo M que devuelve Emparejamiento-Bipartito no cubre L. Sea f su flujo, S el conjunto de vértices alcanzables desde s en G'_f, el mismo que aparece en la prueba del teorema 10.15, y $X = L \cap S$. Con la proposición 11.10 se leen tres hechos:

1. Todo vértice libre de L está en X, por la arista (s, u).
2. $N(X) \subseteq R \cap S$. Sean $x \in X$ y $\{x, v\} \in E$. Si la arista no está en M, (x, v) está en la residual y v es alcanzable. Si está en M, la única arista que entra a x en la residual es (v, x), así que a x se llegó desde v, y $v \in S$.
3. Todo $v \in R \cap S$ está emparejado, porque si no, (v, t) haría alcanzable a t y habría camino aumentante. Y su pareja está en X, por la arista (v, u).

Por 2 y 3, la pareja es una biyección entre $R \cap S$ y los vértices emparejados de X, y además $N(X) = R \cap S$. Por 1,

$$|N(X)| = |X| - (\text{número de libres de } L) < |X|\,,$$

y X viola la condición. □

Observación. La misma cuenta da el tamaño del máximo:

$$|M^*| = |L| - \max_{A \subseteq L}(|A| - |N(A)|).$$

Para cualquier A, a lo más $|N(A)|$ de sus vértices quedan emparejados, así que al menos $|A| - |N(A)|$ quedan libres; el X de la prueba alcanza la igualdad. Con $A = \emptyset$ se ve que el máximo de la derecha nunca es negativo.

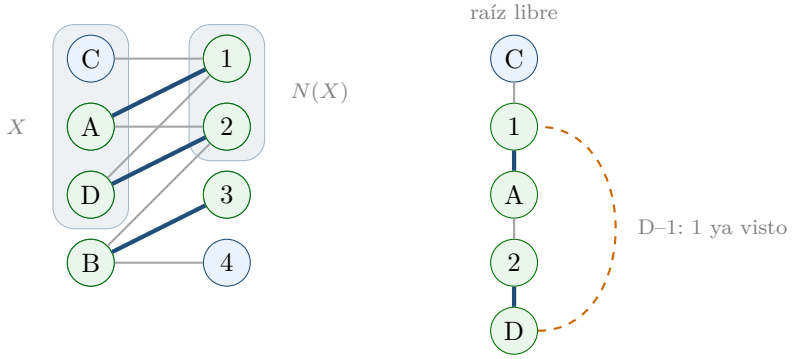
Ejemplo 11.16 (Tres alumnos para dos proyectos)**.** Los mismos cuatro alumnos, con otras preferencias:

Alumno	Proyectos que acepta
A (Ana)	1, 2
B (Beto)	2, 3, 4
C (Caro)	1
D (Dani)	1, 2

Así $E = \{A1, A2, B2, B3, B4, C1, D1, D2\}$, con $n = 8$ y $m = 8$. Emparejamiento-Máximo, con el mismo orden de vecinos, encuentra $A1$, después $B2$, y en la tercera iteración el camino D–2–B–3: Dani toma el 2 y Beto se muda al 3. Queda $M = \{A1, B3, D2\}$. La cuarta búsqueda arranca del único libre de L, que es Caro, y falla:

Sale	Descubre	Cola después
C	1, que es de A; se encola A	[A]
A	2, que es de D; se encola D	[D]
D	nada: 1 ya visto y 2 es su pareja	[]

Los alcanzables son $S = \{s, C, 1, A, 2, D\}$, así que $X = \{A, C, D\}$ y $N(X) = \{1, 2\}$: $|N(X)| = 2 < 3 = |X|$. Tres alumnos compiten por dos proyectos, y ningún algoritmo puede acomodar a los cuatro. La deficiencia $|X| - |N(X)| = 1$ es justo $|L| - |M| = 4 - 3$. El proyecto 4 queda libre: Beto podría tomarlo, pero eso no ayuda a nadie de X. El subconjunto se llama X y no A porque Ana ya es A.



A la izquierda, X y $N(X)$ sombreados. A la derecha, el árbol de la búsqueda que falla: alterna una arista fuera de M y una de M, y se agota sin tocar un libre de R.

Del mismo conjunto S sale un segundo certificado. Una **cubierta de vértices** de G es un conjunto $C \subseteq V$ que toca todas las aristas.

Teorema 11.17 (König)**.** *En un grafo bipartito, el tamaño de un emparejamiento máximo es igual al de una cubierta de vértices mínima.*

Demostración. Para cualquier emparejamiento M' y cualquier cubierta C', $|M'| \leq |C'|$: las aristas de M' no comparten extremos, y cada una necesita su propio vértice de C'. Basta entonces exhibir una cubierta del tamaño del máximo M. Los tres hechos de la prueba de Hall no usan que M deje libres en L, así que valen aquí. Sea $C = (L - S) \cup (R \cap S)$. Toca todas las aristas: si $\{u, v\} \in E$ con $u \in L$, la toca u cuando $u \notin S$, y cuando $u \in S$ el hecho 2 pone a v en $R \cap S$. Y mide $|M|$: todo vértice de C está emparejado (los libres de L están en S por el hecho 1, y los de $R \cap S$ están emparejados por el hecho 3), y cada arista de M tiene exactamente un extremo en C, porque por los hechos 2 y 3 sus dos extremos están en S o los dos fuera. □

Cormen no lo menciona, pero vale la pena: para convencer a alguien de que M es máximo basta enseñarle $|M|$ vértices que tocan todas las aristas, y eso se revisa a mano. En la figura 26.8, $\{3, 6, 8\}$ toca las ocho aristas, así que ningún emparejamiento pasa de 3. En el ejemplo 11.16, la cubierta es $\{B\} \cup \{1, 2\}$. La cubierta es el corte mínimo leído en G: las aristas de G' que van de S a $V' - S$ son exactamente las (s, u) con $u \in L - S$ y las (v, t) con $v \in R \cap S$.

Hopcroft-Karp: varios caminos por fase

Emparejamiento-Máximo hace un BFS completo para agregar una sola arista. Hopcroft y Karp (problema 26-6 de Cormen) aprovechan cada BFS para varios caminos. Sea ℓ la longitud del camino aumentante más corto respecto a M. Una **fase** encuentra un conjunto *maximal* de caminos aumentantes de longitud ℓ, ajenos en vértices, y los aplica todos juntos.

Algoritmo 11.4 Hopcroft-Karp(G)

```
1: procedimiento HOPCROFT-KARP(G)
2:     M ← ∅
3:     repetir
4:         ℓ ← longitud del camino aumentante más corto respecto a M
5:         P ← un conjunto maximal de caminos aumentantes de longitud ℓ, ajenos en vértices
6:         M ← M ⊕ (P_1 ∪ ··· ∪ P_k), donde P = {P_1, ..., P_k}
7:     hasta que P = ∅                                  // no queda camino aumentante
8:     devolver M
9: fin procedimiento
```

Que se puedan aplicar juntos sale del lema 11.11: como los P_i no comparten vértices, aplicar uno no toca a los otros, y $M \oplus (P_1 \cup \cdots \cup P_k)$ es un emparejamiento de tamaño $|M| + k$. Una fase cuesta $\mathcal{O}(m)$. Un BFS desde todos los libres de L a la vez, como en Camino-Aumentante, arma las capas y da ℓ; se detiene al completar la capa donde aparece el primer libre de R. Después, un DFS que solo baja de una capa a la siguiente saca los caminos uno por uno y borra cada vértice que usa o que resulta un callejón sin salida, así que cada arista se revisa una sola vez en toda la fase.

El número de fases descansa en tres hechos, que Cormen deja como partes del problema:

1. Después de una fase, el camino aumentante más corto mide más de ℓ. Las longitudes son impares, así que ℓ sube al menos 2 por fase. Esta es la parte larga del problema y su prueba se omite.
2. Si el más corto mide ℓ, entonces $|M^*| \leq |M| + n/(\ell + 1)$. En la prueba del teorema de Berge, cada componente de $M \oplus M^*$ tiene a lo más una arista más de M^* que de M, así que hay al menos $|M^*| - |M|$ componentes que son caminos aumentantes, y son ajenos en vértices. Cada uno tiene al menos ℓ aristas, o sea $\ell + 1$ vértices, y no hay más de n vértices.
3. Después de $\sqrt{n}$ fases, $\ell > \sqrt{n}$, y por el hecho 2 faltan menos de $n/\sqrt{n} = \sqrt{n}$ aristas. Cada fase agrega al menos una, así que quedan menos de $\sqrt{n}$ fases: a lo más $2\sqrt{n}$ en total.

El costo total es $\mathcal{O}(\sqrt{n}\, m)$, contra $\mathcal{O}(nm)$ de Emparejamiento-Máximo.

Ejemplo 11.18 (Hopcroft-Karp sobre alumnos y proyectos)**.** En el ejemplo 11.14 los caminos que aplicó Edmonds-Karp medían 1, 1, 3 y 5, y eso ya sugiere las fases. El DFS recorre los vecinos en orden alfabético.

Fase	ℓ	Caminos más cortos	Conjunto aplicado	$\lvert M\rvert$
1	1	las 7 aristas	A–1, B–3	2
2	3	C–1–A–2, C–3–B–4, D–1–A–2	C–1–A–2	3
3	5	D–1–C–3–B–4	el mismo	4

En la fase 1, A toma el 1 y B el 3, y C y D quedan bloqueados. ℓ crece estrictamente, 1, 3, 5, y hay tres fases en lugar de cuatro búsquedas. La fase 2 enseña la diferencia entre maximal y máximo para conjuntos de caminos: {C–1–A–2} es maximal, porque los otros dos caminos de longitud 3 chocan con él, pero C–3–B–4 y D–1–A–2 son ajenos y habrían terminado el trabajo en esa misma fase.

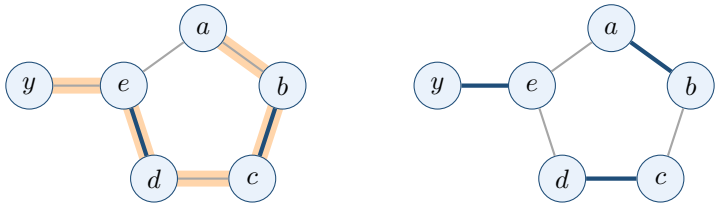
Ojito…

Hopcroft-Karp pide en cada fase un conjunto maximal de caminos, no uno máximo. Encontrar el máximo número de caminos ajenos sería otro problema de flujo; el maximal sale con un DFS voraz en $\mathcal{O}(m)$ y basta para la cota. La primera fase, con $M = \emptyset$ y $\ell = 1$, es exactamente el voraz del principio de la sección: construye un emparejamiento maximal.

Grafos que no son bipartitos

La reducción a flujo necesita la partición en L y R para orientar las aristas. En un triángulo el emparejamiento máximo mide 1, pero no hay manera de repartir sus vértices en dos lados. El teorema de Berge sigue valiendo, con camino aumentante definido como un camino alternante entre dos vértices libres: su prueba usó la bipartición solo para nombrar los extremos. Lo que se rompe es la búsqueda.

Ejemplo 11.19 (Una búsqueda alternante que se pierde)**.** El grafo es el ciclo a, b, c, d, e de longitud 5 más la arista $\{e, y\}$, con $M = \{\{b, c\}, \{d, e\}\}$. Los libres son a e y, y $P = a$–b–c–d–e–y es un camino aumentante: $M \oplus P = \{\{a, b\}, \{c, d\}, \{e, y\}\}$ es perfecto.



M y el camino P $\quad$ $M \oplus P$

Una búsqueda como Camino-Aumentante, que desde un vértice emparejado salta a su pareja, sale de a y descubre b y e por aristas fuera de M; de ahí salta a sus parejas, c y d. Desde c y d la única arista fuera de M es $\{c, d\}$, que une dos vértices ya vistos, y la búsqueda se detiene sin encontrar nada. El problema es e: se le marcó como visto al llegar desde a, en el papel de un vértice de R, y desde ahí solo se siguió a su pareja. El camino aumentante necesita llegar a e por su arista de M, desde d, y salir por $\{e, y\}$. En un bipartito cada vértice tiene un solo papel posible; aquí e tiene dos, porque el ciclo a, b, c, d, e es impar.

La arista $\{c, d\}$ une dos vértices a los que se llegó por su arista de M y cierra el ciclo impar; por el teorema 3.12, eso no puede pasar en un bipartito. Edmonds (1965) llamó **flor** (*blossom*) a un ciclo impar así, y su algoritmo la contrae a un solo vértice, sigue buscando en el grafo más chico y, cuando encuentra un camino aumentante, lo expande por el lado correcto de la flor. En el ejemplo, contraer a, b, c, d, e deja un vértice libre unido a y: un camino aumentante de longitud 1, que expandido es a–b–c–d–e–y. El algoritmo de flores corre en tiempo polinomial y queda fuera del capítulo 26 de Cormen.

Por eso, antes de aplicar la reducción hay que verificar que el grafo es bipartito y obtener L y R, con

el algoritmo 3.4 de la sección 3, en $\Theta(n + m)$. El grafo G de la libreta, por ejemplo, no es bipartito: tiene el ciclo A, B, E, F, C de longitud 5.

12. Empujar y reetiquetar

Complemento: Cormen, cap. 26.4 y 26.5

Resumen rápido. Ford-Fulkerson mejora un flujo buscando caminos completos de s a t. Los algoritmos de empujar y reetiquetar trabajan al revés: saturan de golpe todo lo que sale de la fuente y después cada vértice resuelve, mirando solo a sus vecinos, qué hacer con lo que le sobra. Lo empuja cuesta abajo, o sube de altura cuando no tiene a dónde empujarlo. Durante la ejecución no hay un flujo sino un preflujo, y una función de altura garantiza desde el primer paso que no existe camino aumentante; cuando ningún vértice interno guarda exceso, el preflujo es un flujo y es máximo. El algoritmo genérico cuesta $\mathcal{O}(n^2m)$ sin importar las capacidades. Reetiquetar al frente fija el orden de las operaciones con una lista que siempre es orden topológico de las aristas por las que se puede empujar, y baja a $\mathcal{O}(n^3)$. La sección cierra con una tabla de todos los algoritmos de la parte de grafos.

La idea: agua que corre cuesta abajo

El método de Ford-Fulkerson (algoritmo 10.1) mantiene un flujo válido en todo momento y, en cada iteración, busca en la red residual un camino completo de s a t. Para mover unas cuantas unidades mira la red entera. Los algoritmos de **empujar y reetiquetar** (*push-relabel*, de Goldberg y de Goldberg y Tarjan) son locales: operan sobre un vértice a la vez y solo miran las aristas residuales que salen de él. A cambio, durante la ejecución renuncian a la conservación de flujo. A un vértice le puede entrar más de lo que sale, y lo que sobra se queda guardado en él.

Cormen lo explica con fluidos. Las aristas son tuberías y los vértices son uniones de tuberías. Cada vértice tiene además un depósito donde se acumula el agua que entró y todavía no sale. Todo el sistema está montado sobre plataformas de altura ajustable, y el agua solo corre cuesta abajo. La fuente está fija a altura n y el sumidero a altura 0; los demás vértices empiezan en 0 y solo suben.

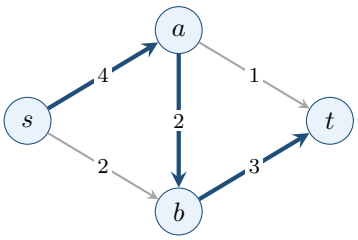
- Al inicio se abre la llave por completo: todas las tuberías que salen de s se llenan a su capacidad y el agua cae en los depósitos de los vecinos de s.
- Un vértice con agua en su depósito la empuja hacia un vecino que esté exactamente un nivel abajo, por una tubería que todavía tenga holgura.
- Si todas sus tuberías con holgura llevan a vecinos de su nivel o más altos, el vértice se levanta: se **reetiqueta** a un nivel por encima del más bajo de esos vecinos, y entonces sí puede vaciarse.

Lo que cabe hasta t llega a t. Cuando las tuberías que entran a t se saturan, el agua que sobra se queda atorada en los depósitos; esos vértices siguen subiendo, rebasan la altura de s y el sobrante regresa a la fuente por las mismas tuberías por las que llegó. Al final ningún depósito interno tiene agua y lo que llegó a t es un flujo máximo.

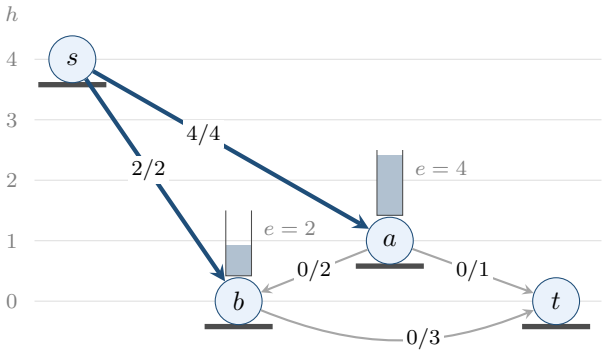
Idea clave

Ford-Fulkerson conserva la **factibilidad** (siempre tiene un flujo) y trabaja hasta alcanzar la **optimalidad** (que no quede camino aumentante). Empujar y reetiquetar conserva la optimalidad desde el primer paso (nunca hay camino de s a t en la red residual) y trabaja hasta alcanzar la factibilidad (que se vacíen los depósitos). Los dos terminan en el mismo punto, un flujo sin camino aumentante, que es máximo por el teorema de flujo máximo y corte mínimo (teorema 10.15).

Ejemplo 12.1 (Caminos contra tuberías)**.** La misma red de cuatro vértices vista por los dos métodos. A la izquierda, lo que hace Ford-Fulkerson: un camino aumentante de s a t, un chorro sin ramas. A la derecha, la red como tuberías justo después del primer reetiquetado del ejemplo 12.5: s en lo alto, las dos tuberías que salen de s saturadas, los depósitos de a y b con su exceso, y a ya levantado un nivel para vaciarse hacia b y t. b sigue al nivel de t y todavía no puede empujar.



Ford-Fulkerson: un camino s, a, b, t en G_f, un chorro sin ramas



Empujar y reetiquetar: s fijo a altura $n = 4$. a ya subió a 1 y puede vaciar su depósito cuesta abajo; b sigue al nivel de t

En esta edición Cormen describe las tuberías solo en prosa; la figura es propia. En todas las figuras que siguen la altura de cada vértice es su coordenada vertical, con líneas grises en cada nivel; dentro del círculo va el exceso y en cada arista el par f/c.

Preflujo, exceso y alturas

Se trabaja con una red de flujo $G = (V, E)$ como la de la sección 10: dirigida, con capacidades $c(u, v) \geq 0$, fuente s, sumidero t, sin aristas antiparalelas y con cada vértice en algún camino de s a t, de donde $m \geq n - 1$. La capacidad residual c_f y la red residual $G_f = (V, E_f)$ son las de la definición 10.5. Esa definición solo usa que $0 \leq f(u, v) \leq c(u, v)$, así que vale tal cual para lo que sigue.

Definición 12.2 (Preflujo)**.** Un **preflujo** (*preflow*) es una función $f : V \times V \to \mathbb{R}$ que cumple la restricción de capacidad, $0 \leq f(u, v) \leq c(u, v)$, y la conservación relajada

$$\sum_{v \in V} f(v, u) - \sum_{v \in V} f(u, v) \;\geq\; 0 \qquad \text{para toda } u \in V - \{s\}\,.$$

El **exceso** (*excess flow*) de u es lo que entra menos lo que sale,

$$e(u) = \sum_{v \in V} f(v, u) - \sum_{v \in V} f(u, v).$$

Un vértice $u \in V - \{s, t\}$ con $e(u) > 0$ está **desbordado** (*overflowing*).

En un preflujo $e(u) \geq 0$ para todo $u \neq s$. Como cada $f(x, y)$ suma en $e(y)$ y resta en $e(x)$, los excesos suman cero y por lo tanto $e(s) \leq 0$. Por definición s y t nunca están desbordados, aunque t acumule exceso: ahí es justamente donde debe terminar el agua. Un preflujo sin vértices desbordados cumple conservación en todo $V - \{s, t\}$, es decir, es un flujo.

Definición 12.3 (Función de altura)**.** Una **función de altura** (*height function*) es una función $h : V \to \mathbb{N}$ con

$$h(s) = n, \qquad h(t) = 0, \qquad h(u) \leq h(v) + 1 \;\text{ para toda arista residual } (u, v) \in E_f.$$

La condición dice que una arista residual puede subir cuanto quiera, pero baja a lo más un nivel. Leída al revés: si $h(u) > h(v) + 1$, entonces (u, v) no es arista residual (Cormen, lema 26.12). En la literatura a h se le llama también *distance function*: si en G_f hay un camino de u a t con k aristas, sumando la condición a lo largo de él sale $h(u) \leq k$, así que $h(u)$ nunca supera la distancia en aristas de u a t. La última parte de la sección vuelve sobre esto.

Observación. Cormen usa atributos en el pseudocódigo ($u.h$, $u.e$, $(u,v).f$, $u.N$, $u.current$) y funciones en los lemas ($h(u)$, $e(u)$, $f(u,v)$). Aquí se usa la forma funcional en todas partes: $h(u)$, $e(u)$, $f(u,v)$, $N(u)$ y actual(u).

Las dos operaciones básicas

Empujar

Empujar(u,v) aplica cuando u está desbordado, $c_f(u,v) > 0$ y $h(u) = h(v)+1$: hay holgura de u a v y v está exactamente un nivel abajo. Mueve $\Delta = \min(e(u), c_f(u,v))$ unidades de u a v. Es un **empuje saturante** si después $c_f(u,v) = 0$, y entonces (u,v) desaparece de la red residual; si no, es **no saturante**.

Algoritmo 12.1 Empujar(u,v) (Push en Cormen)

```
1: procedimiento EMPUJAR(u, v)          // aplica si e(u) > 0, c_f(u,v) > 0 y h(u) = h(v) + 1
2:     Δ ← mín(e(u), c_f(u,v))
3:     si (u,v) ∈ E entonces
4:         f(u,v) ← f(u,v) + Δ                                   // arista original
5:     si no
6:         f(v,u) ← f(v,u) − Δ                                   // arista de regreso
7:     fin si
8:     e(u) ← e(u) − Δ,    e(v) ← e(v) + Δ
9: fin procedimiento
```

- **Línea 2.** Δ es lo más que se puede mover sin dejar $e(u)$ negativo y sin rebasar la capacidad residual.
- **Líneas 3 a 7.** La arista residual (u,v) puede ser original o de regreso. Si es original, se aumenta su flujo. Si la original es (v,u), empujar de u a v cancela flujo que iba de v a u. Por eso Cormen prohíbe las antiparalelas: con (u,v) y (v,u) en E, la pregunta de la línea 3 no decide nada.
- El procedimiento no revisa alturas; $h(u) = h(v)+1$ es una precondición que garantiza quien lo llama. Cuesta $\mathcal{O}(1)$ si c_f se calcula en tiempo constante a partir de c y f.

Lema 12.4 (Cormen, lema 26.13)**.** *Después de un empuje no saturante de u a v, u ya no está desbordado.*

Demostración. Si el empuje no saturó, después queda $c_f(u,v) - \Delta > 0$, así que $\Delta \neq c_f(u,v)$ y el mínimo lo dio $e(u)$. Se empujó todo el exceso y $e(u)$ queda en 0. □

Reetiquetar

Reetiquetar(u) aplica cuando u está desbordado y $h(u) \leq h(v)$ para toda arista residual (u,v): tiene agua y no hay nada abajo hacia dónde empujarla.

Algoritmo 12.2 Reetiquetar(u) (Relabel en Cormen)

```
1: procedimiento REETIQUETAR(u)          // aplica si e(u) > 0 y h(u) ≤ h(v) para toda (u,v) ∈ E_f
2:     h(u) ← 1 + mín {h(v) : (u,v) ∈ E_f}
3: fin procedimiento
```

La nueva altura es la mayor que permite la condición $h(u) \leq h(v) + 1$ sobre las aristas residuales que salen de u. Con ella queda al menos una arista por la que se puede empujar: la que va al vecino que dio el mínimo. El conjunto sobre el que se toma el mínimo no es vacío. Como $e(u) > 0$, a u le entra flujo por alguna arista (v, u), y entonces $c_f(u, v) \geq f(v, u) > 0$. Recorrer las aristas residuales de u cuesta $\mathcal{O}(\text{gr}(u))$ con listas.

Inicializar el preflujo y el algoritmo genérico

Algoritmo 12.3 Inicializar-Preflujo(G, s)

```
1:  procedimiento Inicializar-Preflujo(G, s)
2:      para todo v ∈ V hacer
3:          h(v) ← 0,    e(v) ← 0
4:      fin para
5:      para todo (u, v) ∈ E hacer
6:          f(u, v) ← 0
7:      fin para
8:      h(s) ← n
9:      para todo v adyacente a s hacer
10:         f(s, v) ← c(s, v)                    // satura todo lo que sale de s
11:         e(v) ← c(s, v),    e(s) ← e(s) − c(s, v)
12:     fin para
13: fin procedimiento
```

Algoritmo 12.4 Empujar-Reetiquetar-Genérico(G) (Generic-Push-Relabel en Cormen)

```
1: procedimiento Empujar-Reetiquetar-Genérico(G)
2:     Inicializar-Preflujo(G, s)
3:     mientras existe una operación Empujar o Reetiquetar que aplique hacer
4:         elegir una cualquiera y ejecutarla
5:     fin mientras
6: fin procedimiento
```

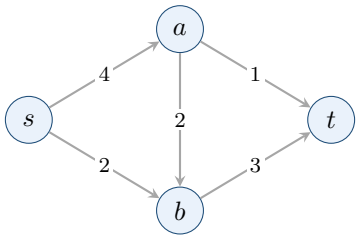
- **Líneas 9 a 11 de la inicialización.** Saturan todas las aristas que salen de s; es mandar la capacidad completa del corte $(\{s\}, V - \{s\})$, y por eso $e(s)$ queda en menos esa capacidad.
- $h(s) = n$ **es una altura válida** justamente porque esas aristas quedaron saturadas. Las únicas aristas con $h(u) > h(v) + 1$ son las que salen de s, y ninguna es residual. Las de regreso (v, s) suben, y subir está permitido.
- **El genérico no fija orden.** Cualquier secuencia de operaciones válidas termina en un flujo máximo. El orden cambia cuántas operaciones se hacen y cuál flujo máximo se obtiene, nunca su valor.

Ojito...

- **Se empuja exactamente un nivel.** Empujar exige $h(u) = h(v) + 1$, no "cualquier vecino más bajo". De todos modos no existen aristas residuales que bajen más de un nivel.
- **s y t nunca se empujan ni se reetiquetan.** $h(s) = n$ y $h(t) = 0$ quedan fijas. $e(t)$ solo crece.
- **A media ejecución f no es un flujo**, así que hablar de su valor $|f|$ no tiene sentido. El valor del flujo final es $e(t) = -e(s)$.

El genérico en una red de cuatro vértices

Ejemplo 12.5 (Empujar y reetiquetar genérico)**.** La red tiene $V = \{s, a, b, t\}$, $n = 4$, $m = 5$ y capacidades $c(s, a) = 4$, $c(s, b) = 2$, $c(a, b) = 2$, $c(a, t) = 1$, $c(b, t) = 3$. Su flujo máximo vale 4: el corte $(\{s, a, b\}, \{t\})$ tiene capacidad $1 + 3 = 4$.



Como el genérico no fija orden, hace falta una regla para que la traza sea reproducible: *se atiende el desbordado que va primero en el orden a, b; si tiene una arista admisible (residual y un nivel abajo) se empuja por la primera en el orden $\langle s, b, t\rangle$ para a y $\langle s, a, t\rangle$ para b; si no tiene, se reetiqueta.*

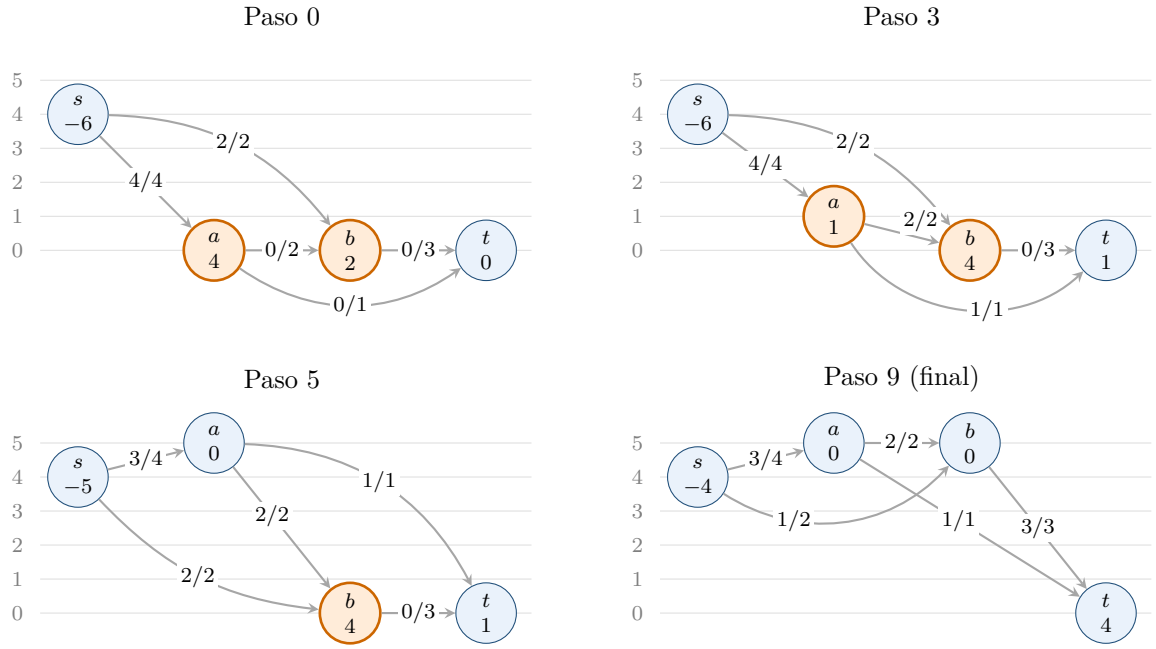
Paso	Operación	$h(a)$	$h(b)$	$e(s)$	$e(a)$	$e(b)$	$e(t)$
0	Inicializar-Preflujo	0	0	-6	4	2	0
1	Reetiquetar(a): $0 \to 1$	**1**	0	-6	4	2	0
2	Empujar(a, b): 2, saturante	1	0	-6	2	4	0
3	Empujar(a, t): 1, saturante	1	0	-6	1	4	1
4	Reetiquetar(a): $1 \to 5$	**5**	0	-6	1	4	1
5	Empujar(a, s): 1, no saturante	5	0	-5	0	4	1
6	Reetiquetar(b): $0 \to 1$	5	**1**	-5	0	4	1
7	Empujar(b, t): 3, saturante	5	1	-5	0	1	4
8	Reetiquetar(b): $1 \to 5$	5	**5**	-5	0	1	4
9	Empujar(b, s): 1, no saturante	5	5	-4	0	0	4

Paso a paso:

- **Paso 0.** $h(s) = 4$ y se saturan (s, a) y (s, b). Los dos internos quedan desbordados.
- **Paso 1.** a está a altura 0 y sus aristas residuales son (a, s) de regreso con $c_f = 4$, (a, b) con 2 y (a, t) con 1, hacia alturas 4, 0 y 0. Ninguna baja, así que se reetiqueta a $1 + \min\{4, 0, 0\} = 1$.
- **Pasos 2 y 3.** Ahora (a, b) y (a, t) son admisibles. Empuja $\min(4, 2) = 2$ a b y $\min(2, 1) = 1$ a t; los dos empujes saturan.
- **Paso 4.** A a le queda 1 y su única arista residual es (a, s), hacia altura 4. Se reetiqueta a 5: queda *arriba de la fuente*.
- **Paso 5.** Devuelve 1 a s. No satura, porque $c_f(a, s)$ baja de 4 a 3, y como dice el lema 12.4, a se vacía.

- **Pasos 6 a 9.** Lo mismo con b, que tiene 4. Sube a 1, manda 3 a t saturando (b,t), sube a $1+\min\{h(s),h(a)\}=1+\min\{4,5\}=5$ y devuelve 1 a s.

Cuatro instantáneas, cada vértice a su altura y, en naranja, los desbordados:



El resultado es $f(s,a)=3$, $f(s,b)=1$, $f(a,b)=2$, $f(a,t)=1$, $f(b,t)=3$, con valor $e(t)=-e(s)=4$.

Edmonds-Karp (algoritmo 10.3) resuelve la misma red con tres caminos completos: s,a,t con 1, s,b,t con 2 y s,a,b,t con 1. Empujar y reetiquetar nunca construyó un camino. Mandó 6 unidades hacia adelante, dejó que cada vértice resolviera con sus vecinos qué hacer con lo suyo y devolvió a la fuente las 2 que no cabían.

Las alturas finales también dicen dónde está el corte mínimo: $h(t)=0$, $h(s)=4$ y $h(a)=h(b)=5$. Ningún vértice quedó a altura 1, y ninguna arista residual puede bajar de un nivel mayor que 1 a uno menor que 1 sin pasar por él, así que $\{v : h(v)>1\}=\{s,a,b\}$ es el lado de s de un corte mínimo. Cormen lo generaliza en el ejercicio 26.4-4.

Ojito...

Las alturas sí pasan de n. Es tentador pensar que nadie sube más que la fuente, y no es así. El exceso que no puede llegar a t solo regresa a s si los vértices que lo guardan rebasan $h(s)$: aquí a y b terminan en 5 con $n=4$. El tope real es $2n-1$, y se prueba más abajo.

Por qué el resultado es un flujo máximo

La corrección descansa en dos hechos: a un vértice desbordado siempre se le puede hacer algo, y una función de altura impide que haya camino de s a t en la red residual.

Lema 12.6 (Cormen, lema 26.14)**.** *Si f es un preflujo, h una función de altura y u está desbordado, entonces a u le aplica un empuje o un reetiquetado.*

Demostración. Para toda arista residual (u,v) se tiene $h(u)\le h(v)+1$. Si ningún empuje aplica, ninguna cumple la igualdad, así que $h(u)<h(v)+1$, es decir, $h(u)\le h(v)$, para todas ellas. Esa es la

condición de Reetiquetar. El mínimo se toma sobre un conjunto no vacío porque $e(u) > 0$ obliga a que entre flujo por alguna (v, u), y entonces $(u, v) \in E_f$. □

Lema 12.7 (Cormen, lemas 26.15 y 26.16)**.** *Durante el algoritmo genérico las alturas nunca bajan, cada reetiquetado sube $h(u)$ en al menos 1, y h es una función de altura en todo momento.*

Idea de la prueba. Solo Reetiquetar cambia alturas. Antes de reetiquetar, $h(u) \leq h(v)$ para todo vecino residual, así que $h(u) < 1 + \min h(v)$: la altura sube. Que h sigue siendo función de altura se prueba por inducción en el número de operaciones; tras la inicialización ya lo es.

- **Reetiquetar u.** Las aristas residuales que salen de u cumplen la condición por la elección del mínimo. Para una que entra, (w, u), ya se tenía $h(w) \leq h(u) + 1$, y $h(u)$ solo subió.
- **Empujar de u a v.** Puede crear la arista residual (v, u), que cumple porque $h(v) = h(u) - 1 < h(u) + 1$, y puede quitar (u, v), lo que solo elimina una restricción. □

Lema 12.8 (Cormen, lema 26.17)**.** *Si f es un preflujo y h una función de altura, no existe camino de s a t en la red residual G_f.*

Demostración. Supóngase que hay uno, $\langle v_0, v_1, \ldots, v_k \rangle$ con $v_0 = s$ y $v_k = t$. Se puede tomar simple, así que tiene a lo más n vértices y $k \leq n - 1$ aristas. Cada (v_i, v_{i+1}) es arista residual, luego $h(v_i) \leq h(v_{i+1}) + 1$. Sumando las k desigualdades, los términos intermedios se cancelan y queda

$$h(s) \;\leq\; h(t) + k \;=\; k \;\leq\; n - 1,$$

lo que contradice $h(s) = n$. □

La prueba es una cuenta de escalones: de s a t hay que bajar n niveles, cada arista residual baja a lo más uno y un camino simple no tiene n aristas.

Teorema 12.9 (Corrección del genérico; Cormen, teorema 26.18)**.** *Si el algoritmo genérico (algoritmo 12.4) termina, el preflujo que calcula es un flujo máximo.*

Demostración. El invariante es "f es un preflujo". Vale tras la inicialización. Reetiquetar no toca f. Empujar lo conserva: $\Delta \leq c_f(u, v)$ respeta la capacidad y $\Delta \leq e(u)$ deja $e(u) \geq 0$. Al terminar no queda ningún vértice desbordado, porque si hubiera uno le aplicaría alguna operación (lema 12.6) y el ciclo no habría terminado. Entonces $e(u) = 0$ en todo $V - \{s, t\}$ y f es un flujo. Por el lema 12.7, h sigue siendo función de altura, y por el lema 12.8 no hay camino aumentante en G_f. Por el teorema 10.15, f es un flujo máximo. □

Falta ver que termina. Eso sale de las cotas de la siguiente subsección, que no dependen de las capacidades: el genérico termina siempre, incluso con capacidades irracionales, caso en el que Ford-Fulkerson con caminos arbitrarios puede no terminar.

Cuánto cuesta

Todo el análisis cuelga de un tope para las alturas, y el tope sale de que todo exceso vino de s y tiene por dónde regresar.

Lema 12.10 (Cormen, lema 26.19)**.** *Si x está desbordado, hay un camino simple de x a s en G_f.*

Demostración. Sea U el conjunto de vértices alcanzables desde x en G_f y supóngase $s \notin U$. Al sumar los excesos de los vértices de U, cada $f(x', y')$ con los dos extremos en U suma en uno y resta en otro, y queda

$$\sum_{u \in U} e(u) = \sum_{u \in U} \sum_{v \notin U} f(v, u) - \sum_{u \in U} \sum_{v \notin U} f(u, v).$$

El lado izquierdo es positivo, porque $e(x) > 0$ y ningún vértice de U tiene exceso negativo (el único que puede tenerlo es s). Como el segundo término de la derecha no es negativo, el primero es positivo: hay $u' \in U$ y $v' \notin U$ con $f(v', u') > 0$. Pero entonces $c_f(u', v') \geq f(v', u') > 0$, la arista de regreso (u', v') es residual y v' sería alcanzable desde x. Esto contradice $v' \notin U$. □

Lema 12.11 (Tope de las alturas; Cormen, lema 26.20)**.** *En todo momento $h(u) \leq 2n - 1$ para todo $u \in V$.*

Demostración. $h(s) = n$ y $h(t) = 0$ no cambian. Un vértice interno u solo cambia de altura al reetiquetarse, y en ese momento está desbordado. Reetiquetar no cambia f, así que después sigue desbordado y, por el lema 12.10, hay un camino simple $\langle u = v_0, \ldots, v_k = s \rangle$ en G_f con $k \leq n - 1$ aristas. Sumando $h(v_i) \leq h(v_{i+1}) + 1$ a lo largo del camino, igual que en el lema 12.8,

$$h(u) \;\leq\; h(s) + k \;\leq\; n + (n - 1) \;=\; 2n - 1.$$ □

Corolario 12.12 (Reetiquetados; Cormen, corolario 26.21)**.** *Cada vértice se reetiqueta a lo más $2n - 1$ veces, y hay a lo más $(2n - 1)(n - 2) < 2n^2$ reetiquetados en total.*

Idea de la prueba. Solo los $n - 2$ vértices internos se reetiquetan. Cada uno empieza en 0, sube al menos 1 por reetiquetado y no pasa de $2n - 1$. □

Lema 12.13 (Empujes saturantes; Cormen, lema 26.22)**.** *El algoritmo genérico hace menos de $2nm$ empujes saturantes.*

Idea de la prueba. Se cuentan juntos los empujes saturantes entre u y v en los dos sentidos; para que haya alguno, (u, v) o (v, u) está en E. Tras uno de u a v queda $h(v) = h(u) - 1$ y la arista (u, v) deja de ser residual. Para volver a empujar de u a v, primero tiene que empujarse algo de v a u, lo que exige $h(v) = h(u) + 1$. Como $h(u)$ no baja, $h(v)$ tuvo que subir al menos 2. Lo mismo vale para $h(u)$ entre dos saturantes de v a u. Así, entre dos saturantes consecutivos del par, uno de los dos vértices subió al menos 2 niveles. Con alturas entre 0 y $2n - 1$, cada vértice sube de dos en dos menos de n veces, y el par admite menos de $2n$ saturantes. Los pares que cuentan son las m aristas de E. □

El ejemplo 12.16 muestra esta escalera en acción. El último conteo es el que domina, y su prueba usa una herramienta que vale la pena ver completa: una **función potencial**, un número que nunca es negativo, que algunas operaciones suben y otras bajan. Si se sabe cuánto puede subir en total y cuánto baja cada operación de cierto tipo, se acota cuántas de esas operaciones hay.

Lema 12.14 (Empujes no saturantes; Cormen, lema 26.23)**.** *El algoritmo genérico hace menos de $4n^2(n + m)$ empujes no saturantes.*

Demostración. Sea

$$\Phi \;=\; \sum_{v \text{ desbordado}} h(v),$$

la suma de las alturas de los vértices desbordados. Tras la inicialización los desbordados son los vecinos de s, todos a altura 0, así que $\Phi = 0$; y como las alturas son naturales, $\Phi \geq 0$ siempre. Se revisa cómo cambia Φ con cada tipo de operación.

- **Reetiquetar u.** u está desbordado antes y después, el conjunto de desbordados no cambia y solo cambia $h(u)$. Φ sube exactamente lo que sube $h(u)$. La altura de cada interno empieza en 0, nunca baja y no pasa de $2n-1$ (lema 12.11), así que la suma de todas estas subidas en toda la ejecución es a lo más $(n-2)(2n-1) < 2n^2$.
- **Empuje saturante de u a v.** Ninguna altura cambia. u puede dejar de estar desbordado, lo que solo resta, y v puede volverse desbordado, lo que suma $h(v) \leq 2n-1$. Cada uno sube Φ menos de $2n$, y por el lema 12.13 hay menos de $2nm$, así que en total suben Φ menos de $4n^2m$.
- **Empuje no saturante de u a v.** Ninguna altura cambia. Por el lema 12.4, u deja de estar desbordado y su término $h(u)$ sale de la suma. A lo más entra v, si antes no estaba desbordado y no es s ni t, con $h(v) = h(u) - 1$. El cambio es a lo más $-h(u) + (h(u)-1) = -1$: cada no saturante baja Φ en al menos 1.

Sea N el número de empujes no saturantes. Φ empieza en 0, sube en total menos de $2n^2 + 4n^2m$, baja al menos N y termina en un valor no negativo:

$$0 \;\leq\; \Phi_{\text{final}} \;<\; 2n^2 + 4n^2m - N, \qquad \text{luego} \qquad N \;<\; 2n^2 + 4n^2m \;\leq\; 4n^2(n+m). \qquad \square$$

Cormen acota la subida por reetiquetados con $2n \cdot 2n^2$ en lugar de $2n^2$, y por eso enuncia la cota como $4n^2(n+m)$; la cuenta de arriba es un poco más fina, pero la conclusión asintótica es la misma.

Teorema 12.15 (Cormen, teorema 26.24 y corolario 26.25)**.** *El algoritmo genérico hace $\mathcal{O}(n^2m)$ operaciones básicas y se puede implementar en tiempo $\mathcal{O}(n^2m)$.*

Derivación. Se cuentan las operaciones y después cuánto cuesta cada una con la estructura adecuada.

1. *Reetiquetados.* Menos de $2n^2$ (corolario 12.12). Cada uno recorre los vecinos de u en $\mathcal{O}(\text{gr}(u))$. Como u se reetiqueta a lo más $2n-1$ veces, el total es $\sum_u (2n-1)\,\text{gr}(u) = (2n-1)\cdot 2m = \mathcal{O}(nm)$, con la misma suma de grados que da el $\Theta(n+m)$ de BFS.
2. *Empujes saturantes.* Menos de $2nm$, a $\mathcal{O}(1)$ cada uno: $\mathcal{O}(nm)$.
3. *Empujes no saturantes.* Menos de $4n^2(n+m)$, a $\mathcal{O}(1)$ cada uno. Como $m \geq n-1$, esto es $\mathcal{O}(n^2m)$.
4. *Elegir la operación.* Los desbordados se guardan en una pila o en una cola, y para el vértice de turno se usa un apuntador al vecino que se está revisando, como en Descargar (algoritmo 12.5). El apuntador recorre la lista de u una vez entre dos reetiquetados de u: otra vez $\mathcal{O}(nm)$ en total. Meter y sacar de la pila cuesta $\mathcal{O}(1)$ por operación.

Total: $\mathcal{O}(nm + nm + n^2m + nm) = \mathcal{O}(n^2m)$. Domina el conteo de empujes no saturantes, y ninguna cuenta depende de las capacidades. $\square$

Ojito...

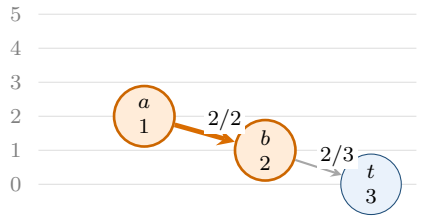
El paso de $4n^2(n+m)$ a $\mathcal{O}(n^2m)$ usa $m \geq n-1$, que sale del supuesto de que todo vértice está en un camino de s a t; Cormen no lo dice en la prueba. Si hay vértices que no llegan a t, el algoritmo funciona igual: lo que les entra sube y regresa a s. Lo único que se pierde es poder escribir la cota como $\mathcal{O}(n^2m)$; queda $\mathcal{O}(n^2(n+m))$.

Ejemplo 12.16 (La misma red, empezando por b: ping-pong)**.** Misma red y misma regla que en el ejemplo 12.5, pero atendiendo primero a b. Ahora son 15 operaciones (7 reetiquetados, 4 saturantes y 4 no saturantes) en lugar de 9, y el valor sigue siendo 4.

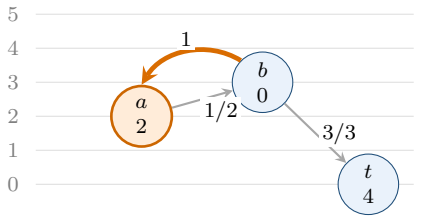
Paso	Operación	$h(a)$	$h(b)$	$e(s)$	$e(a)$	$e(b)$	$e(t)$
0	Inicializar-Preflujo	0	0	-6	4	2	0
1	Reetiquetar(b): $0 \to 1$	0	**1**	-6	4	2	0
2	Empujar(b, t): 2, no saturante	0	1	-6	4	0	2
3	Reetiquetar(a): $0 \to 1$	**1**	1	-6	4	0	2
4	Empujar(a, t): 1, saturante	1	1	-6	3	0	3
5	Reetiquetar(a): $1 \to 2$	**2**	1	-6	3	0	3
6	Empujar(a, b): 2, saturante	2	1	-6	1	2	3
7	Empujar(b, t): 1, saturante	2	1	-6	1	1	4
8	Reetiquetar(b): $1 \to 3$	2	**3**	-6	1	1	4
9	Empujar(b, a): 1, no saturante	2	3	-6	2	0	4
10	Reetiquetar(a): $2 \to 4$	**4**	3	-6	2	0	4
11	Empujar(a, b): 1, saturante	4	3	-6	1	1	4
12	Reetiquetar(b): $3 \to 5$	4	**5**	-6	1	1	4
13	Empujar(b, s): 1, no saturante	4	5	-5	1	0	4
14	Reetiquetar(a): $4 \to 5$	**5**	5	-5	1	0	4
15	Empujar(a, s): 1, no saturante	5	5	-4	0	0	4

Tres instantáneas del ping-pong entre a y b; solo se dibujan a, b y t, y en naranja grueso el empuje de ese paso.

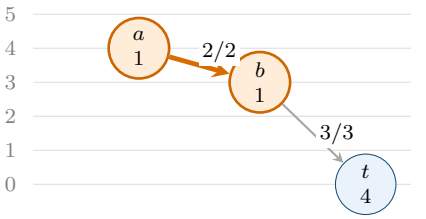
Paso 6: a empuja a b



Paso 9: b devuelve a a



Paso 11: a empuja a b



a satura (a, b) en el paso 6 y otra vez en el 11. Entre los dos, b tuvo que devolverle 1 por la arista de regreso (paso 9, que baja $f(a, b)$ de 2 a 1), y para eso subió de 1 a 3; después a tuvo que subir de 2 a 4 para volver a empujar. Cada ida y vuelta le cuesta dos niveles a cada uno: es exactamente el argumento del lema 12.13. Como las alturas están topadas en $2n - 1$, el ping-pong no puede durar para siempre; al final los dos rebasan $h(s) = 4$ y el sobrante regresa a la fuente.

En este caso hasta el flujo final sale igual que con el otro orden, aunque la red tiene otro flujo máximo entero, con $f(a, b) = 1$ y $f(s, a) = f(s, b) = 2$. En general el orden sí puede cambiar el flujo que se obtiene, nunca su valor. Los conteos de los dos ejemplos quedan muy por debajo de las cotas, que con $n = 4$ y $m = 5$ son 14 reetiquetados, 40 saturantes y 576 no saturantes: las cotas son para el peor caso y en redes chicas son holgadas.

Reetiquetar al frente

El genérico deja libre el orden de las operaciones, y el orden importa: en la misma red un orden hizo 9 operaciones y otro 15. **Reetiquetar al frente** (*relabel-to-front*) fija uno con dos ideas. La primera es atender un vértice hasta vaciarlo, en vez de alternar entre vértices. La segunda es el orden en que se atienden: una lista L de los vértices internos que se recorre de principio a fin, y cada vez que un vértice sube de altura pasa al frente de la lista y el recorrido sigue detrás de él. Para ver por qué ese orden

funciona hay que mirar qué aristas sirven para empujar.

Aristas admisibles

Definición 12.17 (Arista admisible)**.** Con un preflujo f y una función de altura h, una arista (u, v) es **admisible** (*admissible*) si $c_f(u, v) > 0$ y $h(u) = h(v) + 1$; las demás son **inadmisibles**. La **red admisible** es $G_{f,h} = (V, E_{f,h})$, con $E_{f,h}$ el conjunto de aristas admisibles.

Las aristas admisibles son exactamente aquellas por las que se puede empujar. Tres hechos sobre ellas sostienen todo el algoritmo.

Lema 12.18 (Cormen, lema 26.26)**.** *La red admisible $G_{f,h}$ es acíclica.*

Demostración. Supóngase un ciclo $\langle v_0, v_1, \ldots, v_k \rangle$ de aristas admisibles, con $v_0 = v_k$ y $k > 0$. Cada arista cumple $h(v_{i-1}) = h(v_i) + 1$. Sumando las k igualdades,

$$\sum_{i=1}^{k} h(v_{i-1}) = \sum_{i=1}^{k} h(v_i) + k.$$

Las dos sumas recorren los mismos vértices del ciclo, cada uno una vez, y son iguales. Queda $0 = k$, una contradicción. □

Lema 12.19 (Cormen, lemas 26.27 y 26.28)**.**

1. *Empujar de u a v no crea aristas admisibles, aunque puede quitar (u, v).*
2. *Si u está desbordado y no tiene aristas admisibles de salida, Reetiquetar(u) aplica. Después, u tiene al menos una arista admisible de salida y ninguna de entrada.*

Demostración. (1) La única arista residual que puede aparecer al empujar es (v, u), y no es admisible porque $h(v) = h(u) - 1$. Las alturas no cambian.

(2) Sin admisibles de salida, ningún empuje aplica a u y, por el lema 12.6, aplica el reetiquetado. Después, la arista hacia el vecino que dio el mínimo es admisible. Si hubiera una admisible de entrada (v, u), se tendría $h(v) = h(u) + 1$ con la altura nueva de u. Como $h(u)$ subió al menos 1, antes era $h(v) > h(u) + 1$ y (v, u) no era residual. Reetiquetar no cambia la red residual, así que tampoco lo es después. □

Listas de vecinos y la operación Descargar

La **lista de vecinos** $N(u)$ de un vértice interno contiene cada v con $(u, v) \in E$ o $(v, u) \in E$: los posibles extremos de una arista residual que sale de u. Su orden es arbitrario pero fijo durante todo el algoritmo. El apuntador actual(u) señala el vecino que se está revisando; empieza en el primero y vale nulo cuando se pasa del último.

Descargar un vértice desbordado es empujar todo su exceso por aristas admisibles, reetiquetándolo cuando haga falta.

Algoritmo 12.5 Descargar(u) (Discharge en Cormen)

```
1:  procedimiento DESCARGAR(u)
2:      mientras e(u) > 0 hacer
3:          v ← actual(u)
4:          si v = nulo entonces                          // se recorrió toda N(u)
5:              REETIQUETAR(u)
6:              actual(u) ← primer vértice de N(u)
7:          si no si c_f(u, v) > 0 y h(u) = h(v) + 1 entonces
8:              EMPUJAR(u, v)                             // (u, v) es admisible
9:          si no
10:             actual(u) ← vértice que sigue a v en N(u)
11:         fin si
12:     fin mientras
13: fin procedimiento
```

Cada vuelta del mientras hace exactamente una de tres cosas:

1. v **nulo (líneas 4 a 6).** Se acabó la lista: todas las aristas de salida resultaron inadmisibles en esta pasada. Se reetiqueta u y el apuntador regresa al inicio.
2. (u, v) **admisible (líneas 7 y 8).** Se empuja, y el apuntador *no* avanza. Si el empuje no saturó, $e(u) = 0$ y la llamada termina; si saturó, en la vuelta siguiente la arista aparece como inadmisible y entonces sí avanza.
3. (u, v) **inadmisible (línea 10).** Se avanza al siguiente vecino.

Ni reetiquetar ni avanzar cambian $e(u)$, así que la última acción de toda llamada sobre un vértice desbordado es un empuje. Y actual(u) se conserva entre llamadas: la siguiente descarga de u sigue donde se quedó la anterior.

Lema 12.20 (Cormen, lema 26.29)**.** *Cuando* Descargar(u) *llama a* Reetiquetar(u) *en la línea 5, el reetiquetado aplica.*

Idea de la prueba. Los empujes están protegidos por la condición de la línea 7. Para el reetiquetado, basta ver que u no tiene aristas admisibles de salida (lema 12.19). En la pasada completa por $N(u)$ que acaba de terminar, y que pudo repartirse entre varias llamadas porque el apuntador no se reinicia, cada arista de salida se encontró inadmisible en algún momento. Ninguna pudo volverse admisible después: empujar no crea admisibles, u no se reetiquetó en esa pasada, y un vértice v que se reetiquetó no tiene admisibles de entrada, en particular no tiene (u, v). □

El algoritmo

Algoritmo 12.6 Reetiquetar-al-Frente(G, s, t) (Relabel-To-Front en Cormen)

```
1:  procedimiento REETIQUETAR-AL-FRENTE(G, s, t)
2:      INICIALIZAR-PREFLUJO(G, s)
3:      L ← V − {s, t} en cualquier orden
4:      para todo u ∈ V − {s, t} hacer
5:          actual(u) ← primer vértice de N(u)
6:      fin para
7:      u ← primer vértice de L
8:      mientras u ≠ nulo hacer
9:          h_antes ← h(u)
10:         DESCARGAR(u)
11:         si h(u) > h_antes entonces                    // u se reetiquetó
12:             mover u al frente de L
13:         fin si
14:         u ← vértice que sigue a u en L
15:     fin mientras
16: fin procedimiento
```

- **Línea 3.** L tiene solo los vértices que pueden desbordarse; s y t quedan fuera.
- **Líneas 9 a 13.** h_{antes} sirve para saber si Descargar reetiquetó a u. Si lo hizo, u pasa al frente de L.
- **Línea 14.** Se sigue con el sucesor de u en su posición *actual*. Si u acaba de pasar al frente, el recorrido vuelve a pasar por todos los que quedaron detrás de él.

Teorema 12.21 (Invariante y corrección de reetiquetar al frente)**.** *Cada vez que se evalúa la condición de la línea 8, L es un orden topológico de la red admisible $G_{f,h}$ y ningún vértice anterior a u en L tiene exceso. En consecuencia, al terminar no hay vértices desbordados y f es un flujo máximo.*

Idea de la prueba. Que $G_{f,h}$ tenga orden topológico lo garantiza el lema 12.18. Al inicio no hay aristas admisibles: $h(s) = n \geq 2$ y todos los demás están en 0, así que cualquier orden sirve. Empujar no crea admisibles. Reetiquetar sí, pero solo de salida del vértice reetiquetado, que además se queda sin admisibles de entrada (lema 12.19): ponerlo al frente de L conserva el orden topológico. Para la segunda parte, Descargar(u) empuja solo por aristas admisibles, que en L van hacia adelante (o a s o t, que no están en L); los vértices anteriores a u no reciben nada y siguen sin exceso, y u queda vacío. Si u pasó al frente, no hay nadie antes de él. Cuando el recorrido se sale de la lista ($u =$ nulo), ningún vértice de L tiene exceso y el teorema 12.9 da la conclusión. □

Por qué cuesta $\mathcal{O}(n^3)$

Se llama **fase** al tramo de ejecución entre dos reetiquetados consecutivos (Cormen, teorema 26.30).

1. *Llamadas a Descargar.* Hay menos de $2n^2$ reetiquetados, luego $\mathcal{O}(n^2)$ fases. Dentro de una fase, una descarga que no reetiqueta deja a u en su lugar y la siguiente es sobre un vértice más adelante en L; una que reetiqueta termina la fase. Como L tiene $n-2$ vértices, una fase tiene a lo más $n-1$ llamadas, y hay $\mathcal{O}(n^3)$ en total. Eso también acota el trabajo del mientras de la línea 8 fuera de Descargar.

2. *Reetiquetados dentro de Descargar.* $\mathcal{O}(nm)$ en total, con la misma cuenta que en el teorema 12.15.
3. *Avances del apuntador.* Entre dos reetiquetados de u, actual(u) recorre $N(u)$ una vez: $\mathcal{O}(\text{gr}(u))$ por reetiquetado, $\mathcal{O}(n\,\text{gr}(u))$ por vértice y $\mathcal{O}(nm)$ en total.
4. *Empujes saturantes.* $\mathcal{O}(nm)$ (lema 12.13).
5. *Empujes no saturantes.* Uno no saturante deja $e(u) = 0$ y termina la llamada, así que hay a lo más uno por llamada: $\mathcal{O}(n^3)$.

El total es $\mathcal{O}(n^3 + nm)$, que es $\mathcal{O}(n^3)$ porque sin antiparalelas $m \leq n(n-1)/2$. Comparado con el genérico, como $m \geq n-1$, $\mathcal{O}(n^3)$ nunca es peor que $\mathcal{O}(n^2 m)$: empatan en grafos dispersos y reetiquetar al frente gana en densos. La mejora está toda en el quinto renglón. El orden de L topa los empujes no saturantes en uno por llamada, en lugar de los $\mathcal{O}(n^2 m)$ que permite la función potencial.

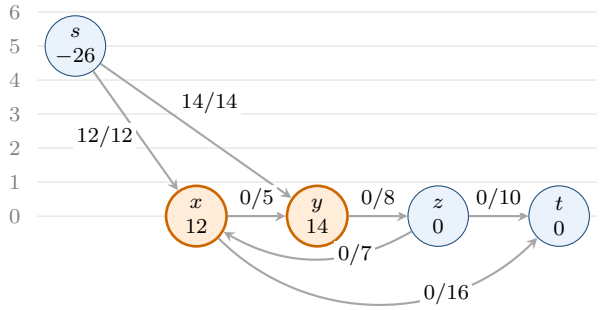
Ojito...

En reetiquetar al frente el apuntador actual(u) **no se reinicia** entre llamadas a Descargar. Si se reiniciara en cada llamada el algoritmo seguiría siendo correcto, pero se perdería la cuenta de avances que da el $\mathcal{O}(nm)$: cada llamada podría volver a recorrer la lista completa.

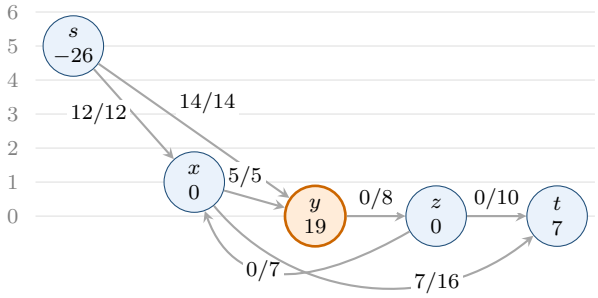
La figura 26.10 de Cormen

Ejemplo 12.22 (Reetiquetar al frente paso a paso)**.** La red tiene $V = \{s, x, y, z, t\}$, $n = 5$ y capacidades $c(s,x) = 12$, $c(s,y) = 14$, $c(x,y) = 5$, $c(y,z) = 8$, $c(z,x) = 7$, $c(z,t) = 10$, $c(x,t) = 16$. Su flujo máximo vale 20. Las listas de vecinos son las de Cormen: $N(x) = \langle s, y, z, t\rangle$, $N(y) = \langle s, x, z\rangle$, $N(z) = \langle x, y, t\rangle$, y la lista inicial es $L = \langle x, y, z\rangle$. En las instantáneas, los desbordados van en naranja.

(a) inicio, $L = \langle x, y, z\rangle$

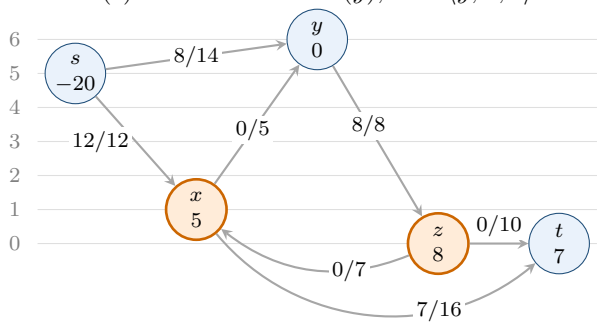


(b) tras Descargar(x)

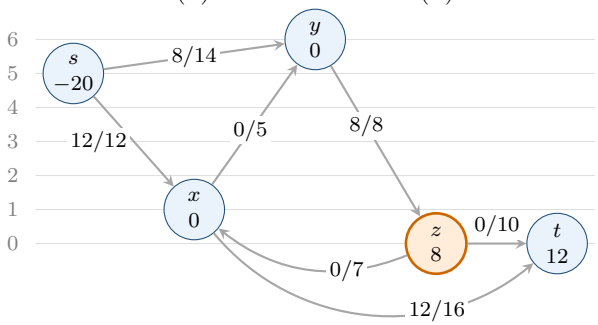


De (a) a (b). Se descarga x. No tiene admisibles, sube a 1, empuja 5 a y (satura (x,y)) y 7 a t. Se reetiquetó, así que pasa al frente, donde ya estaba; sigue y.

(c) tras Descargar(y), $L = \langle y, x, z \rangle$



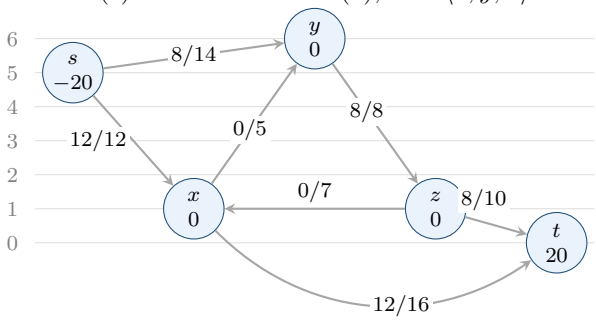
(d) tras Descargar(x)



De (b) a (c). Se descarga y con 19 unidades: sube a 1, manda 8 a z, sube a 2, devuelve 5 a x cancelando el flujo de (x, y), sube a 6 (arriba de s) y devuelve 6 a la fuente. Pasa al frente: $L = \langle y, x, z \rangle$.

De (c) a (d). Sigue x, que recibió 5. Su apuntador se había quedado en t desde su descarga anterior, y (x, t) es admisible: empuja 5 a t sin reetiquetarse y se queda en su lugar.

(e) tras Descargar(z), $L = \langle z, y, x \rangle$



De (d) a (e). Sigue z con 8. Sube a 1 y manda 8 a t. Pasa al frente: $L = \langle z, y, x \rangle$. Después se recorren y y x sin exceso y la lista se acaba. El flujo final tiene $f(s, x) = 12$, $f(s, y) = 8$, $f(x, t) = 12$, $f(y, z) = 8$, $f(z, t) = 8$ y $f(x, y) = f(z, x) = 0$, con valor 20. Hubo 5 reetiquetados, 3 empujes saturantes y 4 no saturantes.

El estado después de cada llamada a Descargar:

	Descarga	**Reetiq.**	L después	$h(x)$	$h(y)$	$h(z)$	$e(s)$	$e(x)$	$e(y)$	$e(z)$	$e(t)$
(a)	inicio		x, y, z	0	0	0	−26	12	14	0	0
(b)	x	sí, $0 \to 1$	x, y, z	1	0	0	−26	0	19	0	7
(c)	y	sí, $0 \to 6$	y, x, z	1	6	0	−20	5	0	8	7
(d)	x	no	y, x, z	1	6	0	−20	0	0	8	12
(e)	z	sí, $0 \to 1$	z, y, x	1	6	1	−20	0	0	0	20
	y, luego x	no	z, y, x	1	6	1	−20	0	0	0	20

La descarga de y es la figura 26.9 de Cormen. Al empezar, $h(s) = 5$, $h(x) = 1$ y $h(z) = 0$; la columna de acción dice a qué vecino apunta actual(y) y qué se hace en cada vuelta del mientras.

Vuelta	**actual**(y) **y acción**	$h(y)$	$e(y)$
1	s: $h(y) = 0 \neq h(s) + 1$, avanza	0	19
2	x: $h(x) = 1$, avanza	0	19
3	z: $h(z) = 0 = h(y)$, avanza	0	19
4	nulo: Reetiquetar, $1 + \text{mín}\{5, 1, 0\}$	1	19
5	s: avanza	1	19
6	x: $h(x) = h(y)$, avanza	1	19
7	z: **empuja 8**, satura (y, z)	1	11
8	z: $c_f(y, z) = 0$, avanza	1	11
9	nulo: Reetiquetar, $1 + \text{mín}\{5, 1\}$	2	11
10	s: avanza	2	11
11	x: **empuja 5** de regreso, $f(x, y)$ vuelve a 0	2	6
12	x: $c_f(y, x) = 0$, avanza	2	6
13	z: $c_f(y, z) = 0$, avanza	2	6
14	nulo: Reetiquetar, $1 + \text{mín}\{5\}$	6	6
15	s: **empuja 6** de regreso a la fuente	6	0

Las vueltas 7 y 11 muestran por qué el apuntador no avanza tras un empuje: en la vuelta siguiente la arista ya saturada se descubre inadmisible y entonces sí avanza. La vuelta 11 es un empuje por arista de regreso: $(x, y) \in E$ y y le devuelve a x las 5 unidades que x le había mandado.

Comparación con Ford-Fulkerson y Edmonds-Karp

Los dos enfoques llegan al mismo certificado de optimalidad por lados opuestos. La primera tabla compara lo que mantiene cada uno; la segunda, cuánto cuesta.

	Ford-Fulkerson y Edmonds-Karp	**Empujar y reetiquetar**
Mantiene	un flujo: conserva en cada vértice	un preflujo: a cada vértice le entra al menos lo que le sale
Paso básico	aumentar por un camino completo de s a t en G_f	empujar o reetiquetar en un vértice, mirando solo a sus vecinos
Invariante	f es un flujo factible	no hay camino de s a t en G_f
Termina cuando	ya no hay camino aumentante	ya no hay vértices desbordados
Certificado	el corte de los alcanzables desde s en G_f	el mismo corte, o el que da un hueco en las alturas

Algoritmo	**Qué elige**	**Costo**	**Denso**	**Disperso**	**Usa** c
Ford-Fulkerson	cualquier camino aumentante	$\mathcal{O}(m\,\lvert f^*\rvert)$	depende de $\lvert f^*\rvert$		sí
Edmonds-Karp	el camino aumentante más corto (BFS)	$\mathcal{O}(nm^2)$	$\mathcal{O}(n^5)$	$\mathcal{O}(n^3)$	no
Genérico	cualquier operación que aplique	$\mathcal{O}(n^2m)$	$\mathcal{O}(n^4)$	$\mathcal{O}(n^3)$	no
Reetiquetar al frente	descargar en el orden de L	$\mathcal{O}(n^3)$	$\mathcal{O}(n^3)$	$\mathcal{O}(n^3)$	no

"Denso" quiere decir $m = \Theta(n^2)$ y "disperso" $m = \Theta(n)$. La cota de Ford-Fulkerson vale con capacidades enteras y depende del valor $|f^*|$ del flujo máximo, no del tamaño de la red; la columna "Usa c" dice si

la cota depende de las capacidades. En grafos densos la diferencia entre n^5 y n^3 es la razón de ser de esta sección.

En la práctica

Las notas del capítulo de Cormen dicen que las variantes de empujar y reetiquetar con buenas heurísticas son los algoritmos de flujo máximo más rápidos en la práctica. Hay dos órdenes alternativos con la misma cota $\mathcal{O}(n^3)$: atender los desbordados en una cola FIFO (Goldberg y Tarjan) y descargar siempre el desbordado más alto (Cheriyan y Maheshwari), que según Cherkassky y Goldberg es la mejor variante. Y dos heurísticas que cambian mucho el tiempo real:

- **Reetiquetado global.** $h(u)$ nunca supera la distancia en aristas de u a t en G_f, y si $h(u) \geq n$, $h(u) - n$ no supera la distancia de u a s (ejercicio 26.4-8). De vez en cuando conviene fijar las alturas en esas distancias exactas, calculadas con un BFS desde t (y otro desde s para los que ya no llegan a t) sobre la red residual con las aristas volteadas: es el $d[v]$ del BFS de la sección 4.
- **Brecha** (*gap*). Si ninguna altura vale k, con $0 < k < n$, los vértices de altura mayor que k ya no pueden mandar nada a t, y se les puede subir de golpe por encima de n (Cormen, ejercicio 26.5-5).

Problemas del capítulo 26

Cada uno en una línea; los dos últimos conectan con la sección 11.

- **26-1, problema de escape.** En una cuadrícula con varios puntos de inicio, decidir si hay caminos ajenos en vértices desde todos ellos hasta la frontera; se reduce a flujo partiendo cada vértice en uno de entrada y otro de salida, unidos por una arista de capacidad 1.
- **26-2, cobertura mínima por caminos.** En un DAG, partir los vértices en el menor número de caminos ajenos; se arma un bipartito con una copia de salida y una de entrada por vértice y la respuesta es n menos el emparejamiento máximo.
- **26-3, consultoría algorítmica.** Escoger expertos con costo y trabajos con ganancia para maximizar la ganancia neta; la respuesta es la suma de ganancias menos la capacidad de un corte mínimo.
- **26-4, actualizar el flujo máximo.** Si una capacidad entera sube o baja en 1, recuperar un flujo máximo en $\mathcal{O}(n + m)$ con a lo más una búsqueda de camino aumentante.
- **26-5, flujo máximo por escalamiento.** Aumentar solo por caminos de capacidad residual al menos K, con K una potencia de 2 que se va partiendo a la mitad; con C la capacidad máxima, cuesta $\mathcal{O}(m^2 \log C)$.
- **26-6, Hopcroft-Karp.** Emparejamiento bipartito máximo en $\mathcal{O}(\sqrt{n}\, m)$, aumentando en cada fase por un conjunto maximal de caminos aumentantes más cortos y ajenos en vértices.

El ejercicio 26.4-5 pide resolver con empujar y reetiquetar el emparejamiento bipartito de la sección 11. Sobre la red de capacidades 1 de esa sección, reetiquetar al frente lo resuelve sin cambiar nada, en $\mathcal{O}(n^3)$.

Resumen de la parte de grafos

La tabla junta todos los algoritmos de la parte, en el orden en que aparecen, y leída de arriba abajo cuenta la historia de la parte. Los recorridos lineales, BFS y DFS, son la base de casi todo lo demás:

el orden topológico, las componentes, los puentes, el coloreo bipartito, los caminos en un DAG, la cerradura transitiva por recorridos y la búsqueda de caminos aumentantes. Donde aparece un logaritmo hay una cola de prioridad o un ordenamiento; donde hay pesos negativos aparece Bellman-Ford; y el único renglón exponencial, Held-Karp, es el de un problema NP-difícil.

Tres convenciones de la tabla. "Red de flujo" quiere decir grafo dirigido con capacidades $c \geq 0$, fuente y sumidero, sin aristas antiparalelas (sección 10). En restricciones de diferencia, p es el número de incógnitas y r el de restricciones, para no confundirlos con n y m. Los costos son con listas de adyacencia salvo que se diga otra cosa.

Algoritmo	**Problema**	**Requisitos**	**Estructura**	**Costo**	**Secc.**
Recorridos y estructura del grafo					
BFS	recorrido por capas; distancias en aristas	ninguno; sin pesos	cola	$\mathcal{O}(n+m)$; $\mathcal{O}(n^2)$ con matriz	4
DFS	recorrido en profundidad; tiempos y tipos de arista	ninguno	pila o recursión	$\Theta(n+m)$	4
Orden topológico con DFS	toda arista hacia adelante	dirigido acíclico (DAG)	DFS e inserción al frente	$\Theta(n+m)$	5
Kahn	lo mismo, quitando vértices sin entrada	dirigido; detecta ciclos	cola y arreglo gr^-	$\Theta(n+m)$	5
Componentes fuertemente conexas	clases de alcanzabilidad mutua	dirigido	dos DFS y grafo transpuesto	$\Theta(n+m)$	5
Puntos de articulación y puentes	vértices y aristas que desconectan	no dirigido, conexo	DFS con entra$[v]$ y low$[v]$	$\Theta(n+m)$	5
Hierholzer	circuito o camino euleriano	conexo; grados pares, o dos impares y salir de uno	pila y apuntador por lista	$\Theta(n+m)$	2
Coloreo bipartito con BFS	decidir si es bipartito; las dos partes	no dirigido	BFS con color por vértice	$\Theta(n+m)$	3
Held-Karp	agente viajero: ciclo hamiltoniano mínimo	completo y pesado; n chico	tabla $C[S][j]$ por subconjuntos	$\Theta(n^2 2^n)$ tiempo, $\Theta(n2^n)$ memoria	3
Árboles de expansión mínima					
Kruskal	árbol de expansión de peso mínimo	no dirigido, conexo, pesado	ordenamiento y conjuntos disjuntos	$\mathcal{O}(m \log n)$	6
Prim	lo mismo, creciendo un solo árbol	no dirigido, conexo, pesado	montículo con decrementar llave	$\mathcal{O}(m \log n)$ binario; $\mathcal{O}(m + n \log n)$ Fibonacci; $\Theta(n^2)$ arreglo	6

Algoritmo	Problema	Requisitos	Estructura	Costo	Secc.
Caminos más cortos					
Bellman-Ford	desde una fuente; detecta ciclos negativos	dirigido, pesos de cualquier signo	arreglos d y π; $n-1$ pasadas	$\mathcal{O}(nm)$	7
Caminos en un DAG	desde una fuente	DAG, pesos de cualquier signo	orden topológico	$\Theta(n+m)$	7
Dijkstra	desde una fuente	pesos no negativos	montículo de mínimos	$\mathcal{O}((n+m)\log n)$ binario; $\mathcal{O}(n^2)$ arreglo; $\mathcal{O}(n\log n+m)$ Fibonacci	7
Restricciones de diferencia	resolver $x_j - x_i \leq b$ o probar que no hay solución	dos incógnitas por restricción	Bellman-Ford en el grafo de restricciones	$\mathcal{O}(p^2+pr)$	7
Floyd-Warshall	entre todos los pares	sin ciclos negativos	matriz $n \times n$	$\Theta(n^3)$	9
Cerradura transitiva	alcanzabilidad entre todos los pares	dirigido	matriz booleana	$\Theta(n^3)$; $\Theta(n(n+m))$ con n recorridos	9
Johnson	entre todos los pares, grafos dispersos	sin ciclos negativos	Bellman-Ford y n veces Dijkstra	$\mathcal{O}(nm\log n)$ binario; $\mathcal{O}(n^2\log n+nm)$ Fibonacci	9
Flujo y emparejamiento					
Ford-Fulkerson	flujo máximo	red de flujo; capacidades enteras	red residual; BFS o DFS	$\mathcal{O}(m\,\lvert f^*\rvert)$	10
Edmonds-Karp	flujo máximo	red de flujo	red residual y BFS	$\mathcal{O}(nm^2)$	10
Emparejamiento bipartito por flujo	emparejamiento máximo	bipartito	red de capacidades 1 y Ford-Fulkerson	$\mathcal{O}(nm)$	11
Hopcroft-Karp	emparejamiento máximo	bipartito	BFS por capas y DFS	$\mathcal{O}(\sqrt{n}\,m)$	11
Empujar y reetiquetar genérico	flujo máximo	red de flujo	alturas, excesos y pila de desbordados	$\mathcal{O}(n^2m)$	12
Reetiquetar al frente	flujo máximo	red de flujo	lista L y listas de vecinos	$\mathcal{O}(n^3)$	12